

multica-ai / multica

github · main · 0db0055

文档导出 · 25 个文件

目录

总览 (3)

Multica 项目概览

系统架构

改进建议

详细文档 (22)

智能体协作看板

任务与问题管理

自动化规则

智能体对话

小队管理

技能管理

频道集成

版本控制集成

工作区仪表盘

通知中心

自定义属性

项目管理

Web 应用

桌面应用

移动应用

自部署方案

CLI 命令行工具

智能体运行时

实时通信基础设施

用户认证体系

定时调度器

国际化框架

Multica 项目概览

overview · overview.md

Multica 项目概览

智能体，也在看板上。

Multica 是一个开源的团队工作区。你像给同事派活一样，把任务交给 AI 编码智能体——它自己接手、边做边汇报、卡住了主动说，做完交回来给你审。可自部署，支持 20 种智能体 CLI，不绑定任何厂商。

核心声明源自 [README.zh.md](#)。

Multica 解决什么问题

今天的使用者手中同时开着 Claude Code、Codex 和另外几个智能体，每一个都关在自己的终端标签页里，会话一关就什么都不记得，同一段上下文可能一天讲四遍。智能体越加越多，人越忙。

Multica 的回答是：把这些智能体和人类队友放进同一个工作区。任务派给智能体，它自己接手，在用户自己的机器上跑，边做边评论，做完挪到审核中等验收。从最初的想法，到中间的每一次执行、每一个决定，再到最后的 diff，全都挂在同一个任务下——没有人需要重新捋一遍上下文，也没有任何东西能不经人点头就上线。

来源：[README.zh.md](#)，"Multica 是什么" 一节。

愿景：人机混编的团队操作系统

Multica 的全称是 **Multiplexed Information and Computing Agent**。这个名字向 20 世纪 60 年代的操作系统 Multics 致意——Multics 首创了分时系统，让多个人共享同一台机器，却又都像独占它一样。后来的 Unix 则是对 Multics 的有意简化：一个用户、一个任务、一种优雅哲学。

Multica 认为同样的转折点正在重演。此后几十年，软件团队一直是单线程的：一个工程师、一个任务、一次一个上下文切换。AI 智能体把这个前提打破了。Multica 把"分时"带回今天——只不过这一次，被多路复用的"用户"，既是人，也是能自己干活的智能体。

在 Multica 里，智能体就是队友。它们接任务、报进展、提阻塞、交代码，和人类同事没有两样。负责人选择器、动态时间线、任务生命周期，还有底下那套运行时，从第一天起就是照着这个想法搭的。和当年的 Multics 一样，核心理念还是"多路复用"：小团队不该因为人少，就只能干出小团队的量。有了合适的系统，两个工程师带一队智能体，能有二十个人的推进速度。

来源：[VISION.zh.md](#)，"为什么叫 Multica" 一节。

Multica 的方向是成为人机协作的**记录系统与行动中枢**。人设定方向并对结果负责，智能体持续推动工作，Multica 保存团队的理解并转化为协调一致的行动。它不是要造一家脱离人类控制、自己运转的"无人公司"，而是要构建人和智能体共同完成重要工作的共享操作系统。

来源：[VISION.zh.md](#)，"愿景，落到真实工作中" 一节。

核心能力全景

Multica 的核心业务能力围绕四个维度展开：

组一支队伍

Claude Code、Codex、Cursor、Kimi——不用挑一个，全部可以招进工作区。

- **20 种智能体 CLI 开箱支持**：Claude Code、Codex、Cursor、GitHub Copilot、OpenCode、Kimi、Qwen Code、Antigravity、CodeBuddy、Grok、Hermes、Pi、Trae CLI 等共 20 种 AI 编码工具，守护进程自动扫描本机已安装的 CLI 并注册为运行时。
- **智能体即队友**：起个名字、选个提供方、配台运行时，它就上了看板，与人类成员出现在同一个负责人选择器里。
- **小队 (Squad)**：人和智能体混编成队，由 leader 决定谁来接活。
- **技能 (Skills)**：解决过一次的问题沉淀为可复用技能，全团队智能体都能调用。
- **本地运行时**：智能体的"工位"就是用户自己的机器——守护进程跑在笔记本或云主机上，代码不出门。

来源：[README.zh.md](#)，"组一支队伍" 及 "运行时" 章节。

把活交出去

一开始只是任务里潦草的三句话，最后变成一个 pull request。

- **分配任务**：像挑同事一样从下拉菜单中选一个智能体当负责人，剩下的它自己来。
- **自动化 (Autopilot)**：日报、巡检、周报等定时任务按 cron 表达式自动执行，无需人工催促。
- **Chat 对话**：直接在聊天频道里问工作区，或者不建任务就把活派出去。
- **项目管理**：把工作归类到项目中，顺手挂上智能体需要访问的仓库和文档。

来源：[README.zh.md](#)，"把活交出去" 章节。

看得见，也管得住

这活哪个智能体动过？它到底跑了什么？花了多少？点开那次运行即可查看。

- **执行日志**：每次工具调用、命令和报错都带时间戳，可以完整回放。
- **Token 用量追踪**：每次运行的花费按智能体、按任务计算，一目了然。
- **人来验收**：智能体完成的活先进入"审核中"状态，不直接合入 main，上线由人说了算。
- **收件箱 (Inbox)**：只在智能体需要人拍板时才提醒，而不是每一步都来打扰。
- **重试与超时**：失败的 task 会自行重试，或停下来给出失败原因。

来源：[README.zh.md](#)，"看得见，也管得住" 章节。

整套都归你

用户的机器、用户的 Git 服务、用户的规矩——还有一份把智能体也算进去的审计记录。

- **整套自部署**：Docker Compose 或 Helm，装在用户自己的基础设施上。
- **任意 Git 服务**：GitHub、GitLab、Gitee、Forgejo，自建实例也行。
- **工作区隔离**：按团队隔离智能体、任务和设置。
- **细粒度权限**：owner、admin、member 三级角色，再精确到谁能跑哪些智能体。
- **安全模型**：智能体碰得到什么、碰不到什么，边界清晰。
- **频道集成**：Slack、飞书、钉钉、企业微信，在团队本来就在聊天的地方触发和跟进智能体工作。
- **CLI 与 API**：界面上能点的，CLI 和 API 里都能调。智能体操作 Multica 用的就是同一套 CLI。

来源：[README.zh.md](#)，"整套都归你" 章节。

三端客户端矩阵

Multica 提供三种形态的客户端，覆盖不同的使用场景。

客户端	技术栈	覆盖平台	说明
Web 应用	Next.js 16 (App Router)	浏览器	完整功能，无需安装即可使用
桌面应用	Electron，复用 Web UI	macOS / Windows / Linux	下载即用，自动注册本机为运行时
移动应用	Expo / React Native	iOS	同一工作区随身携带，需从源码编译

桌面端下载安装后，会自动把本机注册为运行时并检测已安装的 AI 编程工具，无需额外配置。移动端目前尚未上架 App Store，可通过 `pnpm ios:mobile:device:prod:release` 命令从源码编译安装到 iPhone。

来源：技术栈信息来自 [README.zh.md](#) "架构" 章节。移动端构建说明来自 [apps/mobile/README.md](#)。

自部署方案

Multica 完全支持在自己的基础设施上运行，与云服务提供相同的功能。

快速安装（两条命令）：

macOS / Linux 上：

```
curl -fsSL https://raw.githubusercontent.com/multica-ai/multica/main/scripts/install.sh | bash -s -- --with-server
multica setup self-host
```

Windows PowerShell 上先设 `$env:MULTICA_MODE="with-server"`，再运行对应的安装脚本。

这会拉取 GHCR 上的官方镜像，需要 Docker。也支持通过 Homebrew 单独安装 CLI：`brew install multica-ai/tap/multica`。

手动部署：

执行 `make selfhost`，自动创建 `.env`、生成 `JWT_SECRET`，通过 Docker Compose 启动全部服务。默认拉取最新稳定版 GHCR 镜像，也可以 `make selfhost-build` 从当前源码构建。

来源：[README.zh.md](#) "开始使用" 章节以及 [SELF_HOSTING.md](#)。

开放式架构理念

不绑定任何厂商

Multica 不自带模型，不锁定任何 LLM 提供方。它驱动的是用户本来就装好、登录好的那些智能体 CLI，换提供方就是切一个下拉框，谈不上迁移。架构上自带 LLM 提供方接入能力，用户可以更换智能体后端、扩展 API，拥有整个技术栈的控制权。

来源：[README.zh.md](#) "运行时" 章节。

完全开源

Multica 完全开源，采用 Apache License 2.0 并附加针对托管服务和商业嵌入的条件。每一行代码都可审计——用户可以确切了解智能体如何做决策、任务如何路由、数据流向何方。自部署、改代码、在它之上做东西均可。

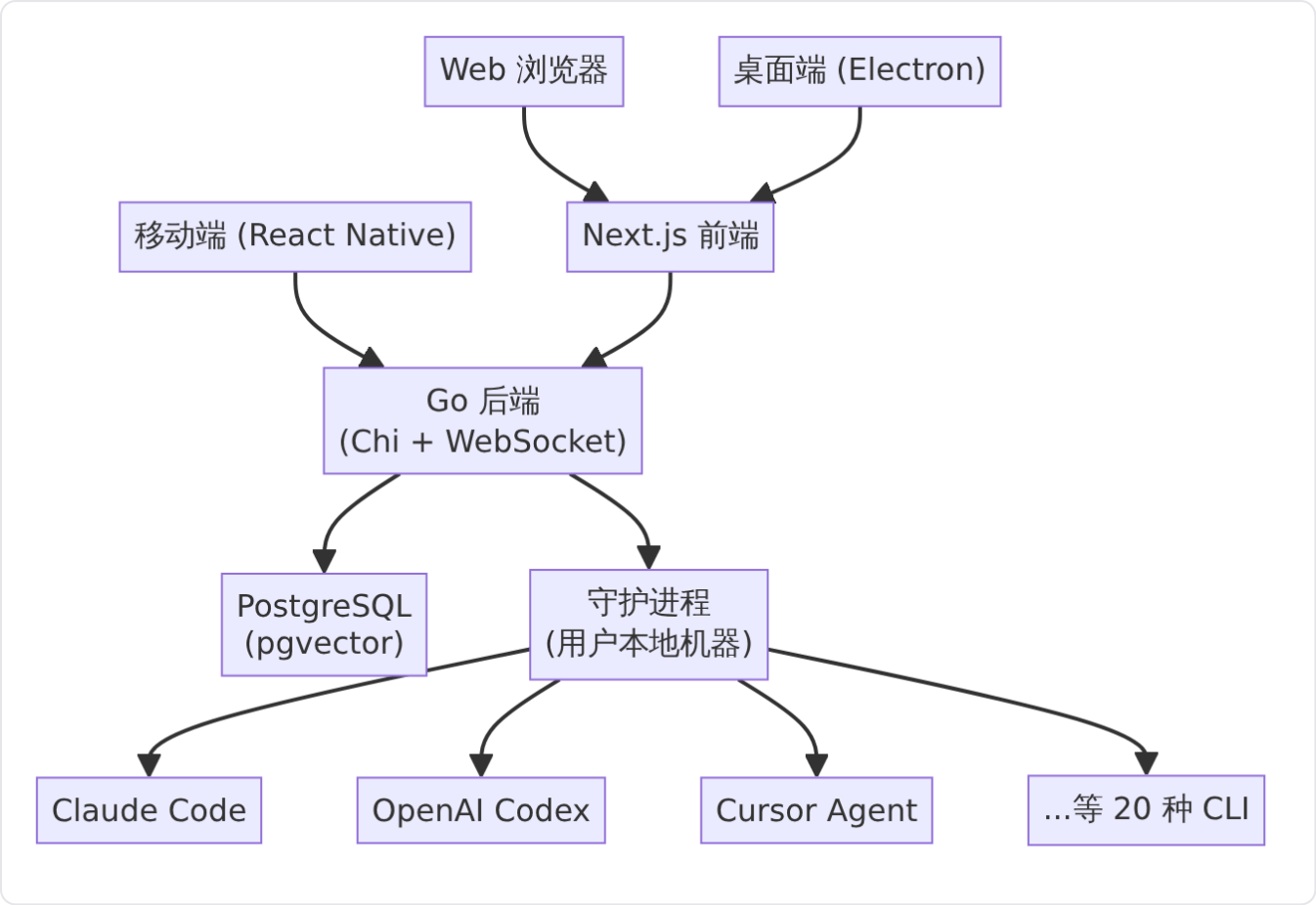
来源：[README.zh.md](#) "开源协议" 章节。

社区驱动

Multica 与社区一起建设，而不仅仅是为社区建设。用户可以贡献技能、集成和智能体后端，让每个人受益。项目通过 GitHub、Discord 和 X 与社区保持沟通。

来源：[README.zh.md](#) 首部。

整体架构概览



Multica 的架构分为四个核心层：

层级	技术栈
前端	Next.js 16 (App Router)，桌面端通过 Electron 复用 Web UI 包，移动端通过 Expo / React Native 独立构建
后端	Go 语言，Chi 路由器，sqlc 数据访问，gorilla/websocket 实时通信
数据库	PostgreSQL 17 + pgvector 向量扩展
智能体运行时	本地守护进程，在用户机器上拉起 20 种受支持的智能体 CLI

Web、桌面端、移动端三条通道最终汇集到同一 Go 后端。后端通过 WebSocket 向本地守护进程下发 task，守护进程再拉起对应的智能体 CLI 执行实际工作。代码从不过 Multica 服务器，平台只负责任务状态协调和事件广播。

来源：[README.zh.md](#) "架构" 章节。

开始使用

无需打开终端：直接在 [multica.ai](#) 注册，或下载 [Multica 桌面端](#) (macOS / Windows / Linux)。唯一前提是跑智能体的机器上需安装并登录至少一个受支持的智能体 CLI——Multica 负责驱动它们，但不替用户安

装。

四步跑通第一个智能体：

1. **登录**：浏览器或桌面端均可。
2. **接入电脑**：桌面端自动注册本机并检测已安装的 CLI；网页版则在"运行时"页面手动添加。
3. **创建智能体**：命名、选提供方、选运行时，或通过 AI 对话描述需求自动生成配置。
4. **派任务**：建一个任务，负责人选成这个智能体，它会自己接手、执行、评论、完成并提交审核。

来源：[README.zh.md](#) "五分钟跑通第一个智能体" 章节。

相关页面

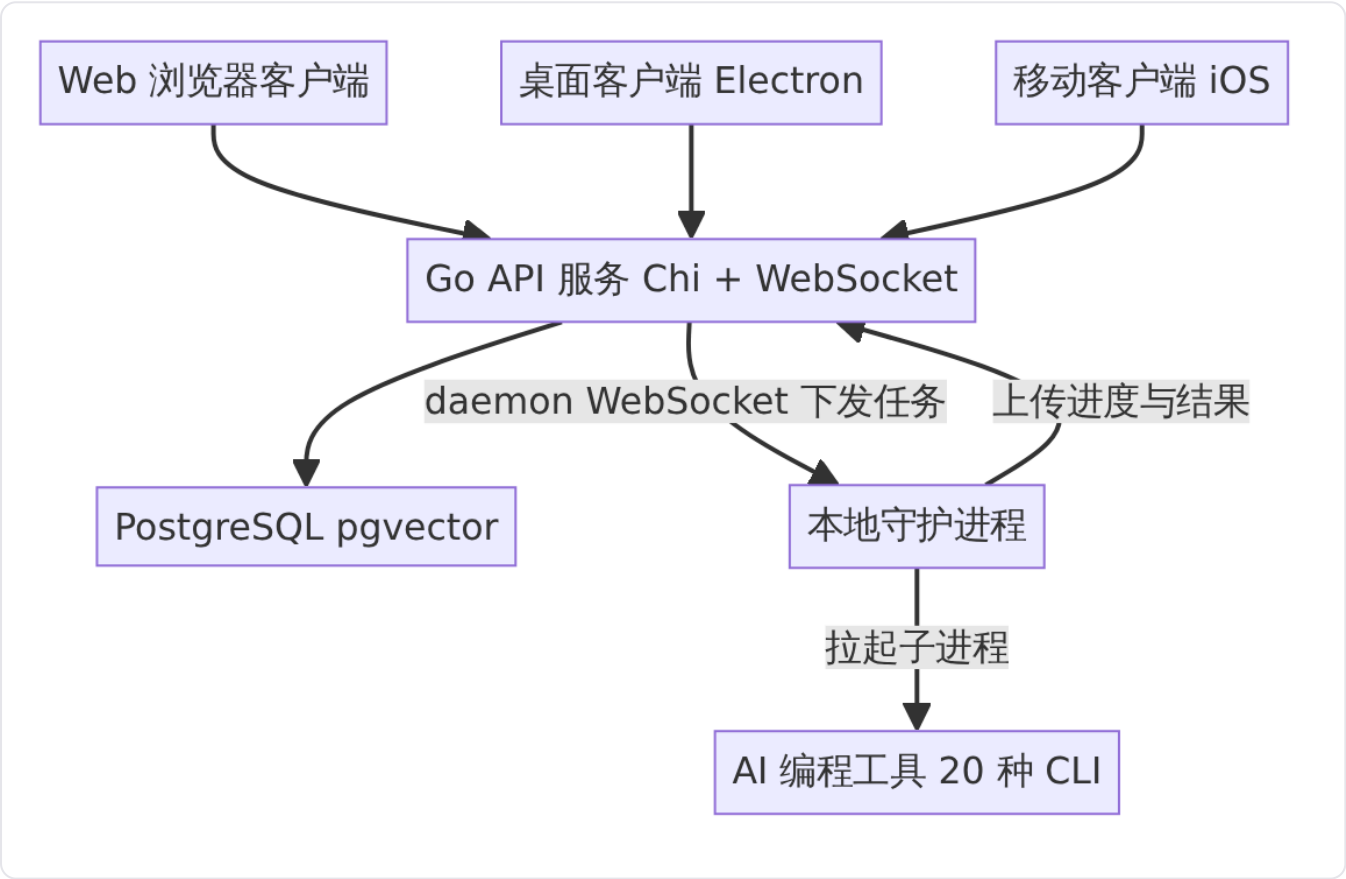
- [系统架构](#) — Multica 的完整技术架构和 monorepo 布局
- [智能体协作看板](#) — 核心业务能力：将 AI 智能体纳入看板协作
- [任务与问题管理](#) — 任务生命周期、分配、优先级与状态流转
- [Web 应用](#) — Next.js 前端的技术细节
- [自部署方案](#) — Docker Compose 与 Helm 部署指南

系统架构

Multica 由一个 Go 后端、三种客户端（Web、桌面、移动）以及运行在用户本地机器上的守护进程组成。PostgreSQL 是业务数据的权威来源，守护进程通过 WebSocket 从服务端领取任务并在本地拉起 AI 编程工具执行。本章从代码分层、数据流向和组件关系三个维度说明系统如何协作。

整体架构概览

从产品视角看，Multica 的系统拓扑分为四个层次：客户端、API 网关层、执行层和数据持久层。



来源：[README.zh.md](#) 和 [apps/docs/content/docs/developers/architecture.zh.md](#)

- **客户端层**：Web (Next.js 16 App Router)、桌面 (Electron，复用 Web UI 包) 和移动端 (Expo / React Native iOS)，三者通过 HTTP + WebSocket 与后端通信。
- **Go API 层**：基于 Chi 路由器的 HTTP 服务，同时托管两条独立的 WebSocket 路径——面向用户客户端的 `realtime` 推送和面向守护进程的 `daemonws` 控制通道。
- **执行层**：本地守护进程 (`multica daemon`) 常驻在用户机器上，从服务端认领任务后在本地拉起对应的 AI 编程工具 CLI 进程。

- 持久层：PostgreSQL 17 + pgvector 存储所有业务数据，Redis 为可选组件，用于跨实例实时事件、缓存和临时协调。

仓库结构与 Monorepo 布局

Multica 采用 pnpm + Turborepo 管理的 monorepo 架构。仓库顶层 `package.json` 定义了 pnpm@10.28.2 作为包管理器，并通过 `turbo.json` 编排构建流水线。各模块职责与技术选型如下：

目录	职责	主要技术
server/	API、认证、任务调度、集成、CLI 和守护进程	Go、Chi、sqlc、gorilla/websocket
apps/web/	浏览器客户端和 landing page	Next.js App Router
apps/desktop/	桌面客户端和本机进程管理	Electron、electron-vite
apps/mobile/	独立的 iOS 客户端	Expo、React Native、NativeWind
apps/docs/	多语言文档站	Next.js、Fumadocs
packages/core/	API client、类型、查询、mutation 和平台无关业务逻辑	TanStack Query、Zustand
packages/ui/	不包含业务逻辑的基础 UI	shadcn、Base UI
packages/views/	Web 与 Desktop 共用的业务页面和组件	React
packages/tsconfig/、 packages/eslint-config/	共享工具配置	TypeScript、ESLint

来源：[apps/docs/content/docs/developers/architecture.zh.mdx](#)

共享包直接导出 `.ts` 和 `.tsx` 源文件，由使用它们的应用编译。依赖方向是 `views → core + ui`；`core` 和 `ui` 互不依赖。

Web 与 Desktop 的代码共享

Web 和 Desktop 共用三层代码，平台差异被限制在应用层：

- `packages/core` ——处理 API 请求、缓存、权限和平台无关状态；
- `packages/ui` ——提供基础 UI 组件（不含业务逻辑）；
- `packages/views` ——组合业务页面和组件。

路由、cookie 和 Electron IPC 等平台能力留在应用层。共享页面通过 `NavigationAdapter` 导航，不直接导入 `next/*` 或 `react-router-dom`。[apps/docs/content/docs/developers/architecture.zh.mdx](#) 描述了这一分层模式。

移动端不复用这些 React 页面。它可以导入 `@multica/core` 的类型和纯函数，但拥有自己的 UI、query key、状态、实时订阅和发布流程。[apps/mobile/CLAUDE.md](#) 明确定义了移动端从 `packages/` 可导入的

范围：仅限 `import type` 和纯函数。

前端状态管理

服务端数据和客户端状态在架构上明确分离：

- **TanStack Query** 持有 `issue`、智能体、成员、收件箱等服务端数据；
- **Zustand** 持有筛选条件、草稿、弹窗和布局等客户端状态；
- 当前工作区由路由决定，只在平台层为请求、持久化命名空间和重连做必要镜像；
- **React Context** 只传递工作区 ID、导航适配器等平台 `plumbing`。

`WebSocket` 事件更新或失效 `TanStack Query` 缓存，不把服务端对象复制进 `Zustand`。API 返回值在 `packages/core/api/` 边界通过 `zod schema` 解析——已安装的桌面端可能连接更新的后端，因此不能直接把网络 JSON 强制转换成 `TypeScript` 类型。

Web 端的工作区布局入口在 `apps/web/app/[workspaceSlug]/layout.tsx`，它通过 `useAuthStore` 检查认证状态，通过 `workspaceBySlugOptions` 查询工作区信息，并在用户无权访问时渲染 `NoAccessPage`。

后端分层架构

Go 后端的入口位于 `server/cmd/`，主程序在 `server/cmd/server/main.go` 启动 HTTP API、`WebSocket`、调度器和集成 worker。主要入口点如下：

入口	作用
<code>server</code>	启动 HTTP API、 <code>WebSocket</code> 、调度器和集成 worker
<code>multica</code>	CLI 与本机守护进程
<code>migrate</code>	执行数据库 migration
<code>backfill_*</code>	特定版本的数据回填工具

来源：[apps/docs/content/docs/developers/architecture.zh.mdx](https://github.com/antlabs/antlabs/blob/main/apps/docs/content/docs/developers/architecture.zh.mdx)

请求在服务端内部按以下方向流动：

```
router → middleware → handler → service → sqlc query → PostgreSQL
```

其中 `server/cmd/server/router.go` 中的 `NewRouterWithOptions` 函数构建完整的 `Chi` 路由器，装配认证、`CORS`、限流等中间件，注册所有 API 路由。路由按功能分组：`/auth/*`（认证）、`/api/*`（业务 API）、`/ws`（用户实时连接）、`/api/webhooks/*`（外部集成回调）等。

中间件层在 `server/internal/middleware/` 下实现了认证（`auth.go`）、工作区解析（`workspace.go`）、守护进程认证（`daemon_auth.go`）、客户端识别（`client.go`）、限流（`ratelimit.go`）、`CSP`（`csp.go`）和 `CloudFront`（`cloudfront.go`）等横切关注点。

业务逻辑的分层协议为：

- `internal/handler/` ——解析 HTTP 输入并生成响应；
- `internal/service/` ——负责跨查询的业务流程和事务；
- `pkg/db/queries/` ——手写 SQL；
- `pkg/db/generated/` ——sqlc 生成的 Go 代码，不可直接编辑；
- `internal/integrations/` ——处理 GitHub、Slack、飞书、钉钉、企业微信等外部事件；
- `internal/storage/` ——处理本地或 S3 附件。

PostgreSQL 是业务数据的权威来源。Redis 是可选组件，用于跨实例实时事件、缓存或临时协调；未配置时单实例开发环境使用进程内实现。

智能体运行时体系

智能体运行时的统一接口定义在 [server/pkg/agent/agent.go](#)。Backend 接口是所有 AI 编程工具提供方的抽象：

```
type Backend interface {
    Execute(ctx context.Context, prompt string, opts ExecOptions) (*Session, error)
}
```

`ExecOptions` 结构体承载了丰富的执行控制参数：工作目录（`Cwd`）、模型选择（`Model`）、超时控制（`Timeout`）、会话恢复（`ResumeSessionID`）、思维级别（`ThinkingLevel`）、MCP 配置（`McpConfig`）等。每个提供方后端按需消费这些参数，不支持的字段被静默忽略而非报错，使得运行时能力可以渐进式扩展。

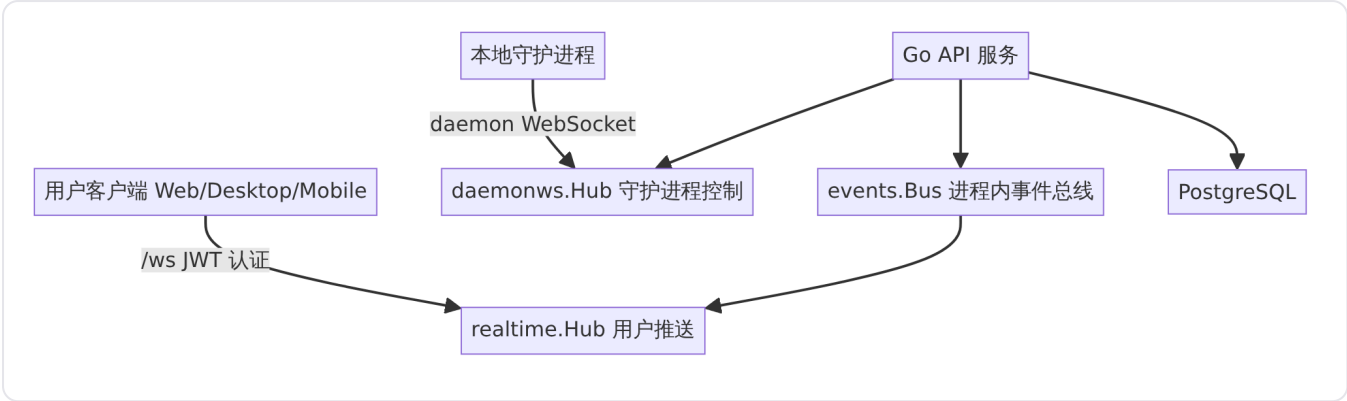
执行过程通过 `Session` 暴露两个通道：`Messages` 流式推送智能体的实时活动（文本输出、工具调用、思考过程），`Result` 在运行结束时发送最终结果。

模型发现由 [server/pkg/agent/models.go](#) 定义。每个提供方 CLI 通过运行时的 `models` 子命令列出可用模型，按供应商分组展示，支持默认模型标记和服务等级（如 Codex 的 Fast/Priority）。

守护进程的核心逻辑在 [server/internal/daemon/daemon.go](#)，它负责管理运行时注册、任务认领与调度、Git 仓库准备、技能包下载、执行环境构建，以及将 `ExecOptions` 映射到具体提供方 CLI 的参数。

实时通信基础设施

Multica 有两条独立的 WebSocket 路径，分别服务于不同的通信场景：



realtime 路径——用户推送

[server/internal/realtime/hub.go](#) 定义了面向用户客户端的 WebSocket Hub。它支持按工作区路由消息，通过 JWT 认证连接，校验成员资格，并将工作区 Slug 解析为 UUID。当 issue、评论、任务状态等发生变化时，服务端通过此通道向对应工作区的所有在线客户端广播更新。

daemonws 路径——守护进程控制

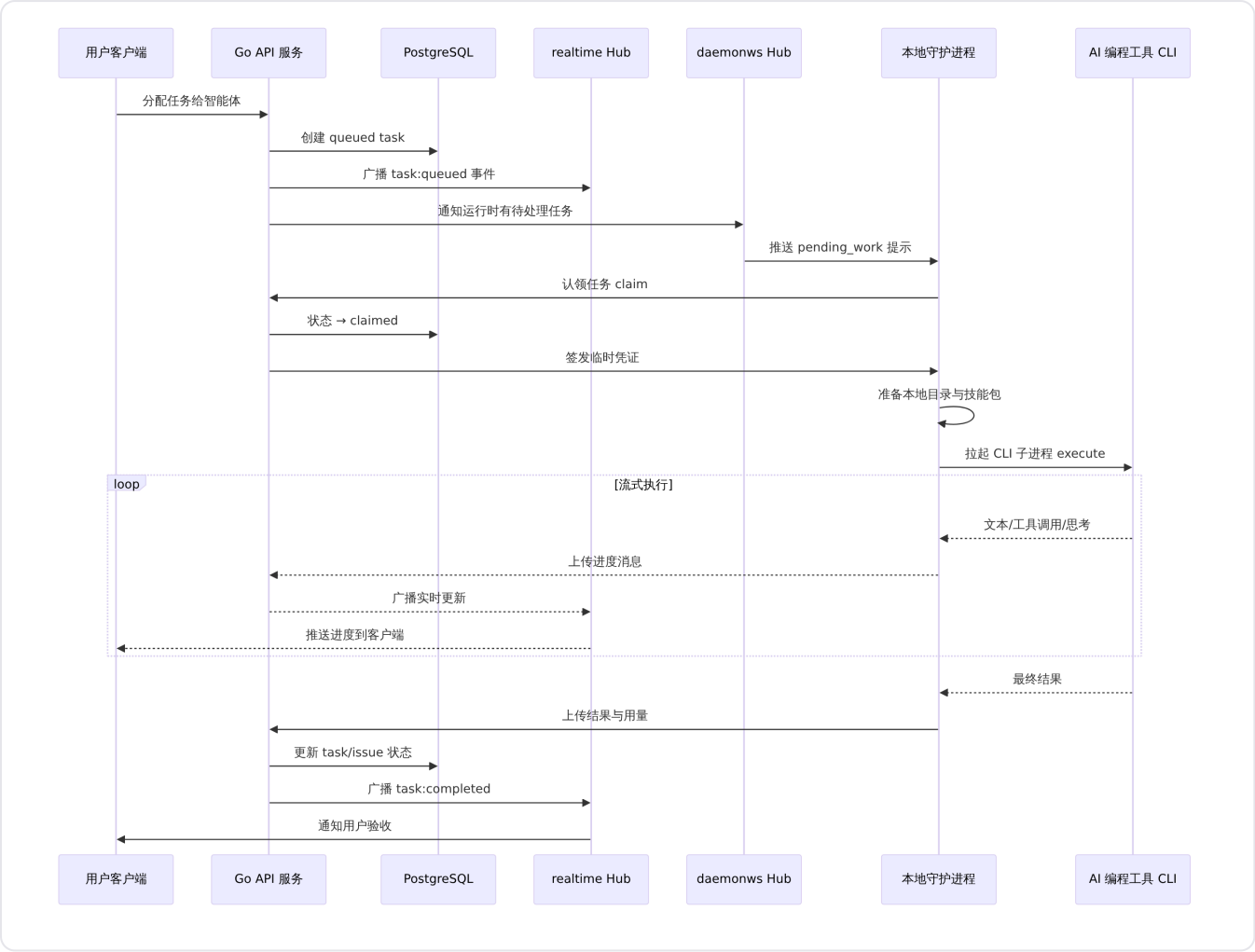
[server/internal/daemonws/hub.go](#) 定义了守护进程侧的 WebSocket Hub。它维护已认证的守护进程连接，按 DaemonID、UserID 和 WorkspaceID 确定连接范围，支持任务下发、运行时唤醒和守护进程 RPC。

事件总线

两条 WebSocket 路径的背后是进程内同步发布/订阅事件总线，定义在 [server/internal/events/bus.go](#)。Bus 支持按事件类型（如 issue:created）和全局订阅（SubscribeAll），事件携带 WorkspaceID 用于路由到正确的 Hub 房间，并可选携带 TaskID 和 ChatSessionID 做更细粒度的作用域提示。

一次智能体执行的完整路径

下面的序列图展示了一个从用户分配任务到智能体完成工作的完整数据流：



来源：此流程综合了 [apps/docs/content/docs/developers/architecture.zh.mdx](#) 的实时连接说明、[server/pkg/agent/agent.go](#) 的 Backend 接口定义以及 [server/internal/daemon/daemon.go](#) 的任务处理逻辑。

定时调度器

[server/internal/scheduler/](#) 实现了进程内定时任务管理器。Manager 按可配置的 `TickInterval`（默认 30 秒）周期性扫描注册的 job，对到期计划执行对应的业务逻辑。当前注册的 job 包括：

- **autopilot 定时触发**（`jobs_autopilot.go`）：按 cron 表达式评估自动化规则，符合条件的触发智能体执行；
- **任务用量统计**（`jobs_task_usage.go`）：定期汇总每个运行时的 token 消耗和执行时长。

调度器使用 PostgreSQL 作为协调后端，通过唯一 claim 机制保证多实例部署时同一计划只被一个节点执行。

桌面端特有架构

桌面端基于 Electron，核心入口在 [apps/desktop/src/main/index.ts](#)。主进程负责：

- 原生窗口管理：多窗口支持（主窗口 + 独立 issue 窗口），窗口位置/大小持久化；
- 守护进程管理（`daemon-manager.ts`）：启动、监控和升级本地 `multica` 守护进程；
- 认证协调（`auth-session-coordinator.ts`）：管理浏览器登录与桌面端的认证状态共享；
- 自动更新（`updater.ts`）：基于 `electron-updater` 的增量更新；
- 键盘快捷键（`keyboard-shortcuts.ts`）：全局快捷键注册；
- 本地目录访问（`local-directory.ts`）：文件系统桥接；
- 上下文菜单（`context-menu.ts`）：原生右键菜单；
- 导航守卫（`navigation-guard.ts`）：防止意外离开。

桌面端的渲染进程复用 `packages/views/` 中的业务页面，通过 Electron IPC 调用主进程的原生能力而非直接访问 Node.js API。

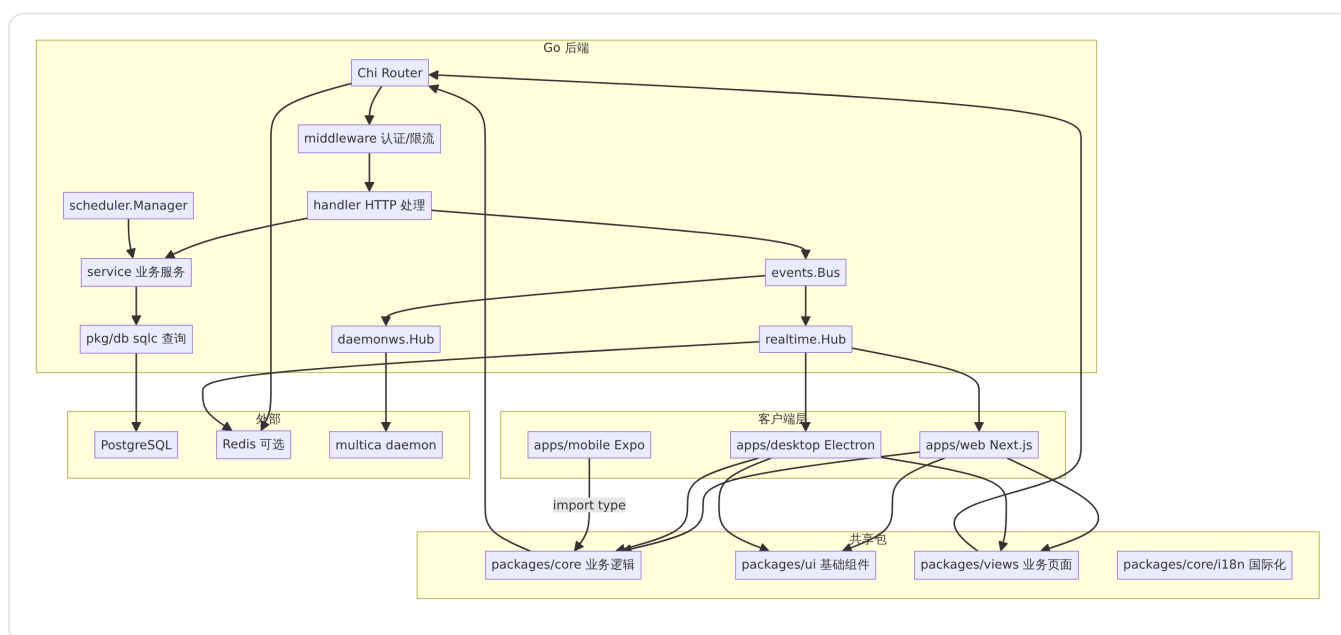
国际化架构

国际化框架位于 [packages/core/i18n/index.ts](#)，采用了分层设计：

- 核心层（`index.ts`）：纯 TypeScript，零 React 依赖，可在 RSC / Edge / Node.js 中间件中安全导入；
- **React 层**（`i18n/react`）：`I18nProvider`、`useLocaleAdapter` 等 React 绑定，与核心层分离以避免 RSC 构建问题；
- 浏览器层（`i18n/browser`）：基于 cookie 的 Locale 适配器，仅在 DOM 环境中可用。

这一分层保证了国际化能力在服务端渲染、客户端和桌面端的 Electron 渲染进程中均能正常工作。

关键组件关系总览



来源：组件关系综合自 [server/cmd/server/router.go](#)、[server/cmd/server/main.go](#)、[server/internal/events/bus.go](#)、[server/internal/scheduler/manager.go](#) 和 [apps/docs/content/docs/developers/architecture.zh.mdx](#)。

改进建议

overview · improvements.md

改进建议

本文档基于对 Multica 代码库的静态分析，识别出以下可改进的架构与可靠性问题。所有发现均附有已验证的源码引用，不涉及推测性性能瓶颈或未经证实的漏洞。

1. 事件总线在 HTTP 请求路径中同步派发且无超时保护

类别

架构 / 可靠性

优先级

中

置信度

高

观察

`events.Bus` 是进程内同步发布/订阅事件总线，其 `Publish` 方法在当前 goroutine 中依次调用所有匹配的类型处理器和全局处理器，直至全部执行完毕才返回：

```
// server/internal/events/bus.go (第 60–88 行)
func (b *Bus) Publish(e Event) {
    b.mu.RLock()
    handlers := b.listeners[e.Type]
    globals := b.globalHandlers
    b.mu.RUnlock()

    for _, h := range handlers {
        func() {
            defer func() {
                if r := recover(); r != nil { ... }
            }()
            h(e)
        }()
    }

    for _, h := range globals {
        func() {
            defer func() { ... }()
            h(e)
        }()
    }
}
```

同一 `Publish` 方法被 `Handler` 的便捷方法在 HTTP 请求处理过程中直接调用:

```
// server/internal/handler/handler.go (第 541–584 行)
func (h *Handler) publish(eventType, workspaceID, actorType, actorID string, payload any) {
    h.Bus.Publish(events.Event{
        Type: eventType, WorkspaceID: workspaceID,
        ActorType: actorType, ActorID: actorID, Payload: payload,
    })
}

func (h *Handler) publishTask(eventType, workspaceID, actorType, actorID, taskID string, payload any) {
    h.Bus.Publish(events.Event{
        Type: eventType, WorkspaceID: workspaceID,
        ActorType: actorType, ActorID: actorID, TaskID: taskID, Payload: payload,
    })
}

func (h *Handler) publishChat(eventType, workspaceID, actorType, actorID, chatSessionID string, payload any) {
    h.Bus.Publish(events.Event{
        Type: eventType, WorkspaceID: workspaceID,
        ActorType: actorType, ActorID: actorID, ChatSessionID: chatSessionID, Payload: payload,
    })
}
```

此外, `TaskService` 和 `AutopilotService` 也在内部调用中同步发布事件。整个代码库中 `Bus.Publish` 共有 34 处调用点, 横跨 `handler` 层和 `service` 层。

影响

当任意事件订阅者执行缓慢（例如持有锁等待、执行 I/O 操作、或逻辑陷入死循环）时，整个 `Publish` 调用被阻塞，进而直接阻塞调用它的 HTTP 处理器，导致客户端响应延迟增加或请求超时。虽然 `Publish` 对单个 handler 的 panic 做了恢复保护，但**没有任何超时或截止时间机制**——一个永不返回的 handler 会使 HTTP 响应永久挂起。

当前代码库中的订阅者负责将事件转发到 WebSocket Hub（用于实时推送），如果 WebSocket 写入路径发生阻塞，HTTP 响应时间将与 WebSocket 写入时间耦合。

建议

为 `Bus.Publish` 增加可选的超时上下文参数，或提供异步发布变体（例如 `PublishAsync`），允许 HTTP 层调用者在有限的时间内等待派发完成。同时可以考虑为每个 handler 包装独立的超时 context，防止单个慢速订阅者拖累所有后续订阅者。

证据

- [server/internal/events/bus.go](https://github.com/GoogleCloudPlatform/gcp-go/blob/master/server/internal/events/bus.go)
- [server/internal/handler/handler.go](https://github.com/GoogleCloudPlatform/gcp-go/blob/master/server/internal/handler/handler.go)

局限性

未进行运行时性能测量。当前 WebSocket 写入通常极快（仅写入连接缓冲区），实际 HTTP 延迟影响可能较小。风险主要体现在未来新增订阅者的场景中：一旦有订阅者执行数据库查询或外部 API 调用，延迟问题会立即暴露。

2. 事件总线缺少背压与发布者超时反馈

类别

可靠性

优先级

低

置信度

中

观察

`Bus.Publish` 方法对所有发布者一视同仁：无论调用方是 HTTP handler（需要低延迟）、service 层（中等延迟容忍）还是 daemon 内部（高延迟容忍），都使用完全相同的同步派发路径。发布者无法获知派发是否在合理时间内完成，也无法在超时后放弃等待。

当前架构中，发布者无法表达"我只愿意等 100ms，超时则跳过事件派发"这一类需求。

影响

如果未来有订阅者因为外部依赖（例如调用 Redis、写入文件系统）而变慢，所有事件发布点（包括 HTTP handler）都会受到同等程度的影响，且无法通过调用侧的超时策略进行隔离。

建议

在 `Bus` 上增加 `PublishContext(ctx context.Context, e Event) error` 方法，允许调用者传入带超时的 context。HTTP handler 可使用 `context.WithTimeout(r.Context(), 100*time.Millisecond)` 调用，service 层可传入 `context.Background()` 保持当前行为不变。

证据

- [server/internal/events/bus.go](#)（整体设计）
- [server/internal/service/task.go](#)（service 层发布点示例）
- [server/internal/service/autopilot.go](#)（autopilot 发布点示例）

局限性

当前所有已知订阅者均为轻量级操作（WebSocket 写入、内存状态更新），实际阻塞风险较低。此建议主要是防御性架构改进，而非针对已确认的线上问题。

3. 调度器 `TickInterval` 默认值 30 秒在单 tick 多作业场景下可能延迟累积

类别

可靠性

优先级

低

置信度

中

观察

调度器 `Manager` 的默认 `TickInterval` 为 30 秒，所有注册的 Job 在同一个 tick 内串行执行：

```
// server/internal/scheduler/manager.go (第 50–51 行)
if opts.TickInterval <= 0 {
    opts.TickInterval = 30 * time.Second
}
```

```
// server/internal/scheduler/manager.go (第 126–141 行)
func (m *Manager) RunOnce(ctx context.Context) error {
    now, err := dbNow(ctx, m.pool)
    if err != nil { return err }
    for _, job := range m.snapshot() {
        if err := m.runJob(ctx, job, now); err != nil {
            m.logger.Warn("job tick error", "job", job.Name, "error", err)
        }
    }
    return nil
}
```

而在 `runJob` 内部，每个 `scope` 下的每个 `plan` 也是串行 `processPlan` 调用。对于 `CatchUpEveryPlan` 模式且累积了多个 `plan` 的作业，一次 `tick` 可能消耗相当长的时间。

影响

如果注册了多个 `Job`（例如 `Autopilot` 调度触发 + 其他定时任务），且其中某个 `Job` 的 `handler` 执行时间接近或超过 30 秒，则后续 `Job` 的有效调度间隔会被拉长，偏离期望的 `cron` 语义。在极端情况下（多个慢 `Job` 叠加），可能导致调度漂移。

建议

考虑为每个 `Job` 启用独立的 `goroutine` 执行（`go m.runJob(...)`），并使用 `errgroup` 或 `sync.WaitGroup` 等待所有 `Job` 完成，同时为每个 `Job` 设置独立的超时 `budget`，防止一个慢 `Job` 阻塞整个调度循环。

证据

- [server/internal/scheduler/manager.go](#)（默认 `TickInterval`）
- [server/internal/scheduler/manager.go](#)（串行 `job` 执行）
- [server/internal/scheduler/manager.go](#)（串行 `plan` 迭代）

局限性

未进行运行时测量。当前注册的 `Job` 数量和每个 `Job` 的执行时间未知。如果当前只有少量轻量级 `Job`，则此问题不会在生产环境中暴露。此建议适用于未来 `Job` 数量增长的场景。

智能体协作看板

detail · agent-collaboration.md

智能体协作看板

智能体协作看板是 Multica 的核心业务理念：将 AI 编码智能体作为一等协作者纳入看板工作区，人类像分配任务给同事一样把工作派给智能体。智能体自己接手、边做边汇报、卡住了主动说、做完提交审核。同一块看板上，人类成员和多个智能体并行推进工作，所有人的进展、决策和产出都汇聚在同一个 issue 时间线中。

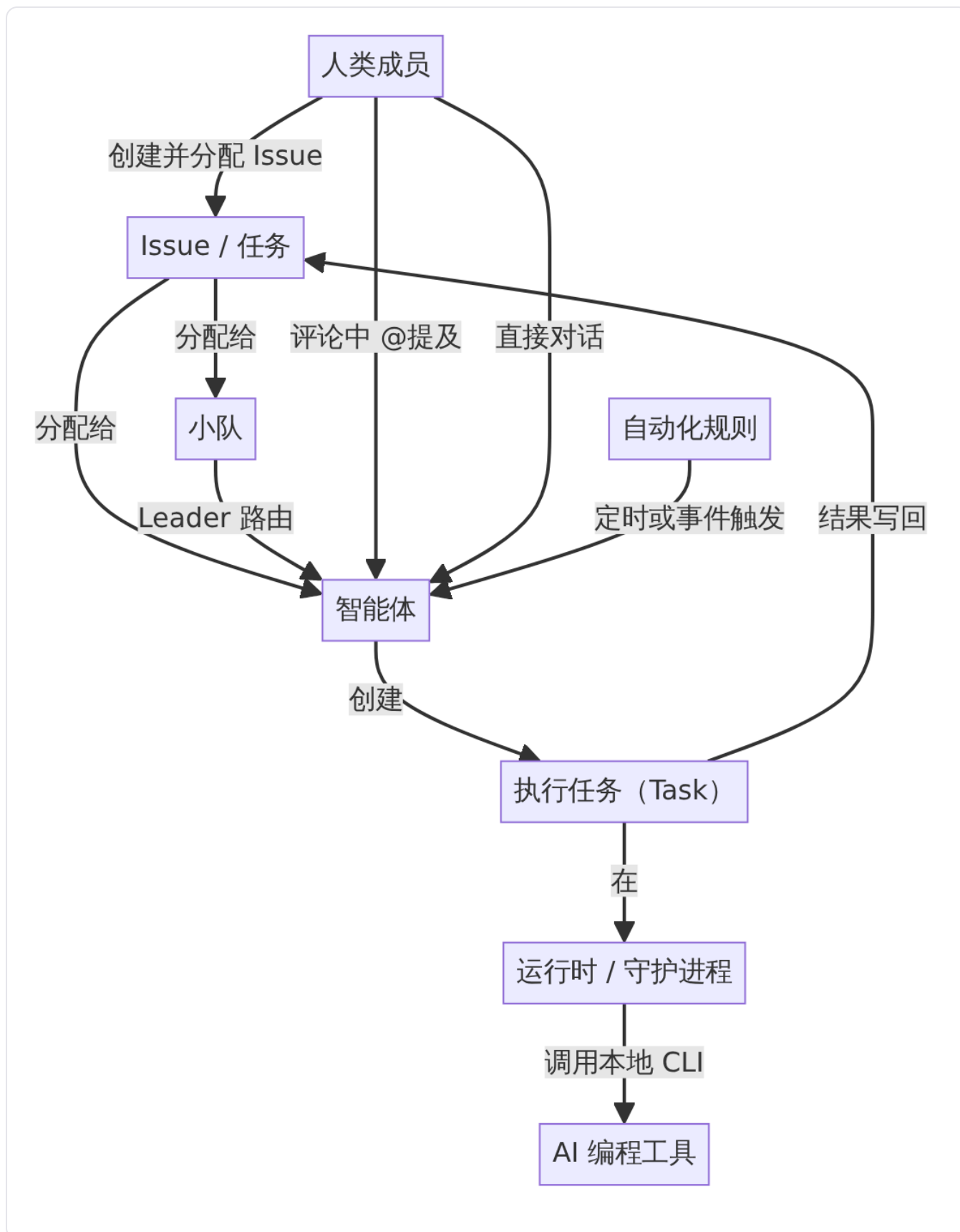
业务目标

软件团队长期面临一个矛盾：每多用一个 AI 智能体，就多了一个需要人类盯着的窗口。智能体的工作散落在终端、对话框和私人会话里，上下文留在个人脑中，关键决定淹没在聊天记录中。任务一旦换人或换智能体，大家就要从头再解释一遍。

智能体协作看板要解决的核心问题是：**让智能体成为真正的队友，而不是又一个工具窗口**。实现方式是把智能体放进人类团队本来就在使用的看板工作区——起个名字、选个提供方、配台运行时，它就上了看板，跟其他同事没两样。从最初的想法，到中间的每一次执行、每一个决定，再到最后的代码改动，全都挂在同一个任务下，没人需要重新捋一遍上下文，也没有任何东西能不经人点头就上线。

[README.zh.md](#) 开篇即阐明："你像给同事派活一样，把任务交给 AI 编码智能体——它自己接手、边做边汇报、卡住了主动说，做完交回来给你审。" [VISION.zh.md](#) 进一步定义了 Multica 的终极方向："人设定方向，并对结果负责；智能体持续推动工作；Multica 保存团队的理解，并把这种理解转化为协调一致的行动。"

核心角色



人类成员

人类成员负责三件事：**定方向、判断什么值得做、对最终结果负责**。人类不需要盯住智能体的每一步——边界清楚的事情，智能体可以继续往前推；一旦涉及产品方向、风险或重要取舍，智能体会主动把人请进来，拍了

板再往下走。验收的也不再是一句"已经完成"，而是工作本身：计划、文档、代码改动、预览、测试结果，以及尚未解决的问题。人可以随时纠正方向、提高标准、批准下一阶段，或者叫停工作。

智能体

智能体是工作区中的正式协作者，是一份可复用的配置：名称、头像、指令、模型、Skill、Access 和运行时。智能体不是一个持续运行的进程——它只有收到工作时才会产生具体的执行任务。智能体可以被分配为 issue 负责人、在评论中被 @提及、参与直接对话、成为小队 leader 或成员，以及被自动化规则触发。智能体可以创建 issue、发表评论和修改工作状态，但它没有收件箱，也不会收到 @all 通知。

来源：[agents.zh.mdx](#) 定义了智能体的完整配置项、参与工作方式和权限模型。

小队

小队由一名 leader 智能体和若干成员（可以是智能体，也可以是人类成员）组成。把 issue 分配给小队后，Multica 会先唤醒 leader，由它阅读上下文并决定下一步交给谁。小队解决的是"把工作交给谁"的问题，不会把多个智能体合并成一个新智能体，也不会自动提高并发。适用场景是工作需要多种能力、且不能在创建 issue 时就确定具体负责人时。

来源：[squads.zh.mdx](#) 详细规定了小队的组成、分配后的执行流程和队长再次触发的时机。

运行时

运行时是智能体实际执行工作的地方——连接的电脑和其中安装的 AI 编程工具。智能体是身份，运行时是执行它的电脑。Multica Cloud 和自托管部署中，代码始终在用户自己的机器上运行，代码不出门。一个运行时可以承载多个智能体；一个智能体也可以先后完成多条执行任务。

触发智能体的四种方式

智能体不会自己开始工作，每次执行都由一个明确的动作触发。四种触发方式覆盖了从正式委派到临时询问的全部协作场景。

触发方式	适用场景	对 Issue 的影响
分配 Issue	让智能体从头到尾负责整条 Issue	智能体成为负责人，Issue 状态按约定流转
评论中 @提及	在不变更负责人的前提下处理一条具体请求	不改变负责人和状态，仅处理当前评论
直接对话	不需要建 Issue 的提问或快速尝试	不涉及 Issue，每条消息触发一轮执行
自动化规则	定时或 webhook 触发的重复工作	可选"创建 Issue"或"仅运行"两种模式

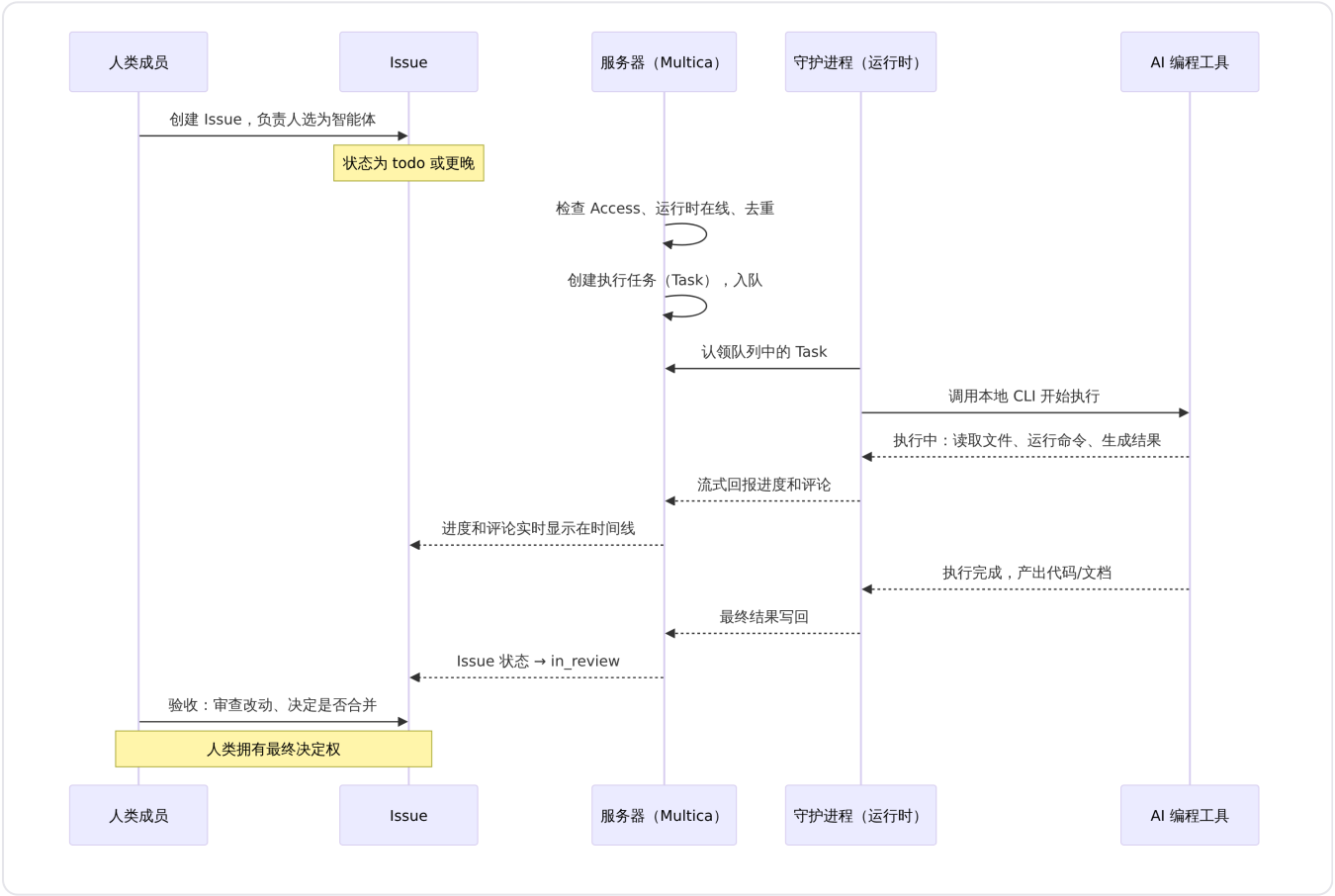
来源：[triggering-agents.mdx](#) 列举了四种触发方式及共享规则。

触发前置检查

每次触发执行前，系统会做一系列前置检查，这些检查由 [dispatch/reason.go](#) 中定义的标准化调度原因码描述：

- **调用权限 (invocation_not_allowed)**：触发者是否有权运行该智能体，取决于智能体的 Access 设置，与工作区角色无关。
- **目标可用性 (target_unavailable)**：智能体是否已归档、小队是否已删除或 leader 不可解析。
- **运行时在线 (runtime_offline)**：绑定的运行时是否在线；离线时任务入队等待，不会丢失。
- **运行时绑定 (agent_runtime_required)**：智能体是否绑定了运行时；未绑定时永远不会有机器领取工作。
- **去重合并 (coalesced)**：同一智能体在同一条 Issue 上已有排队中的执行任务时，新评论合并进已有任务。
- **自我触发抑制 (self_trigger_suppressed)**：小队 leader 的 @提及不会再次唤醒自己。

主流程：从分配 Issue 到验收交付



执行链路详解

一次完整的执行链路分五步，来源：[how-multica-works.zh.mdx](#)：

1. **Issue 提供上下文**：Issue 保存工作说明、讨论和负责人。分配给智能体后，系统使用该智能体的指令、模型、Skill 和运行时配置。

- 2. **Multica 创建执行任务**：执行任务进入队列；没有在线运行时，它在队列中等待。
- 3. **运行时领取任务**：连接电脑上的在线运行时领取任务，调用智能体配置的 AI 编程工具。
- 4. **AI 编程工具在本地执行**：工具读取工作目录、运行命令并生成结果。过程中智能体可以读取 Issue 的描述和评论、使用绑定的 Skill 和 MCP 服务器、在本地工作目录中读写文件和运行命令、发表评论和更新 Issue 状态。
- 5. **结果写回 Issue**：进度、评论和结果显示在 Issue 的时间线与执行日志中。

数据与执行的边界

Multica 服务器和连接的电脑之间有明确边界：

Multica（服务器端）	连接的电脑（客户端）
工作区、Issue、评论和状态	AI 编程工具及其凭据
智能体配置与 Skill	代码目录和本地文件
执行任务的状态、执行记录和结果	实际的文件修改与命令执行

这条边界在 Multica Cloud 和自托管中相同，唯一的例外是智能体的 `custom_env` 内容会存放在服务器端，执行时传给运行时。

智能体的状态模型

智能体在看板上呈现两种正交的状态维度：

可用性状态（来自运行时）

状态	含义
在线	运行时守护进程与 Multica 保持心跳连接，可以立即接受任务
离线	运行时守护进程断开，已入队任务保留等待恢复，新任务不会被领取
不稳定	运行时心跳间歇性丢失，任务可能延迟或需要重试

离线不表示智能体被删除。已经入队的任务会等待运行时恢复；已归档才表示它不能再接收新工作。

工作负载状态（来自执行任务）

状态	含义
处理中	当前有一条或多条执行任务正在运行
排队中	有执行任务在队列中等待，但尚未被运行时领取
空闲	没有活跃的执行任务

来源：[agents.zh.md](#)x 将这两类状态并称为"在线、离线和工作状态"。

Issue 状态约定

智能体在执行过程中被期望按约定管理 Issue 状态。这不是系统强制执行，而是智能体指令中的协作习俗：

- 首次接单：`todo` → `in_progress`
- 交付成果：`in_progress` → `in_review`
- 人类验收后：`in_review` → `done`（由人手动操作或既有集成自动流转）

执行任务结束不等于 Issue 完成。Issue 仍然可以继续讨论、补充要求或再次触发智能体。`done` 留给人工确认——没有不经人点头就能上线的东西。

小队场景下的特殊约定：队长首次接单把父 Issue 推到 `in_progress`，分发成员不等于完成——小队干活期间父 Issue 保持 `in_progress`。队长在整体目标达成后才推 `in_review`。

权限与 Access 控制

智能体的运行权限由三层控制：

Access 级别	谁可以运行
仅自己 (Only me)	只有智能体的 owner
指定成员 (Specific people)	owner 和指定的成员
整个工作区 (Entire workspace)	工作区中的所有成员

新建智能体默认使用"仅自己"。工作区 `owner` 和 `admin` 可以查看并管理所有智能体，但不能绕过 Access 运行别人的"仅自己"智能体。普通成员只会看到自己拥有或有权运行的智能体。

在分配 Issue 或 @提及时，触发预览会明确告知哪些智能体会被唤醒、哪些因权限不足或已归档而不会启动。

异常分支

运行时离线

当智能体的运行时离线时，为其创建的执行任务不会丢失——任务进入队列等待。用户的修复动作是将那台机器重新上线，队列中的任务会被自动领取。但任务永远绑定到创建时指定的运行时，不会转移到其他机器。

智能体已归档

归档的智能体不能接收新工作，所有尚未结束的执行任务（排队中和运行中）都会被取消。历史记录保留，之后可以恢复。归档操作由智能体 `owner` 或工作区 `owner/admin` 执行。

未绑定运行时

当智能体的运行时被删除后，`agent.runtime_id` 变为 `NULL`，这是一个与"运行时离线"不同的状态。此时没有任何机器能够领取该智能体的工作，唯一的修复方式是将智能体重新绑定到一个运行时。

去重合并

同一个智能体在同一条 Issue 上已有排队中的执行任务时，新评论会合并进这条任务，而不是创建第二条独立任务。智能体已经在执行时，后续评论等当前任务结束后合并为一次跟进执行。一条评论 @多个不同智能体时，每个智能体各得一条执行任务；多个提及指向同一个智能体时，只保留一次。

自我触发抑制

小队 leader 自己的 @提及不会再次唤醒自己。智能体之间可以互相 @提及（A @ B、B 又 @ A），系统会合并同一时刻的重复执行，但不会判断这段协作是否应该结束——协作的终止条件需要写进智能体指令。

不触发智能体的留言

两种不触发智能体的方式：以 `/note` 开头的评论作为成员之间的备注；使用 `@all` 通知全部工作区成员并关闭对负责人的自动触发。

业务边界

智能体能做什么

- 读取 Issue 的描述、字段和评论，理解工作上下文
- 使用绑定的 Skill、MCP 服务器和项目上下文
- 在本地工作目录中读写文件、运行命令、做出改动
- 发表评论报告进度、提出问题、请求决策
- 更新 Issue 状态按约定流转
- 通过 CLI 调用 Multica 自身 API，创建 Issue、查询工作区

智能体不能做什么

- 不能自行开始工作——每次执行必须由明确的触发动作启动
- 不能绕过 Access 控制——权限不足的成员无法触发
- 不能绕过人类验收——`done` 状态留给人工确认或既有集成
- 不能收到通知——智能体没有收件箱，`@all` 不包含智能体
- 不能在未绑定运行时的情况下执行——必须有运行时才能领取任务
- 不能访问 Multica 服务器之外的数据——代码执行始终在用户本地机器上

人类保留的职责

人不需要盯住智能体的每一步，而是负责：定方向、判断什么值得做、定义什么叫做好、对最终结果负责。涉及产品方向、风险或重要取舍时，人类必须参与决策。验收的对象不再是"已经完成"这句汇报，而是工作本身：计划、文档、代码改动、预览、测试结果，以及尚未解决的问题。

如 [VISION.zh.md](#) 所述："Multica 不是要造一家脱离人类控制、自己运转的'无人公司'。我们要做的，是人和智能体共同完成重要工作的那套共享操作系统。"

任务与问题管理

detail · task-management.md

任务与问题管理

Issue 是 Multica 中安排工作的基本单位：一项功能、一个 bug、一次调研——任何需要成员或智能体负责推进的工作。围绕它的讨论、状态变化和每次执行都留在同一处，无需再从聊天记录或终端日志中拼接上下文。

Issue 的组成

每条 issue 由以下内容构成：

内容	用途
标题和描述	目标、背景、要求和验收条件
状态和优先级	工作处于哪个阶段，先处理什么
负责人	工作区成员、智能体或小队
日期、标签和自定义属性	计划、分类和团队自己的字段
项目和父子关系	归入更大的工作范围，或拆成子 issue
动态和执行日志	评论、状态变化、执行任务和智能体返回的结果

每条 issue 有一个编号，例如 MUL-123。数字在工作区内递增；管理员修改工作区的 issue 前缀后，编号会用新前缀显示。

创建 Issue

在 **Issues** 页面或项目中新建，填写标题即可，其余属性随时补充。创建时可指定的字段包括标题（必填）、描述、状态、优先级、负责人类型和 ID、父 issue、阶段（stage）、开始日期、截止日期、附件和标签。

默认状态下，新创建的 issue 状态为 `todo`。显式指定 `backlog` 时，该状态会被保留。创建子 issue 时，若未显式指定项目，会自动继承父 issue 的项目归属。

状态

状态	含义
<code>backlog</code>	暂不启动。已分配给智能体的 issue 需离开 <code>backlog</code> 后才会创建执行任务
<code>todo</code>	已明确，等待开始
<code>in_progress</code>	正在处理

状态	含义
in_review	已有结果，等待检查
done	已完成
blocked	暂时无法继续
cancelled	不再继续，保留记录

状态之间没有固定流转，成员和智能体都可以直接修改。

两个变化由系统执行：执行失败且该 issue 没有其他执行、也没有触发重试时，in_progress 回到 todo；关联的 GitHub PR 带关闭意图合并并且没有其他仍处于 open 或 draft 的关联 PR 时，issue 变为 done。

优先级

优先级	含义
urgent	紧急
high	高
medium	中
low	低
none	未设置

选择负责人

Issues 支持三种负责人类型：成员、智能体和小队。选择不同负责人会产生不同的后续行为：

负责人	效果
成员	由这位成员负责跟进，不创建执行任务
智能体	为该智能体创建一条执行任务
小队	由小队 leader 接收，再决定交给谁处理

分配给智能体或小队时，issue 不在 backlog 就会立即入队；运行时离线时，任务留在队列中等待。已归档的智能体和小队不能被分配。分配不会绕过智能体的访问范围（Access）；分配给小队时，校验的是 leader 的 Access。

分配确认窗口中提供两个选项：**开始**——立即创建执行任务并开始第一次执行；**暂不开始**——保存负责人但不创建执行任务，适合先明确归属、等背景或依赖准备好后再触发。

更换负责人时，若新负责人是智能体，会为新负责人启动一次新的执行；选择成员只改变负责人标记，不会触发 AI 编程工具。移除负责人也不会创建新任务。修改 issue 的负责人或状态不会停止已经开始的执行——需要中断时，必须在执行日志中手动停止对应的执行任务。

Issue 与执行任务

Issue 是持续存在的工作记录；执行任务（task）是智能体的一次具体执行。一个 issue 可以先后产生多条执行任务——第一次实现、补充修改、再次检查。执行任务"已完成"只表示这一次执行结束；issue 是否完成，以 issue 的状态为准。

	Issue	执行任务
记录什么	一项持续推进的工作	智能体的一次执行
持续多久	可以反复讨论、补充和重新分配	从触发开始，到完成、失败或取消为止
数量关系	一个 issue 可以包含多次执行	每次执行都有独立记录

项目和子 Issue

一个 issue 最多属于一个项目，项目为其中的执行提供共享说明和资源；移到另一个项目不会产生副本。较大的工作可以拆成子 issue：父 issue 保留整体目标，子 issue 各自独立推进。父子状态互不联动。

阶段（Stage）

子 issue 可以设置阶段（stage），按 1、2、3 分批推进。当前最早未完成阶段的全部子 issue 到达 `done` 或 `cancelled` 时，父 issue 会收到"子任务完成"通知；父 issue 的负责人是智能体时，它会被唤醒并决定是否推进下一阶段。未设置阶段的子 issue 视为同一批，全部结束时通知一次。

视图

Issues 页面提供五种视图，可按状态、负责人、项目和其他属性筛选、排序，展示的都是同一批 issue：

- **列表视图**：传统列表展示，支持排序和筛选
- **看板视图**：按状态分列，支持拖拽移动调整状态
- **表格视图**：结构化表格，支持自定义列、分组和批量操作
- **甘特图**：按时间线展示，适合跟踪进度和依赖
- **泳道视图**：交叉分组展示，按负责人或其他维度分泳道

API 与实时推送

REST API

Issue 的 REST API 路由为 `/api/issues`，所有接口需要工作区成员身份。完整 API 包括对 issue 的创建、查询、更新、移动、删除，以及围绕 issue 的子资源操作。

来源：[server/cmd/server/router.go](#)

核心接口如下：

方法	路径	用途
GET	/api/issues	列表查询，支持按状态、优先级、负责人、项目、involves_user_id 等筛选
POST	/api/issues/query	与 GET 列表等价，用于超大筛选参数集
POST	/api/issues	创建 issue
POST	/api/issues/quick-create	快速创建（精简流程）
POST	/api/issues/batch-update	批量更新
POST	/api/issues/batch-delete	批量删除
GET	/api/issues/search	全文搜索
GET	/api/issues/grouped	按负责人分组查询
POST	/api/issues/table/groups	表格视图分组数据
POST	/api/issues/table/rows	表格视图行数据
POST	/api/issues/table/facets	表格视图筛选面数据
POST	/api/issues/preview-trigger	预览分配后的触发效果
GET	/api/issues/child-progress	子 issue 进度查询
GET	/api/issues/children	批量查询父 issue 下的子 issue
GET	/api/issues/{id}	获取单条 issue
PUT	/api/issues/{id}	更新 issue
POST	/api/issues/{id}/move	在看板列内移动排序
DELETE	/api/issues/{id}	删除 issue
POST	/api/issues/{id}/comments	创建评论
GET	/api/issues/{id}/comments	列出评论
GET	/api/issues/{id}/timeline	查看时间线
POST	/api/issues/{id}/subscribe	订阅 issue
POST	/api/issues/{id}/unsubscribe	取消订阅
POST	/api/issues/{id}/rerun	重新入队执行
GET	/api/issues/{id}/task-runs	查看执行历史
GET	/api/issues/{id}/usage	查看聚合 token 用量
GET	/api/issues/{id}/children	查看子 issue 列表
GET	/api/issues/{id}/labels	查看 issue 上的标签
POST	/api/issues/{id}/labels	添加标签

方法	路径	用途
DELETE	/api/issues/{id}/labels/{labelId}	移除标签

更新时的关键控制字段

UpdateIssueRequest 包含两个控制字段，仅在服务端消费，不会写入 issue 记录：

- suppress_run：设为 true 时，分配或状态变更不触发智能体执行（即"暂不启动"）
- handoff_note：交接说明文本，注入到新启动执行的上下文中

来源：[packages/core/types/api.ts](#)

实时推送

Issue 的创建、更新、删除等变更通过 WebSocket 实时推送给同一工作区内的在线客户端。事件由内部事件总线驱动：服务层完成写入后发布事件，总线上的订阅者将事件序列化并通过 WebSocket 广播到对应的工作区房间。非个人事件（如 issue 变更）向整个工作区房间广播；个人事件（如收件箱、邀请）仅发送给目标用户。

来源：[server/cmd/server/listeners.go](#)

客户端 WebSocket 连接通过 JWT 认证和工作区成员资格校验后建立，按事件类型分发消息到各订阅者。连接断开后自动重连，重连后通过 REST API 重新查询数据校准状态。

删除 Issue

任何工作区成员都可以删除 issue。删除不可恢复：issue 及其评论、附件和关联记录会被永久移除，尚未结束的执行任务会被取消。不再继续时建议将状态改为 cancelled，讨论和结果仍可查阅。

在 CLI 中使用

CLI 提供完整的 issue 管理命令：

```

# 列出 issue
multica issue list --status todo --priority high --assignee "Alice"

# 查看单条
multica issue get MUL-42

# 创建
multica issue create --title "添加登录功能" --priority high --assignee "Agent name"

# 更新
multica issue update MUL-42 --status in_progress

# 指派
multica issue assign MUL-42 --to "Agent name"
multica issue assign MUL-42 --unassign

# 搜索
multica issue search "登录"

# 查看子 issue (按阶段分组)
multica issue children MUL-42

# 重新入队
multica issue rerun MUL-42

# 取消 task
multica issue cancel-task <task-id> --issue MUL-42

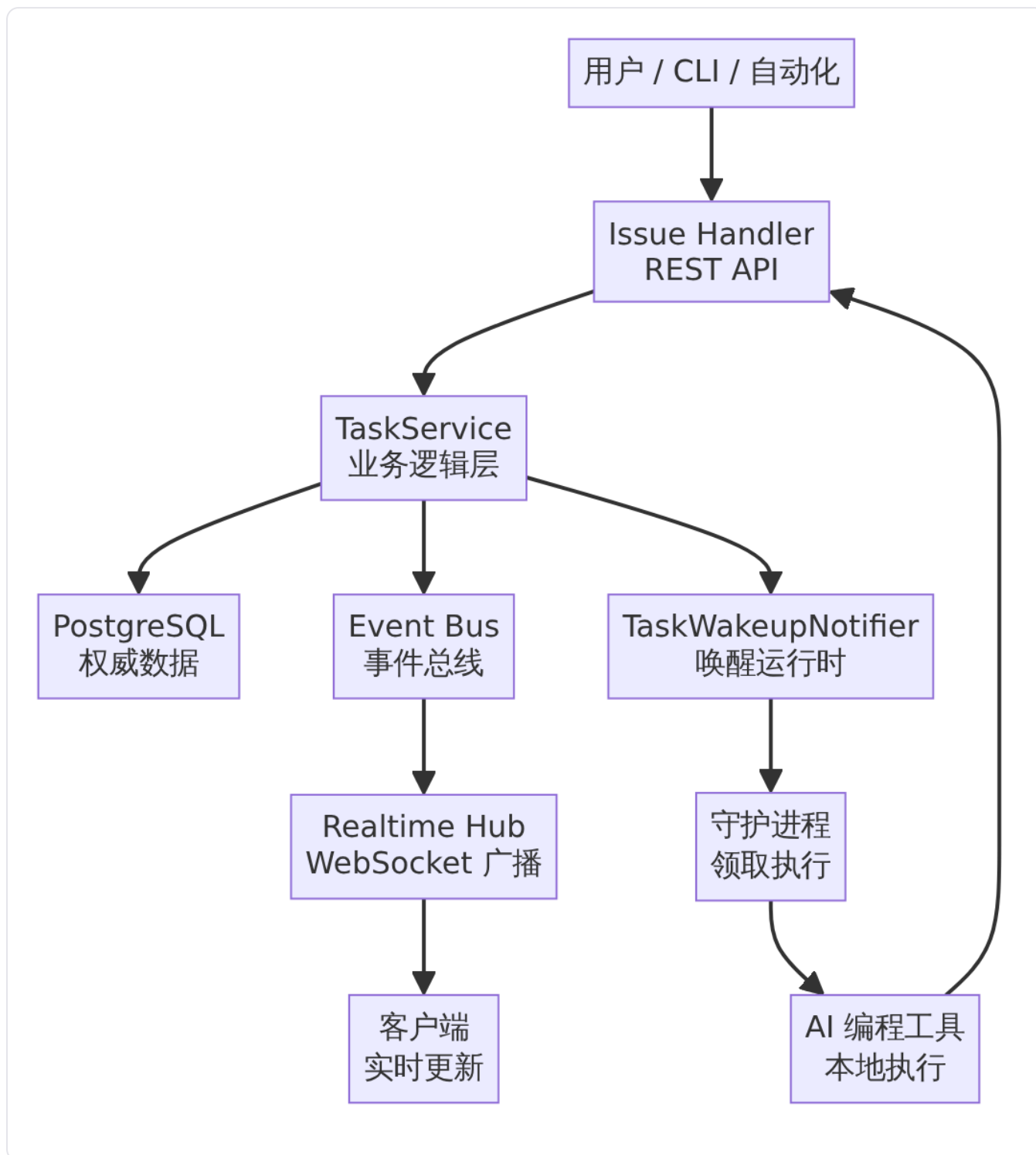
```

来源：apps/docs/content/docs/cli.zh.mdx

与其他模块的关系

- **智能体协作**：将 issue 分配给智能体触发执行任务，智能体读取 issue 上下文、使用绑定的 skill 完成工作、将进展和结果写回 issue。详见 [智能体协作看板](#)。
- **自动化规则**：通过定时或 Webhook 触发器自动为 issue 创建执行任务，支持周期性重复工作。详见 [自动化规则](#)。
- **项目管理**：issue 归入项目后，项目描述和资源进入执行上下文；项目进度按关联 issue 自动计算。详见 [项目管理](#)。
- **通知中心**：issue 的订阅动态、@提及和分配事件进入收件箱，通过 WebSocket 实时推送。详见 [通知中心](#)。
- **实时通信**：issue 变更事件通过 WebSocket 广播到工作区，客户端即时更新看板、列表等视图。详见 [实时通信基础设施](#)。

架构概览



请求按 `router` → `middleware` → `handler` → `service` → `sqlc query` → `PostgreSQL` 方向流动。`handler` 层解析 HTTP 输入并生成响应，`service` 层负责跨查询的业务流程和事务。Issue 的负责人是多态关系，可以指向成员、智能体或小队，所有业务查询必须限定 `workspace_id` 并在请求进入工作区路由前校验成员身份。

来源：apps/docs/content/docs/developers/architecture.zh.mdx

限制与约束

- 一条 issue 最多属于一个项目。项目被删除时，其中的 issue 解除关联继续保留。

- 父子 issue 状态互不联动。阶段只控制"子任务完成"通知的触发时机，不强制状态同步。
- 修改 issue 的负责人或状态不会停止已开始的执行任务，需在执行日志中手动停止。
- 已归档的智能体和小队不能被分配为负责人。
- 删除 issue 不可恢复，关联的评论、附件和执行记录会被永久移除。
- backlog 状态下的 issue 分配给智能体或小队时不会创建执行任务，移出 backlog 后才会触发。
- 分配给小队时，校验小队 leader 的 Access 权限，而非直接校验小队中所有成员的权限。

自动化规则

detail · autopilot.md

自动化规则

自动化规则 (Autopilot) 是 Multica 的重复任务调度引擎。它根据 cron 时间表或 Webhook 外部事件触发，自动将工作分配给指定的智能体或小队，适用于日报汇总、定期巡检、依赖审计等周期性场景。每次触发都会创建运行记录，支持完整的审计追溯和持续失败自动暂停。

核心概念

每条自动化规则包含以下要素：

要素	说明
名称	描述该自动化负责什么工作
Runbook	智能体每次运行时读取的目标、背景、约束和执行步骤
执行方	一个智能体或一个小队；小队模式下自动解析到队长智能体
输出模式	创建任务 (create_issue) 或 仅运行 (run_only)
触发器	一条或多条时间表或 Webhook
关联项目	可选，自动创建的任务归入指定项目
订阅者	自动创建任务后需要接收通知的成员

保存后自动化默认为启用状态。立即运行 可随时手动执行一次完整流程。

来源：[apps/docs/content/docs/autopilots.zh.mdx](https://github.com/multica/docs/blob/main/content/docs/autopilots.zh.mdx)

输出模式

两种输出模式决定了每次触发后的行为路径：

模式	行为	适合
创建任务 (create_issue)	每次触发先创建一条 issue，再分配给执行方；讨论、状态和执行记录都保留在 issue 中	需要团队查看、确认或继续处理的工作
仅运行 (run_only)	直接创建执行任务，不产生 issue；结果只在自动化的运行历史中查看	无需协作记录的后台执行任务

创建任务 模式与普通 issue 使用相同的任务队列。运行时离线时，issue 仍会被创建，执行任务等待运行时上线后再执行。

仅运行 模式要求运行时在触发时可用；否则本次运行会标记为"已跳过"，不会留下等待中的 issue。这是 dispatchAutopilot 中的准入检查逻辑——若执行方智能体的运行时不在线，则记录 skipped 运行并直接

返回，避免因定时触发而堆积数千个无法完成的任务（MUL-1899）。

来源：[server/internal/service/autopilot.go](#) —— DispatchAutopilot 核心执行入口，包含 run_only 模式的运行时准入检查逻辑。

时间表触发

时间表编辑器提供可视化配置：选择运行时间、重复日期（每天/每周特定天数/每月）、时间窗口和时区，并可预览接下来的运行时间。

需要更复杂规则时，可直接编辑标准 5 字段 cron 表达式：

分 时 日 月 星期

Cron	时区	含义
0 9 * * 1-5	Asia/Shanghai	工作日 9:00
*/30 * * * *	UTC	每 30 分钟
0 3 * * *	UTC	每天 3:00

Cron 不包含秒字段，时区使用 Asia/Shanghai 等 IANA 名称。保存前应通过页面显示的"接下来"时间核对结果。一条自动化可以添加多个时间表；单独启停某个触发器可通过 CLI 的 `autopilot trigger-update` 命令完成。

调度器集成

时间表触发由进程级调度器驱动。调度器注册了 `autopilot_schedule_dispatch Job`，每个启用的时间表触发器作为一个独立的作用域（`scope_kind= autopilot_trigger`，`scope_id=触发器 UUID`），按 cron 表达式计算计划时间（`plan_time`）并触发执行。

调度器每 30 秒评估一次到期计划，通过 `sys_cron_executions` 实现多副本环境下的租约协调。最大延迟容忍窗口为 5 分钟——超出此窗口的滞后触发将被跳过，等待下一个配置的时间槽，避免在错误的时间运行。

来源：[server/internal/scheduler/jobs_autopilot.go](#) —— AutopilotScheduleDispatchJob 定义，包括作用域模型和最大延迟窗口。

来源：[server/internal/scheduler/manager.go](#) —— 调度器 Manager：默认 TickInterval 为 30 秒，RunnerID 标识进程实例。

Webhook 触发

添加 Webhook 触发器后，Multica 会生成一个唯一 URL。向该 URL 发送 POST 请求并附带 JSON 对象或数组作为 body，即可触发自动化：

```
curl -X POST "$MULTICA_WEBHOOK_URL" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: demo-001" \
  -d '{"event": "build.completed", "eventPayload": {"status": "success"}}'
```

Payload 会保存在投递和运行记录中，并交给智能体。`创建任务` 模式还会把事件内容附在 issue 描述中。

Webhook 请求约束

- Body 必须是有效的 JSON 对象或数组，最大 256 KiB
- `Idempotency-Key` 请求头可避免发送方重试时重复运行；GitHub 的 `X-GitHub-Delivery` 同样被支持
- 没有稳定幂等键时，Multica 无法保证重复请求只运行一次
- 触发器已禁用或事件不匹配时，投递会记录为"已忽略"，不会创建运行

来源：apps/docs/content/docs/autopilots.zh.mdx

Webhook 响应速查

调试发送方时可对照以下响应：

HTTP 状态	响应状态	含义
200	<code>accepted</code>	已受理并创建运行，返回投递和运行 ID
200	<code>skipped</code>	已受理，但本次运行被跳过（例如仅运行模式下运行时离线）
200	<code>ignored</code>	未创建运行：触发器已禁用、自动化已暂停/归档，或事件被过滤
200	<code>duplicate</code>	幂等键命中已有投递，返回原投递 ID，不会重复运行
400	错误信息	Body 为空、不是有效 JSON，或不是 JSON 对象/数组
401	<code>rejected</code>	触发器配置了签名密钥，但请求缺少签名或签名不匹配
404	错误信息	URL 中的 token 无效或已被重新生成
413	错误信息	Body 超过 256 KiB
429	错误信息	请求过于频繁，按响应头 <code>Retry-After</code> 稍后重试
500	错误信息	Multica 内部错误，发送方可稍后重试

暂停、归档和事件过滤这类业务性忽略返回的是 200 而不是 4xx，避免发送方反复重试。

来源：apps/docs/content/docs/autopilots.zh.mdx

Webhook URL 安全

Webhook URL 中的 token 就是调用凭证，不应放入公开仓库、issue 或截图。URL 泄露后可通过"重新生成 URL"按钮立即轮换，旧 URL 即刻失效。只有自动化创建者、工作区 owner/admin 和获得访问权限的协作者可以查看完整 URL。

事件过滤

同一个来源会发送多种事件时，可为触发器添加事件过滤。每行包含一个事件名和可选的 action 列表，任意一行命中即运行，全部留空则接收所有事件。Multica 按以下顺序从请求头和 body 中推断事件与 action：

1. body 中带有字符串 `event` 字段时直接使用
2. 否则尝试 `X-GitHub-Event` 请求头，与 body 中的 `action` 拼成 `github.<event>.<action>`
3. 再尝试 `X-Gitlab-Event` 请求头
4. 再尝试 `X-Event-Type` 请求头
5. 再到 body 中的 `event`、`type`、`action` 字段
6. 全部缺失时记为 `webhook.received`

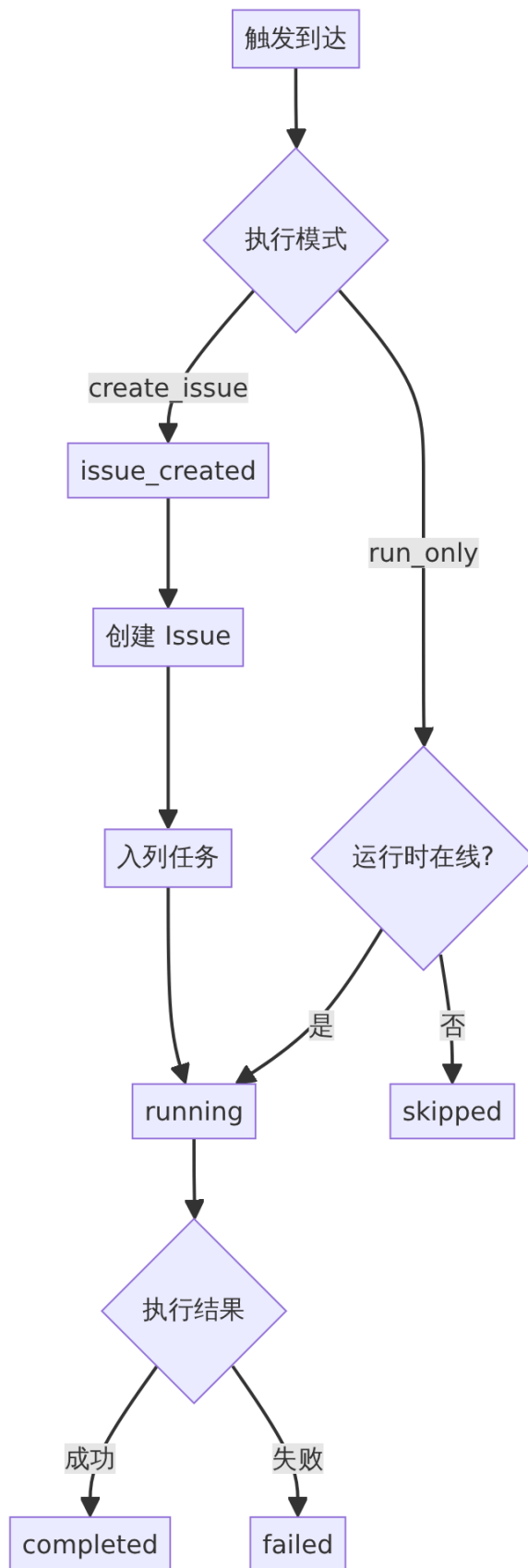
投注重放

处理完成的 Webhook 投递可以从详情中重放。重放会创建一次新的投递和运行，不会改写原记录。签名校验失败或仍在排队的投递不能重放，重放也不参与去重。

来源：[server/internal/handler/webhook_delivery.go](#) —— `ReplayAutopilotDelivery` 的处理入口。

运行生命周期

每次触发（时间表、Webhook、手动或 API）都会创建一条运行记录（`AutopilotRun`），其状态流转如下：



来源：[server/internal/service/autopilot.go](#) —— `dispatchAutopilot` 和 `dispatchAutopilotRun` 实现运行状态转换和下游副作用。

运行记录的来源字段 (source) 标识触发方式： `schedule` (时间表)、 `manual` (手动立即运行)、 `webhook` (Webhook)、 `api` (API 调用)。

对于时间表触发，调度器通过 `DispatchAutopilotForPlan` 入口提交，该入口基于 (`trigger_id`, `planned_at`) 部分唯一索引实现幂等——同一计划时间点不会产生两条成功的运行。若前次尝试在创建运行行后因进程崩溃而部分完成，下次重试时会先标记该部分运行行为 `failed`，再重新创建一次完整运行。

来源：[server/internal/service/autopilot.go](#) —— `DispatchAutopilotForPlan` 的幂等和部分运行恢复策略。

手动触发通过 `DispatchAutopilotManual` 入口，运行归因设置为 `direct_human`，指向触发成员。若执行方智能体缺少运行时、不可用或已离线，UI 会以 `reason_code` 字段返回具体原因，用于展示对应的 toast 提示。

来源：[server/internal/handler/autopilot.go](#) —— `TriggerAutopilot` 的 handler 实现。

失败自动暂停

Multica 内置后台故障监控器，定期检查近期运行是否持续失败。默认配置如下：

- 检查间隔：每 24 小时
- 回溯窗口：过去 7 天
- 最小运行次数：50 次（低于此值不触发暂停，避免统计偏差）
- 失败率阈值：90%（失败数 / 完成或失败总数 ≥ 90%）

满足以上条件时，系统自动暂停该自动化，向创建者发送收件箱通知，并通过 WebSocket 推送状态变更事件。统计时 `skipped` 状态不计入分子也不计入分母——跳过表示准入阶段拒绝（如运行时离线），不应稀释失败率也不应夸大失败率。

来源：[server/cmd/server/autopilot_failure_monitor.go](#) —— `failureMonitorConfig` 的默认值和 `runAutopilotFailureMonitor` 启动逻辑。

可通过环境变量覆盖监控参数：

环境变量	含义
<code>AUTOPILOT_FAIL_MONITOR_INTERVAL</code>	检查间隔，设为 0 则禁用监控
<code>AUTOPILOT_FAIL_MONITOR_LOOKBACK</code>	回溯窗口
<code>AUTOPILOT_FAIL_MONITOR_MIN_RUNS</code>	最小运行次数阈值
<code>AUTOPILOT_FAIL_MONITOR_FAIL_RATIO</code>	失败率阈值 (0~1)
<code>AUTOPILOT_FAIL_MONITOR_STARTUP_DELAY</code>	启动延迟（避免多副本同时查库）

来源：[server/cmd/server/autopilot_failure_monitor.go](#) —— `envFailureMonitorConfig` 环境变量配置。

暂停与删除

手动暂停会停止时间表、Webhook 触发和"立即运行"，自动化状态变为 `paused`。修复导致暂停的原因后可手动恢复为 `active`。

删除实际上是归档：将自动化标记为 `archived`，停止后续触发，但运行历史、投递记录、订阅者和协作者信息全部保留，作为执行历史可查询。归档是一个实质性的状态变更，会被记录到规则版本快照中。

来源：[server/internal/handler/autopilot.go](#) —— `DeleteAutopilot` 的归档实现。

规则版本管理

每次对自动化的实质性（substantive）变更都会记录一个规则版本快照（rule-version snapshot），包含变更前后的核心配置摘要和发布者身份。以下字段的变更被视为实质性变更，会在原地事务中原子地追加版本记录：

- `assignee_type` / `assignee_id`（执行方）
- `status`（启用状态：`active` / `paused` / `archived`）
- `execution_mode`（输出模式）
- `description`（Runbook 内容）
- `issue_title_template`（任务标题模板）

外观性或路由性字段（`title`、`project_id`）的变更不会产生新的规则版本，因为它们不改变指令内容或执行方。触发器表级编辑（`cron`、`timezone`、`enabled`、`event_filters`）在其各自的 `UpdateAutopilotTrigger` 路径中处理，也会触发重新发布。

来源：[server/internal/handler/autopilot.go](#) —— `autopilotRuleSubstantiveChange` 定义了哪些字段构成实质性变更。

来源：[server/internal/service/autopilot.go](#) —— `RecordAutopilotRuleVersion` 的实现，发布者类型为 `"member"` 或 `"system"`。

权限模型

自动化规则的访问控制分为三个层级：

角色	可编辑/运行/管理触发器	可管理协作者访问列表
创建者	✓	✓
工作区 owner/admin	✓	✓
被授予访问权限的协作者	✓	✗
其他工作区成员	✗（只能查看）	✗

任何工作区成员都可以创建自动化。能管理自动化不代表一定能运行它使用的智能体——智能体的 Access 控制仍然独立生效。API 响应中包含 `can_write` 和 `can_manage_access` 布尔字段，前端据此控制 UI 的操作

入口显示。

来源：[apps/docs/content/docs/autopilots.zh.mdx](#) —— 权限说明段落。

来源：[server/internal/handler/autopilot.go](#) —— `CanWrite` 和 `CanManageAccess` 响应字段定义。

CLI 操作

自动化规则支持通过 CLI 进行管理：

```
# 列出自动化
multica autopilot list

# 查看单条自动化详情
multica autopilot get <id>

# 立即触发运行
multica autopilot trigger <autopilot-id>

# 查看运行历史
multica autopilot runs <autopilot-id>

# 重新生成 Webhook URL
multica autopilot trigger-rotate-url <autopilot-id> <trigger-id>
```

触发器管理子命令支持添加时间表或 Webhook 触发器、更新触发器标签/启用状态/cron 表达式/时区。

来源：[server/cmd/multica/cmd_autopilot.go](#) —— CLI 命令实现。

内置模板

创建自动化时，可从以下预置模板中选择作为起点：

模板	用途
每日新闻摘要	搜索并汇总今天的新闻给团队
PR Review 提醒	标记需要 review 的滞留 Pull Request
Bug 分诊	评估并安排新提交的 bug
每周进度报告	汇总团队本周进展
依赖审计	扫描安全漏洞和过时的依赖包
文档检查	检查近期改动是否缺失文档

来源：[packages/views/locales/zh-Hans/autopilots.json](#)

约束与限制

- **Cron 精度**：分钟级，不支持秒级调度
- **Webhook Body 上限**：最大 256 KiB

- **仅运行模式**：运行时离线时触发会被跳过，不重试
- **创建任务模式**：仅运行模式中的执行任务失败后不会自动重试；下一次时间表仍按原计划触发。创建任务模式产生的是普通 issue 的执行任务，基础设施故障按任务系统的规则处理
- **删除不可逆**：删除实为归档，运行和投递历史保留但自动化不可恢复
- **执行方变更**：变更执行方（assignee）属于实质性变更，会生成新的规则版本快照

智能体对话

detail · agent-chat.md

智能体对话

智能体对话（Chat）是 Multica 中与智能体进行一对一独立沟通的方式。与通过任务（Issue）分配工作不同，对话不绑定任何任务、不产生工作记录，适合梳理想法、讨论方案、查询工作区信息或让智能体完成临时小任务。

每条消息触发一轮执行，智能体通过绑定的运行时调用 AI 编程工具完成回复。对话完全私有——工作区内其他成员（包括管理员）无法查看。整段对话会尽量维持 AI 编程工具的原会话，后续消息无需重复上文。

对话与任务的对比

对话和 Issue 是 Multica 中两种互补的工作方式：

需要	更适合
快速提问、探索方案或处理私人草稿	对话
明确负责人、状态、优先级和交付结果	Issue
让团队成员查看背景并继续讨论	Issue
临时查看工作区信息，不需要建立工作记录	对话

[apps/docs/content/docs/chat.zh.mdx](#)

开始对话

创建会话

通过侧边栏 **对话** 入口新建会话。创建时需要选择一个有权运行的智能体。服务端 API 为 `POST /api/chat/sessions`，由 `CreateChatSession` 处理，接收 `agent_id`、可选 `title` 和可选 `project_id`。

创建时执行以下校验：

1. 验证智能体存在于当前工作区；
2. 验证智能体未被归档；
3. 通过 `canInvokeAgent` 检查调用权限——启动对话会产生智能体运行，因此使用 `invoke` 权限而非更宽松的 `view` 权限。

[server/internal/handler/chat.go](#)

发送消息

在输入框中输入消息，也可以附加图片或文件，点击发送。服务端 API 为 `POST /api/chat/sessions/{sessionId}/messages`，由 `SendChatMessage` 处理。

发送前执行多层前置检查，全部通过后才持久化消息并创建执行任务：

1. 会话所有权和智能体访问权限（`gatePublicChatSessionForUser`）；
2. 会话状态必须为 `active`（已归档会话为只读）；
3. 智能体未被归档；
4. 智能体已绑定运行时；
5. 调用权限（`canInvokeAgent`）——发送消息会入队一次执行，必须满足 `invoke` 权限，即使管理员也不能绕过。

此外，如果这是会话的第一条用户消息，服务器会在发送成功后异步触发 LLM 自动生成会话标题（MUL-4295），且采用 CAS（Compare-And-Swap）策略：如果用户手动重命名了标题，自动生成不会覆盖。

[server/internal/handler/chat.go](#)

项目上下文

对话可以携带项目背景。点击输入框左下角的 **+**，选择 **项目上下文**，为对话挂上一个项目。智能体执行时会收到该项目的描述和仓库等资源。

项目上下文通过创建会话时的 `project_id` 参数设置，也可通过 `PATCH /api/chat/sessions/{sessionId}` 修改。修改时会在事务中锁定目标项目，确保项目存在且属于同一工作区。

[apps/docs/content/docs/chat.zh.mdx](#)

[server/internal/handler/chat.go](#)

会话生命周期

会话列表

`GET /api/chat/sessions` 返回当前用户创建的会话列表，支持 `status=all` 参数以包含归档会话。列表会过滤掉用户已失去访问权限的智能体对应的会话——当成员角色被降级、智能体所有权转移或智能体从共享改为私有时，相应会话不再出现在列表中。

[server/internal/handler/chat.go](#)

重命名

通过 `PATCH /api/chat/sessions/{sessionId}` 设置 `title` 字段。标题最长 200 个字符。修改后广播 `chat:session_updated` 事件，其他设备/标签页同步更新。

[server/internal/handler/chat.go](#)

置顶与取消置顶

`PATCH /api/chat/sessions/{sessionId}/pin` 将常用对话固定在列表前面。置顶状态不触发 `updated_at` 更新，这样取消置顶后对话不会跳到活动排序列表的首位。

[server/internal/handler/chat.go](#)

归档与取消归档

`PATCH /api/chat/sessions/{sessionId}/archive` 将对话设为只读状态。归档后：

- 不能继续发送新消息（`SendMessage` 拒绝 `status='archived'`）；
- 如果绑定了外部渠道（Slack/飞书等），绑定关系会被断开——防止后续进站消息继续向已归档会话写入回复（MUL-4372）；
- 取消归档不会自动恢复渠道绑定——如果后续流量已创建了新的会话，恢复旧绑定会导致抢夺。

[server/internal/handler/chat.go](#)

删除

`DELETE /api/chat/sessions/{sessionId}` 永久删除对话及其所有消息。删除在事务内执行：先锁定会话行阻止并发写入，取消进行中的任务，再删除会话行。删除后广播 `chat:session_deleted` 事件。

删除不可撤销——暂时不需要时优先使用归档。

[server/internal/handler/chat.go](#)

[apps/docs/content/docs/chat.zh.mdx](#)

消息与执行

消息列表与分页

消息支持两种查询方式：

- **全量列表**：`GET /api/chat/sessions/{sessionId}/messages` 返回所有消息。
- **分页查询**：`GET /api/chat/sessions/{sessionId}/messages/page` 基于游标分页，每页默认 50 条（可调 1-100），通过 `before_created_at` 和 `before_id` 游标向前翻页。SQL 按从新到旧获取窗口，序列化前反转为时间顺序。

两种接口均过滤掉服务端内部消息（如 `onboarding_kickoff` 类型的引导首轮），用户不可见。

[server/internal/handler/chat.go](#)

消息已读标记

`POST /api/chat/sessions/{sessionId}/mark-read` 清除会话的未读计数，并广播 `chat:session_read` 事件，使同一用户的其他设备同步消除未读标记。

[server/internal/handler/chat.go](#)

排队消息

对话支持消息排队：当前一条消息还在处理中时，后续发送的消息不会丢失，而是进入排队状态。API 支持：

- **查看排队**：GET /api/chat/sessions/{sessionId}/pending-task 返回当前执行任务及排队中的后续任务。
- **调整优先级**：POST /api/chat/sessions/{sessionId}/queued-tasks/{taskId}/prioritize 可以将某条排队消息提前，替代当前回复。
- **清空队列**：DELETE /api/chat/sessions/{sessionId}/queued-tasks 取消所有排队中的后续任务，不影响当前正在执行的任务。

[server/internal/handler/chat.go](#)

停止与草稿恢复

停止执行后，有两种可能的结果，通过 chat:cancel_finalized 事件广播：

- **stopped**：执行已产生内容，一个 "已停止" 的助手消息被持久化。
- **restored**：执行尚未产生任何内容，用户的消息被删除，其内容和附件被持久化为草稿恢复 (draft restore)。只有触发该任务的用户客户端会通过 GET /api/chat/sessions/{sessionId}/draft-restores 获取可恢复的草稿，消费后调用 DELETE 清理。

[server/pkg/protocol/messages.go](#)

[server/internal/handler/chat.go](#)

失败原因

当智能体无法完成回复时，界面会展示具体的失败原因。支持的错误类型覆盖以下场景：

失败类型	含义
agent_error	智能体运行时内部错误
timeout	智能体处理超时
codex_semantic_inactivity	智能体长时间无响应
runtime_offline	智能体当前离线
runtime_recovery	智能体在完成前重启
manual	用户手动取消
skill_bundle_unavailable	无法下载 skill
provider_network	与模型服务的连接中断
provider_auth_or_access	模型账号登录失效

失败类型	含义
provider_quota_limit	模型账号额度用尽
provider_capacity_or_rate_limit	模型服务繁忙或限流
context_overflow	超出模型上下文长度
runtime_missing_executable	找不到智能体的 CLI
runtime_version_unsupported	CLI 版本过低

[packages/views/locales/zh-Hans/chat.json](#)

多轮上下文与会话恢复

同一段对话会尽量继续 AI 编程工具中的原会话。服务器保存会话标识（`session_id`、`work_dir`、`runtime_id`），后续消息的执行被路由到能访问该会话的运行时。

如果本地会话已不存在、上下文无法继续，或者上一次错误不适合复用，运行时会从新会话开始——对话中的历史消息仍然保留。

[apps/docs/content/docs/chat.zh.mdx](#)

权限与隐私

会话创建与访问控制

- 对话只对创建者开放，其他工作区成员和管理员不能读取。
- 创建时验证 `canInvokeAgent`（调用权限），而非更宽松的 `canAccessPrivateAgent`（查看权限）。
- 即使管理员也无法绕过调用权限运行"仅自己"（Only me）权限的智能体。

运行时权限衰减

每次发送消息时重新验证权限。如果用户的角色被降级、智能体被转为私有或从允许列表中移除，已存在的会话无法继续发送新消息——基于 `canInvokeAgent` 的前置检查在消息持久化之前执行，确保不会留下孤立的消息行。

[server/internal/handler/chat.go](#)

会话列表的访问过滤

列出会话时，通过 `accessibleAgentIDs` 计算用户当前有权访问的智能体集合，过滤掉无权访问的智能体对应的会话——用户的角色降级或智能体权限变更后，不再能看到相关会话。

[server/internal/handler/chat.go](#)

消息读取的权威检查

`gatePublicChatSessionForUser` 在每次读取消息时同时验证：会话所有权（必须是创建者）和智能体访问权限（`canAccessPrivateAgent`）。但注意：读取使用较宽松的 `canAccessPrivateAgent` ——管理员可以读取但不可以发送。

[server/internal/handler/chat.go](#)

离线与运行时状态

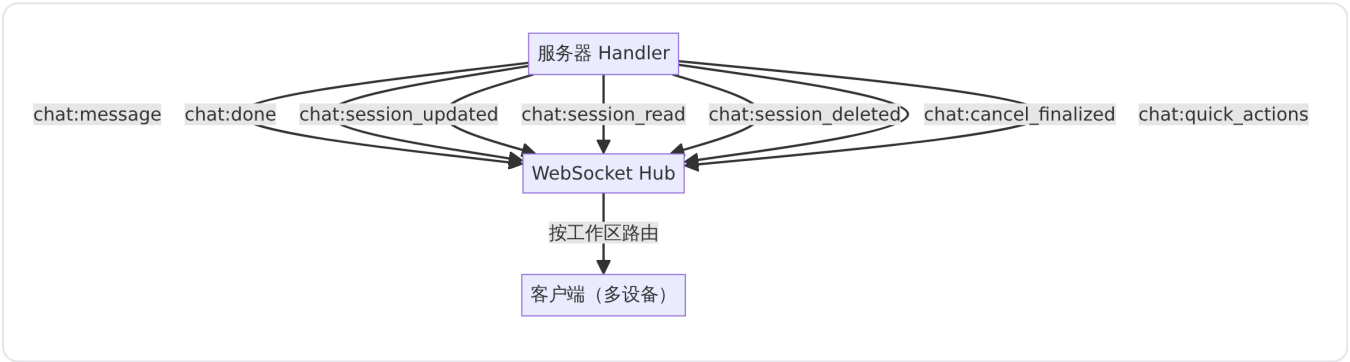
运行时离线时，发送的消息会等待它重新上线。智能体被归档后，不能继续发送新消息；已入队的任务需要等待运行时恢复。界面状态指示器会显示：

- **离线**：智能体的运行时不可达，消息等待发送。
- **重连中**：运行时正在恢复连接。
- **不稳定**：连接波动，回复可能延迟。

[packages/views/locales/zh-Hans/chat.json](#)

实时事件

对话模块通过 WebSocket 推送以下实时事件，由 [实时通信基础设施](#) 承载：



来源：[server/pkg/protocol/events.go](#)

事件	触发时机	关键载荷
<code>chat:message</code>	新消息创建	<code>chat_session_id</code> 、 <code>message_id</code> 、 <code>role</code> 、 <code>content</code> 、 <code>task_id</code>
<code>chat:done</code>	智能体完成回复	<code>message_id</code> 、 <code>content</code> 、 <code>elapsed_ms</code> 、 <code>message_kind</code> 、 <code>quick_actions</code>
<code>chat:session_updated</code>	会话字段变更	<code>title</code> 、 <code>project_id</code> 、 <code>pinned</code> 、 <code>status</code>
<code>chat:session_read</code>	标记已读	<code>chat_session_id</code>
<code>chat:session_deleted</code>	会话删除	<code>chat_session_id</code>

事件	触发时机	关键载荷
chat:cancel_finalized	取消执行的结果确定	outcome (stopped / restored) 、 initiator_user_id
chat:quick_actions	后续提问建议就绪	附加到对应助手消息的快捷操作列表

来源：[server/pkg/protocol/messages.go](#)

事件通过 `publishChat` 方法发布到工作区的 `member` 路由作用域，确保只有该工作区的成员能接收。

执行阶段与状态指示

智能体运行时，界面会实时展示当前阶段和工具活动：

阶段	含义
排队中	消息已入队，等待运行时领取
等待本地目录释放	等待本地工作目录变为可用
启动中	运行时正在启动 AI 编程工具
思考中	智能体正在分析问题
输入中	智能体正在生成回复

工具活动包括：执行命令、读取文件、搜索代码、修改文件、搜索网页等。

[packages/views/locales/zh-Hans/chat.json](#)

快速操作与后续提问

对话完成后，服务器可自动生成后续提问建议（quick actions），以 `chat:quick_actions` 事件推送。用户可以点击建议继续对话，也可以点击 **重新生成建议** 触发服务端重新计算（`POST /api/chat/sessions/{sessionId}/quick-actions/regenerate`）。

重新生成采用异步模式：服务端返回 202，刷新后的建议通过 `chat:quick_actions` 事件送达，客户端无需轮询。

[server/internal/handler/chat.go](#)

全局运行状态指示

对话窗口关闭时，FAB（Floating Action Button）通过独立端点获取运行状态：

- `GET /api/chat/pending-tasks` 返回当前用户在工作区中的所有进行中的对话任务。
- `GET /api/chat/pending-tasks/has-any` 用 EXISTS 查询快速判断是否有进行中的任务，用于点亮"运行中"指示器。

两个端点均过滤掉用户已失去访问权限的私有智能体任务（MUL-4159）。

[server/internal/handler/chat.go](#)

限制与边界

- 对话是创建者私有的，无法共享或转让给其他成员。
- 删除操作不可撤销，暂时不需要时优先使用归档。
- 智能体被归档后，所有进行中的对话任务会被取消。
- 超出模型上下文长度时，对话无法继续——需要开启新会话或拆分需求。
- 消息标题自动生成是尽最大努力（best-effort）的：如果 LLM 层不可用或失败，静默保留原标题，不会阻塞发送。
- 会话的 `agent_id`、`creator_id` 和 `workspace_id` 创建后不可变。
- 会话恢复指针（`session_id`、`work_dir`、`runtime_id`）由守护进程管理，不通过 API 暴露。

[apps/docs/content/docs/chat.zh.mdx](#)

[server/internal/handler/chat.go](#)

小队管理

detail · squad-management.md

小队管理

小队是将人类成员和 AI 智能体混编而成的协作单元。当任务分配给小队时，Multica 不会同时运行所有成员，而是唤醒小队 Leader 智能体，由 Leader 阅读上下文后决定下一步交给谁。

小队解决的核心问题是「把工作交给谁」。它不是将多个智能体合并成一个新智能体，也不自动提高并发。工作范围明确时，直接分配给对应的智能体即可；需要多种能力且无法在创建任务时确定具体负责人时，使用小队更合适 [apps/docs/content/docs/squads.zh.mdx](#)。

用户目标

- **混编队伍**：将不同能力的人类成员和 AI 智能体编入同一小队，统一管理和调度。
- **自动路由**：任务分配给小队后，Leader 智能体自动判断哪位成员最适合接手，无需人工协调。
- **透明协作**：所有派活、执行和交付记录留在任务时间线中，团队随时可回溯。
- **灵活触发**：支持通过分配任务或评论中 @小队两种方式唤醒 Leader，适应不同协作场景。

入口点

小队功能通过以下入口访问：

- **Web 应用小队列表页**：查看所有小队、新建小队、筛选和排序。列表展示小队名称、Leader、成员数、创建者和创建时间 [packages/views/locales/zh-Hans/squads.json](#)。
- **任务负责人选择器**：在任务详情中，负责人选择器可选取小队作为负责人。
- **评论 @提及菜单**：在评论编辑器中 @小队，仅唤醒 Leader 处理该条评论而不改变任务负责人。
- **CLI 命令行**：通过 `multica squad create`、`multica squad member add` 等命令管理小队 [apps/docs/content/docs/squads.zh.mdx](#)。
- **自动化规则**：Autopilot 可将执行者配置为小队，按 cron 或 webhook 触发时自动解析到 Leader 执行 [server/internal/service/builtin_skills/multica-squads/SKILL.md](#)。

小队的组成

小队由以下要素构成：

配置	作用
Leader	必须是一名智能体。接收分配给小队的工作，并决定如何处理。
成员	可以是智能体，也可以是人类成员。同一成员可以加入多个小队。

配置	作用
角色说明	告诉 Leader 每个成员适合处理什么工作。它只是上下文，不会授予权限或自动触发成员。
小队指令	保存路由规则、协作方式和需要贯穿全队的背景，只会提供给 Leader。

创建小队时需要填写名称并选择 Leader。Leader 会自动成为成员；之后可以继续添加成员、填写角色说明和小队指令 apps/docs/content/docs/squads.zh.mdx。

分配后的执行流程

将非 backlog 状态的任务分配给小队后，Multica 只会给 Leader 智能体入队一个执行任务（task），而不是给每个成员都入队。执行流程如下：

- Leader 的运行时领走 task**，流程与普通智能体分配一致。
- Leader 拿到 briefing**：Multica 在 Leader 指令后追加三段内容——Squad Operating Protocol（小队工作规范）、Squad Roster（小队花名册）和 Squad Instructions（小队指令，仅当非空时注入）。
- Leader 将父任务推到 in_progress**：首次接单应离开 todo 进入 in_progress。分发成员不等于完成，小队干活期间父任务保持 in_progress。
- Leader 发一条派活评论**：评论中使用花名册提供的 mention markdown 来 @ 选中成员，该 @ 会触发被派成员入队新 task。
- Leader 记录 evaluation**：通过 `multica squad activity <issue-id> <outcome> --reason "..."` 将评估写入任务活动时间线，方便人类回溯。
- Leader 停下**：派完活后 Leader 不亲自动手。被派成员有回复时 Leader 自动被唤醒，决定下一步动作。

如果任务仍在 backlog 状态，分配本身不会触发 Leader；移出 backlog 后才会开始执行 apps/docs/content/docs/squads.zh.mdx。

Leader 每次执行看到的内容

每次 Leader 被触发时，三段内容会附加到其指令上：

- Squad Operating Protocol**（小队工作规范）：硬编码的规则集——读任务 → 首次接单把父任务推到 in_progress → 用 @ 派活 → 保持简洁（不复述任务内容，被派成员自己能读）→ 每次记 evaluation → 派完就停 → 整体目标达成后才推 in_review。该内容由系统管理，不可编辑。
状态相关规则仅对确实分配给本小队的任务生效。Leader 被别人任务中的 @小队 唤醒时，同样拿到花名册和派活规则，但会被明确告知不要改动那条任务的状态——状态仍归其自己的负责人。
- Squad Roster**（小队花名册）：Leader 一行 + 每个未归档成员一行，每行带可直接复制的 mention markdown。对于智能体成员，花名册还会列出其分配的技能（如 `skills: a, b` 或 `no skills`）

assigned) ，让 Leader 可以按能力而非角色标签来委派工作。人类成员不携带技能段。内置的 multica-* 技能不在花名册中列出，仅展示显式附加给智能体的工作区技能。已归档的智能体成员从花名册中跳过 [server/internal/service/builtin_skills/multica-squads/SKILL.md](#)。

- **Squad Instructions** (小队指令)：用户为小队编写的自定义内容，如路由规则（「数据库相关派给 Alice，前端派给 Bob」）、上报策略或任务本身不会有的背景 [apps/docs/content/docs/squads.zh.mdx](#)。

Leader 的再次触发时机

第一次派活之后，大多数后续评论会自动唤醒 Leader：

事件	触发 Leader
非小队成员（人类、外部智能体）发评论	会
小队成员发进展更新，不带任何 @	会——Leader 重新评估下一步
任何评论里显式 @ 了智能体、成员、小队或 @all	不会——显式 @ 就是路由信号，Leader 让位
Leader 自己发的评论	不会——防自触发
评论里只有任务互链	会——任务引用不算路由

在这些规则之上还有去重：Leader 在该任务上已有排队或已领取的 task 时，新触发不会重复入队 [apps/docs/content/docs/squads.zh.mdx](#)。

成员发的 @ 评论不唤醒 Leader，是因为直接 @ 已经是明确的交接，再唤醒 Leader 只会多一轮空转。例外：智能体发结果时顺手 @ 了另一个智能体，Leader 仍会被唤醒以便协调整条线程。

分配小队与 @小队

操作	是否改变负责人	结果
把任务分配给小队	是	小队成为负责人，并触发 Leader。
在评论中 @小队	否	只让 Leader 处理这条评论，原负责人不变。

直接 @某名成员时，工作会交给该成员处理。Multica 会合并重复触发并阻止 Leader 的评论再次触发自己，避免形成执行循环 [apps/docs/content/docs/squads.zh.mdx](#)。

小队与自动化

Autopilot 可以将执行者配置为小队。当 assignee_type = "squad" 时：

- 可执行智能体从 squad.leader_id 解析。
- 准入和就绪检查针对 Leader 执行。

- 已归档小队会跳过调度 (fail closed) 。
- 运行归属记录在适用时包含小队 ID。

对于 `create_issue` 类型的 Autopilot，创建的任务保留 `assignee_type = "squad"` 和 `assignee_id = <squad-id>`，实际执行的智能体是解析出的 Leader。对于 `run_only` 类型，不创建任务，直接为解析出的 Leader 智能体创建执行任务 [server/internal/service/builtin_skills/multica-squads/SKILL.md](#)。

小队与 Access

将智能体加入小队不会绕过其 Access 权限设置。普通成员创建或管理小队时，只能选择自己有权运行的智能体。成员能否分配或 @某个小队，也取决于其是否有权运行小队的 Leader。若 Leader 已归档，或当前成员无权运行它，这个小队就无法被分配或 @提及 [apps/docs/content/docs/squads.zh.mdx](#)。

分配校验会拒绝：缺少类型/ID 对、不存在的小队、已归档的小队、已归档的 Leader，以及操作者无权访问的私有 Leader [server/internal/service/builtin_skills/multica-squads/SKILL.md](#)。

创建与管理权限

任何工作区成员都可以创建小队。创建者可以修改和归档自己创建的小队；工作区 `owner` 和 `admin` 可以管理所有小队 [apps/docs/content/docs/squads.zh.mdx](#)。

小队名称不要求在工作区中唯一。更换 Leader 时，新 Leader 会自动加入小队；当前 Leader 不能直接移除，需要先指定另一名 Leader [apps/docs/content/docs/squads.zh.mdx](#)。

归档小队

归档后，小队会从列表、负责人选择器和 @菜单中消失，且不可恢复。

为避免现有工作失去负责人，当前分配给小队的任务和自动化会转交给原 Leader 智能体。历史评论和活动记录仍然保留。如果之后还需要相同的路由，需要重新创建小队 [apps/docs/content/docs/squads.zh.mdx](#)。

警告： 归档操作无法撤销。

CLI 操作

```
multica squad create --name "Product Delivery" --leader delivery-lead
multica squad member add <squad-id> \
  --member-id <agent-or-member-id> \
  --type agent \
  --role "负责前端实现"
```

完整命令参考 CLI 文档 [apps/docs/content/docs/squads.zh.mdx](#)。

设计边界与限制

小队机制有明确的设计边界，理解这些有助于正确使用：

- **不合并智能体**：小队不会将多个智能体合并成一个新的智能体。每个成员仍是独立的智能体或人类成员。
- **不提高并发**：小队本身不带来并发执行。Leader 一次派活给一个成员，但多个成员可以各自在自己的运行时并行工作。
- **不广播任务**：分配给小队的任务仅入队给 Leader，不会入队给每个成员。成员只在被 Leader 显式 @ 后才收到自己的 task。
- **Leader 不亲自动手**：Leader 的角色是协调和路由，派完活即停。实际执行由被派成员完成。
- **状态由 Leader 管理**：父任务的状态由 Leader 按协议管理（`in_progress` → 最终 `in_review`），成员只负责自己收到的子任务。Leader 的 task 完成本身不改变任务状态
server/internal/service/builtin_skills/multica-squads/SKILL.md。
- **更换负责人会取消现有任务**：修改任务负责人时，该任务已有的执行任务会被取消，然后为新负责人路径入队。
- **任务状态写入权限限定**：Leader 仅对确实分配给本小队的任务拥有状态写入权限。通过 @小队 在他人任务中被唤醒时，协议明确禁止修改该任务的状态。

与其他功能的关系

- **智能体协作看板**：小队将智能体作为成员纳入看板，智能体配置（名称、指令、Skills、Access）决定其在队内的行为边界。智能体是队友，小队是编队方式。
- **任务与问题管理**：任务分配给小队时，`assignee_type` 设为 "squad"、`assignee_id` 设为小队 ID。任务状态流转、评论和执行日志均在任务生命周期内记录。
- **自动化规则**：Autopilot 可将执行者指向小队，定时或通过 webhook 触发时自动解析到 Leader 执行，适合周期性的巡检、日报等重复工作。

技能管理

detail · skill-management.md

技能管理

技能（Skill）是 Multica 工作区中可复用的工作方法集合。它以 `SKILL.md` 为主文件，可附带脚本、模板和参考资料等支持文件。将技能添加到智能体后，智能体会在执行相关任务时使用其中的说明——相同的方法不必在每个智能体的指令中重复维护。

技能与智能体指令的区别

技能和智能体指令各有侧重，不能互相替代：

	智能体指令	技能
适合保存	该智能体长期不变的职责、边界和交付要求	某类工作可复用的步骤、资料 and 工具
使用范围	只属于一名智能体，每次执行都会提供	可以添加给多名智能体，并单独启用或停用
典型例子	"只审查前端代码，不直接修改"	一套无障碍检查清单和报告模板

智能体指令定义"这名智能体是谁"，技能说明"这类工作怎么做"。修改技能后，所有绑定该技能的智能体从下一次任务执行开始使用新内容——不会影响已经在运行中的任务。

创建技能

在工作区的 **Skills** 页面选择"新建 skill"，有三种入口：

方式	适用场景
手动创建	从空白的 <code>SKILL.md</code> 开始编写，填写名称、描述、正文内容以及可选的支持文件。
从 URL 导入	从 GitHub、ClawHub 或 Skills.sh 导入已发布的技能。
从运行时复制	扫描连接电脑上的本地技能快照，将选中内容复制到工作区。

通过 CLI 也可以从本地 `.skill` 或 `.zip` 文件导入：

```
multica skill import --file ./review-helper.skill
```

来源：[apps/docs/content/docs/cli.zh.mdx](#)

技能的内容结构

每个技能包含一个主文件 `SKILL.md`，它是技能的入口。在 `SKILL.md` 中应明确描述：

- 技能适用的场景；
- 开始前需要检查的事项；
- 具体执行步骤；
- 结果的提交形式；
- 需要停止并向成员确认的情况。

此外还可以添加任意支持文件，例如：

```
SKILL.md
references/api-conventions.md
templates/review-report.md
scripts/check.sh
```

`SKILL.md` 是保留路径，不能再创建同名的支持文件。支持文件会和主文件一起提供给智能体。在服务端实现中，路径 `SKILL.md` 由 `skill.IsReservedContentPath` 强制执行——任何大小写变体（如 `./SKILL.md`）都会被拒绝，避免与守护进程写入的主内容文件冲突。

来源：server/internal/skill/reserved.go

导入技能

支持的导入源

URL 导入通过 URL 的域名自动判断来源：

来源	URL 格式	说明
ClawHub	<code>https://clawhub.ai/{owner}/{slug}</code>	公开技能市场，也支持搜索
Skills.sh	<code>https://skills.sh/{owner}/{repo}/{skill-name}</code>	基于 GitHub 仓库的技能目录
GitHub	<code>https://github.com/{owner}/{repo}</code>	直接从 GitHub 仓库导入

没有域名的裸 slug 默认按 ClawHub 处理。导入源识别逻辑由 `detectImportSource` 实现，对 `clawhub.ai`、`skills.sh`、`github.com` 三个域名分别路由。

导入时的冲突处理

当工作区已存在同名技能时，通过 `on_conflict` 参数控制行为：

策略	行为	适用场景
<code>fail</code> （默认）	停止导入，不修改现有技能	不确定是否需要替换时

策略	行为	适用场景
overwrite	用导入的内容覆盖现有技能，保留原 ID 和智能体绑定	更新到最新版本
rename	以新名称保存，最多尝试 50 次	同时保留新旧版本
skip	跳过该技能，不导入	已有足够新的本地版本

overwrite 仅限该技能的创建者本人操作。工作区 admin 虽然可以在应用内编辑技能，但不能通过导入方式覆盖他人创建的技能。这是有意为之的限制——覆盖权限比编辑权限更窄。

来源：[server/internal/handler/skill.go](#)

导入限制

为防止超大文件导致问题，导入时有以下硬性限制：

限制项	值	说明
单文件最大体积	1 MiB	超过则导入失败
支持文件总大小	8 MiB	不含 SKILL.md 本身
最大支持文件数	256	超过则导入失败

二进制文件（图片、字体、压缩包等）在导入时会被静默跳过，因为它们无法安全存入数据库的 TEXT 列，且智能体也不会以文本形式读取它们。

来源：[server/internal/handler/skill.go](#)

输入净化

所有通过 API 写入的文本内容在进入数据库前会经历两层净化，由 sanitizeNullBytes 统一处理：

- **NULL 字节 (0x00)**：PostgreSQL 会拒绝含 NUL 字节的 TEXT 值 (SQLSTATE 22021)，因此直接移除。
- **无效 UTF-8 字节序列**：例如 Windows-1252 编码的弯引号 (0x91)，曾导致智能体模板导入技能时崩溃。使用 strings.ToValidUTF8 将其丢弃。

净化范围覆盖技能名称、描述、正文内容以及每个支持文件的路径和内容，在创建和更新路径中均强制执行。

来源：[server/internal/handler/skill.go](#)

添加到智能体

创建或导入技能后，还需将其绑定到具体智能体才会生效。绑定操作可以在两个入口完成：

- 在技能详情页选择目标智能体；
- 在智能体的 **Skills** 标签页中管理。

绑定规则：

- 一名智能体可以绑定多个技能；
- 同一个技能可以绑定给多名智能体；
- 可以暂时停用绑定，不必删除技能；
- 修改后的内容从后续任务执行开始生效，不影响已运行中的任务；
- 只有有权修改该智能体的成员，才能为它添加、移除或停用技能。

通过 CLI 管理绑定的两种方式——全量替换和增量添加：

```
multica agent skills set <agent-id> --skill-ids <id1>,<id2>  
multica agent skills add <agent-id> --skill-ids <id3>
```

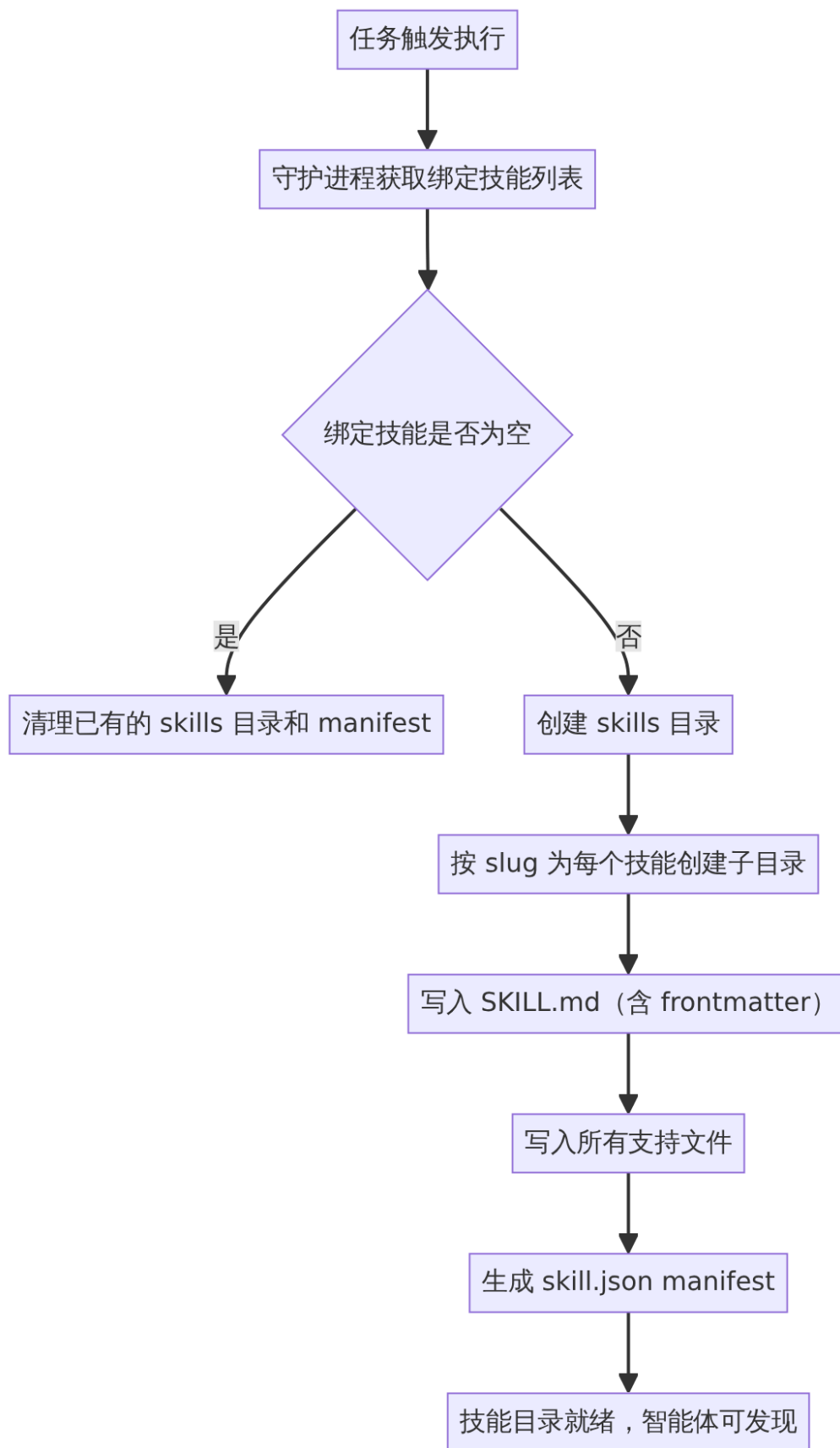
来源：apps/docs/content/docs/cli.zh.mdx

守护进程中的技能准备

当任务被触发执行时，守护进程在准备执行环境的阶段将绑定的技能下载并写入工作目录，确保智能体在执行任务时可以访问技能内容。

技能写入流程

以 QwenPaw 运行时的执行环境准备为例（`prepareQwenpawWorkspace`），流程如下：



该流程是幂等的：每次准备时先删除已有的 skills 目录和 manifest，再重建。这意味着添加、移除或编辑技能都能正确反映，不会累积过期状态。当所有技能都被解绑时，已有的技能文件会被彻底清理——解绑即撤

销。

来源：[server/internal/daemon/execenv/qwenpaw_workspace.go](#)

技能 Slug 生成

技能名称在写入执行环境时会转换为文件系统安全的 slug（如 "Deploy Helper" → `deploy-helper`）。重复准备同一套技能的测试验证了：多次准备同一套技能后，slugs 不会出现碰撞后缀（如 `deploy-helper-multica`），manifest 保持与技能数量一致。

来源：[server/internal/daemon/execenv/qwenpaw_workspace_test.go](#)

SkillContextForEnv 数据结构

守护进程在执行环境准备时使用的技能上下文结构：

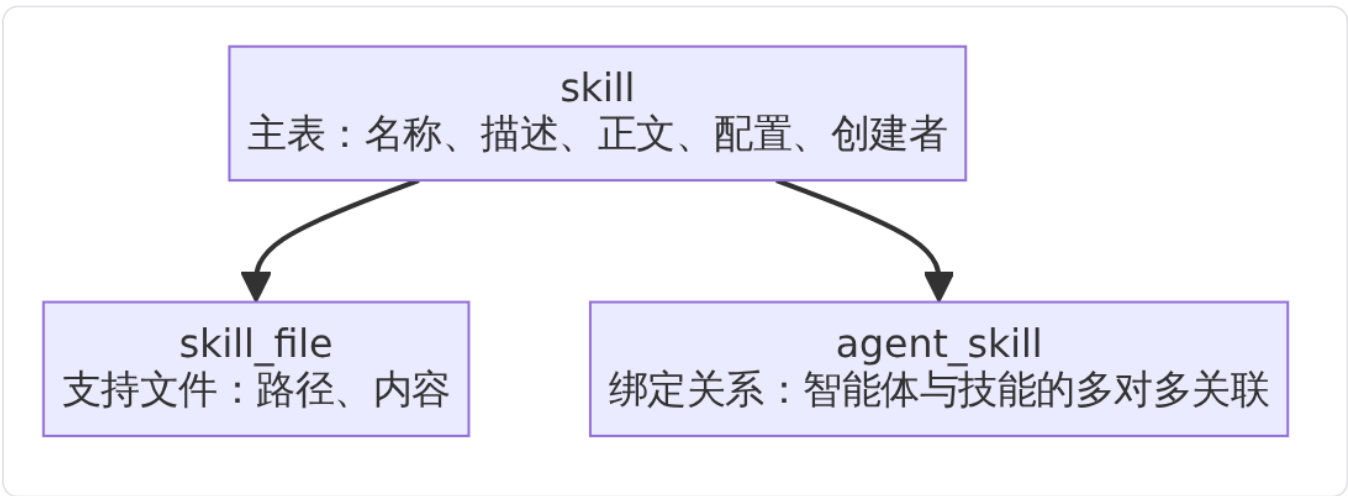
```
type SkillContextForEnv struct {
    Name      string
    Description string
    Content    string
    Files     []SkillFileContextForEnv
}

type SkillFileContextForEnv struct {
    Path    string
    Content string
}
```

来源：[server/internal/daemon/execenv/execenv.go](#)

数据模型

技能在数据库中以三张核心表存储，通过迁移脚本 `008_structured_skills` 引入：



- **skill**：存储技能本身的元数据和主内容（`SKILL.md` 正文作为 `content` 字段）。
- **skill_file**：存储技能下的支持文件，每条记录包含相对路径和内容。

- **agent_skill**：智能体与技能的多对多绑定关系表，支持启用/停用。

来源：[server/migrations/008_structured_skills.down.sql](#)

修改权限

操作	谁可以执行
创建/导入	任何工作区成员
编辑	创建者本人、工作区 <code>owner</code> 或 <code>admin</code>
删除	创建者本人、工作区 <code>owner</code> 或 <code>admin</code>
通过导入覆盖	仅创建者本人（比编辑更窄）
使用	任何有智能体修改权限的成员可以将技能绑定到该智能体

删除是永久操作，会同时从所有绑定的智能体上移除该技能。删除前会先清理技能上的标签分配。

来源：[server/internal/handler/skill.go](#)

工作区技能与仓库本地技能

工作区技能保存在 Multica 服务端数据库中，适合由团队共同维护并跨智能体共享。

部分 AI 编程工具也会直接读取代码仓库中的项目级技能目录，例如 `.claude/skills/` 或 `.agents/skills/`。这些文件仍由 Git 仓库管理，Multica 不会自动将它们登记为工作区技能。

连接电脑上已安装的运行时本地技能（Claude Code 和 Codex）会显示在智能体的 **Skills** 标签页中，可以按智能体单独停用。从运行时复制的技能是该时刻的文件快照——之后本地文件发生变化不会自动同步到工作区，需要重新导入或手动编辑。

来源：[apps/docs/content/docs/skills.zh.mdx](#)

安全注意事项

从外部导入的技能可能包含脚本、命令或不可信的指令。Multica 不会对导入的内容进行审查、签名或沙箱隔离——导入的内容会原样提供给智能体执行。因此，导入来源必须可信，特别是当智能体拥有执行 `shell` 命令的权限时。

来源：[apps/docs/content/docs/skills.zh.mdx](#)

限制

- 单个支持文件最大 1 MiB，整个导入包支持文件总大小上限 8 MiB，最多 256 个支持文件——超过任一项都会导致整个导入失败。
- `SKILL.md` 为保留文件名，不能作为支持文件路径上传。

- 从运行时复制的技能是快照，不会自动同步本地后续变更。
- 导入覆盖仅限技能创建者本人，即使 `admin` 也无法通过导入覆盖他人创建的技能。
- 删除操作不可撤销，会同时从所有智能体上移除该技能。
- 二进制文件（图片、字体、压缩包等）在导入时会被静默跳过。

频道集成

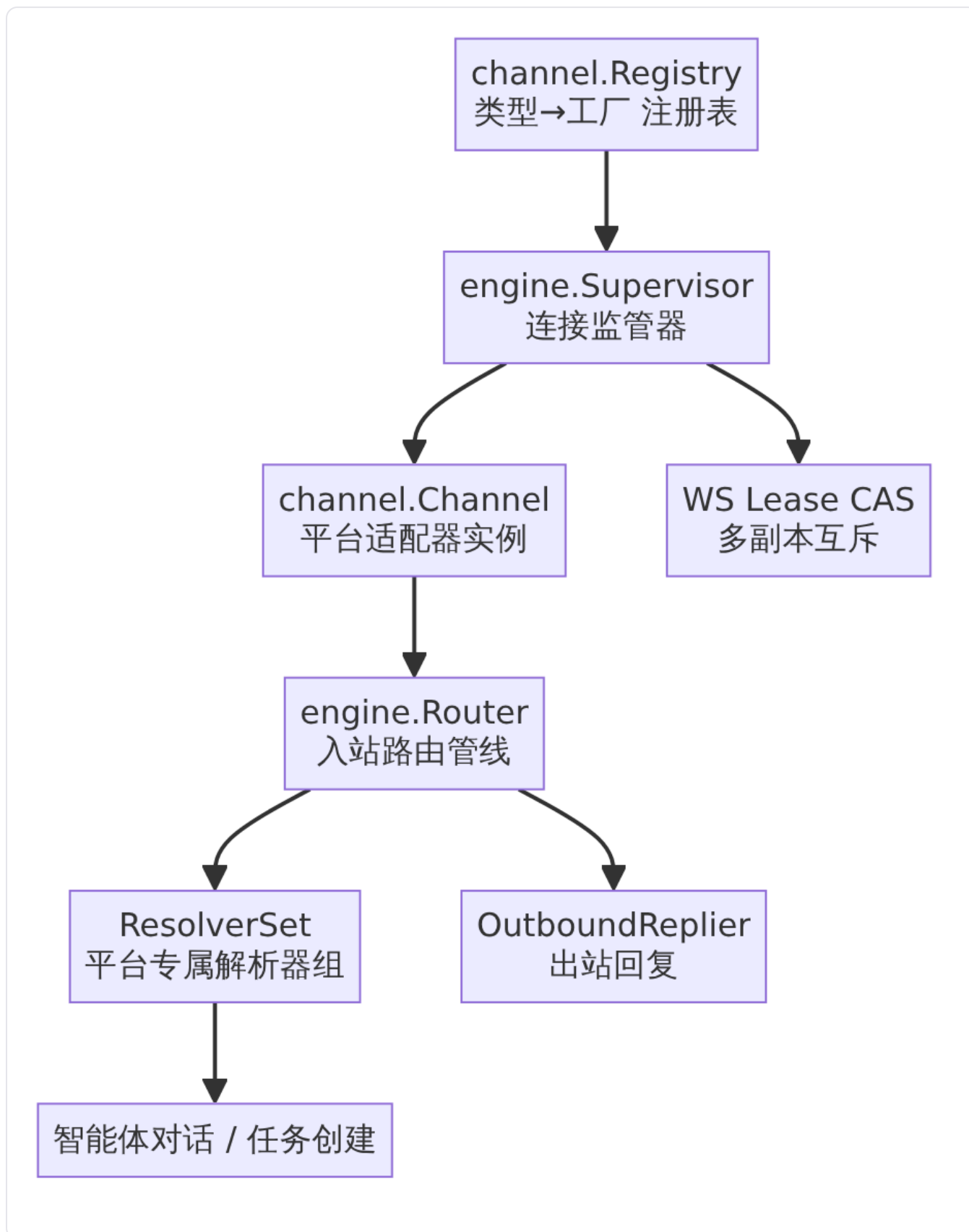
detail · channel-integration.md

频道集成

频道集成是 Multica 的跨平台即时消息（IM）接入引擎，使用户能够通过 Slack、飞书、钉钉、企业微信等外部聊天平台与 Multica 智能体交互。它提供了一套频道无关的通用运行时，各平台适配器只需实现统一契约即可接入，无需修改引擎核心代码。

架构概览

频道集成分为两层：**契约层**（`channel` 包）定义跨平台抽象，**引擎层**（`engine` 包）提供连接管理与消息路由的通用运行时。



来源：engine/doc.go 定义了 Supervisor 和 Router 两大组件的职责边界。

支持的平台

当前引擎支持四种频道类型，每种平台通过常量 `channel.Type` 标识：

平台	Type 常量	连接模式	安装模型
飞书 / Lark	"feishu"	WebSocket 长连接（Lark 协议）	OAuth 设备流 / 应用市场
Slack	"slack"	Socket Mode（每安装独立连接）	自带应用（BYO）
企业微信	"wecom"	智能机器人 WebSocket	自带应用
钉钉	"dingtalk"	—	自带应用（BYO）

[channel/channel.go](#) 定义 `channel.Type` 及其飞书常量；[slack/channel.go](#)、[wecom/types.go](#)、[dingtalk/config.go](#) 分别定义各自的 Type 常量。

核心契约

Channel 接口

所有平台适配器必须实现 `channel.Channel` 接口的五个方法：

- **Type()**：返回平台标识，与注册时的 Type 一致，实例存活期内稳定不变。
- **Connect(ctx)**：建立平台连接并阻塞运行接收循环，直到连接结束。返回 `nil` 表示正常退出（`ctx` 取消），非 `nil` 错误将触发监管器的指数退避重连。
- **Disconnect(ctx)**：断开连接并释放资源，重复调用安全。
- **Send(ctx, out)**：投递一条出站消息，返回平台消息 ID。仅网络、认证、限流等真实投递失败才返回错误。
- **Capabilities()**：声明该平台支持的能力位掩码（纯声明，无副作用）。

[channel/channel.go](#) 定义了完整的 Channel 接口及其语义契约。

消息信封

入站消息统一建模为 `channel.InboundMessage`，这是引擎唯一的消息处理单元：

- **路由信息**：`Source` 结构包含 `ChannelType`、`ChatID`、`ChatType`（`p2p` 或 `group`）、`SenderID`——所有字段在所有平台均成立。
- **消息内容**：`Text` 为智能体可读文本，`CommandText` 为命令分类用的原始文本，`Type` 为归一化消息类型（`text / image / file / audio / video / unknown`）。
- **去重与追踪**：`EventID` 和 `MessageID` 支撑幂等层，以（`installation, MessageID`）为去重键。
- **上下文关联**：`ReplyTo` 携带引用/回复上下文，`AddressedToBot` 标记群聊中 @机器人的判定，`ForceFresh` 请求开启新会话，`SkipAgentRun` 抑制智能体运行。
- **平台原始数据**：`Raw` 字段（`json.RawMessage`）存储平台特有数据，核心代码永不读取。

[channel/message.go](#) 定义了 `InboundMessage` 和 `OutboundMessage` 的完整结构。核心设计原则（MUL-3515 §2）明确规定：信封仅包含跨平台通用字段，平台特有数据仅存在于 `Raw` 中且仅由适配器读取。

能力声明

`channel.Capability` 是 64 位掩码，用于声明平台支持的功能：

能力	含义
<code>CapText</code>	纯文本消息（所有平台应至少声明此项）
<code>CapRichCard</code>	富交互卡片（Lark 交互卡片、Slack Block Kit）
<code>CapThreadReply</code>	话题/讨论串回复
<code>CapQuoteReply</code>	引用回复
<code>CapAttachment</code>	媒体附件收发
<code>CapVoice</code>	语音/音频消息
<code>CapTypingIndicator</code>	输入状态指示器
<code>CapMessageEdit</code>	消息编辑（卡片补丁、 <code>chat.update</code> ）

能力声明是纯声明式，核心代码不含降级逻辑——调用方自行读取位掩码并决定降级策略（例如富卡片 → 纯文本），因此新增平台无需修改核心代码。

[channel/capability.go](#) 定义了完整的能力位掩码。

Registry 注册表

`channel.Registry` 维护 `Type → Factory` 映射，采用“最后注册者胜出”（last-writer-wins）语义。新增平台仅需调用 `Register(type, factory)`，不需编辑引擎代码。注册表并发安全。

[channel/registry.go](#) 定义了 `Registry` 的完整实现。

连接监管器（Supervisor）

`engine.Supervisor` 是每个安装实例的连接监管器，从飞书专用的 `lark.Hub` 泛化而来。它负责：

- **枚举活跃安装**：跨所有频道类型扫描需要建立连接的安装实例，无平台硬编码。
- **多副本互斥**：通过 WS 租约 CAS（Compare-And-Swap）保证全局最多一个副本持有某安装的连接，杜绝重复事件消费。
- **连接生命周期**：通过 `channel.Registry` 构建平台 `Channel`，驱动 `Connect / Disconnect`。
- **指数退避重连**：失败后从 `MinBackoff`（默认 2s）开始加倍，上限 `MaxBackoff`（默认 60s），一旦连接存活超过 `ResetBackoffAfter`（默认 60s）则重置退避。每次重连间隔增加 `[0, backoff)` 均匀分布的随机抖动。
- **凭证轮转感知**：检测到安装凭证指纹变化时，重建连接并使用新凭证。

监管器关键配置及其默认值：

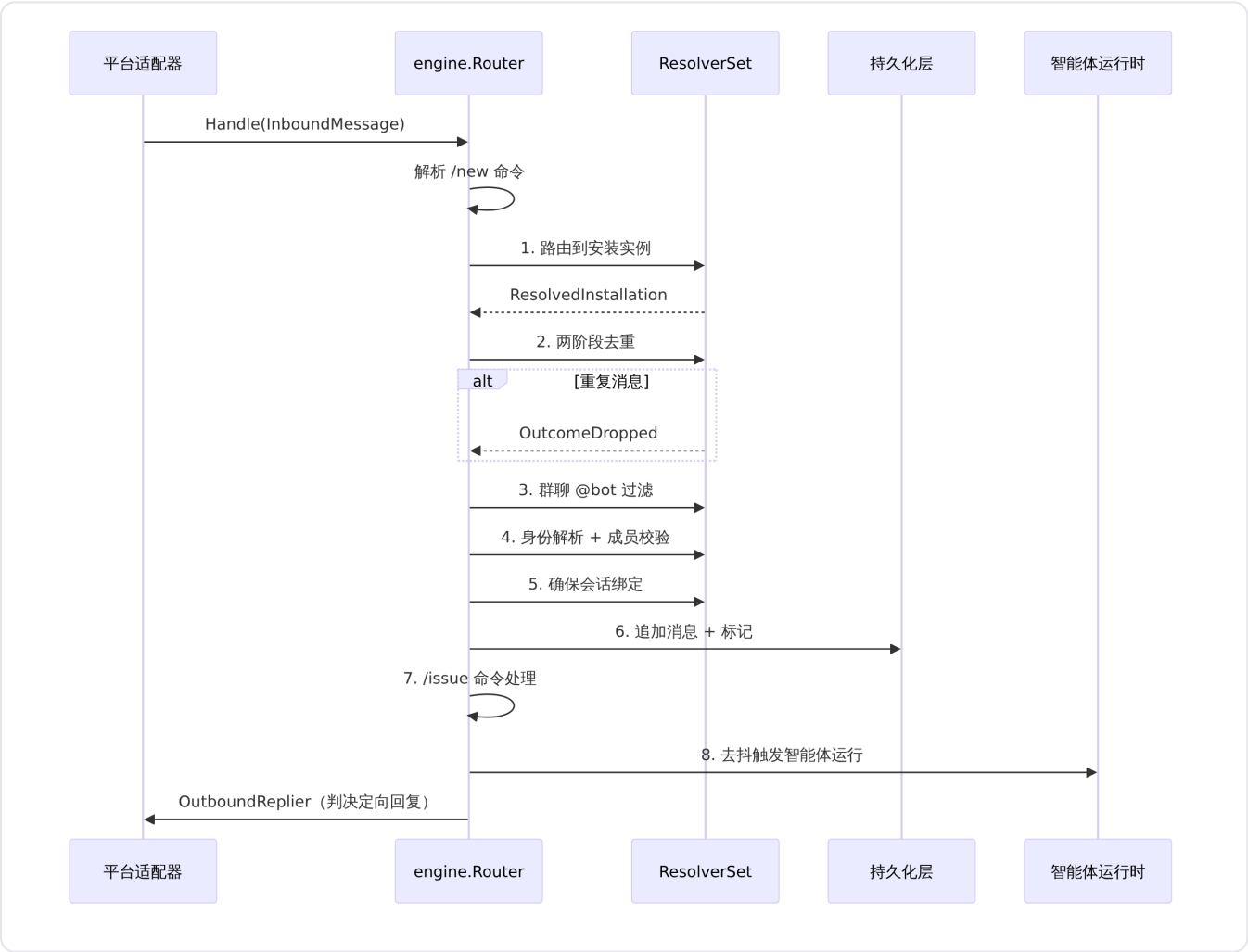
配置项	默认值	说明
LeaseTTL	90s	租约有效期
LeaseRenewInterval	30s	租约续期间隔
PollInterval	30s	扫描新安装/过期租约间隔
MinBackoff	2s	重连最小退避
MaxBackoff	60s	重连最大退避
ResetBackoffAfter	60s	正常连接多久后重置退避

engine/supervisor.go 定义了 Config 和 Supervisor 的完整实现。

入站路由管线 (Router)

engine.Router 是统一的 channel.InboundHandler 实现，从飞书专用的 lark.Dispatcher 泛化而来。监管器将其注入每个 Channel，所有平台通过同一管线处理入站消息。

管线流程



管线步骤详解：

- 1. **安装路由**：通过 `InstallationResolver` 将消息路由到对应的安装实例。
- 2. **两阶段去重**：以 `(安装ID, MessageID)` 为键执行 `idempotency` 检查，重复消息标记为 `OutcomeDropped`（原因 `DropReasonDuplicate`）。
- 3. **群聊 @bot 过滤**：群聊场景下，仅处理 `AddressedToBot` 为真的消息；未 @机器人的消息丢弃（原因 `DropReasonNotAddressedInGroup`）。
- 4. **身份解析与成员校验**：通过 `IdentityResolver` 将平台用户映射到 Multica 用户；非工作区成员丢弃（原因 `DropReasonNonWorkspaceMember`）；未绑定用户进入绑定提示流程（`OutcomeNeedsBinding`）。
- 5. **会话绑定**：通过 `SessionBinder` 确保消息关联到有效的聊天会话——首次自动创建，后续复用。
- 6. **追加与标记**：通过 `Auditor` 将消息持久化到 `channel_chat_message` 表，并写入占位媒体记录。
- 7. **/issue 命令**：解析共享命令（跨平台一致），创建任务并关联到会话。
- 8. **去抖运行触发**：通过 `pendingBatcher` 合并短时间窗口内的多条消息，去抖后触发智能体对话运行。

管线判决结果

Outcome	含义
<code>dropped</code>	消息被丢弃（去重重、非成员、未@等）
<code>needs_binding</code>	用户未绑定 Multica 账号，需引导
<code>ingested</code>	已摄入，触发智能体运行
<code>agent_offline</code>	智能体离线
<code>agent_archived</code>	智能体已归档

[engine/router.go](#) 定义了 `Router` 的完整实现；[engine/resolvers.go](#) 定义了 `Outcome`、`Result`、`ResolvedInstallation` 等类型。

出站回复与输入指示器

`Router` 在摄入消息后会异步驱动两个出站行为：

- **OutboundReplier**：根据管线判决（`Result`）定向回复——绑定引导、智能体离线/归档通知、`/issue` 确认等。在 `ACK` 关键路径之外独立执行。
- **TypingNotifier**：摄入成功时显示"处理中"指示器（`OnIngested`），会话因智能体离线等原因被静默终止时清除指示器（`OnSettled`）。

二者均为可选接口，平台 `ResolverSet` 按需提供。

ResolverSet：平台专属解析器组

每个平台的平台特有逻辑封装在 `engine.ResolverSet` 中，包含以下接口：

解析器	接口	必须	职责
Installation	InstallationResolver	是	将入站消息路由到正确安装实例
Identity	IdentityResolver	是	将平台用户映射到 Multica 用户
Dedup	Deduper	是	幂等去重
Session	SessionBinder	是	会话绑定
Media	MediaResolver	是	媒体下载/上传/附件绑定
Audit	Auditor	是	消息追加持久化
Replier	OutboundReplier	否	判决定向回复
Typing	TypingNotifier	否	输入状态指示
OriginType	string	是	/issue 来源标签 (如 "lark_chat")

[engine/resolvers.go](#) 定义了 `ResolverSet` 结构体。

新增平台只需注册一个 `Factory` (构建 `Channel`) 和一个 `ResolverSet` (注入管线) , 无需编辑引擎代码。

平台适配器详情

飞书 / Lark

飞书是首个也是功能最完整的适配器。它同时服务中国大陆飞书云和 Lark 国际云，区域配置存储在每条安装记录中，不作为独立 `Type` 。连接模式为 `WebSocket` 长连接协议。

- 安装方式：OAuth 设备流 (RFC 8628) ，通过飞书应用市场扫码安装。
- 注册入口：`lark.RegisterFeishu(channelRegistry, deps) + channelRouter.Register(channel.TypeFeishu, lark.NewFeishuResolverSet(...))`
- 环境变量：`MULTICA_LARK_SECRET_KEY` 、 `MULTICA_LARK_CALLBACK_BASE_URL`

[server/cmd/server/router.go](#) 展示了飞书适配器的完整注册流程。

Slack

Slack 采用自带应用 (BYO) 模型：工作区管理员在 Slack 中创建并安装自己的 Slack App，将其 bot token (`xoxb-`) 和 app-level token (`xapp-`) 粘贴到 Multica。每个安装拥有独立的 `Socket Mode` 连接，由引擎按安装实例分别监管。

- 安装方式：BYO 手动配置。
- 注册入口：`slack.RegisterSlack(channelRegistry, deps)`
- 环境变量：`MULTICA_SLACK_SECRET_KEY`

Slack 适配器设计参考了 Nous Research 的 Hermes Agent 中经过验证的 Slack 适配器模式。

[slack/config.go](#) 定义了 Slack 安装配置的 JSON 结构，包含加密存储的 bot 和 app token。

企业微信

企业微信适配器实现智能机器人 (aibot) WebSocket 长连接协议。每个安装一份 (bot_id, secret)，连接 `wss://openws.work.weixin.qq.com`。

- 安装方式：BYO 手动配置。
- 注册入口：`wecom.RegisterWecom(channelRegistry, deps)` + `channelRouter.Register(wecom.TypeWecom, wecom.NewResolverSet(...))`
- 环境变量：`MULTICA_WECOM_SECRET_KEY`

多副本限制：企业微信的出站投递（智能体回复）仅能由持有 WebSocket 租约的副本完成。在多副本部署中，其他副本产生的 `EventChatDone` / `EventInboxNew` 事件无法路由到租约持有者，因此这些回复将被丢弃。建议将企业微信后端作为单副本运行，直到实现跨副本出站路由。

[wecom/types.go](#) 完整说明了企业微信适配器的设计原理和维护策略（社区维护，由 @leroy-chen 和 @seacen 共同拥有）。

企业微信的 `Channel.Send` 方法返回 `ErrSendNotSupported`，因为所有出站均通过 `OutboundReplier` / `Outbound` 路径（基于 `EventChatDone` / `EventInboxNew` 事件），不经过通用 `Send` 接口。

[wecom/wecom_channel.go](#) 定义了 `Send` 的存根实现和工厂注册逻辑。

钉钉

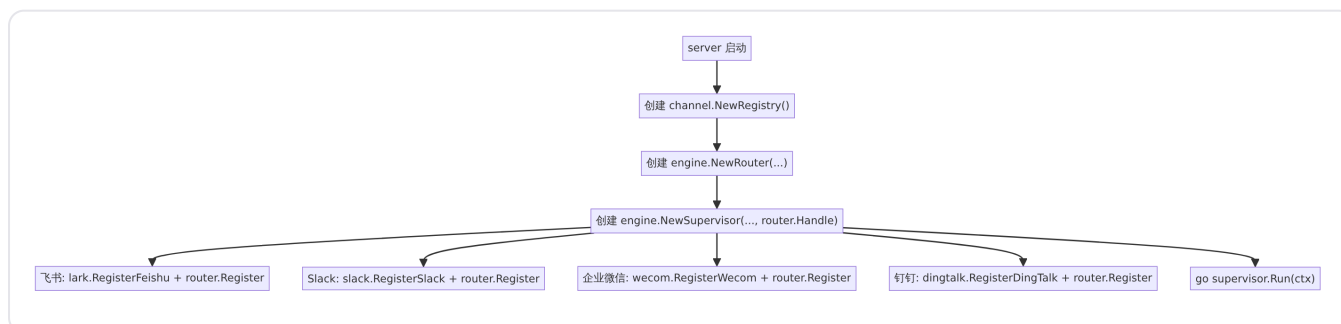
钉钉适配器同样采用 BYO 模型，工作区管理员创建自己的钉钉应用并配置到 Multica。

- 安装方式：BYO 手动配置。
- Type 常量：`dingtalk.TypeDingTalk = "dingtalk"`

[dingtalk/config.go](#) 定义了钉钉的 Type 常量。

启动注册流程

所有平台在服务器启动时统一注册到 `channel.Registry` 和 `engine.Router`。以 `server/cmd/server/router.go` 中的启动代码为例：



[server/cmd/server/router.go](#) 展示了所有平台适配器的导入和注册上下文。

配置方式

各平台通过环境变量控制启用/禁用。未设置对应密钥时，平台静默禁用并输出 Info 日志。

平台	环境变量	安装 API
飞书	MULTICA_LARK_SECRET_KEY	POST /lark/install/device-flow
Slack	MULTICA_SLACK_SECRET_KEY	POST /slack/install/byo
企业微信	MULTICA_WECOM_SECRET_KEY	POST /wecom/install/byo

[server/cmd/server/router.go](#) 展示了 Slack 和 Wecom 的 BYO 安装 API 路由注册。

与相关功能的关系

频道集成是用户与 Multica 智能体交互的**外部入口之一**，与其他功能紧密协作：

- **智能体对话**：频道消息摄入后，引擎触发智能体对话运行——等同于在 Multica 界面中发起聊天。会话通过 `channel_chat_session_binding` 与平台聊天一一对应。
- **任务与问题管理**：用户通过 `/issue` 命令在频道中直接创建任务，引擎调用共享的 `IssueCreator` 接口完成创建，并返回确认消息。创建的任务与 Web 界面中的任务完全一致。
- **通知中心**：频道相关的安装状态变更、绑定提示等通过实时事件总线推送至前端。

架构约束与设计边界

- **平台无关性**：引擎核心（`engine` 包）不导入任何具体平台适配器包，仅依赖 `channel` 包的接口定义。
- **Raw 字段隔离**：`InboundMessage.Raw` 仅由适配器读写，核心代码永不访问——这是 MUL-3515 §2 的硬边界。
- **无降级逻辑**：`Capability` 仅声明平台能力，降级决策（如富卡片 → 纯文本）由调用方自行处理，不在核心代码中实现。
- **出站消息范围**：`OutboundMessage` 仅包含 `ChatID`、`Text`、`ThreadID`、`ReplyTo` 四个字段。富卡片、媒体上传、出站 Webhook 等高级出站能力由适配器自行暴露，不通过此跨平台信封。
- **纯函数契约层**：`channel` 包无数据库、网络、平台依赖，不导入任何集成包。

[channel/doc.go](#) 系统性地阐述了上述设计边界。

版本控制集成

detail · vcs-integration.md

版本控制集成

Multica 为自部署场景提供与自托管 Git 实例的深度集成，支持 **Forgejo**、**Gitea** 和 **GitLab** 三种代码托管平台。工作区连接 Git 实例后，Pull Request（或 Merge Request）会根据其中引用的 issue 编号自动关联到对应任务，合并时自动推进任务状态，并实时镜像 CI/CD 流水线状态。该能力与 GitHub 集成并行运作，一个工作区可同时组合使用多个提供商。

适用场景与限制

版本控制集成仅在自部署 **Multica** 中可用——Multica Cloud 不提供此功能。其设计目标是让部署在自有网络中的 Multica 实例能够访问同一网络内的自托管 Git 服务器。

`server/internal/handler/vcs.go` 中 `isVCSAvailable()` 方法定义了产品边界：该开关决定部署是否提供此功能，与加密密钥是否配置（`isVCSConfigured()`）相互独立。未启用时，前端设置页中整个集成板块隐藏，相关 API 端点均返回拒绝。

与 GitHub 的 App/Installation 模型不同，这些 Git 提供商没有「App」概念。每个工作区独立存储自己的实例地址和访问令牌，并在对应仓库或组织上注册 Webhook。令牌和 Webhook 密钥在服务端均经过加密存储。

前置条件

服务端需同时满足两个条件：

1. **启用功能开关**——设置环境变量 `MULTICA_VCS_INTEGRATION_ENABLED=true`。官方自部署 `docker-compose.selfhost.yml` 已预设此值。
2. **配置加密密钥**——设置一个 Base64 编码的 32 字节密钥作为 `MULTICA_VCS_SECRET_KEY`，服务端使用它对存储的访问令牌和 Webhook 密钥进行加密。未配置时连接表单不可用。

此外，建议设置 `MULTICA_PUBLIC_URL` 为服务端的公网基础地址，这样 Multica 可以直接显示完整的 Webhook URL 供用户复制粘贴到 Git 实例。未设置时界面仅显示 Webhook 路径，用户需自行拼接来源地址。

来源：用户文档 [vcs-integration.mdx](#) 对前置条件做了完整说明。

连接生命周期

连接工作区

用户在工作区 **设置** → **集成** → **Git 代码托管** 中执行连接操作：

1. 在 Git 实例中创建具备仓库读取权限的访问令牌：Forgejo/Gitea 通过设置 → 应用，GitLab 需要 `read_api` 权限的个人或群组访问令牌。
2. 选择提供商、填写实例 URL 和访问令牌，点击连接。
3. 服务端先向实例发起校验请求确认令牌有效，校验通过后才保存连接。
4. 连接成功后一次性返回 **Webhook URL** 和 **Webhook 密钥**——后者仅在此时显示，离开页面后无法再次获取。

连接操作对应 [server/internal/handler/vcs.go](#) 中的 `ConnectVCS` 处理器（`POST /workspaces/{id}/vcs/connections`）。它会验证提供商种类（`Kind.Valid()`），调用对应 provider 的 `ValidateToken` 方法做实例端校验，为连接生成唯一的 Webhook 密钥，并将令牌和密钥加密后存入数据库。

查看连接列表

工作区成员均可查看已有连接列表（`GET /workspaces/{id}/vcs/connections`），返回 `ListVCSConnectionsResponse`，其结构在 [packages/core/types/vcs.ts](#) 中定义。响应包含两个关键部署级标志：

- `available`：部署是否启用了集成（自部署且开关打开时为 `true`）
- `configured`：加密密钥是否已设置（决定连接表单是否可用）
- `can_manage`：当前用户是否为 `owner/admin`（决定是否显示连接/断开控件）

轮换 Webhook 密钥

`POST /workspaces/{id}/vcs/connections/{connectionId}/rotate-webhook` 执行密钥轮换。调用时服务端生成新的 Webhook 密钥，加密存储后返回一次性明文值。用户需将新密钥更新到 Git 实例的 Webhook 设置中。前端在轮换前会弹出确认对话框，提示用户旧密钥将立即失效。

断开连接

`DELETE /workspaces/{id}/vcs/connections/{connectionId}` 删除连接及其关联的加密凭据。前端同样要求确认操作，并提醒用户手动移除 Git 实例侧的 Webhook 注册。

所有变更类操作（连接、轮换、断开）均仅限 `owner` 和 `admin` 角色执行。

Webhook 处理

Webhook 端点

每个连接拥有唯一的 Webhook 端点：`POST /api/webhooks/vcs/{connectionId}`。该路径由 [server/internal/handler/vcs.go](#) 中的 `vcsWebhookPath` 方法生成，前缀为 `/api/webhooks/vcs/`。

Webhook 注册指南

在 Git 实例的仓库或组织上注册 Webhook 时，各提供商配置差异如下：

Forgejo / Gitea（设置 → Webhook → 添加 Webhook → Forgejo/Gitea）：

- HTTP 方法 `POST`，内容类型 `application/json`
- **密钥**：Webhook 密钥，用于验证 `X-Gitea-Signature` HMAC-SHA256
- 触发事件：勾选 **Pull Request** 和 **Commit Status**

GitLab（设置 → Webhooks）：

- **Secret token**：Webhook 密钥，作为 `X-Gitlab-Token` 头逐字比对
- 触发器：启用 **Merge request events** 和 **Pipeline events**

签名验证与事件路由

`server/internal/handler/vcs_webhook.go` 中的 `HandleVCSWebhook` 处理器完成以下流程：

1. 从路径中解析 `connectionId`，从数据库加载连接和解密后的 Webhook 密钥。
2. 根据连接的 `provider` 字段获取注册的 `Provider` 实现，调用其 `VerifySignature` 校验签名。
3. 通过 HTTP 头判定事件类别（`EventKind`）：`pull_request` → `EventPullRequest`，`status / pipeline` → `EventCIStatus`，其余事件归类为 `EventOther` 并静默确认不处理。
4. 根据事件类别调用 `ParsePullRequest` 或 `ParseCIStatus` 将各提供商的原始负载转换为统一内部结构，再持久化并广播。

提供商差异

各 Git 提供商通过 `vcs.Provider` 接口注册，共享的抽象层定义在 `server/internal/integrations/vcs/vcs.go` 中：

差异点	Forgejo / Gitea	GitLab
Webhook 签名	<code>X-Gitea-Signature</code> ，HMAC-SHA256 的十六进制表示	<code>X-Gitlab-Token</code> ，明文逐字比对
事件头	<code>X-Gitea-Event</code> （也兼容 <code>X-GitHub-Event</code> ）	<code>X-Gitlab-Event</code>
PR 事件名称	<code>pull_request</code>	Merge Request Hook
CI 事件名称	<code>status</code>	Pipeline Hook
终端 PR 动作	<code>closed</code> 、 <code>merged</code>	<code>merge</code> 、 <code>close</code>

Forgejo 和 Gitea 在线上完全一致，因此 `server/internal/integrations/vcs/forgejo.go` 中的 `forgejoProvider` 同时注册为 `KindForgejo` 和 `KindGitea`。GitLab 实现在 `server/internal/integrations/vcs/gitlab.go` 中独立定义。

镜像内容

Pull Request / Merge Request

Webhook 送达的 PR/MR 事件经提供商解析后统一为 `vcs.PullRequestEvent` 结构，包含以下信息：

- 状态：`open`、`closed`、`merged`、`draft`
- 元数据：仓库、编号、标题、正文、HTML 链接、分支、作者
- Diff 统计：新增行数、删除行数、变更文件数（提供商支持时）
- 时间戳：创建、更新、合并、关闭时间

终端事件由 `PullRequestEvent.Terminal()` 方法判定：Forgejo/Gitea 的 `closed` / `merged` 和 GitLab 的 `merge` / `close` 均为终端动作。

Issue 自动关联与自动关闭

PR 标题、正文或分支名中引用 issue 编号（如 `MUL-123`）时，Multica 自动将 PR 关联到对应 issue，出现在 issue 侧栏的 **Pull requests** 区域。

当 PR 合并且满足以下条件时，关联的 issue 被自动移动到「完成」状态：

- PR 正文或合并提交消息中包含关闭关键字（`Closes`、`Fixes`、`Resolves` 后跟 issue 编号）
- 该 issue 没有其他仍处于 `open` 状态的关联 PR

CI / 流水线状态

Forgejo/Gitea 的 Commit Status 和 GitLab 的 Pipeline 事件经统一后汇总为 head commit 的检查状态：

- **通过** (`passed`)：所有上下文均成功
- **失败** (`failed`)：至少一个上下文失败
- **进行中** (`pending`)：有上下文正在执行且无失败

状态聚合结果以检查条形式展示在 PR 卡片上，通过 `aggregateChecksConclusion` 函数计算，包含 `checks_total`、`checks_passed`、`checks_failed`、`checks_pending` 四个统计值。来源：[server/internal/handler/vcs_webhook.go](#) 中 `vcsPullRequestRowToResponse` 展示了该聚合逻辑。

智能体创建 Pull Request

智能体创建 PR 不需要 Multica 侧做任何 Git 提供商配置。智能体在运行时中执行 `git checkout`，使用 daemon 主机自身的 Git 凭据（SSH 部署密钥、Git credential helper 中的令牌等）完成认证和推送。要让智能体在某个代码托管上工作，只需确保 daemon 主机可对其完成认证，然后在仓库地址中正常添加 Git URL 即可。

仓库检出支持任意 Git URL，因此无需提供商专属的前置配置。这与工作区的 VCS 连接是两条独立的路径：VCS 连接负责镜像 PR 和 CI 状态，而智能体直接操作 Git 仓库。

安全设计

- **静态加密**：访问令牌和 Webhook 密钥在数据库中以加密形式存储。加密使用 AES-GCM，密钥来自服务端配置的 `MULTICA_VCS_SECRET_KEY`。加解密方法 `sealVCSecret` 和 `openVCSecret` 定义在 [server/internal/handler/vcs.go](#) 中。
- **一次性展示**：Webhook 密钥创建时以明文返回一次，之后无法通过 API 再次获取。重新连接同一实例会触发令牌和密钥的双重轮换。
- **签名校验**：每次 Webhook 投递均使用存储的密钥校验签名。Forgejo/Gitea 使用 HMAC-SHA256，GitLab 使用常量时间字符串比对。签名不匹配的请求被拒绝。
- **访问控制**：连接列表对工作区所有成员可见，但连接、断开和轮换操作仅限 owner 和 admin 执行。部署级开关（`MULTICA_VCS_INTEGRATION_ENABLED` 为 false）直接屏蔽整个功能，无论角色。

工作区仪表盘

detail · dashboard.md

工作区仪表盘

工作区仪表盘是 Multica 的聚合数据首页，面向工作区成员和管理者，提供对智能体使用量、运行时间、任务吞吐和失败率的全景视图。仪表盘完全只读，不产生任何写操作；其数据来源于服务端预计算的聚合查询，成本在客户端根据模型定价表实时折算。

仪表盘由六个只读 REST 端点驱动，全部定义在 [server/internal/handler/dashboard.go](#)：三个按日期分桶的序列和三个按智能体分桶的序列，各自接受可选的 `?project_id=<uuid>` 将范围缩小到单个项目。

用户目标

仪表盘回答四类核心问题：

- **花了多少钱、用了多少 Token？**——按日和按智能体维度的成本与 Token 消耗趋势。
- **智能体跑了多久、完成了多少任务？**——运行时长和终端任务计数，含成功、失败与取消。
- **失败率是否在恶化？**——按日期和失败原因的终端任务分布，以及按智能体排名的"高频失败者"列表。
- **哪个智能体消耗最大？**——按 Token / 成本 / 时间 / 任务数排序的排行榜。

这些指标在 [packages/views/dashboard/components/dashboard-page.tsx](#) 中被组织为两个标签页："用量" (Usage) 和"错误" (Errors)，每个标签页独立加载其所需的聚合数据，不会因为另一个标签页的查询延迟而阻塞渲染。

入口与导航

仪表盘位于工作区级别的导航路径 `/slug/dashboard`，是工作区首页的默认登录落地视图。用户通过工作区侧边栏或顶部导航进入。

页面顶部提供两个全局筛选器：

- **时间范围选择器**：可选 1 天、7 天、30 天、90 天、180 天。其中 1 天和 7 天仅支持按日粒度展示，180 天仅支持按周粒度，30 天和 90 天同时支持日和周两种粒度。时间范围配置定义在 [packages/views/dashboard/components/dashboard-shared.tsx](#)。1 天的语义为"今天"（以查看者时区的自然日历日 00:00 为界），而非"过去 24 小时"。
- **项目过滤器**：可选择"全部项目"或某个具体项目，将仪表盘所有六个查询的范围限定到该项目。与 [项目管理](#) 中创建的项目相关联。

页面头部还展示数据新鲜度信息：上一次数据更新时间（以查看者时区显示）以及一个手动刷新按钮。刷新按钮会同时触发全部六个查询的重新获取，按钮在任一查询处于加载中状态时显示加载动画。

用量标签页

用量标签页展示智能体的使用量、成本和运行效率。数据来自四个查询端点，对应四个 React Query hook：`dashboardUsageDailyOptions`、`dashboardUsageByAgentOptions`、`dashboardAgentRunTimeOptions`、`dashboardRunTimeDailyOptions`，定义在 [packages/core/dashboard/queries.ts](#)。

KPI 卡片

页面顶部排列四张 KPI 卡片，每张卡片显示当前时间窗口内的汇总数值：

KPI	数据来源	说明
成本 (Cost)	<code>usage/daily</code> 聚合	客户端根据每条 (provider, model) 行的 Token 数和模型定价表计算
Token	<code>usage/daily</code> 聚合	含输入、输出、缓存读取和缓存写入四类 Token 的总和
运行时间 (Run time)	<code>runtime/daily</code> 聚合	仅计入有 <code>started_at</code> 和 <code>completed_at</code> 的终端任务
任务数 (Tasks)	<code>runtime/daily</code> 聚合	终端任务总数 (含成功、失败和取消)

成本计算发生在客户端而非服务端——服务端在 [server/internal/handler/dashboard.go](#) 的注释中明确保留了 model 维度以便客户端使用定价表折算。这使得定价模型更新无需后端部署。

趋势图

趋势图卡片支持四种指标切换：Token、成本 (Cost)、时间 (Time)、任务 (Tasks)。图表粒度可在日 (Daily) 和周 (Weekly) 之间切换 (受时间范围限制)。当时间范围设为 30 天或 90 天时，用户可以在日柱状图和周柱状图之间切换；设为 1 天或 7 天时仅支持日粒度；设为 180 天时仅支持周粒度。

趋势图的数据来源于 `usage/daily` 端点。为了让周粒度图表有足够的柱状数据填充，日期分桶查询的实际获取窗口为 `Math.ceil(days / 7) * 7` 天，即向上取整到完整周数。客户端通过 `dailyCutoffIso` 筛选器将 KPI 卡片和日粒度图表的展示截断到用户选择的天数，该筛选器以查看者时区的当天日期为锚点——这与服务端按 `bucket_hour` 切片使用的时区一致，确保在午夜边界处不会多计或少计一天。

排行榜

排行榜以列表形式展示当前工作区 (或项目) 内智能体的消耗排名。定义在 [packages/views/dashboard/components/leaderboard.tsx](#)。

排行榜的特性：

- **四种排序维度**：Token、成本、时间、任务数。切换排序指标会重新排列列表、更新进度条宽度并高亮当前排序列的表头。
- **进度条**：每个智能体的进度条宽度以排序指标的最大值为基准 (跨所有行计算，不限于可见行)，因此展开/折叠尾部智能体时进度条不会重新缩放。

- **默认展示 10 个智能体**：排行榜默认显示前 10 名，超出部分通过"显示全部"按钮展开。这个限制是为了防止拥有数十个智能体的工作区将整个页面撑出屏幕。
- **已删除智能体的合并桶**：服务端检测到智能体已被删除的行会被合并为一个"已删除智能体"的汇总行，保留其运行时间和任务数，以便整个工作区的 KPI 数据仍然可对账。
- **受限制智能体的合并桶**：当前用户无权查看的智能体行被折叠到一个 `__restricted_agents__` 桶中，保留完整的 (provider, model) 或 failure_reason 拆分，但隐藏真实智能体 ID。该机制由 [server/internal/handler/dashboard.go](#) 中的 `foldRestrictedAgents` 函数实现。

排行榜的排序指标切换仅影响排行榜自身，不会影响 KPI 卡片或趋势图的展示。

错误标签页

错误标签页展示工作区内智能体任务的失败情况。定义在 [packages/views/dashboard/components/errors-tab.tsx](#)。数据来自 `failures/daily` 和 `failures/by-agent` 两个端点。

错误率趋势

错误趋势图展示每日的失败率。关键设计决策：两个失败端点返回**全部**终端任务行（不限于失败的任务），其中 `FailureReason == ""` 的行代表成功任务数。客户端利用成功行作为分母计算错误率，保证分子和分母在完全一致的筛选条件下得出——如果从运行时间端点推导分母，会因为运行时间端点要求 `started_at IS NOT NULL` 而与失败端点产生偏差（例如在队列中过期的任务从未启动，只有失败端点会计入它）。

失败原因分类

服务端写入了 22 种规范的失败原因值，定义在 [server/pkg/taskfailure/failure.go](#)。这 22 种值分为两组：

- **8 个平台侧原因**（无 `agent_error.` 前缀）：`queued_expired`、`runtime_offline`、`runtime_recovery`、`timeout`、`iteration_limit`、`agent_blocked`、`api_invalid_request`、`skill_bundle_unavailable`。这些原因归因于 Multica 基础设施或调度层，而非智能体进程自身的行为。
- **14 个智能体侧原因**（带 `agent_error.` 前缀）：涵盖 LLM 提供商认证失败、配额耗尽、上下文溢出、运行器崩溃等场景。

客户端将这些 22 种原始原因折叠为少数几个展示类别（如超时、提供商错误、配额不足等），使错误分析卡片易于阅读。原始原因字符串保留在线路上传输，因此展示类别的映射可以在不部署后端的情况下调整。

高频失败智能体列表

以列表形式展示失败次数最多的智能体，默认显示前 8 名（`TOP_OFFENDER_LIMIT = 8`）。与排行榜类似，超出部分通过切换按钮展开。列表按绝对失败数排序，包含每个智能体的失败率（失败数 / 终端任务总数）。

数据刷新机制

仪表盘采用定时轮询策略保持数据新鲜度。轮询参数定义在 [packages/core/dashboard/queries.ts](#) :

- **轮询间隔 (REFETCH_INTERVAL)** : 5 分钟。服务端以约 5 分钟的节奏物化聚合数据，更频繁的轮询只会重读未变化的聚合结果。
- **陈旧时间 (STALE_TIME)** : 60 秒。重新进入页面时，超过 60 秒的缓存数据会在挂载时触发重新获取，而不是等待轮询间隔到期。
- 轮询仅在页面挂载且浏览器窗口处于前台时生效；未挂载的查询或后台窗口不会触发请求。

时间范围变更时，使用 `keepPreviousData` 策略保留上一次的数据，使 KPI 卡片和图表在原地过渡，而非回退到全页骨架屏。工作区、项目或时区变更时则明确不使用此策略，以避免短暂展示错误数据。

后端端点详解

六个只读端点

全部端点定义在 [server/internal/handler/dashboard.go](#)。

端点	返回内容	维度
GET /api/dashboard/usage/daily	按日期和模型的 Token 行	(date, model)
GET /api/dashboard/usage/by-agent	按智能体和模型的 Token 行	(agent, model)
GET /api/dashboard/agent-runtime	按智能体的运行时间和任务计数	(agent)
GET /api/dashboard/runtime/daily	按日期的运行时间和任务计数	(date)
GET /api/dashboard/failures/daily	按日期和失败原因的终端任务计数	(date, failure_reason)
GET /api/dashboard/failures/by-agent	按智能体和失败原因的终端任务计数	(agent, failure_reason)

通用查询参数

- `days` : 时间窗口天数，默认 30，上限 365。
- `project_id` : 可选的 UUID，将聚合范围限定到指定项目。省略时覆盖整个工作区。
- `tz` : IANA 时区标识符，用于确定日期分桶的日边界。省略时回退到用户配置的时区，最终回退到 UTC。

时间窗口约定

三个按日期分桶的序列使用 `parseSinceParamInTZ`，返回 $N+1$ 个日历日的数据（多出的 1 天由客户端通过 `-(days-1)` 筛选器裁剪）。三个按智能体分桶的序列使用 `parseExactSinceParamInTZ`，严格返回恰好 N 个日历日的数据——因为这些序列不含日期维度，客户端无法按日期裁剪，必须由服务端精确控制窗口以确保排行榜与 KPI 卡片使用一致的时间跨度。

访问控制

工作区级别的序列（`usage/daily`、`runtime/daily`、`failures/daily`）不含智能体维度，仅需工作区成员身份即可访问——Token 消耗、运行时间和失败数量被视为工作区级别的运营指标。

三个按智能体分桶的序列额外施加按智能体的可见性控制：当前用户无权查看的智能体会被 `foldRestrictedAgents` 合并到一个名为 `__restricted_agents__` 的合成桶中。该桶保留完整的 `failure_reason` 拆分，使客户端可以像对待任何其他智能体一样推导其失败率和类别分布。折叠而非丢弃这些行，确保按智能体的细分始终与旁边的工作区总计可对照。

限制与注意事项

- **完全只读**：仪表盘不提供任何写入或修改操作，所有六个端点均为 HTTP GET。
- **时间窗口上限 365 天**：`days` 参数最大值为 365。
- **聚合延迟**：服务端约每 5 分钟物化一次聚合数据，因此仪表盘显示的数据最多滞后约 5 分钟。
- **成本为客户估算**：成本由客户端根据 Token 数和模型定价表计算，与账单系统可能有微小差异。
- **运行时间仅计终端任务**：只有同时填充了 `started_at` 和 `completed_at` 的终端任务（已完成、失败或取消）才贡献运行时长。排队中和运行中的任务没有有限时长，不计入。
- **失败端点包含成功行**：`failures/daily` 和 `failures/by-agent` 返回全部终端任务行，其中 `failure_reason == ""` 代表成功任务。客户端需要自行过滤以计算失败率。
- **智能体可见性限制**：如果当前用户无权查看某些智能体，这些智能体的数据会被合并到受限桶中，保留聚合值但隐藏身份。

通知中心

detail · notification-center.md

通知中心

通知中心是 Multica 工作区的集中收件箱，按成员身份路由通知，汇聚与你有关的任务动态。它不是完整的活动日志，而是提醒你“哪里发生了需要关注的变化”——完整上下文始终保存在对应的 issue 中。

通知的来源与类型

收件箱中的每条通知都携带一个类型、一个严重级别和一个操作者身份。通知的产生时机如下：

分配相关

- **issue_assigned**：任务分配给你。
- **unassigned**：任务从你名下移走。
- **assignee_changed**：你订阅的任务负责人发生变更。

来源：[server/cmd/server/notification_listeners.go](#) 中的 `registerNotificationListeners` 函数根据事件总线的 `issue:updated` 等事件计算变更字段，匹配到订阅者后通过 `notifySubscribers` 或 `notifyDirect` 写入 `inbox_item` 表。

状态与优先级变更

- **status_changed**：你订阅的任务状态发生变化（如 `todo` → `in_progress` → `in_review` → `done`）。
- **priority_changed**：优先级变更。
- **start_date_changed / due_date_changed**：开始日期或截止日期变更。

这些通知由事件总线监听器在检测到对应字段变化时创建。子任务的状态变更会冒泡通知父任务的订阅者，但子任务的评论、优先级和日期变更不会冒泡。

评论与互动

- **new_comment**：你订阅的任务出现新评论。
- **mentioned**：有人在任务描述或评论中 @你（包括 `@all` 和 `@squad`）。
- **reaction_added**：你创建的任务或评论收到表情回应。
- **review_requested**：请求你审阅。

提及通知通过 `notifyMentionedMembers` 独立处理，不受评论组的静音控制——即使你静音了评论通知，直接 @你仍会单独通知。

智能体活动

- **task_completed**：智能体执行完成。
- **task_failed**：你订阅的任务中智能体执行失败。
- **agent_blocked**：智能体被阻塞。
- **agent_completed**：智能体已完成。

`task_failed` 通知的严重级别为 `action_required`。当任务状态变为 `in_review`、`done` 或 `cancelled` 后，该任务的 `task_failed` 通知会自动归档。

快速创建

- **quick_create_done**：通过智能体成功创建任务。
- **quick_create_failed**：通过智能体创建任务失败。
- **quick_create_unconfirmed**：通过智能体创建的结果未确认。

自动化暂停

当自动化规则因连续失败被暂停时，其订阅者会收到通知。该通知通过 [server/cmd/server/autopilot_failure_monitor.go](#) 发布 `EventInboxNew` 事件。

数据模型

每条通知的核心结构如下，定义于 [server/internal/handler/inbox.go](#)：

字段	类型	说明
<code>id</code>	UUID	通知唯一标识
<code>workspace_id</code>	UUID	所属工作区
<code>recipient_type</code>	string	始终为 <code>"member"</code>
<code>recipient_id</code>	UUID	接收者的用户 ID
<code>type</code>	string	通知类型（如 <code>"mentioned"</code> 、 <code>"status_changed"</code> ）
<code>severity</code>	string	严重级别（ <code>"info"</code> 或 <code>"action_required"</code> ）
<code>issue_id</code>	UUID null	关联任务
<code>title</code>	string	通知标题
<code>body</code>	string null	通知正文
<code>read</code>	bool	是否已读
<code>archived</code>	bool	是否已归档
<code>created_at</code>	timestamp	创建时间

字段	类型	说明
actor_type	string null	操作者类型 ("member" 或 "agent")
actor_id	UUID null	操作者 ID
details	JSON	结构化详情

自动订阅

以下参与者会自动订阅任务的通知：

- 任务的创建者。
- 被分配为负责人的成员。
- 发表过评论的成员。
- 在任务描述中被新增 @提及的成员。
- 自动化规则中预先配置的订阅成员。

负责人变更不会取消原有订阅。你可以在任务页面随时订阅或取消订阅。取消订阅后，普通更新不再进入收件箱，但直接 @你仍会单独通知。

来源：apps/docs/content/docs/inbox.zh.mdx

通知偏好设置

在 **设置 → 通知** 中，可以按六组开关控制收件箱通知：

偏好组	键名	涵盖的通知类型
分配	assignments	issue_assigned 、 unassigned 、 assignee_changed
状态变更	status_changes	status_changed
评论	comments	new_comment 、 review_requested
提及	mentions	mentioned
优先级与截止日期	updates	priority_changed 、 start_date_changed 、 due_date_changed
智能体活动	agent_activity	task_completed 、 task_failed 、 agent_blocked 、 agent_completed

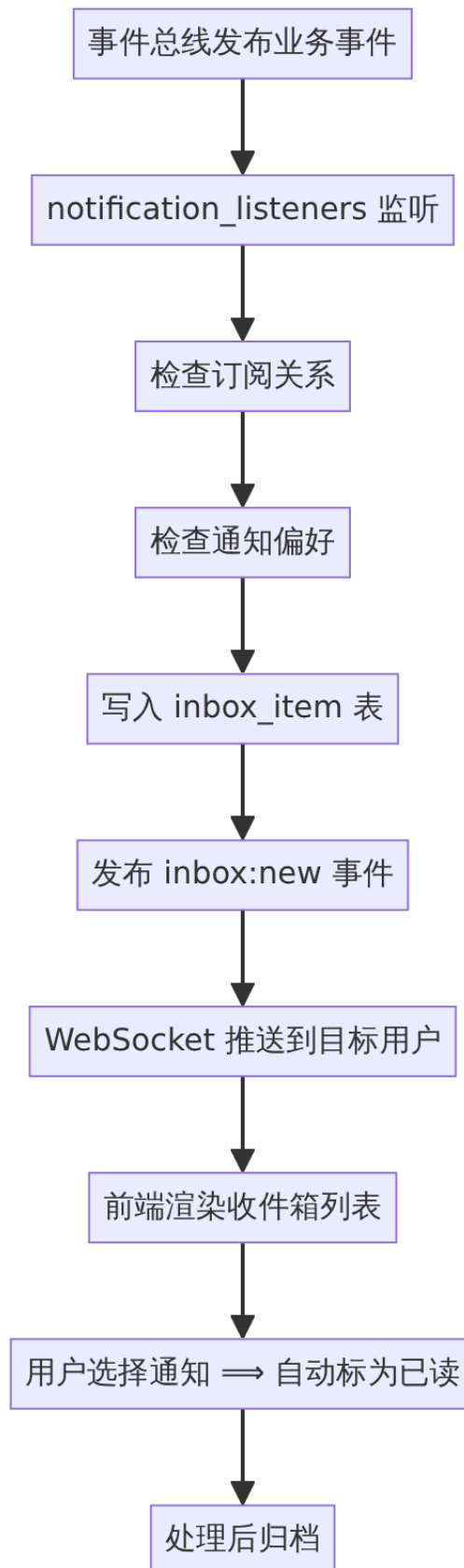
每组可设为 `all`（接收）或 `muted`（静音）。关闭某一组后，相关变化仍记录在任务中，只是不再创建收件箱通知。表情回应、快速创建结果和自动化暂停等类型不在六组控制范围内，始终投递。

偏好通过 REST API 管理，定义在 [server/internal/handler/notification_preference.go](#)：

- GET /api/notification-preferences — 获取当前偏好。
- PUT /api/notification-preferences — 全量替换（兼容旧客户端）。
- PATCH /api/notification-preferences — 原子合并指定键，防止多设备覆盖。

静音判断逻辑在 [server/cmd/server/notification_listeners.go](#) 中的 `isNotifMuted` 函数实现，将通知类型映射到偏好组后检查是否为 `"muted"`；不在映射表中的类型（如 `reaction_added`、`quick_create_done`）始终投递。

通知的生命周期



通知由服务端事件总线驱动。各类业务事件（任务创建/更新、评论、反应等）触发 `registerNotificationListeners` 中注册的监听器，这些监听器通过 `notifyDirect` 或

`notifySubscribers` 将通知写入数据库，并发布 `inbox:new` 事件。

实时推送层将 `inbox:new` 等个人事件通过 `WebSocket` 仅投递给目标用户（而非广播到整个工作区），定义在 [server/cmd/server/listeners.go](https://github.com/nextcloud/server/blob/master/cmd/server/listeners.go)。事件类型包括：

事件	说明
<code>inbox:new</code>	新通知到达
<code>inbox:read</code>	单条已读
<code>inbox:unread</code>	单条标为未读
<code>inbox:archived</code>	单条归档
<code>inbox:unarchived</code>	单条取消归档
<code>inbox:batch-read</code>	批量已读
<code>inbox:batch-archived</code>	批量归档

操作与交互

列表查询

- `GET /api/inbox` — 获取当前成员的未归档通知列表。
- `GET /api/inbox/archived` — 获取已归档通知（独立端点，仅返回活跃收件箱中不存在对应通知的 `issue` 分组，上限 200 条）。
- `GET /api/inbox/unread-count` — 获取当前工作区未读计数。
- `GET /api/inbox/unread-summary` — 获取跨工作区未读摘要，用于侧边栏的工作区切换器标记。

单条操作

- `POST /api/inbox/{id}/read` — 标为已读。选择通知时会自动触发。
- `POST /api/inbox/{id}/unread` — 标为未读。用于"看过了但还没处理完"的场景。通知重新变为未读，未读计数回升；即使通知详情仍打开，状态也保持未读，直到再次选择。
- `POST /api/inbox/{id}/archive` — 归档。归档范围为 `issue` 级别：同一条 `issue` 的所有通知同时归档。归档不修改或删除对应任务。
- `POST /api/inbox/{id}/unarchive` — 取消归档。同样为 `issue` 级别：恢复整组通知。已读状态在归档时被保留，恢复时原样返回。

批量操作

- `POST /api/inbox/mark-all-read` — 全部标为已读。
- `POST /api/inbox/archive-all` — 归档全部通知。
- `POST /api/inbox/archive-all-read` — 归档全部已读通知。

- **POST /api/inbox/archive-completed** — 归档状态为 `done` 或 `cancelled` 的任务的通知。

以上所有操作均通过 [server/internal/handler/inbox.go](#) 中的 `Handler` 方法实现，对应路由注册于 [server/cmd/server/router.go](#)。

前端交互

前端为收件箱提供完整的交互支持：

- **列表视图**：显示通知类型图标、标题、时间（"刚刚"、"X 分钟"、"X 小时"、"X 天"）。空状态显示"暂无通知"。
- **已归档视图**：独立子视图，显示已归档通知，空状态显示"暂无已归档通知"。
- **右键菜单**：支持标为已读/未读、归档/取消归档。
- **收件箱菜单**：提供全部标为已读、归档全部、归档全部已读、归档已完成。
- **详情面板**：显示通知正文和原始输入，提供"在完整表单中编辑"的入口。

来源：[packages/views/locales/zh-Hans/inbox.json](#)

智能体不使用收件箱

收件箱只面向工作区成员。即使智能体是负责人或订阅者，它也不会读取收件箱通知。在评论中 `@智能体`、把任务分配给智能体或触发自动化，都会直接创建一次执行，而不是给智能体留一条通知。`@all` 也只通知成员。

来源：[apps/docs/content/docs/inbox.zh.md](#)

企业 IM 集成

当工作区绑定了企业微信时，收件箱通知会通过智能机器人以 Markdown 卡片形式推送给用户。推送格式为：

```

**[{类型}] {标题}**
{正文}
[查看详情]({应用URL}/.../inbox?issue={issueID})

```

实现于 [server/internal/integrations/wecom/inbox_message.go](#) 和 [server/internal/integrations/wecom/outbound.go](#)。应用 URL 解析优先级为 `WECOM_APP_URL` → `MULTICA_APP_URL` → `FRONTEND_ORIGIN`，仅接受 HTTPS 值。

与相关功能的关系

- **任务与问题管理**：通知始终关联一个 `issue`。选择通知可跳转到对应任务详情页，完整上下文保留在任务中。任务状态进入 `in_review`、`done` 或 `cancelled` 后，其执行失败通知自动归档。

- **实时通信基础设施**：所有收件箱事件通过 WebSocket 按用户精确推送，不广播到工作区。事件总线为同步发布模型，在 HTTP 请求 goroutine 内直接分发。

自定义属性

detail · custom-properties.md

自定义属性

自定义属性是工作区级别的类型化字段系统，允许团队为任务定义结构化的元数据字段。与系统内置字段（状态、优先级、负责人等）不同，自定义属性完全由工作区管理员按需创建，并可绑定到每个任务上。智能体（Agent）可以读写属性值，并在类型校验的约束下进行自我纠错。

功能概览

自定义属性的核心使用场景包括：

- 任务元数据建模**：为任务补充业务特有字段，如 `Severity`（严重等级）、`Environment`（环境）、`Customer`（客户）等。
- 智能体数据交换**：Agent 可以通过属性读写结构化数据，属性值的类型校验确保 Agent 输出永远是合法值，Agent 消费校验错误信息即可自我纠正。
- 表格视图列与分组**：属性可作为任务列表的独立列展示，也可用作分组维度，让团队按业务字段聚合任务。

属性类型

每个自定义属性必须指定一种基础类型，创建后不可更改。支持七种类型：

类型标识	中文名称	值格式	存储形式
<code>text</code>	文本	任意字符串（最长 2000 字符）	JSON string
<code>number</code>	数字	数值	JSON number
<code>select</code>	单选	选项 ID	JSON string（选项 ID）
<code>multi_select</code>	多选	选项 ID 数组	JSON string[]
<code>date</code>	日期	<code>YYYY-MM-DD</code> 格式	JSON string
<code>checkbox</code>	勾选	<code>true / false</code>	JSON boolean
<code>url</code>	链接	URL 字符串（最长 2048 字符）	JSON string

类型定义位于 `packages/core/types/property.ts` 中，`select` 和 `multi_select` 需要预定义选项列表（Option），每个选项包含 `id`、`name`、`color`。

属性生命周期

创建属性

工作区 Owner 或 Admin 可以在"设置 → 属性"页面创建新属性。创建时需要提供：

- **名称**：最长 32 字符，不可使用系统保留名称（如 `status`、`priority`、`assignee`、`due_date` 等）。比较时忽略大小写和空格，`Due Date` 与 `due_date` 同被视为冲突。
- **类型**：七选一，选定后不可修改。
- **图标**（可选）：从预定义图标目录中选择，用于在列表中快速区分属性。
- **描述**（可选）：最长 500 字符，解释属性用途。
- **选项**（仅 `select` / `multi_select`）：至少一个选项，每个选项包含名称和颜色，最多 50 个选项。

每个工作区最多启用 20 个属性，到达上限后"新建属性"按钮被禁用。该上限在前端和后端双重校验，如 [packages/views/settings/components/properties-tab.tsx](#) 和 [server/internal/handler/property.go](#) 中均定义了 `MAX_ACTIVE_PROPERTIES = 20` 常量。

编辑与归档

管理员可以编辑属性的名称、描述、图标和选项。类型不可修改。

归档是一种软删除：归档后属性从新建任务的选择器和筛选器中隐藏，已有值保留并继续展示为只读。归档属性拒绝新值写入。可以随时从归档状态恢复。

属性的操作菜单提供"编辑"和"归档/恢复"两个动作。

国际化字符串位于 [packages/views/locales/zh-Hans/settings.json](#)，覆盖完整的创建、编辑、归档对话框文案。

为任务设置属性值

设置与取消

属性值绑定到每个任务上，通过 `POST /api/issues/{id}/properties/{propertyId}` 设置，通过 `DELETE /api/issues/{id}/properties/{propertyId}` 取消。所有成员和智能体均可写值。

值必须通过后端校验：

- **类型匹配**：`text` 必须是字符串，`number` 必须是数字，`checkbox` 必须是布尔值，等等。
- **非空约束**：空字符串和 `null` 被拒绝，提示使用 `DELETE` 取消属性。
- **长度限制**：`text` 最长 2000 字符，URL 最长 2048 字符。
- **选项有效性**：`select` / `multi_select` 的值必须是定义的选项 ID。
- **日期格式**：`date` 必须是有效的 `YYYY-MM-DD`。

如果校验失败，API 返回具体错误原因并枚举合法值，智能体可以直接读取错误信息进行自我纠正。校验逻辑见 [server/internal/handler/property.go](#) 中的 `validatePropertyValue` 函数。

前端 picker

在任务详情和表格视图中，自定义属性通过专用 picker 组件展示。不同类型的 picker 行为如下：

- **文本**：单行文本输入框，placeholder 为"输入值..."。
- **数字**：数字输入框，placeholder 为 0 。
- **单选**：下拉菜单，展示所有选项。
- **多选**：多选下拉，展示所有选项。
- **日期**：日期选择器。
- **勾选**：二元切换 "是/否"。
- **链接**：URL 输入框，placeholder 为 https://... ，已设置值时提供"打开链接"按钮。

已归档属性的 picker 显示为只读，并带有归档提示。国际化字符串位于 [packages/views/locales/zh-Hans/issues.json](#) 的 pickers.custom_property 节点。

React Query 集成

前端通过 @multica/core/properties 包导出 React Query 钩子，对属性定义和属性值操作进行了完整的缓存管理。

查询钩子

propertyListOptions(wsId, includeArchived) 返回 queryOptions 配置，按工作区缓存属性列表。查询键为 ["properties", wsId, "list", includeArchived] ，支持包含/排除已归档属性。

来源：[packages/core/properties/queries.ts](#)

变更钩子

四个变更钩子均基于 useMutation：

钩子	对应 API	用途
useCreateProperty()	POST /api/properties	创建新的属性定义
useUpdateProperty()	PATCH /api/properties/{id}	更新属性定义或归档/恢复
useSetIssueProperty()	POST /api/issues/{id}/properties/{propertyId}	为任务设置属性值
useUnsetIssueProperty()	DELETE /api/issues/{id}/properties/{propertyId}	取消任务的某个属性值

useSetIssueProperty 采用乐观更新策略：先乐观合并单个属性值到缓存的任务属性集合中，失败时回滚快照。同一任务上的多个写操作通过 TanStack Query 的 scope 机制串行化，避免并发写入导致的交错覆

盖。最后一次 settled mutation 会做一次权威的 invalidate ，修复因 WebSocket 推送产生的缓存差异。

来源：[packages/core/properties/mutations.ts](#)

入口文件 [packages/core/properties/index.ts](#) 统一导出所有公开 API。

服务器 API

所有属性 API 位于 `/api/properties` 路径下，处理函数位于 [server/internal/handler/property.go](#)。

属性定义 API

方法	路径	处理函数	说明
GET	<code>/api/properties</code>	ListProperties	列出工作区所有属性， <code>?include_archived=true</code> 可包含已归档
POST	<code>/api/properties</code>	CreateProperty	创建属性（需 owner/admin）
PATCH	<code>/api/properties/{id}</code>	UpdateProperty	更新属性定义或归档（需 owner/admin）
GET	<code>/api/properties/{id}</code>	GetProperty	获取单个属性详情

属性值 API

方法	路径	处理函数	说明
POST	<code>/api/issues/{id}/properties/{propertyId}</code>	SetIssueProperty	为任务设置属性值
DELETE	<code>/api/issues/{id}/properties/{propertyId}</code>	UnsetIssueProperty	取消任务的属性值

权限模型

- 属性定义管理：仅工作区 Owner 和 Admin。智能体即使其运行时拥有者具有管理员角色，也不能创建或编辑属性定义——这是有意设计，防止 Agent 批量创建属性导致字段膨胀。
- 属性值读写：所有成员和智能体均可为任务设置和取消属性值。

并发控制

属性定义更新和属性值写入共享同一个 PostgreSQL advisory lock (`prop:{id}`) ，确保并发场景下值校验和定义更新之间不存在 TOCTOU 问题。

实时推送

属性定义变更（创建、更新、归档）通过 WebSocket 以 `property.updated` 事件推送到工作区所有在线客户端。

前端管理界面

属性管理界面位于工作区"设置 → 属性"标签页，由 `PropertiesTab` 组件实现。

界面结构：

- **工具栏**：搜索框（按名称筛选）、"显示已归档"开关、启用中属性计数 `{count}/20`、新建属性按钮。
- **权限提示**：非管理员看到"仅工作区 Owner 和 Admin 可以管理属性定义"提示文案。
- **属性列表**：表格形式展示，包含以下列：
 - **名称**：属性名（含图标，已归档属性带"已归档"徽章）
 - **类型**：中文类型名称
 - **选项**：仅 `select/multi_select` 显示选项列表
 - **使用数量**：已设置该属性的任务数（`{{count}}` 个任务）
 - **更新时间**：最后更新时间
 - **操作**：编辑、归档/恢复下拉菜单
- **空状态**：无属性时展示引导文案。

来源：<packages/views/settings/components/properties-tab.tsx>

编辑器对话框

创建和编辑属性均使用同一套编辑器表单，在对话框内完成：

- **图标选择器**：从预定义图标目录中选择，支持移除。
- **名称输入**：文本输入，placeholder 为"例如 Severity"。
- **类型选择**：下拉菜单，创建时可选（编辑时锁定并提示"创建后类型不可修改"）。
- **描述输入**：文本域，placeholder 为"这个属性的含义以及何时设置"。
- **选项管理**（仅 `select/multi_select` 类型）：可添加/删除选项，每个选项含名称和颜色。

e2e 测试文件 <e2e/property-icons.spec.ts> 覆盖了图标的创建、持久化和清除流程。

与任务管理的集成

自定义属性深度集成到任务管理功能中，体现在以下几个层面：

表格视图列

在表格视图中，自定义属性可作为独立列添加。列的"添加列"面板中将属性分为两个区域：**任务属性**（系统字段）和**自定义属性**（工作区定义的字段）。用户可以显示、隐藏和重新排序列。

分组

属性支持按自定义属性分组任务。分组值基于属性类型展示：`select` 按选项名称分组，`multi_select` 按选项组合分组，其他类型按原始值分组。如果用于分组的属性已被归档或删除，分组自动清除并显示提示："用于分组的属性已不可用，分组已清除。"

筛选

任务筛选器支持按自定义属性值过滤。属性筛选通过 `issue_table_facets.go` 中的 `facet` 机制实现，如 [server/internal/handler/issue_table_facets.go](#) 所示。

计算

表格视图支持对数值类型的属性列进行表尾计算，可显示计数、总和或平均值。

约束与限制

约束项	值	说明
每个工作区最多启用属性数	20	前后端双重校验
属性名称最大长度	32 字符	UTF-8 字符计数
属性描述最大长度	500 字符	
属性图标最大长度	32 字符	
<code>select/multi_select</code> 最多选项数	50	
<code>text</code> 值最大长度	2000 字符	
<code>url</code> 值最大长度	2048 字符	
图标来源	预定义目录 (约 35 个 Lucide 图标键)	不允许自定义文本或 emoji
保留名称	<code>status</code> , <code>priority</code> , <code>assignee</code> , <code>project</code> , <code>parent</code> , <code>stage</code> , <code>label</code> , <code>labels</code> , <code>start_date</code> , <code>due_date</code> , <code>title</code> , <code>description</code> , <code>creator</code> , <code>created_at</code> , <code>updated_at</code> , <code>metadata</code> , <code>properties</code>	大小写和空格不敏感
属性定义管理权限	Owner / Admin	智能体不可管理
属性值写入权限	所有成员和智能体	
类型修改	创建后不可修改	
删除行为	不删除，仅归档	归档后已有值保留为只读

项目管理

detail · project-management.md

项目管理

项目是 Multica 中组织多条 issue 协同推进的容器。一次产品发布、一次系统迁移、一个分阶段推进的功能——当一项工作涉及的 issue 数量超出单个 issue 能承载的范围，且它们共享目标、资源或进度跟踪需求时，就适合用项目来归类和管理。

项目为智能体提供了共享的执行上下文：项目名称和描述会注入到该项目下每条 issue 的执行中，关联的资源（代码仓库和本地目录）则告诉智能体去哪里工作，无需在每条 issue 中重复配置。

项目的构成

每个项目由以下要素组成：

要素	说明
名称、图标和描述	描述项目的目标、范围和技术边界。描述会作为上下文提供给项目内执行任务的智能体。
状态	planned（计划中）、in_progress（进行中）、paused（已暂停）、completed（已完成）或 cancelled（已取消）。
优先级	urgent（紧急）、high（高）、medium（中）、low（低）或 none（无优先级）。
负责人（lead）	工作区成员或智能体，表示谁负责协调该项目。
开始和截止日期	项目计划的时间范围，日期格式为 YYYY-MM-DD。
Issue 和进度	项目中的工作项及其完成比例。
资源	关联的 GitHub 仓库或本地目录，为智能体执行提供代码和工作目录。

项目状态和优先级的具体取值由服务端约束校验。[server/internal/handler/project.go](#) 中定义了 `validProjectStatuses` 和 `validProjectPriorities`，创建或更新时传入非法值将返回 400 错误及合法取值列表。

项目与 issue 的关系

一个项目可以包含多条 issue，但一条 issue 最多只能属于一个项目。可以在项目内直接创建 issue，也可以在 issue 的属性面板中选择或更换所属项目。

项目进度按关联的 issue 自动计算：

$$\text{进度} = (\text{状态为 done 或 cancelled 的 issue 数}) \div (\text{项目中的全部 issue 数})$$

cancelled 表示该项工作已从项目范围中结束，因此计入进度。项目状态与 issue 状态彼此独立：

- 所有 issue 都完成后，项目不会自动变为 completed。
- 修改项目状态不会批量修改其中 issue 的状态。

来源：apps/docs/content/docs/projects.zh.md#source

项目描述与智能体执行上下文

智能体执行项目内的 issue 时，Multica 会将项目名称和描述注入执行上下文。在服务端，任务领取处理程序 (claim handler) 在 `server/internal/handler/daemon.go` 中将 `proj.Description` 写入领取响应的 `ProjectDescription` 字段；守护进程将其携带至执行环境的 `## Project Context` 节和 `.multica/project/resources.json` 中。

描述适合保存所有相关任务都需要的信息：总体目标、技术边界、交付约定。只与单条 issue 相关的要求应写在该 issue 自身的描述中，而非项目描述里。

项目资源

项目资源告诉智能体使用哪些代码和工作目录。两类资源均持续关联在项目上，不必在每条 issue 中重复配置：

资源类型	适用场景	执行位置
GitHub 仓库 (github_repo)	团队共享的代码库，由运行时管理 checkout	运行时管理的 worktree 工作目录
本地目录 (local_directory)	已存在的 checkout、大型仓库，或需直接查看本地改动	指定电脑上的原目录

本地目录仅能通过桌面端添加（浏览器无法选择本地文件夹），且每个项目在同一台守护进程上最多关联一个本地目录。使用本地目录时，同一真实路径上的任务串行执行，后来的任务进入 `waiting_local_directory` 状态等待。以下路径会被拒绝：系统根目录（ `/` 、 `C:\` ）、用户主目录及其上级目录（如 `/Users` 、 `/home` ），以及 `/etc` 、 `/var` 、 `/tmp` 、 `/usr` 、 `/opt` 等系统目录。

本地目录是逃生舱，不是默认选项。 如果你的目录只是一个能正常 clone 的 Git 仓库，请使用 GitHub 仓库资源——它默认走 `worktree` 模式，同一仓库上的任务可以并发执行。选择本地目录意味着该目录上的任务只能串行运行。

来源：apps/docs/content/docs/project-resources.zh.md#source

项目负责人 (Lead)

Lead 可以是工作区成员或智能体，表示谁负责协调项目推进。需要注意：

- Lead **不是**权限设置，不会自动分配项目中的 issue。
- 将智能体设为 lead **不会**自动触发其运行。启动执行仍需分配 issue、@提及或自动化规则触发。
- Lead 不影响 issue 的执行范围——智能体的 Access 仍然由工作区配置决定。

创建、编辑和删除

所有工作区成员都可以创建和编辑项目；只有 `owner` 和 `admin` 角色可以删除项目。

删除项目时，其中的 issue 不会被删除——它们解除与项目的关联后继续保留在工作区。项目的描述、状态和资源会被永久删除。

创建项目时，可以同时指定初始资源列表。服务端在 [server/internal/handler/project.go](#) 中将项目和资源放入同一事务，保证原子性——要么全部创建成功，要么全部回滚。

项目列表与视图

项目列表页面支持以下交互方式：

- **视图切换**：表格视图和卡片视图两种展示模式。
- **筛选**：按状态（`planned`、`in_progress`、`paused`、`completed`、`cancelled`）和优先级（`urgent`、`high`、`medium`、`low`、`none`）筛选，也支持按负责人筛选。
- **排序**：支持升序/降序排列。
- **搜索**：支持按名称搜索项目。
- **列控制**：可自定义表格视图显示的列（名称、优先级、状态、进度、负责人、创建时间、任务数）。

来源：[packages/views/locales/zh-Hans/projects.json](#)

固定到侧边栏

可以将常用项目固定到侧边栏以便快速访问。固定是个人偏好设置，不会影响工作区中的其他成员。

在评论中引用项目

项目没有 `MUL-123` 式的短标识符，因此在评论中直接写项目标题不会产生可点击的链接。正确的引用方式是使用 mention 链接格式：

```
[项目显示名](mention://project/<project-id>)
```

Web 和桌面端会将此链接渲染为带有项目图标和名称的 Chip，移动端则渲染为普通链接。与 `@agent` / `@squad` 不同，项目 mention 是纯链接，不触发任何入队或通知——`util.MentionRe` 甚至不包含 `project` 类型。

来源：[server/internal/service/builtin_skills/multica-projects-and-resources/SKILL.md](#)

仪表盘中的项目级聚合

工作区仪表盘的六个只读端点均支持可选的 `project_id` 查询参数。传入项目 ID 后，数据将限定在该项目范围内聚合：

端点	聚合内容
GET <code>/api/dashboard/usage/daily</code>	按天统计 token 用量与费用，限定项目内的 issue 关联任务
GET <code>/api/dashboard/usage/by-agent</code>	按智能体统计 token 用量与费用
GET <code>/api/dashboard/agent-runtime</code>	按智能体统计任务运行时长、成功/失败/取消任务数
GET <code>/api/dashboard/failures/daily</code>	按天统计失败原因分布
GET <code>/api/dashboard/failures/by-agent</code>	按智能体和失败原因统计任务数

项目范围过滤通过 SQL 中的 `($N::uuid IS NULL OR i.project_id = $N)` 模式实现：当 `project_id` 参数为空时返回工作区全部数据。来源：[server/pkg/db/generated/task_usage.sql.go](#)

REST API

项目管理通过以下 REST 端点操作：

方法	路径	说明
GET	<code>/api/projects?workspace_id=<id></code>	列出工作区所有项目，支持 <code>status</code> 和 <code>priority</code> 查询参数过滤
GET	<code>/api/projects/{id}?workspace_id=<id></code>	获取单个项目详情
POST	<code>/api/projects?workspace_id=<id></code>	创建项目，请求体可包含初始资源列表
PATCH	<code>/api/projects/{id}?workspace_id=<id></code>	更新项目属性，所有字段均为可选（PATCH 语义）
DELETE	<code>/api/projects/{id}?workspace_id=<id></code>	删除项目（需 owner 或 admin 角色）
GET	<code>/api/projects/{id}/resources?workspace_id=<id></code>	列出项目的所有资源
POST	<code>/api/projects/{id}/resources?workspace_id=<id></code>	向项目添加资源
PATCH	<code>/api/projects/{id}/resources/{resource_id}?workspace_id=<id></code>	更新项目资源
DELETE	<code>/api/projects/{id}/resources/{resource_id}?workspace_id=<id></code>	移除项目资源

列表端点在返回项目元数据时批量查询 issue 统计（总量和已完成数）以及资源数量，避免 N+1 查询。创建项目时若附带资源列表，整个操作在同一数据库事务中完成，保证原子性。

来源：[server/internal/handler/project.go](#) 和 [server/internal/handler/project_resource.go](#)

CLI 操作

通过 Multica CLI 可以执行完整的项目和资源管理：

```
# 列出和查看项目
multica project list --output json
multica project get <project-id> --output json

# 创建项目
multica project create --title "Agent UX" --repo https://github.com/multica-ai/multica --output json
multica project create --title "Q1 Migration" --start-date 2026-03-01 --due-date 2026-03-31 --output json

# 更新项目
multica project update <project-id> --title "新标题" --output json
multica project update <project-id> --due-date 2026-04-15 --output json
multica project update <project-id> --start-date "" --output json # 清除开始日期

# 修改项目状态
multica project status <project-id> in_progress --output json

# 管理资源
multica project resource list <project-id> --output json
multica project resource add <project-id> --type github_repo --url <github-url> --output json
multica project resource add <project-id> --type github_repo --url <github-url> --ref main --output json
multica project resource add <project-id> --type local_directory --local-path /absolute/path --daemon-id <daemon-id> --output json
multica project resource update <project-id> <resource-id> --ref <branch-or-sha> --output json
multica project resource remove <project-id> <resource-id> --output json
```

--ref 指定 GitHub 仓库的默认 checkout 分支、tag 或 commit。日期参数使用 YYYY-MM-DD 格式，与 issue 日期一致。更新时传空字符串可清除已有日期。

来源：[server/internal/service/builtin_skills/multica-projects-and-resources/SKILL.md](#)

实时事件

项目变更通过 WebSocket 实时推送给工作区成员：

- `EventProjectCreated`：项目创建时广播。
- `EventProjectUpdated`：项目更新时广播。
- `EventProjectDeleted`：项目删除时广播。
- `EventProjectResourceCreated` / `EventProjectResourceUpdated` / `EventProjectResourceRemoved`：项目资源变更时广播。

事件发布逻辑位于 `server/internal/handler/project.go` 的 `h.publish()` 调用中，通过 `protocol` 包定义的事件类型传播。

限制与边界

- 一条 issue 最多属于一个项目。需要跨项目追踪的工作应考虑使用标签或自定义属性辅助分类。
- 项目删除会永久移除项目的描述、状态和资源，但不会删除其中的 issue——它们只是解除关联。
- 本地目录资源仅在最初注册它的电脑上可用，且同一守护进程对同一项目只能关联一个本地目录。
- 使用本地目录时，智能体会直接修改当前分支和未提交的文件。Multica 不会自动切换分支、stash、commit、push 或创建 PR。
- 项目进度仅按 issue 状态统计，不反映子任务或执行任务的内部进展。

与其他功能的关系

- **任务与问题管理**：issue 是项目中的基本工作单元。项目进度由关联 issue 的状态驱动，项目和 issue 的状态彼此独立。
- **工作区仪表盘**：仪表盘的六个只读端点均支持 `project_id` 过滤，可查看项目级别的用量、运行时统计和错误分布。
- **版本控制集成**：项目资源中的 GitHub 仓库依赖于工作区级别的 Git 集成配置，包括自托管 Forgejo、Gitea 或 GitLab。

Web 应用

Multica Web 应用是基于 Next.js App Router 构建的浏览器客户端，运行于 Turborepo 管理的 pnpm monorepo 中。它为最终用户提供工作区路由、认证守卫、权限控制、实时更新以及多语言支持。Web 应用与 [桌面应用](#) 共享 `packages/core`、`packages/ui` 和 `packages/views` 三层代码，仅在路由、Cookie 和平台适配器等层面保持独立。

架构定位

Web 应用（`apps/web/`）是 Multica monorepo 中面向浏览器的交付物，依赖三个共享包：

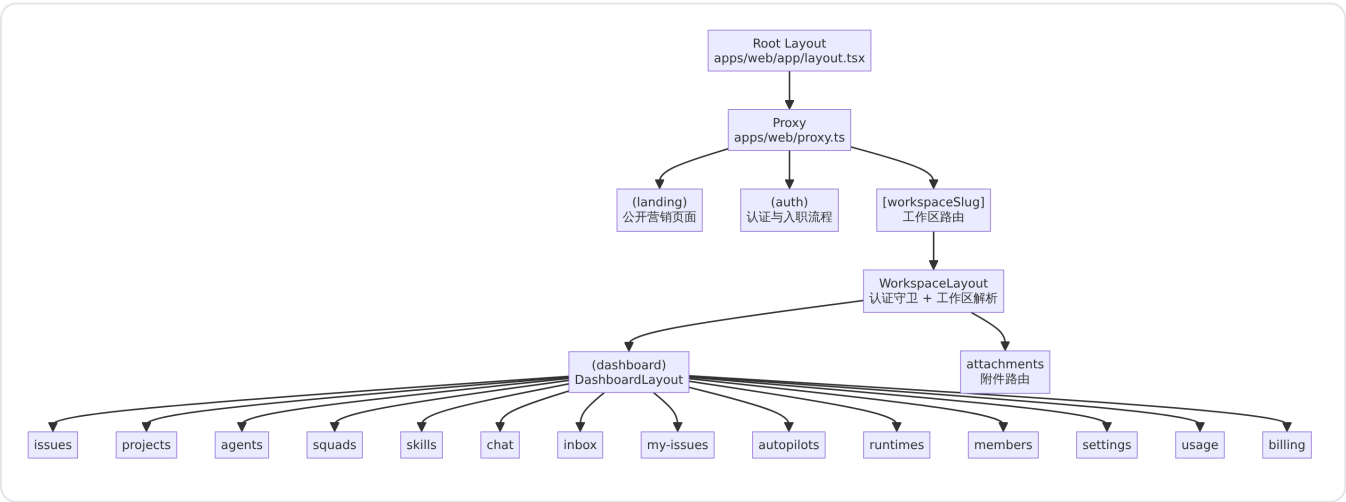
共享包	职责
<code>packages/core</code>	API 客户端、TanStack Query 查询与 mutation、Zustand 状态、权限逻辑、WebSocket 订阅
<code>packages/ui</code>	不含业务逻辑的基础 UI 组件（shadcn、Base UI）
<code>packages/views</code>	Web 与 Desktop 共用的业务页面和组件

依赖方向为 `views → core + ui`，`core` 与 `ui` 互不依赖。共享包直接导出 `.ts` 和 `.tsx` 源文件，由消费应用编译 [apps/docs/content/docs/developers/architecture.mdx](#)。

Web 应用自身的平台层位于 `apps/web/platform/`、`apps/web/components/`、`apps/web/config/` 和 `apps/web/lib/`，负责 Next.js 特有的路由适配、Cookie 管理、运行时 URL 解析等能力。

路由结构

Web 应用使用 Next.js App Router 的文件系统路由，按功能划分为三个路由组：



路由组 (auth) 包含 login、onboarding、invitations、invite、workspaces 页面；(landing) 包含首页、about、changelog、download、contact-sales、usecases 等公开营销页面。

请求代理 (Proxy)

Next.js 16 中 middleware 已重命名为 proxy，API 保持一致。apps/web/proxy.ts 在每个请求到达路由处理器之前执行以下逻辑 apps/web/proxy.ts：

- 1. 运行时 URL 重写：将 /api/*、/auth/*、/uploads/*、/ws、/docs/* 请求代理到后端或文档服务，避免将上游地址编译进构建产物 apps/web/config/runtime-urls.ts。
- 2. 旧版 URL 重定向：将不带 [workspaceSlug] 前缀的旧版路由（如 /issues/...、/projects/...）重定向到新的 /、{slug}、/... 格式，利用 last_workspace_slug Cookie 恢复工作区上下文 apps/web/proxy.ts。
- 3. 根路径重定向：对于已登录的非营销主机访问，将 / 重定向到 /{lastSlug}/issues。
- 4. 语言环境转发：从 Cookie 和 Accept-Language 头解析语言环境，通过 x-multica-locale 请求头传递给 React Server Components apps/web/proxy.ts。

代理的匹配器覆盖所有页面请求以及明确的运行时代理路由 apps/web/proxy.ts。

根布局

apps/web/app/layout.tsx 是 Next.js 的根 Server Component，负责 apps/web/app/layout.tsx：

- 字体加载：Inter (UI)、Geist Mono (代码)、Source Serif 4 (编辑排版)，通过 next/font 以 CSS 变量注入，其中 CJK 回退链在 globals.css 按 <html lang> 覆盖，确保中文使用日文优先的字体栈 apps/web/app/layout.tsx。
- 元数据：设置 SEO 标题模板 (%s | Multica)、描述、Open Graph 和 Twitter Card 标签。
- 语言检测：通过 getRequestLocale() 从代理转发的 x-multica-locale 头或 Cookie/Accept-Language 解析语言环境，映射为 BCP-47 的 lang 属性（如 zh-Hans → zh-CN） apps/web/lib/request-locale.ts。
- 运行时 URL 解析：从环境变量解析 apiBaseUrl 和 wsUrl，传递给 WebProviders。
- WebProviders 包裹：客户端组件，封装 CoreProvider 及平台适配器。

平台提供层 (WebProviders)

apps/web/components/web-providers.tsx 是 Web 应用的关键胶水层，将 @multica/core 的 CoreProvider 与 Web 平台特有的能力对接 apps/web/components/web-providers.tsx：

注入能力	说明
NavigationAdapter	Next.js useRouter 的适配器，使共享的 packages/views 页面无需直接导入

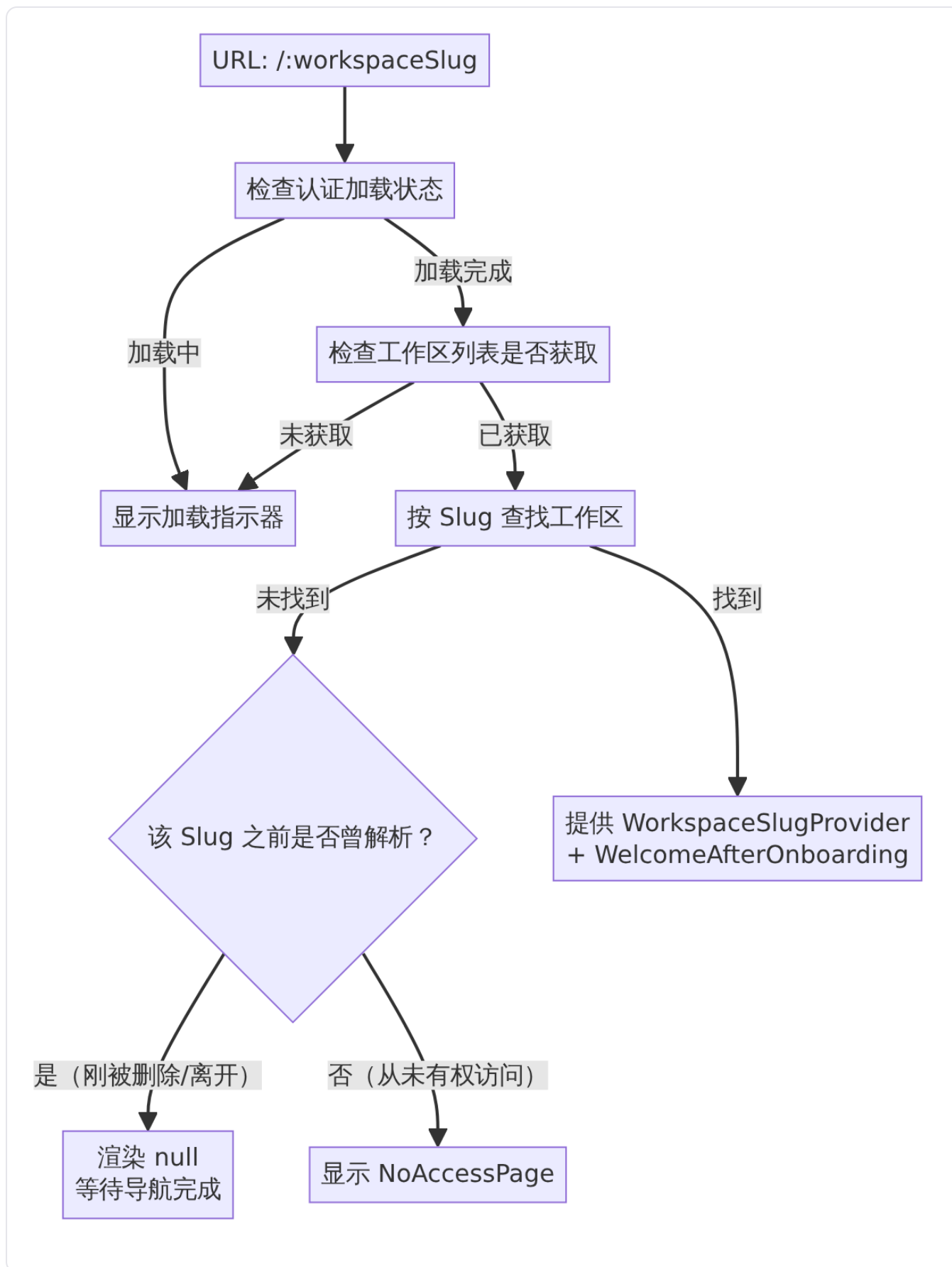
注入能力	说明
	next/navigation
语言适配器	基于浏览器 Cookie 的 LocaleAdapter，与 @multica/core/i18n 对接
认证 Cookie	setLoggedInCookie / clearLoggedInCookie 管理 multica_logged_in Cookie
客户端 OS 检测	detectWebOS() 从 navigator.platform 和 userAgent 推断操作系统
WebSocket URL	从环境变量或 API 基地址推导 WebSocket 连接地址

认证 Cookie 的生命周期为一年，samesite=lax，路径为 / apps/web/features/auth/auth-cookie.ts。客户端 OS 检测支持 macOS、Windows、Linux、iOS、Android、ChromeOS 六种粗粒度分类 apps/web/platform/client-os.ts。

工作区路由与认证守卫

工作区布局

apps/web/app/[workspaceSlug]/layout.tsx 是工作区路由的入口守卫，所有 /[workspaceSlug]/* 页面均在其保护下渲染。该组件执行以下流程 apps/web/app/[workspaceSlug]/layout.tsx：



关键行为：

- **认证强制**：工作区路由要求用户已认证。未认证用户将被重定向到登录页，并保留原始 URL 以便登录后恢复。
- **工作区解析**：通过 `workspaceBySlugOptions` React Query 查询按 slug 查找工作区，结果存入 `setCurrentWorkspace` 以供全局使用 `apps/web/app/[workspaceSlug]/layout.tsx`。
- **无权限页面**：当用户访问从未拥有的工作区 slug 时，显示 `NoAccessPage` 而非静默重定向，不区分 404 与 403 以防止攻击者枚举 slug。
- **闪烁防护**：通过 `useWorkspaceSeen` 追踪 slug 是否在当前会话中曾解析成功。若用户主动删除、离开工作区或被实时驱逐，该机制避免短暂渲染 `NoAccessPage` 的闪烁。
- **入职欢迎**：工作区首次访问时，`WelcomeAfterOnboarding` 组件在子路由外渲染入职引导。

仪表盘布局

`apps/web/app/[workspaceSlug]/(dashboard)/layout.tsx` 为所有仪表盘子路由提供统一的 `DashboardLayout`，包含 `apps/web/app/[workspaceSlug]/(dashboard)/layout.tsx`：

- **全局搜索**：`SearchTrigger`（触发按钮）与 `SearchCommand`（命令面板），来自 `@multica/views/search`。
- **系统通知桥接**：`WebNotificationBridge` 注册系统通知点击处理器，点击浏览器通知时通过 `NavigationAdapter` 导航到对应工作区页面。
- **浮动聊天**：`FloatingChat` 提供跨页面的智能体对话入口。

实时通信

Web 应用的实时更新基于 WebSocket 连接。连接地址由 `apps/web/config/runtime-urls.ts` 中的 `resolveBrowserWsUrl` 函数推导 `apps/web/config/runtime-urls.ts`：

- 优先使用显式配置的 `NEXT_PUBLIC_WS_URL`。
- 否则从 `NEXT_PUBLIC_API_URL` 推导：将 `http(s)` 协议替换为 `ws(s)`，并追加 `/ws` 路径。
- API 基地址中的 `/api` 后缀在推导前被剥离，避免产生错误的 `/api/ws` 路径 `apps/web/config/runtime-urls.ts`。

`WSProvider` 来自 `@multica/core/realtime`，在 `WebProviders` 中初始化，提供事件订阅和重连回调机制 `packages/core/realtime/provider.tsx`。业务模块通过 `useWSEvent` 和 `useWSReconnect` 钩子订阅服务端推送的事件，更新 TanStack Query 缓存 `packages/core/realtime/hooks.ts`。

实时通信的详细机制见 [实时通信基础设施](#)。

状态管理

遵循项目全局规范，Web 应用的状态管理分为两层：

状态类型	管理工具	范围
服务端数据	TanStack Query	issue、智能体、成员、收件箱、项目等
客户端状态	Zustand	筛选条件、草稿、弹窗、布局等

当前工作区身份由路由决定，仅在平台层为请求、持久化命名空间和重连做必要镜像。React Context 仅传递工作区 ID、导航适配器等平台 plumbing。

Zustand store 统一位于 `packages/core/`，不在 `packages/views/` 或应用目录中定义。WebSocket 事件更新 React Query 缓存，不将服务端对象复制进 Zustand。

国际化

Web 应用通过多层机制确定用户语言环境：

1. **代理层**：`proxy.ts` 从 `Cookie (multica_locale)` 和 `Accept-Language` 头解析语言，通过 `x-multica-locale` 请求头传递给 RSC [apps/web/proxy.ts](#)。
2. **RSC 层**：`getRequestLocale()` 优先读取 `x-multica-locale` 头，回退到 `Cookie` 和 `Accept-Language` [apps/web/lib/request-locale.ts](#)。
3. **客户端层**：`WebProviders` 创建浏览器 `Cookie` 适配器，与 `@multica/core/i18n` 的 `I18nProvider` 对接。

支持的语言：英语 (`en`)、简体中文 (`zh-Hans`)、日语 (`ja`)、韩语 (`ko`)。HTML `lang` 属性映射为 BCP-47 区域标签以支持屏幕阅读器和字体回退 [apps/web/app/layout.tsx](#)。

详细的国际化机制见 [国际化框架](#)。

运行时 URL 解析

`apps/web/config/runtime-urls.ts` 提供环境变量驱动的 URL 解析，支持自部署场景：

函数	用途	环境变量
<code>resolveRemoteApiUrl</code>	服务端 API 重写目标	<code>REMOTE_API_URL</code> 、 <code>NEXT_PUBLIC_API_URL</code>
<code>resolveBrowserApiBaseUrl</code>	浏览器端 API 基地址	<code>NEXT_PUBLIC_API_URL</code>
<code>resolveBrowserWsUrl</code>	浏览器端 WebSocket URL	<code>NEXT_PUBLIC_WS_URL</code> 或从 API URL 推导
<code>resolveDocsUrl</code>	文档站 URL	<code>DOCS_URL</code>
<code>runtimeRewriteDestination</code>	代理层运行时重写路径	综合以上

开发模式下提供 `resolveDevRemoteApiUrl` 和 `resolveDevDocsUrl`，默认指向 `localhost:8080` 和 `localhost:4000` [apps/web/config/runtime-urls.ts](#)。

API 基地址自动剥离末尾的 `/api` 后缀，防止自部署用户配置 `NEXT_PUBLIC_API_URL=https://host/api` 时产生 `/api/api/**` 的双重路径错误 [apps/web/config/runtime-urls.ts](#)。

工作区 URL 展示

`packages/core/workspace/workspace-url.ts` 中的 `workspaceUrlHost` 函数确定工作区 URL 的域名展示 [packages/core/workspace/workspace-url.ts](#)：

- 优先使用部署的应用 URL（`daemon_app_url`），自部署实例展示自己的域名。
- 未配置时回退到品牌域名 `multica.ai`。
- 处理带协议、路径、查询参数的输入，仅提取主机部分。

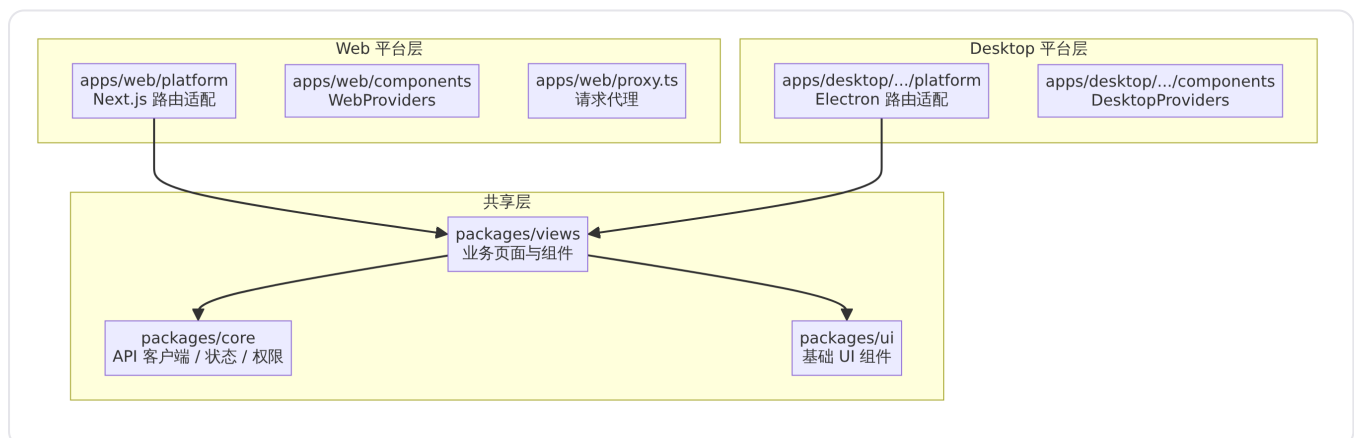
旧版 URL 兼容

代理层维护 `LEGACY_ROUTE_SEGMENTS` 集合，覆盖 URL 重构前的工作区路由段 [apps/web/proxy.ts](#)：

- 旧版路由段：`issues`、`projects`、`agents`、`squads`、`inbox`、`my-issues`、`autopilots`、`runtimes`、`skills`、`settings`、`usage`。
- 当匹配到旧版 URL 时：未登录用户重定向到 `/login`；已登录用户利用 `last_workspace_slug` Cookie 重定向到 `/{$slug}/{$path}`；无 Cookie 的已登录用户重定向到 `/`。

与桌面应用的代码共享

Web 与桌面应用共享三层代码，平台能力保留在应用层 [apps/docs/content/docs/developers/architecture.mdx](#)：



共享页面通过 `NavigationAdapter` 导航，不直接导入 `next/*` 或 `react-router-dom`。Web 平台的适配器实现在 `apps/web/platform/navigation.tsx` 中，封装 Next.js 的 `useRouter`、`usePathname` 和 `useSearchParams` [apps/web/platform/navigation.tsx](#)。

技术栈

apps/web/package.json 定义了 Web 应用的核心依赖 [apps/web/package.json](#) :

类别	技术
框架	Next.js 16 (App Router)
状态管理	TanStack Query、Zustand
UI 组件	shadcn、Base UI、Radix UI (vaul)
编辑器	Tiptap (富文本)、lowlight (代码高亮)
样式	Tailwind CSS、class-variance-authority
图表	Recharts
拖拽	dnd-kit
国际化	@multica/core/i18n (Cookie 适配器)
构建	Webpack (Next.js 内置)
测试	Vitest、Testing Library

桌面应用

detail · desktop-application.md

桌面应用

Multica Desktop 是基于 Electron 的桌面客户端，提供 macOS、Windows 和 Linux 原生应用体验。它复用与 Web 版相同的账号和工作区数据，同时在本机自动管理守护进程，并为每个工作区保留独立的标签页集合。

Desktop 与 Web 的区别

Multica Desktop 和 Web 应用共享同一后端，数据互通，但交互模型有本质差异：

维度	Web	Desktop
打开方式	浏览器	安装桌面应用
工作区标签页	使用浏览器标签页	每个工作区保留独立的标签页集合
守护进程	需单独安装 CLI 并手动启动	登录后由应用自动启动
更新机制	刷新页面	通过内置自动更新器升级
本地目录访问	受浏览器沙箱限制	原生文件对话框选择本地目录

两者可以同时登录。只要连接的是同一个 Multica 服务，看到的数据就是共享的。临时查看或在公共电脑上使用时，Web 更方便。

构建与打包

桌面应用位于 monorepo 的 `apps/desktop` 目录下，通过 `electron-vite` 构建。入口结构分为三层：

- **主进程 (main)**：位于 `apps/desktop/src/main/index.ts`，负责窗口创建、守护进程管理、自动更新、IPC 处理等系统级操作。[apps/desktop/src/main/index.ts](#)
- **预加载 (preload)**：位于 `apps/desktop/src/preload/index.ts`，通过 `contextBridge` 向渲染进程暴露四组安全 API：`electron`、`desktopAPI`、`daemonAPI`、`updater`。
[apps/desktop/src/preload/index.ts](#)
- **渲染进程 (renderer)**：使用 React，复用 monorepo 中 `@multica/ui` 和 `@multica/views` 的共享 UI 组件。

`electron-vite` 配置将三个入口分离构建，渲染进程支持 React + Tailwind CSS，并通过别名 `@` 指向 `src/renderer/src`。开发模式下可通过 `DESKTOP_RENDERER_PORT` 环境变量自定义渲染进程端口，默认 5173。[apps/desktop/electron.vite.config.ts](#)

打包使用 `electron-builder`，配置位于 `apps/desktop/electron-builder.yml`。构建产物按平台和架构分发：macOS (`.dmg`)、Windows (`.exe`)、Linux (`.AppImage`、`.deb`、`.rpm`)。同时注册了

`multica://` 自定义协议用于深度链接。 [apps/desktop/electron-builder.yml](#)

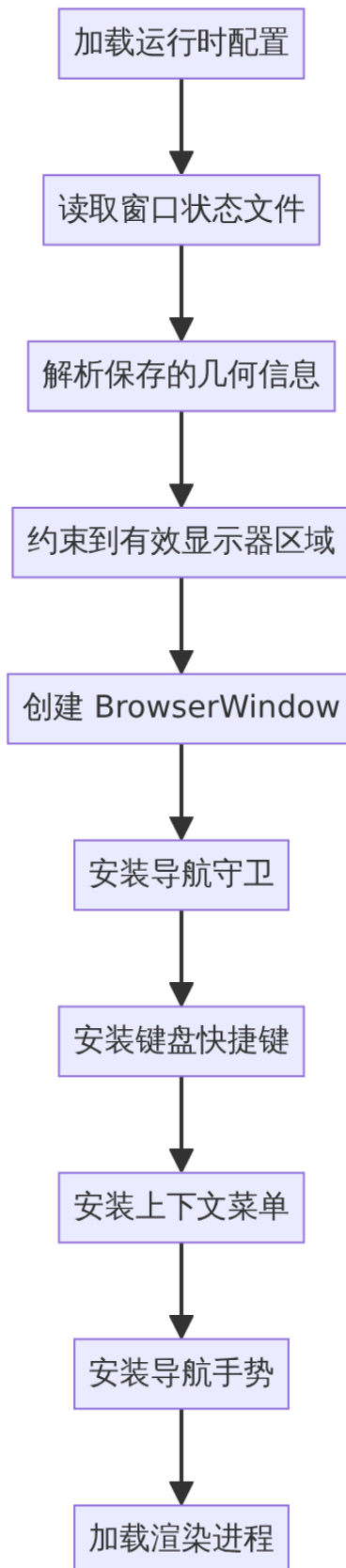
窗口管理

主窗口

主窗口在应用启动时创建，提供完整的 Multica 功能界面。核心行为：

- **窗口状态持久化**：窗口的尺寸、位置、最大化/全屏状态保存到 `userData` 目录下的 `window-state.json`，下次启动时恢复。默认尺寸为 1280×800，最小尺寸为 900×600。
[apps/desktop/src/main/window-state.ts](#)
- **单实例锁**：通过 `app.requestSingleInstanceLock()` 保证同一时刻只有一个 Desktop 实例运行。第二个实例启动时会将深度链接 URL 传递给已有实例并聚焦主窗口。
- **macOS 特性**：使用 `titleBarStyle: "hiddenInset"` 实现沉浸式标题栏，红绿灯按钮位置固定为 (16, 17)。支持通过 IPC 切换沉浸模式（隐藏红绿灯）。
- **开发/生产隔离**：开发模式使用独立的应用名 `Multica Canary` 和独立的 `userData` 路径，避免与生产版冲突。可通过 `DESKTOP_APP_SUFFIX` 环境变量支持多个并行开发实例。

窗口创建流程如下：



来源：[apps/desktop/src/main/index.ts](https://github.com/electron/electron/blob/master/docs/api/browser-window.md)

独立 Issue 窗口

Desktop 支持将单个 Issue 从主窗口分离为独立的 Issue 窗口。窗口尺寸为 960×760，最小 720×520。Issue 窗口共享相同的渲染进程入口，但通过 `additionalArguments` 传递 Issue 上下文，使其加载针对性 UI。主窗口和所有 Issue 窗口通过 `AuthSessionCoordinator` 协同管理认证会话。

导航守卫

所有窗口在加载渲染进程前安装导航守卫，对初始导航进行来源加固。守卫仅作用于外部 URL 的加载行为，应用内部路由切换不受影响。[apps/desktop/src/main/navigation-guard.ts](#)

运行时配置与自托管连接

Desktop 默认连接 Multica Cloud (`api.multica.ai`)。连接自托管实例时，需在用户目录的 `.multica` 下创建 `desktop.json`：

平台	路径
macOS	<code>/Users/<用户名>/.multica/desktop.json</code>
Linux	<code>/home/<用户名>/.multica/desktop.json</code>
Windows	<code>C:\Users\<用户名>\.multica\desktop.json</code>

配置文件格式：

```
{
  "schemaVersion": 1,
  "apiUrl": "https://api.example.com"
}
```

`apiUrl` 为必填字段。`wsUrl` 和 `appUrl` 可选，省略时 Desktop 自动推导：

- `wsUrl`：将 `apiUrl` 协议换为 `ws / wss`，路径追加 `/ws`；
- `appUrl`：当 `apiUrl` 主机名以 `api.` 开头且至少包含三段标签时，剥离 `api.` 前缀（如 `api.example.com` → `example.com`），否则保持与 `apiUrl` 一致。

Desktop 只在启动时读取配置。配置文件的处理逻辑分两种情况：

- **文件未找到**：Desktop 静默使用默认 Cloud 配置，不报错——这表明文件不在 Desktop 查找的位置。
- **文件找到但无效**（JSON 格式、`schemaVersion` 或 URL 有问题）：Desktop 显示配置错误，不会自动回退到 Cloud。

删除该文件并重启即可恢复默认 Cloud 配置。配置文件路径由 `desktopConfigPath()` 函数解析，路径为 `app.getPath("home") + "/.multica/desktop.json"`。[apps/desktop/src/main/runtime-config-loader.ts](#)

注意：Desktop 的 `desktop.json` 与 CLI 的 `~/.multica/config.json` 是两个独立文件，字段名也不同——CLI 使用 `server_url`，Desktop 使用 `apiUrl`。Desktop 不会读取 CLI 配置。

内置守护进程

登录后，Desktop 为当前 Multica 服务创建专用的 CLI profile，并以此启动守护进程。Profile 路径为：

```
~/.multica/profiles/desktop-<host>/
```

该 profile 不会读取或覆盖终端中使用的默认 profile。若同时手动启动了另一个守护进程，Multica 会将它们显示为不同的运行时。

CLI 二进制探测

Desktop 通过以下优先级链查找可用的 `multica` CLI 二进制文件：

1. **已缓存的解析结果：**前次成功解析的路径；
2. **内置二进制：**随 Desktop 打包在 `resources/bin/` 下的 CLI；
3. **托管二进制：**已下载到 `userData/bin/` 的副本；
4. **自动下载安装：**从 GitHub Releases 下载最新版本到 `userData/bin/`；
5. **系统 PATH：**用户自行安装的 `multica`（含 Homebrew 路径）。

每次探测通过执行 `multica version --output json` 验证二进制可用性。优先使用内置二进制，确保 Desktop 与其配套的 CLI 版本对齐。[apps/desktop/src/main/daemon-manager.ts](https://github.com/multica/apps/desktop/src/main/daemon-manager.ts)

健康检查与状态轮询

守护进程管理器以 5 秒间隔轮询守护进程的 `/health` 端点（默认端口 19514），获取运行状态、PID、上线时间、CLI 版本、活跃任务数、检测到的 Agent 和已连接工作区等数据。状态变化通过 `daemon:status` IPC 通道推送给渲染进程。

守护进程状态包括：

- **stopped**：未运行；
- **starting**：正在启动中（守护进程已绑定端口但尚未完成预检）；
- **running**：正常运行，携带完整健康信息；
- **auth_expired**：令牌已过期，需重新登录。

此外，Desktop 能识别外部管理的守护进程（如 Linux-in-WSL2 场景）——当守护进程报告的操作系统与当前主机不同时，UI 会禁用启停控件。[apps/desktop/src/main/daemon-manager.ts](https://github.com/multica/apps/desktop/src/main/daemon-manager.ts)

令牌同步

Desktop 通过 `daemon:sync-token` 和 `daemon:clear-token` IPC 通道实现令牌的同步与清除。渲染进程登录后，主进程将令牌写入 `profile` 目录，供守护进程使用。登出时清空令牌并停止守护进程。

[apps/desktop/src/preload/index.ts](#)

CLI 自举

当内置和托管二进制均不可用时，Desktop 从 GitHub Releases 下载最新 CLI。下载流程包含 SHA256 校验——从 `checksums.txt` 中提取预期哈希值，与下载文件的 SHA256 比对，确保文件完整性。

[apps/desktop/src/main/cli-bootstrap.ts](#)

自动更新

自动更新由 `electron-updater` 驱动，默认开启。更新策略：

- **自动下载**：检测到新版本后在后台静默下载；
- **自动安装**：下载完成后在应用退出时自动安装；
- **启动延迟检查**：应用启动 5 秒后执行首次更新检查；
- **定期检查**：每小时重新检查一次。

更新按平台和架构分发不同更新源：

- Windows arm64 使用独立通道 `latest-arm64`（避免与 x64 在 `latest.yml` 上冲突）；
- macOS x64 (Intel) 使用独立通道 `latest-x64`；
- 其他架构使用默认更新源；
- Linux 上自动更新仅对 `.AppImage` 生效，`.deb` 和 `.rpm` 需手动下载覆盖安装。

渲染进程通过 `updater` API 监听更新事件（`updater:update-available`、`updater:download-progress`、`updater:update-downloaded`）并控制下载与安装操作。用户可在 **设置** → **更新** 中关闭自动检查或手动检查新版本。[apps/desktop/src/main/updater.ts](#)

应用版本号

生产环境中，版本号来自 `electron-builder` 打包时注入到 `package.json` 的 `extraMetadata.version`（由 `git describe` 驱动），与 GitHub Release 标签精确匹配。开发环境中，通过 `git describe --tags --match "v[0-9]*" --always --dirty` 获取描述性版本号，使开发者能准确识别当前构建。

[apps/desktop/src/main/app-version.ts](#)

键盘快捷键

桌面应用通过 `before-input-event` 事件统一处理键盘快捷键。`handleAppShortcut` 函数负责识别与分发：

- **缩放**： `Cmd/Ctrl + + / - / 0` 控制页面缩放。步长为 0.5，有效范围 -3 到 4.5，防止 UI 过小或过大不可用。
- **关闭标签页**： `Cmd/Ctrl + W` 触发 `tab:close-active` IPC 消息，由渲染进程关闭当前标签页。

所有快捷键处理均在主进程中通过 `event.preventDefault()` 阻止默认行为，避免渲染进程和系统菜单的双重触发。[apps/desktop/src/main/keyboard-shortcuts.ts](#)

上下文菜单

桌面应用安装自定义右键上下文菜单，解决 Electron 默认不提供右键菜单的问题。菜单根据当前上下文动态构建：

条件	菜单项
可编辑区域 + 可剪切	剪切 (cut)
已选中文本 + 可复制	复制 (copy)
可编辑区域 + 可粘贴	粘贴 (paste)
可编辑区域 + 可全选	全选 (selectAll)
光标在 http(s) 链接上	打开链接、复制链接地址

`copy`、`cut`、`paste`、`selectAll` 使用 Electron 内置角色（自动跟随系统语言），而"打开链接"和"复制链接地址"为自定义菜单项，根据操作系统语言提供中文、日文、韩文的本地化标签。
[apps/desktop/src/main/context-menu.ts](#)

本地目录访问

Desktop 通过原生文件对话框提供本地目录选择功能，突破浏览器沙箱对文件系统的限制。渲染进程通过 IPC 调用 `local-directory:pick` 触发系统原生目录选择器，主进程返回所选路径的绝对路径和目录名。同时提供 `local-directory:validate` 用于验证给定路径是否可访问、是否为目录。
[apps/desktop/src/main/local-directory.ts](#)

macOS 导航手势

在 macOS 上，Desktop 监听触控板滑动手势（swipe），将其转换为前进/后退导航事件。手势通过 IPC 通道 `navigation:gesture` 发送给渲染进程处理。此功能仅在 macOS (darwin) 平台激活。
[apps/desktop/src/main/navigation-gestures.ts](#)

深度链接与协议处理

Desktop 注册了 `multica://` 自定义协议，支持以下场景：

- **认证回调**： `multica://auth/callback` 处理 OAuth 流程的深度链接回调；

- **冷启动**：应用启动时扫描 `process.argv` 中的协议链接；
- **macOS `open-url` 事件**：监听系统级 URL 打开事件；
- **Windows/Linux 第二实例**：第二个实例通过 `second-instance` 事件传递深度链接。

深度链接在主窗口和渲染进程就绪前被排队缓存，待就绪后分发到渲染进程的对应路由。

[apps/desktop/src/main/index.ts](#)

原生通知

Desktop 在以下条件下通过系统级通知（`Notification` API）弹窗：

- 当前有活跃的认知会话；
- 通知内容通过 `notificationGate` 去重（同一 `itemId` 在任意窗口处于前台时抑制）；
- 用户点击通知后，Desktop 通过深度链接机制导航到对应收件箱条目，并验证通知发出时的认知会话是否仍然有效。

来自不同渲染窗口（主窗口与 Issue 窗口）的通知均统一由主进程管理，避免重复弹窗。

[apps/desktop/src/main/index.ts](#)

渲染进程恢复

Desktop 内置渲染进程崩溃恢复机制。当渲染进程无响应（hung）或崩溃时：

- 主进程写入冻结面包屑（freeze breadcrumb），记录崩溃上下文（路由信息、时间戳、应用版本）；
- 下次渲染进程启动时，读取并上报未送达的崩溃信息到遥测服务；
- 开发模式下，通过 DevTools 日志通道输出渲染进程的控制台消息和加载失败诊断信息。

此机制确保用户侧的白屏/卡死问题可被追溯定位。[apps/desktop/src/main/index.ts](#)

预加载 API 总览

预加载脚本通过 `contextBridge` 向渲染进程暴露四组 API，全部在

`apps/desktop/src/preload/index.d.ts` 中声明类型：

electron API

由 `@electron-toolkit/preload` 提供，暴露 Electron 标准能力（如 `ipcRenderer` 的受限子集）。

desktopAPI

属性/方法	说明
<code>appInfo</code>	应用版本号和标准化操作系统标识
<code>systemLocale</code>	操作系统首选语言（BCP 47），通过启动参数注入

属性/方法	说明
<code>onSystemLocaleChanged</code>	订阅操作系统语言变化事件
<code>runtimeConfig</code>	已验证的运行时端点配置或阻塞性错误
<code>windowContext</code>	当前窗口上下文（主窗口或 Issue 窗口）

daemonAPI

方法	说明
<code>getStatus</code>	获取当前守护进程状态
<code>onStatus</code>	订阅守护进程状态变化
<code>setTargetApiUrl</code>	设置守护进程目标 API 地址
<code>syncToken</code> / <code>clearToken</code>	同步/清除认证令牌
<code>reauthenticate</code>	重新认证守护进程
<code>isCliInstalled</code>	检查 CLI 是否已安装
<code>getPrefs</code> / <code>setPrefs</code>	读取/设置守护进程偏好（自动启动、自动停止）
<code>autoStart</code>	触发守护进程自动启动
<code>retryInstall</code>	重试 CLI 安装
<code>startLogStream</code> / <code>stopLogStream</code>	控制守护进程日志流
<code>onLogLine</code>	订阅日志行输出
<code>openLogFile</code>	在系统编辑器中打开日志文件

updater API

方法	说明
<code>onUpdateAvailable</code>	订阅更新可用事件
<code>onDownloadProgress</code>	订阅下载进度事件
<code>onUpdateDownloaded</code>	订阅更新下载完成事件
<code>downloadUpdate</code>	触发更新下载
<code>installUpdate</code>	触发更新安装（重启应用）
<code>getPreferences</code>	获取更新偏好设置
<code>setAutomaticUpdates</code>	开关自动更新
<code>checkForUpdates</code>	手动检查更新

来源：[apps/desktop/src/preload/index.d.ts](#) 和 [apps/desktop/src/preload/index.ts](#)

PATH 修复

macOS 和 Linux 上，图形界面启动的应用继承自 `launchd` 的最小 PATH，缺少用户 Shell 配置中的路径（Homebrew、`nvm`、`~/.local/bin` 等）。Desktop 在启动时执行两步修复：

- 1. 调用 `fix-path` 库运行用户登录 Shell 恢复完整 PATH；
- 2. 追加常见安装位置作为后备（`/opt/homebrew/bin`、`/usr/local/bin`、`~/.local/bin`）。

此操作必须在任何 `child_process.spawn` / `execFile` 调用之前完成，确保内置 `multica` CLI 能找到 Agent 二进制文件。[apps/desktop/src/main/index.ts](#)

平台差异化处理

功能	macOS	Windows	Linux
标题栏样式	<code>hiddenInset</code> + 红绿灯自定义位置	系统默认	系统默认
窗口图标	Dock 图标（ <code>app.dock.setIcon</code> ）	EXE 资源内嵌	<code>BrowserWindow</code> <code>icon</code> 选项显式指定
导航手势	触控板滑动手势	不支持	不支持
沉浸模式	隐藏红绿灯	不支持	不支持
窗口关闭行为	关闭所有窗口后不退出的 Dock 应用模式	关闭最后窗口后退出	关闭最后窗口后退出
自动更新格式	<code>.dmg</code>	<code>.exe</code>	仅 <code>.AppImage</code> 支持自动更新

移动应用

移动应用是 Multica 的 iOS 原生客户端，基于 **Expo (SDK 55)** 和 **React Native 0.83** 构建。它独立于 Web 和桌面端——仅从 `@multica/core` 导入类型定义与纯函数，拥有自己的 UI 组件、查询键、状态管理、实时订阅和发布流程。目前仅支持 iOS，Android 版本尚未提供。

[apps/mobile/README.md](#) 是移动端的入口文档；[apps/mobile/CLAUDE.md](#) 包含技术栈基线及导入规则。

用户目标

Multica 移动应用面向需要在手机上管理任务的用户，提供以下核心能力：

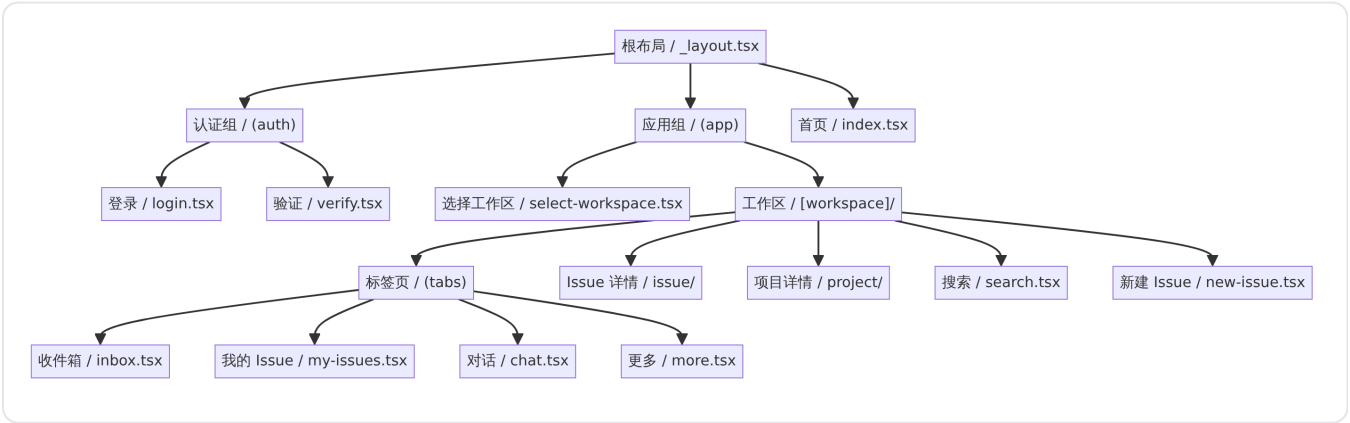
- **邮件验证码登录**：无需密码，输入邮箱后接收一次性验证码完成认证。
- **工作区切换**：在多工作区之间导航，所有数据由服务端保存，无需手动同步。
- **Issue 浏览与管理**：查看、筛选、创建 Issue，支持指派智能体、标签和优先级。
- **智能体对话**：与智能体进行实时聊天，查看执行记录。
- **收件箱**：集中查看与自己相关的通知和更新。
- **实时更新**：Issue、评论、收件箱和智能体状态通过 WebSocket 实时推送。

所有数据——工作区、Issue、评论和执行记录——均保存在服务端。用户可以在 iPhone、Web 和桌面端之间无缝切换，无需手动同步。

[apps/docs/content/docs/mobile-app.zh.md](#) 提供了面向最终用户的安装说明。

入口点与路由

应用采用 Expo Router 的基于文件系统的路由方案，入口点为 `expo-router/entry`。路由结构分为两层：



根布局文件 `apps/mobile/app/_layout.tsx` 负责初始化所有全局 Provider： `GestureHandlerRootView`、`SafeAreaProvider`、`KeyboardProvider`、`QueryClientProvider`、`ThemeProvider`，以及认证初始化器 `AuthInitializer`。认证初始化器在挂载时设置全局 401 拦截器，当服务端返回未授权错误时自动清除认证状态、查询缓存，并将用户导航回登录页。

多环境构建

移动端支持三种环境，通过 `APP_ENV` 环境变量和对应的 `.env` 文件区分：

环境	环境变量文件	后端目标	典型用途
开发 (Dev)	<code>.env.development.local</code>	本地 Go 后端	日常开发，连接 Mac 本机 API
预发布 (Staging)	<code>.env.staging</code> （已提交仓库）	staging 环境	连接预发布服务器进行测试
生产 (Production)	<code>.env.production</code>	multica.ai 生产环境	正式使用或设备安装

每个环境的 bundle identifier 和显示名称随 `APP_ENV` 切换，因此开发、预发布和生产版本可以共存于同一台设备或模拟器上。

`apps/mobile/README.md` 中列出了完整的脚本矩阵：

命令	说明	构建类型
<code>pnpm dev:mobile</code>	仅启动 Metro（复用已有安装）	Debug
<code>pnpm dev:mobile:staging</code>	Metro + staging 后端	Debug
<code>pnpm dev:mobile:prod</code>	Metro + 生产后端	Debug
<code>pnpm ios:mobile</code>	完整重建 + iOS 模拟器安装	Debug
<code>pnpm ios:mobile:staging</code>	模拟器 + staging	Debug
<code>pnpm ios:mobile:prod</code>	模拟器 + 生产	Debug
<code>pnpm ios:mobile:device</code>	USB 真机构建	Debug
<code>pnpm ios:mobile:device:staging</code>	真机 + staging	Debug
<code>pnpm ios:mobile:device:prod</code>	真机 + 生产	Debug
<code>pnpm ios:mobile:device:staging:release</code>	真机独立构建（无 Metro）	Release
<code>pnpm ios:mobile:device:prod:release</code>	真机独立构建 + 生产	Release

`dev:*` 命令仅启动 Metro bundler，假设对应变体已安装。`ios:mobile*` 执行完整原生重建和安装。

Debug 与 Release 构建

Debug 构建内嵌 `expo-dev-client`，每次启动时 App 会探测 Mac 上的 Metro 服务器并拉取最新的 JavaScript bundle——适合需要热重载的开发场景。离开 Mac 或处于不同 WiFi 网络时无法使用。

Release 构建不包含 `expo-dev-launcher`，不探测 Metro。启动流程为 Splash → App，行为与 App Store 安装完全一致。代价是每次 JavaScript 变更都需要重新运行构建命令。

认证流程

移动端采用无密码邮件验证码（OTP）认证，流程如下：

- 1. 用户在登录页输入邮箱地址。
- 2. 调用 `sendCode` 方法向服务端发送验证码请求。
- 3. 成功后导航至 `/verify` 页面，用户输入收到的六位验证码。
- 4. 验证通过后，服务端返回访问令牌和刷新令牌，存储至 iOS Keychain（通过 `expo-secure-store`）。
- 5. 认证状态由 `useAuthStore`（Zustand store）管理。

`apps/mobile/app/(auth)/login.tsx` 包含登录表单的完整实现，使用 `expo-haptics` 提供触觉反馈。认证错误通过 `mapAuthError` 工具函数映射为可读的错误信息。

数据架构

移动端拥有完全独立的数据层，不依赖 `packages/core` 的查询和 mutation hooks。数据流按以下层次组织：

层次	典型文件	职责
API 客户端	<code>data/api.ts</code>	HTTP 请求封装，401 拦截，令牌管理
查询	<code>data/queries/</code>	TanStack Query 选项（ <code>workspaces</code> 、 <code>issues</code> 、 <code>inbox</code> 、 <code>chat</code> 、 <code>agents</code> 、 <code>members</code> 、 <code>labels</code> 、 <code>projects</code> 、 <code>pins</code> 、 <code>runtimes</code> 、 <code>squads</code> 等）
变更	<code>data/mutations/</code>	乐观更新 mutation（ <code>issues</code> 、 <code>chat</code> 、 <code>inbox</code> 、 <code>labels</code> 、 <code>projects</code> 、 <code>pins</code> 、 <code>notification-preferences</code> ）
客户端状态	<code>data/stores/</code>	Zustand stores（草稿、筛选视图、回复目标、会话选择器等）
Schema	<code>data/schemas.ts</code>	Zod schema 解析 API 返回值
实时通信	<code>data/realtime/</code>	WebSocket 客户端与各类实时更新 hook

客户端状态 store 涵盖：聊天草稿、Issue 筛选视图、我的 Issue 视图、新 Issue 草稿、新项目草稿、失败评论、最近查看、提及草稿、回复目标和会话选择器，均定义在 `apps/mobile/data/stores/` 目录内。

导入规则

移动端只能从 `packages/` 导入两类内容：

- `import type` 从 `@multica/core/types/*` ——零运行时耦合
- 纯函数从 `@multica/core/`

其他所有内容（React Query hooks、UI 组件、实时订阅、Zustand stores 等）由移动端独立实现。

[apps/mobile/CLAUDE.md](#) 明确了这些边界规则。

实时通信

移动端通过 WebSocket 接收服务端推送的实时事件。进入工作区时，`RealtimeProvider` 建立 WebSocket 连接。各类 hook 订阅特定事件类型并更新 TanStack Query 缓存：

Hook	订阅范围	文件
<code>useInboxRealtime</code>	收件箱通知	use-inbox-realtime.ts
<code>useIssuesRealtime</code>	Issue 列表变更	use-issues-realtime.ts
<code>useMyIssuesRealtime</code>	我的 Issue	use-my-issues-realtime.ts
<code>useChatSessionsRealtime</code>	聊天会话	use-chat-sessions-realtime.ts
<code>useProjectsRealtime</code>	项目变更	use-projects-realtime.ts
<code>usePinsRealtime</code>	置顶项目	use-pins-realtime.ts
<code>usePresenceRealtime</code>	成员在线状态	use-presence-realtime.ts

WebSocket 事件通过 `*-ws-updaters.ts` 模块更新 React Query 缓存，确保服务端数据与 UI 保持同步。

样式方案

移动端使用 **NativeWind 4.x**（基于 Tailwind CSS 3.4 的 React Native 适配）作为样式方案。组件中使用 `className` 属性编写类似于 Web Tailwind 的工具类样式，NativeWind 在编译时将其转换为 React Native 的 `StyleSheet`。

[apps/mobile/tailwind.config.js](#) 定义了设计令牌，[apps/mobile/global.css](#) 提供全局样式入口。

Markdown 渲染

移动端采用 [react-native-enriched-markdown](#) 作为 Markdown 渲染核心。该库使用 **md4c** 解析器生成 iOS `NSAttributedString`（Android 上为 `Spannable`），完全绕过 React Native 的嵌套 `<Text>` 布局路径，避免了 RN 社区已知的长达十年的嵌套 `<Text>` 渲染 bug。

技术架构决策记录在 [apps/mobile/docs/markdown-rendering-adr.md](#) 中正式记录了选择 `enriched-markdown` 的原因：Expo 官方文档在 [Edit rich text](#) 中推荐该库，且其维护方 Software Mansion 是 React Native 生态的核心贡献者。

对于原生渲染路径无法覆盖的自定义节点（如代码块语法高亮），移动端在 `enriched-markdown` 之上叠加了独立的 React 层：使用 Shiki 引擎提供语法高亮的代码块组件。语法高亮相关的核心实现在 [apps/mobile/lib/markdown/shiki.ts](#)，并在应用启动时通过 `prewarmHighlighter()` 预加载，确保用户首次打开包含代码块的页面时高亮引擎已就绪。

组件体系

移动端组件按功能领域组织在 `apps/mobile/components/` 下：

目录	职责
<code>components/ui/</code>	基础 UI 组件：Button、Text、Input、TextField、Avatar、Card、Tabs、Switch、DropDownMenu、Skeleton 等
<code>components/issue/</code>	Issue 相关组件：列表项、详情页、评论区、描述区、活动时间线、反应栏、属性行
<code>components/chat/</code>	聊天组件：会话列表、消息列表、输入框、智能体选择器、状态指示器
<code>components/composer/</code>	内容编辑器组件
<code>components/inbox/</code>	收件箱展示组件
<code>components/nav/</code>	导航相关组件
<code>components/project/</code>	项目相关组件
<code>components/workspace/</code>	工作区相关组件
<code>components/brand/</code>	品牌标识组件（如 MulticaLogo）

各组件的实际依赖声明在 [apps/mobile/package.json](#) 中。

设备安装与签名

Multica 尚未上架 App Store，用户需从源码构建并安装到 iPhone。构建前需满足以下条件：

- 安装了 Xcode 的 Mac
- 已在 Xcode → Settings → Accounts 中登录的 Apple ID
- 通过 USB 连接的 iPhone，开发者模式已启用
- Node.js、pnpm 和 Git

首次构建需要下载 CocoaPods 并编译 React Native 源码，预计耗时 10-20 分钟。后续构建复用 Xcode 的 DerivedData 缓存。

构建命令：

```
pnpm ios:mobile:device:prod:release
```

该命令生成不依赖 Metro 的 Release 构建，签名使用 Apple ID 自动关联的 Personal Team。

免费签名的 7 天限制

免费 Apple ID 签名的构建有效期为 7 天（Debug 和 Release 均受此限制）。到期后将 iPhone 重新连接 Mac 并再次运行构建命令即可续签。Apple Developer Program 账号（\$99/年）可将有效期延长至 1 年。

签名失败处理

若 Xcode 提示 "No matching provisioning profiles found"，说明默认 bundle identifier

ai.multica.mobile 已被占用。此时设置环境变量覆盖：

```
export EXPO_BUNDLE_IDENTIFIER_PROD=com.yourname.multica
pnpm ios:mobile:device:prod:release
```

更新方式

iOS 自助安装版不会自动更新。拉取最新代码并重新构建：

```
git pull --ff-only
pnpm install
pnpm ios:mobile:device:prod:release
```

连接自托管实例

Release 构建在编译时将 API 地址写入 App。要连接自托管实例，修改 apps/mobile/.env.production：

```
EXPO_PUBLIC_API_URL=https://api.example.com
EXPO_PUBLIC_WEB_URL=https://app.example.com
```

EXPO_PUBLIC_API_URL 为必填项；EXPO_PUBLIC_WEB_URL 为可选项，用于 "复制链接" 和 "在网页打开" 等入口。修改后重新运行构建命令。iPhone 必须能访问该 API 地址——使用局域网地址时不要写 localhost，因为它在 iPhone 上指向手机本身。

[apps/docs/content/docs/mobile-app.zh.mdx](#) 提供了详细的自托管配置说明。

技术栈总览

类别	技术	版本
框架	React Native	0.83.6
平台层	Expo SDK	~55.0.23

类别	技术	版本
路由	Expo Router	~55.0.14
样式	NativeWind + Tailwind CSS	4.1.23 / 3.4.17
服务端状态	TanStack React Query	^5.96.2
客户端状态	Zustand	^5.0.0
实时通信	自定义 WebSocket 客户端	—
Markdown	react-native-enriched-markdown + Shiki	^0.5.0 / ^4.0.2
认证存储	expo-secure-store	~55.0.13
语法高亮	react-native-shiki-engine	^0.3.10
动画	react-native-reanimated	~4.2.1
手势	react-native-gesture-handler	~2.30.1
TypeScript	—	~5.9.0

来源：[apps/mobile/package.json](#)

局限性

- **仅支持 iOS**：Android 客户端暂未提供。项目文档中明确提及 "Android 客户端暂未提供"。
- **未上架 App Store**：用户必须从源码自行构建，无法通过 App Store 直接安装。
- **不可复用 Web/Desktop 的 UI**：移动端拥有独立的 React 页面和组件。`packages/views/` 中的业务页面由 Web 和 Desktop 共享，但不适用于移动端。
- **免费签名有效期 7 天**：使用免费 Apple ID 签名的构建每 7 天需重新签名。
- **Metro 依赖（仅 Debug）**：Debug 构建必须与 Mac 上运行的 Metro 服务器保持连接，离开 Mac 或切换 WiFi 后无法使用。
- **不参与 Turborepo 管线**：根目录的 `dev`、`build`、`typecheck` 等命令默认排除 Mobile。Mobile 拥有独立的脚本和 CI。

自部署方案

detail · self-hosting.md

自部署方案

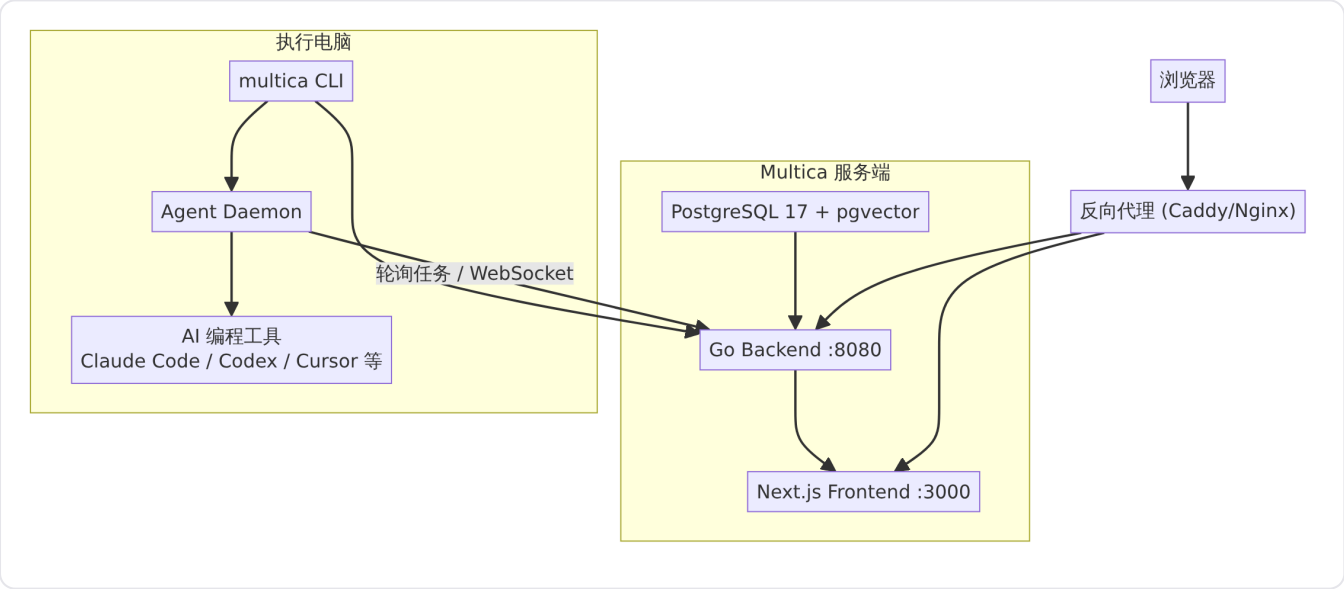
Multica 支持完整的自托管部署，让团队在自己的基础设施上运行 Multica 服务，数据不离开自有网络。自部署不是 Multica Cloud 的简化版——它与云服务共用同一套后端与前端代码，通过 Docker Compose 或 Kubernetes Helm Chart 交付。

整体架构

自部署分为两部分：

部分	运行内容	部署位置
Multica 服务	Web 前端、API 后端、PostgreSQL 数据库	一台安装 Docker 或 Kubernetes 的服务器
执行电脑	Multica 守护进程与 AI 编程工具	每个开发者实际工作的机器

这两部分可以是同一台机器，也可以分开。自部署替换的只是 Multica Cloud 部分，守护进程和执行逻辑与云服务完全相同。



来源：[SELF_HOSTING.md](#)

服务端由三个容器组成：PostgreSQL 17（含 pgvector 扩展）、Go 编写的 REST API + WebSocket 后端、Next.js 16 构建的 Web 前端。容器镜像发布在 GitHub Container Registry（GHCR），`make selfhost` 默认拉取 `latest` 标签的已发布镜像，不会编译本地代码。

执行电脑上的守护进程不在 Docker 内运行——它直接运行在开发者本地机器上，检测已安装的 AI 编程工具，向服务端注册为运行时，并在收到任务时调用对应工具执行。

Docker Compose 部署

这是推荐的自部署方式。整个部署围绕 `docker-compose.selfhost.yml` 编排文件进行，由仓库根目录的 Makefile 封装为 `make selfhost` 一条命令。

前置条件

运行 Multica 服务的机器需要：

- Docker Engine 或 Docker Desktop，且 `docker compose`（Compose v2 插件）可用
- Git、Make、curl 和 OpenSSL
- 本机 3000、8080 端口未被占用

[CLI_AND_DAEMON.md](#) 指出旧版 `docker-compose v1` 不受支持，必须使用 Compose 插件（`docker compose`）。Makefile 内置了检查逻辑，检测不到 Compose 插件时提前报错而非出现晦涩的解析错误。

快速启动

```
git clone --depth 1 https://github.com/multica-ai/multica.git
cd multica
make selfhost
```

首次运行 `make selfhost` 会自动完成以下步骤，如 [Makefile](#) 中 `selfhost` target 所示：

1. 检测 `.env` 是否存在；不存在则从 `.env.example` 创建
2. 用 `openssl rand` 生成随机 `JWT_SECRET`（hex 32 字节）、PostgreSQL 密码（hex 24 字节）和 `MULTICA_VCS_SECRET_KEY`（base64 32 字节）
3. 拉取 PostgreSQL、backend、frontend 镜像
4. 创建持久化数据卷并启动三个容器
5. 等待 backend 健康检查就绪，打印连接信息

之后再次运行 `make selfhost` 复用已有 `.env` 和数据卷，不会重新生成密钥。

拉取镜像失败（如所选 tag 尚未发布）时，脚本会提示使用 `make selfhost-build` 从本地源码构建。

确认服务就绪

查看容器状态：

```
docker compose -f docker-compose.selfhost.yml ps
```

`postgres` 应显示为 `healthy`，`backend` 和 `frontend` 处于运行状态。检查健康端点：

```
curl -fsS http://localhost:8080/readyz
```

正常响应为 `{"status":"ok","checks":{"db":"ok","migrations":"ok"}}`。Backend 容器每次启动时先运行数据库 migration，再启动 HTTP 服务，无需手动执行 migration 命令。

反向代理与远程访问

`docker-compose.selfhost.yml` 中所有服务默认绑定 `127.0.0.1`，不应直接改为 `0.0.0.0` 暴露到公网。根据 [docker-compose.selfhost.yml](#) 的注释说明，Docker 默认会绕过主机防火墙（UFW/iptables），直接暴露端口存在安全风险。应当使用带 HTTPS 的反向代理（Caddy、Nginx 或 Cloudflare Tunnel）。

以两个域名为例——`app.example.com` 和 `api.example.com`——在 `.env` 中设置：

```
FRONTEND_ORIGIN=https://app.example.com
MULTICA_APP_URL=https://app.example.com
MULTICA_PUBLIC_URL=https://api.example.com
```

Caddy 示例配置，将浏览器 WebSocket、API 请求和前端静态资源分别路由：

```
app.example.com {
  @ws path /ws /ws/*
  handle @ws {
    reverse_proxy 127.0.0.1:8080 {
      flush_interval -1
    }
  }
  handle {
    reverse_proxy 127.0.0.1:3000
  }
}

api.example.com {
  reverse_proxy 127.0.0.1:8080 {
    flush_interval -1
  }
}
```

修改 `.env` 后，必须使用 `docker compose -f docker-compose.selfhost.yml up -d` 重新创建容器，因为 `docker compose restart` 不会重新读取 `.env` 文件。

一键安装脚本

除 `make selfhost` 外，Multica 还提供了安装脚本，同时安装 CLI 并部署服务端：

```
# macOS / Linux: 安装 CLI + 部署服务端
curl -fsSL https://raw.githubusercontent.com/multica-ai/multica/main/scripts/install.sh | bash -s
-- --with-server

# 配置 CLI、认证并启动守护进程
multica setup self-host
```

来源于 [scripts/install.sh](#) 的文档头注释。Windows 用户使用 PowerShell 安装器。

脚本会自动完成 CLI 安装（优先 Homebrew，fallback 到 GitHub Releases 二进制下载）、Git 仓库 clone、Docker 环境检查、以及整个服务栈启动。如果只需要 CLI 而服务已在别处运行，可以省略 `--with-server`。

登录与首次使用

服务启动后，打开 `http://localhost:3000`（或配置的域名）。默认没有配置邮件服务时，验证码打印在 backend 容器日志中：

```
docker compose -f docker-compose.selfhost.yml logs backend | grep "Verification code"
```

生产环境建议在 `.env` 中配置 `RESEND_API_KEY`，验证码通过邮件发送。不要在公网实例上设置 `MULTICA_DEV_VERIFICATION_CODE`——该固定验证码仅在 `APP_ENV=development` 时生效，且任何知道邮箱的人都能用它登录。

来源：[SELF_HOSTING.md](#)

连接执行电脑

执行电脑是开发者实际工作的机器。在上面安装 Multica CLI 和至少一款 AI 编程工具，然后通过 `multica setup self-host` 连接到自部署服务。

CLI 安装

CLI 支持三种安装方式：

方式	命令
Homebrew（推荐）	<code>brew install multica-ai/tap/multica</code>
安装脚本	<code>curl -fsSL https://raw.githubusercontent.com/multica-ai/multica/main/scripts/install.sh bash</code>
源码构建	<code>git clone</code> → <code>make build</code> → 将 <code>server/bin/multica</code> 放入 PATH

来源：[CLI_AND_DAEMON.md](#)

Homebrew 安装后可通过 `brew upgrade multica-ai/tap/multica` 更新。非 Homebrew 安装使用 `multica update`，该命令会自动检测安装方式并执行对应升级流程。

连接命令

如果 Multica 服务也在同一台电脑上：

```
multica setup self-host
```

如果服务在另一台机器上，传入服务地址：

```
multica setup self-host \
  --server-url https://api.example.com \
  --app-url https://app.example.com
```

`multica setup self-host` 自动完成四步操作：配置 CLI 连接地址 → 打开浏览器认证 → 发现你的工作区 → 启动后台守护进程。认证通过后，CLI 凭据保存在本地，守护进程向服务端注册为可用运行时。

确认连接状态：

```
multica daemon status
```

输出应显示 `Daemon: running`、已检测到的 AI 工具列表，以及 `Workspaces > 0`。

来源：[SELF_HOSTING.md](#)

Kubernetes 部署

如果已有 Kubernetes 集群，可通过 OCI Helm Chart 部署 Multica，替代 Docker Compose。

Chart 结构

Chart 地址为 `oci://ghcr.io/multica-ai/charts/multica`，源码位于 [deploy/helm/multica/Chart.yaml](#)。它创建以下资源：

资源	说明
<code>multica-postgres</code>	<code>pgvector/pgvector:pg17</code> ，10Gi PVC
<code>multica-backend</code>	Go API/WebSocket 服务，5Gi <code>ReadWriteOnce</code> 上传 PVC（配置 S3 后可禁用）
<code>multica-frontend</code>	Next.js 独立服务
两个 Ingress	分别为 <code>frontend</code> 和 <code>backend</code> 主机名
<code>multica-config</code> ConfigMap	从 <code>values.yaml</code> 渲染

来源：[SELF_HOSTING.md](#)

Chart 针对 `k3s + Traefik + local-path StorageClass` 设计，使用默认 `ReadWriteOnce` 存储，在大多数集群上只需小幅调整即可运行。

基本部署流程

```
# 创建命名空间
kubectl create namespace multica

# 登录 GHCR (如需要)
helm registry login ghcr.io

# 安装
helm install multica oci://ghcr.io/multica-ai/charts/multica \
  --namespace multica \
  -f my-values.yaml
```

安装后，执行电脑同样通过 `multica setup self-host` 指向 Ingress 主机名：

```
multica setup self-host \
  --server-url http://api.multica.dev.lan \
  --app-url http://multica.dev.lan
```

镜像版本管理

Chart 默认使用 `latest` 标签跟踪最新镜像。升级到特定版本时，升级到匹配的 Chart 版本即可——发布版 Chart 的 `appVersion` 对应 Git tag，镜像标签自动匹配：

```
helm upgrade multica oci://ghcr.io/multica-ai/charts/multica \
  --version <chart-version> \
  -n multica \
  -f my-values.yaml
```

也可在 `values.yaml` 中独立覆盖镜像标签：

```
images:
  backend:
    tag: v0.2.4
  frontend:
    tag: v0.2.4
```

回滚命令为 `helm -n multica rollback multica`。

来源：[deploy/helm/multica/values.yaml](#) 和 [SELF_HOSTING.md](#)

桌面应用连接自托管

Multica Desktop 默认连接 Multica Cloud。连接自托管实例时，需要创建 `desktop.json` 配置文件：

平台	路径
macOS	<code>/Users/<用户名>/.multica/desktop.json</code>
Linux	<code>/home/<用户名>/.multica/desktop.json</code>
Windows	<code>C:\Users\<用户名>\.multica\desktop.json</code>

```
{
  "schemaVersion": 1,
  "apiUrl": "https://api.example.com"
}
```

`apiUrl` 是必填项。Desktop 会自动从 `apiUrl` 推导 `wsUrl` 和 `appUrl`：WebSocket 地址由 API URL 的协议替换为 `ws / wss` 并追加 `/ws` 路径；Web 地址在域名以 `api.` 开头时去掉该前缀。推导不符合预期时可显式覆盖三个字段。

注意这个文件**不是** CLI 的 `~/.multica/config.json` ——Desktop 在 `~/.multica/profiles/desktop-<host>/` 下管理独立的 daemon profile，不会读取 CLI 配置。修改 `config.json` 不影响 Desktop。

保存后重启 Desktop，配置仅在启动时读取。文件查找失败时 Desktop 静默回退到 Cloud；文件存在但格式无效时则显示配置错误。

配置与环境变量

自部署的行为由 `.env` 文件中的环境变量控制。核心变量包括：

- **JWT_SECRET**：JWT 签名密钥，`make selfhost` 自动生成随机值
- **POSTGRES_PASSWORD**：PostgreSQL 密码，首次启动自动生成
- **MULTICA_VCS_SECRET_KEY**：自托管 Git 集成的加密密钥
- **RESEND_API_KEY**：配置后通过邮件发送验证码，为空时验证码打印在 backend 日志
- **APP_ENV**：Docker Compose 部署默认为 `production`，不会启用固定验证码
- **FRONTEND_ORIGIN / MULTICA_APP_URL / MULTICA_PUBLIC_URL**：反向代理后的公开地址
- **ALLOW_SIGNUP / DISABLE_WORKSPACE_CREATION / GOOGLE_CLIENT_ID**：签名和工作区创建控制

修改 `.env` 后必须 `docker compose -f docker-compose.selfhost.yml up -d` 重建容器；`restart` 不重新读取 `.env`。

详细环境变量列表和高级配置（手动部署、反向代理、数据库设置等）参见 [SELF_HOSTING_ADVANCED.md](#)。

守护进程自更新

CLI 启动的守护进程会定期比较自身编译版本与当前 `multica` 二进制文件的 `--version` 输出。两者不一致时——如通过 `brew upgrade`、重新下载二进制或本地 `make build` 更新了 CLI——守护进程等待所有正在运行的任务完成后自动重启到新版本。运行中的任务不会被中断；如果守护进程正忙，重启推迟到下一次检查周期。

这个机制独立于 GitHub 自更新轮询器。关闭该行为：

```
MULTICA_DAEMON_AUTO_RELOAD=0 multica daemon start
# 或
multica config set disable_auto_reload true
```

AI 编程工具自身的升级处理方式不同：当工具原地升级后，守护进程会重新探测其版本并重新注册运行时，**无需重启**，后续任务即刻使用新版 CLI。

来源：[CLI_AND_DAEMON.md](#)

升级与维护

Docker Compose 升级

升级到最新发布镜像：

```
cd multica
git pull --ff-only
docker compose -f docker-compose.selfhost.yml pull
docker compose -f docker-compose.selfhost.yml up -d
curl -fsS http://localhost:8080/readyz
```

`git pull` 更新的是编排文件本身（新增环境变量、健康检查等），不是拉取新版本 Multica 的机制。真正决定运行版本的是 `docker compose pull` ——它查询 GHCR 上该 tag 当前指向的镜像。

`make selfhost` 也可以用于升级：它执行同一套 `pull + up -d` 流程，额外在 `.env` 缺失时生成并等待 `/health` 就绪。

要锁定特定版本，在 `.env` 中设置 `MULTICA_IMAGE_TAG=v0.2.4`。如果所选 tag 尚未发布到 GHCR，回退到 `make selfhost-build` 从本地源码构建。

常用管理命令

```
# 查看容器状态
docker compose -f docker-compose.selfhost.yml ps

# 查看 backend 日志
docker compose -f docker-compose.selfhost.yml logs -f backend

# 停止服务，保留数据卷
docker compose -f docker-compose.selfhost.yml down

# 停止一切（含守护进程）
make selfhost-stop
multica daemon stop
```

切换到 Multica Cloud

如果从自托管切换回 Multica Cloud：

```
multica setup
```

这会重新配置 CLI 指向 `multica.ai`，重新认证并重启守护进程。本地的 Docker 服务不受影响，需单独停止。

来源： [SELF_HOSTING.md](#)

手动部署（分步执行）

如果不使用 `make selfhost` 封装，可手动执行每一步：

```
git clone https://github.com/multica-ai/multica.git
cd multica
cp .env.example .env
# 编辑 .env，至少修改 JWT_SECRET
docker compose -f docker-compose.selfhost.yml pull
docker compose -f docker-compose.selfhost.yml up -d
```

手动配置 CLI（替代 `multica setup`）：

```
multica config set server_url http://localhost:8080
multica config set app_url http://localhost:3000
multica login
multica daemon start
```

生产环境使用 TLS 时，将地址替换为 HTTPS 域名。

来源： [SELF_HOSTING.md](#)

存储与文件上传

自部署默认使用本地文件存储。Backend 容器挂载持久化数据卷用于上传文件。配置 S3（包括 MinIO 等兼容 S3 的自定义端点）后，上传走 S3，本地存储作为 fallback。Kubernetes 部署中，设置

`backend.config.s3Bucket` 后可将 `backend.uploads.persistence.enabled` 设为 `false`，不再声明上传 PVC。

[deploy/helm/multica/values.yaml](#) 中明确了 PVC 的 `accessModes` 默认为 `ReadWriteOnce`，对应单节点部署；多副本场景下如需共享上传卷，需切换为 `ReadWriteMany` 并配置兼容的 `StorageClass`（NFS、EFS、CephFS）。

与相关模块的关系

自部署是连接 Multica 平台侧与执行侧的桥梁，依赖以下模块协同工作：

- **CLI 命令行工具**： `multica` CLI 负责安装、认证、守护进程管理和工作区发现。 `multica setup self-host` 一条命令完成 CLI 配置、浏览器登录和守护进程启动的全流程。
- **智能体运行时**：守护进程检测本机 AI 编程工具（Claude Code、Codex、Cursor 等 20 种 CLI），向服务端注册运行时，轮询任务并执行。支持会话恢复、超时控制和 MCP 配置。
- **版本控制集成**：自部署环境支持 Forgejo、Gitea、GitLab 作为 Git 提供商。 `MULTICA_VCS_SECRET_KEY` 是自托管 Git 集成的加密密钥，由 `make selfhost` 自动生成。

CLI 命令行工具

detail · cli-tool.md

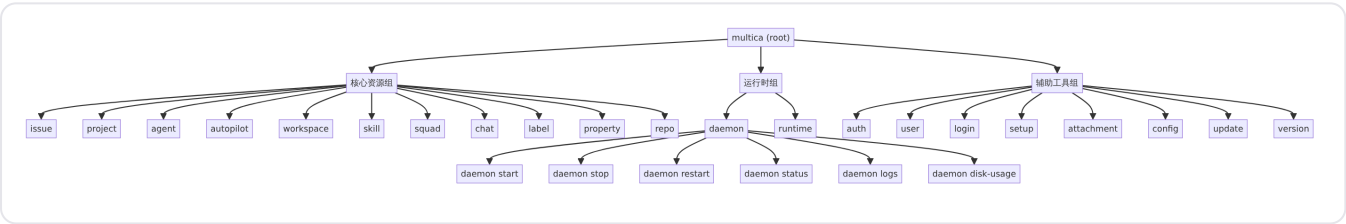
CLI 命令行工具

Multica CLI（可执行文件 `multica`）是将本地机器连接到 Multica 平台的核心客户端工具。它承担三项关键职责：**认证与配置**（登录、Token 管理、服务地址配置）、**工作区与资源管理**（工作区、Issue、智能体、技能、自动化规则等资源的命令行操作），以及**本地守护进程生命周期管理**（启动、停止、重启、状态查询、日志查看）。一条 `multica setup` 命令即可完成从配置到守护进程启动的全流程。

CLI 基于 github.com/spf13/cobra 命令框架构建，入口位于 [server/cmd/multica/main.go](https://github.com/multica-ai/multica/blob/main/server/cmd/multica/main.go)。所有命令按功能分为三个逻辑组：核心资源组（`groupCore`）、运行时组（`groupRuntime`）和辅助工具组（`groupAdditional`）。

命令架构

CLI 的顶层命令树由 `rootCmd` 统一注册，每个子命令以独立的 Go 源文件实现。命令分为三个逻辑分组，在 Help 输出中有序展示：



命令分组和注册位于 [server/cmd/multica/main.go](https://github.com/multica-ai/multica/blob/main/server/cmd/multica/main.go)，全局持久化标志 `--server-url`、`--workspace-id`、`--profile` 和 `--debug` 对所有子命令生效。

安装

Multica CLI 提供三种安装途径：

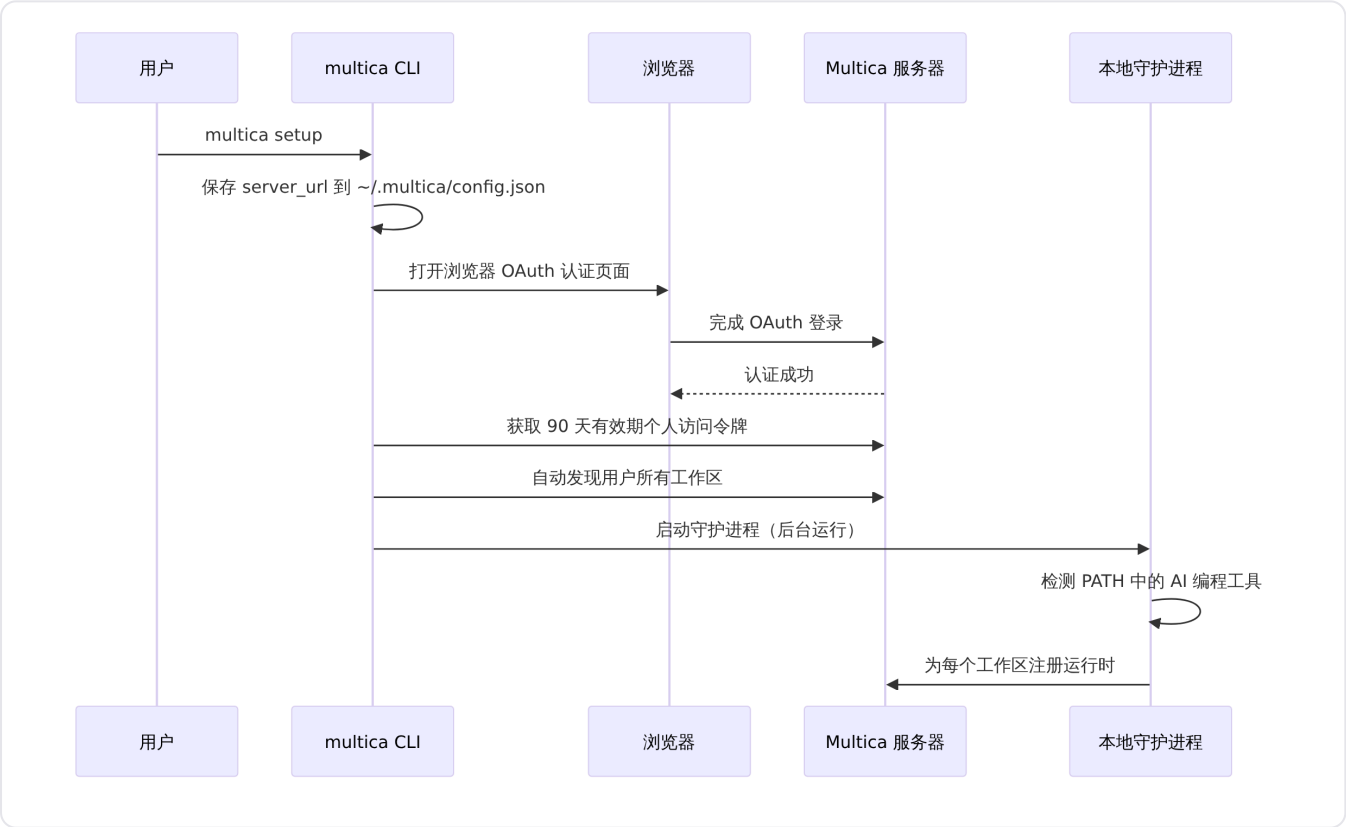
途径	命令	适用平台
Shell 安装脚本	<code>curl -fsSL https://raw.githubusercontent.com/multica-ai/multica/main/scripts/install.sh bash</code>	macOS / Linux
Homebrew	<code>brew install multica-ai/tap/multica</code>	macOS / Linux

途径	命令	适用平台
PowerShell 安装脚本	<code>irm https://raw.githubusercontent.com/multica-ai/multica/main/scripts/install.ps1 iex</code>	Windows
源码构建	<code>make build</code> 然后 <code>cp server/bin/multica /usr/local/bin/multica</code>	所有平台

安装脚本的详细参考见 [CLI_AND_DAEMON.md](#)。安装完成后运行 `multica version` 确认可用，版本命令实现于 [server/cmd/multica/cmd_version.go](#)，支持 `--output json` 输出机器可读的版本信息（包含 version、commit、date、go 运行时版本和 os/arch）。

首次连接：`multica setup`

`multica setup` 是面向新用户的"一键配置"命令，将配置、认证和守护进程启动合并为一条命令。



`setup` 命令定义于 [server/cmd/multica/cmd_setup.go](#)。对于自托管实例，使用子命令：

```
multica setup self-host \
  --server-url https://api.example.com \
  --app-url https://app.example.com
```

`setup self-host` 默认连接 `http://localhost:8080`（后端）和 `http://localhost:3000`（前端）。通过 `--callback-host` 可以指定浏览器回连本机的地址，适用于 SSH 远程执行场景。

认证体系

认证相关的命令包括 `login`、`auth status` 和 `auth logout`。

Browser OAuth 登录

```
multica login
```

打开浏览器完成 OAuth 认证，自动创建 90 天有效期的个人访问令牌，并自动发现用户所属的所有工作区加入守护进程监视列表。

Token 登录（无浏览器环境）

```
multica login --token
```

命令以交互方式提示粘贴令牌，完整值不会写入 Shell 历史记录。也可以直接传入 `--token=<mul_...>`，但推荐使用交互方式确保安全。详细参数见 [CLI_AND_DAEMON.md](#)。

状态检查与登出

```
multica auth status      # 显示当前服务器、用户和令牌有效性
multica auth logout      # 移除存储的认证令牌
```

工作区管理

```
multica workspace list      # 列出可访问的工作区
multica workspace switch <slug> # 切换默认工作区
multica workspace member invite teammate@example.com      # 邀请成员
multica workspace member invite admin@example.com --role admin # 邀请管理员
```

切换后的默认工作区会持久化到配置中。单次命令可通过 `--workspace-id` 临时覆盖，也可设置环境变量 `MULTICA_WORKSPACE_ID`。

Issue 操作

CLI 提供完整的 Issue 生命周期管理：

操作	命令示例
列表	<code>multica issue list</code>
查看	<code>multica issue get MUL-123</code>
搜索	<code>multica issue search "关键词"</code>
创建	<code>multica issue create --title "标题"</code>

操作	命令示例
状态变更	<code>multica issue status MUL-123 in_progress</code>
分配	<code>multica issue assign MUL-123 --to "智能体名"</code>
评论列表	<code>multica issue comment list MUL-123</code>
添加评论	<code>multica issue comment add MUL-123 --content "内容"</code>
执行记录	<code>multica issue runs MUL-123</code>
执行消息	<code>multica issue run-messages <task-id> --issue MUL-123</code>
取消任务	<code>multica issue cancel-task <task-id> --issue MUL-123</code>

长文本内容（描述、评论）可从 stdin 读取，避免 Shell 转义问题：

```
multica issue create --title "标题" --description-stdin < notes.md
multica issue comment add MUL-123 --content-stdin < review.md
```

智能体与技能管理

```
multica agent list           # 列出智能体
multica agent get <agent-id> # 查看智能体详情
multica agent create --help  # 创建智能体（查看参数）
multica agent update <agent-id> --help # 更新智能体（查看参数）

multica skill list          # 列出技能
multica skill get <skill-id> # 查看技能详情
multica skill import --url <url> # 从 URL 导入技能
multica agent skills add <agent-id> --skill-ids <skill-id> # 绑定技能
```

技能导入时的冲突处理策略通过 `--on-conflict` 控制，可选值为 `overwrite`（仅允许技能创建者使用，保留原 ID 与智能体绑定）、`rename` 或 `skip`。

守护进程管理

守护进程是本地智能体运行时的核心。其命令定义于 [server/cmd/multica/cmd_daemon.go](#)，包含七个子命令：

子命令	用途
<code>daemon start</code>	启动守护进程
<code>daemon stop</code>	停止守护进程
<code>daemon restart</code>	重启守护进程（等价于 stop + start）
<code>daemon status</code>	查看守护进程状态

子命令	用途
daemon logs	查看守护进程日志
daemon disk-usage	查看工作区磁盘使用情况
daemon probe-runtimes	探测本地运行时（隐藏命令，供桌面应用使用）

启动参数

daemon start 支持丰富的运行时配置标志，亦可全部通过环境变量设置：

标志	环境变量	默认值	说明
--foreground	—	false	前台运行（便于调试）
--daemon-id	MULTICA_DAEMON_ID	hostname	守护进程唯一标识
--device-name	MULTICA_DAEMON_DEVICE_NAME	hostname	设备可读名称
--runtime-name	MULTICA_AGENT_RUNTIME_NAME	Local Agent	运行时显示名称
--poll-interval	MULTICA_DAEMON_POLL_INTERVAL	3s	任务轮询间隔
--heartbeat-interval	MULTICA_DAEMON_HEARTBEAT_INTERVAL	15s	心跳间隔
--agent-timeout	MULTICA_AGENT_TIMEOUT	0（无上限）	单任务绝对超时
--max-concurrent-tasks	MULTICA_DAEMON_MAX_CONCURRENT_TASKS	20	最大并发任务数
--no-auto-update	MULTICA_DAEMON_AUTO_UPDATE=false	false	禁用周期自更新
--auto-update-interval	MULTICA_DAEMON_AUTO_UPDATE_INTERVAL	—	自更新检查间隔
--no-auto-reload	MULTICA_DAEMON_AUTO_RELOAD=false	false	禁用检测到二进制版本变化后自动重启

配置表格来源：[CLI_AND_DAEMON.md](#) 和 [server/cmd/multica/cmd_daemon.go](#)。

守护进程运行机制

守护进程的工作流程如下：

1. **启动时**：检测 PATH 中受支持的 AI 编程工具，为每个被监视的工作区注册运行时。
2. **轮询**：以可配置间隔（默认 3 秒）向服务器轮询已领取的任务。
3. **任务执行**：收到任务后创建隔离的工作区目录，启动对应的智能体 CLI，流式回传结果。
4. **心跳**：定期（默认 15 秒）发送心跳，服务器据此判断运行时在线状态。

5. 关闭时：注销所有运行时。

守护进程至少需要检测到一款内置支持的 AI 编程工具才能启动。启动时必须在服务器端已完成认证——`daemon start` 在执行前会进行认证前置检查。自定义运行时配置在守护进程启动后才同步，因此不能作为一台空白机器上的唯一启动条件。

二进制自动跟随 (Auto-Reload)

CLI 启动的守护进程会定期比较自身编译时版本与当前 `multica` 二进制文件的 `--version` 输出。当二者不一致时（例如通过 `brew upgrade multica`、重新下载或本地 `make build`），守护进程会等待所有运行中的任务完成后自动重启到新版本，不会中断正在执行的任务。这一机制与基于 GitHub 的自更新轮询器是独立的，禁用后者不会阻止守护进程检测到手动安装的新二进制。

Agent CLI（如 `codex`、`claude` 等）原地升级时，守护进程会重新探测版本并重新注册运行时，**无需重启自身**，后续任务将使用新版本的 CLI。

桌面应用管理的守护进程忽略这两种机制，因为桌面应用拥有其内置 CLI 的完整生命周期控制权。详见 [CLI_AND_DAEMON.md](#)。

工作区垃圾回收

守护进程定期扫描 `MULTICA_WORKSPACES_ROOT`（默认为 `~/multica_workspaces`），按四种模式回收磁盘空间：

模式	触发条件	说明
完整任务清理	Issue 状态为 <code>done / cancelled</code> ，空闲超过 <code>MULTICA_GC_TTL</code> （默认 24h）	移除整个任务目录
孤儿清理	任务目录缺少 <code>.gc_meta.json</code> ，超过 <code>MULTICA_GC_ORPHAN_TTL</code> （默认 72h）	清理守护进程崩溃遗留
仅清理构建产物	任务已完成超过 <code>MULTICA_GC_ARTIFACT_TTL</code> （默认 12h），但 Issue 未关闭	移除匹配 <code>MULTICA_GC_ARTIFACT_PATTERNS</code> 的目录
仓库缓存淘汰	仓库不再被任何工作区使用、无剩余工作树、超过 <code>MULTICA_GC_REPO_TTL</code> （默认 30d）	回收共享 Git 对象存储

来源：[CLI_AND_DAEMON.md](#)。

运行时管理

运行时（Runtime）代表工作区可以使用的具体执行环境。`runtime` 子命令提供：

```
multica runtime list                # 列出运行时
multica runtime rename <id> "新名称" # 重命名运行时
multica runtime usage <runtime-id>   # 查看使用情况
multica runtime activity <runtime-id> # 查看活动
multica runtime delete <id>          # 删除运行时（默认拒绝仍有活跃智能体绑定的运行时）
multica runtime delete <id> --cascade # 级联删除：解绑智能体、保留配置与历史、取消活跃任务
```

运行时配置文件（Profile）管理子命令：

```
multica runtime profile list
multica runtime profile create --help
multica runtime profile set-path <profile-id> --path /absolute/path/to/command
multica runtime profile unset-path <profile-id>
```

配置系统

CLI 配置持久化在 `~/.multica/config.json` 中，核心字段定义于 [server/internal/cli/config.go](#)：

字段	说明
<code>server_url</code>	Multica 服务器地址
<code>app_url</code>	Web 应用地址
<code>workspace_id</code>	默认工作区 ID

配置管理命令：

```
multica config show                # 显示当前配置
multica config set <key> <value>  # 设置配置项
```

`--profile` 标志支持创建隔离的配置环境（如 `--profile staging`），每个 profile 在 `~/.multica/profiles/<name>/` 下拥有独立的配置、守护进程状态和工作区数据。桌面应用在 `~/.multica/profiles/desktop-<host>/` 下管理独立的守护进程 profile，不会读取或覆盖 CLI 的默认配置。

自更新

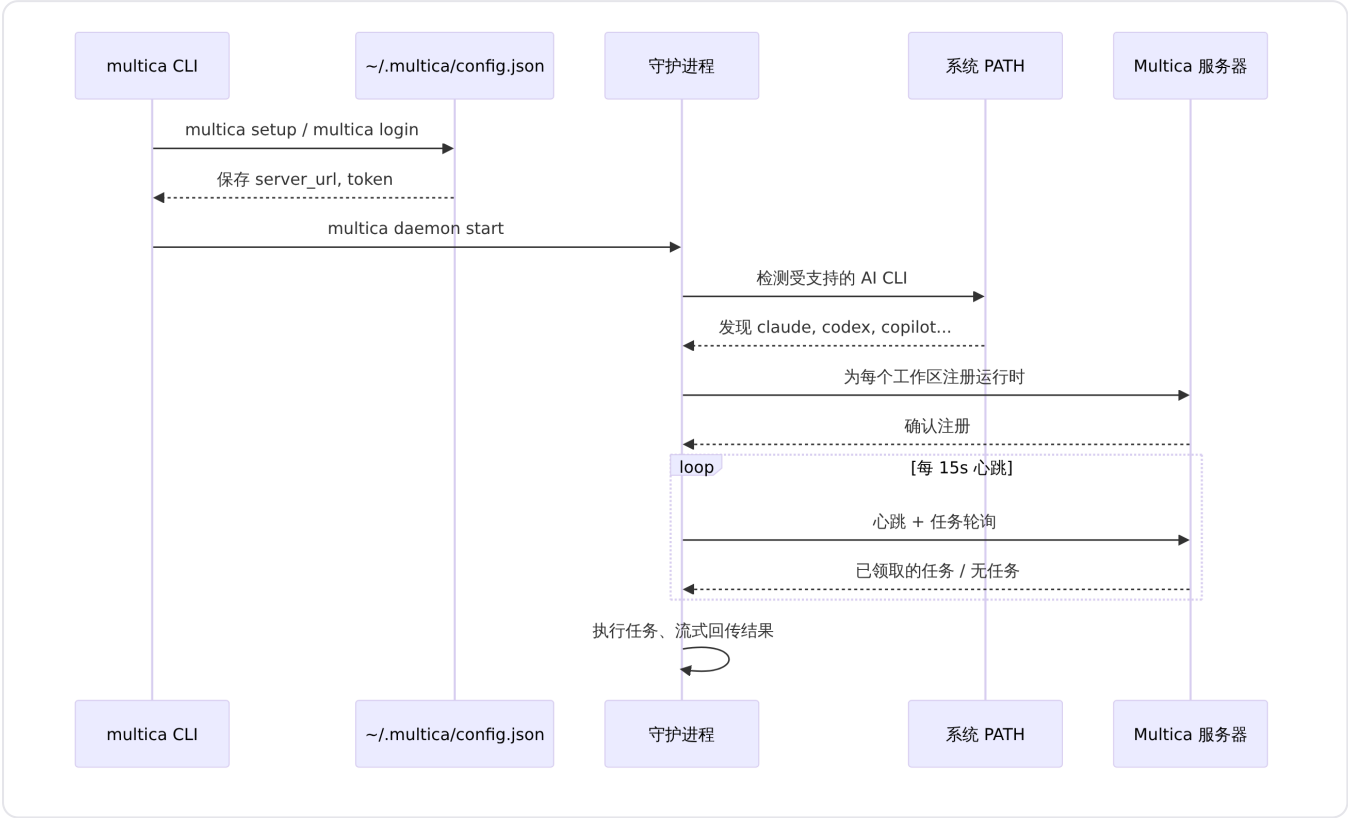
`multica update` 命令自动检测安装方式并执行相应升级策略。实现位于 [server/internal/cli/update.go](#)：通过 GitHub Releases API 获取最新版本，匹配当前操作系统和架构的构建产物，经过 SHA256 校验和验证后下载并替换二进制文件。

守护进程内置的自更新轮询器会定期检查 GitHub Release，版本比较逻辑由 `IsNewerVersion` 和 `IsReleaseVersion` 函数实现——仅当编译版本为正式发布版本（如 `v0.1.13`）时才执行自更新，源码构建版本不会降级到公开发布版本。

Profile 与执行路径解析

CLI 通过 `server/internal/selfexec/selfexec.go` 解析当前可执行文件路径。优先使用 `os.Executable()` 返回的路径；当环境无法提供时（如某些启动器），回退到 `argv[0]` 通过 `exec.LookPath` 查找。这一机制确保守护进程在自动重载或重新执行时能正确找到自身。

守护进程与智能体运行时的关系



守护进程与服务器的交互机制详见 [智能体运行时](#) 页面。运行时的在线/离线状态、并发限制和自定义配置见 [守护进程与运行时](#) 文档。

命令概览

命令	用途
setup	一键配置、认证并启动守护进程
login	浏览器或 Token 登录
auth	认证状态查看与登出
workspace	工作区列表与切换、成员邀请
issue	Issue 创建、更新、分配、搜索、评论、执行记录管理
agent	智能体列表、详情、创建与更新
skill	技能列表、详情与导入
squad	小队管理
chat	智能体对话

命令	用途
autopilot	自动化规则管理
project	项目管理
repo	版本控制仓库管理
label	标签管理
property	自定义属性管理
daemon	守护进程启动、停止、重启、状态、日志、磁盘使用
runtime	运行时列表、重命名、删除、使用情况、Profile 管理
attachment	附件上传与下载
config	CLI 配置查看与修改
update	自更新到最新版本
version	打印版本信息

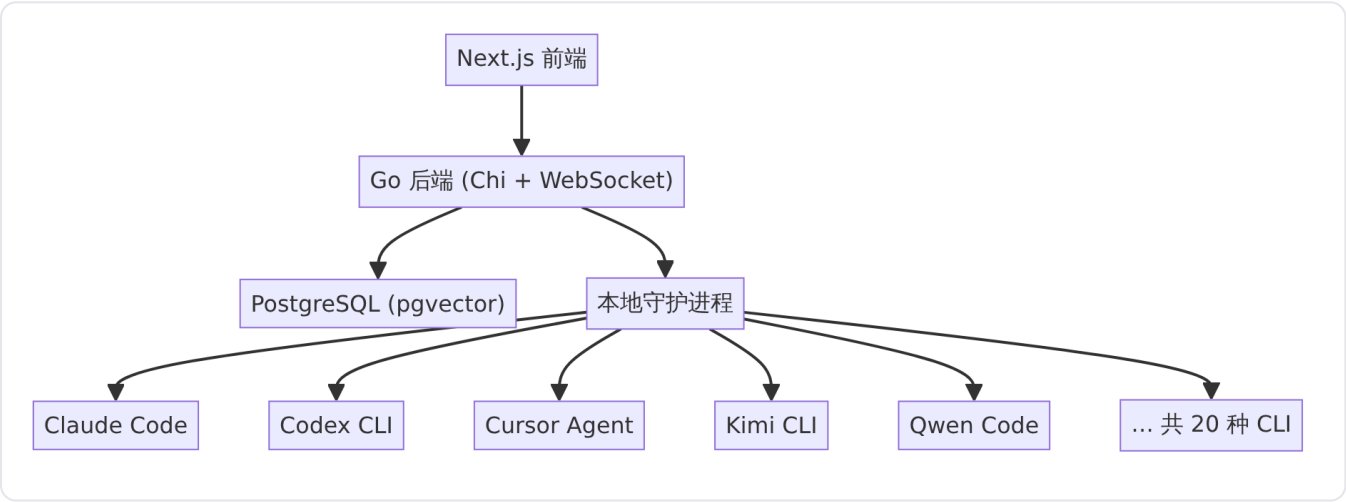
智能体运行时

detail · agent-runtime.md

智能体运行时

智能体运行时是 Multica 将人类意图转化为本地机器上具体开发工作的核心桥梁。它由两部分紧密协作的能力构成：**守护进程驱动的统一后端执行**，负责在本机拉起 20 种 AI 编程 CLI 并管理任务的全生命周期；**模型与供应商管理**，负责从各 CLI 动态发现可用模型列表，按供应商分组呈现，供用户在创建或编辑智能体时选择。

Multica 不自带模型，也不托管任何 LLM。它驱动的是用户早已在本机安装、登录好的那些 AI 编程 CLI——Claude Code、Codex、Cursor、Kimi 等——因此切换提供方只需在下拉框中选一下，不存在迁移成本。本地守护进程跑在用户的机器上，紧挨着代码仓库，通过 WebSocket 从 Multica 平台接收任务，拉起对应的 CLI 执行，并实时回传执行过程中的流式消息。



来源：[server/internal/daemon/daemon.go](#) 和 [README.zh.md](#) 中的架构图。

支持的智能体 CLI

守护进程在启动时自动扫描 PATH 上的 20 种 AI 编程 CLI，为检测到的每一种注册为独立的运行时。注册后的运行时在工作区的运行时页面上显示为在线，可以被分配给任意智能体。

工具	CLI 命令	启动骨架
Antigravity	agy	agy -p (non-interactive)
Claude Code	claude	claude (stream-json)
CodeBuddy Code	codebuddy	codebuddy (stream-json)
Codex CLI	codex	codex app-server
GitHub Copilot CLI	copilot	copilot (json)

工具	CLI 命令	启动骨架
Cursor Agent	<code>cursor-agent</code>	<code>cursor-agent (stream-json)</code>
DevEco Code	<code>deveco</code>	<code>deveco run (json)</code>
Grok Build	<code>grok</code>	<code>grok agent stdio</code>
Hermes Agent	<code>hermes</code>	<code>hermes acp</code>
Kimi CLI	<code>kimi</code>	<code>kimi acp</code>
Kiro CLI	<code>kiro-cli</code>	<code>kiro-cli acp</code>
OpenClaw	<code>openclaw</code>	<code>openclaw agent (json)</code>
OpenCode	<code>opencode</code>	<code>opencode run (json)</code>
Pi	<code>pi</code>	<code>pi (json mode)</code>
Qoder CLI	<code>qodercli</code>	<code>qodercli --acp</code>
Qoder CN	<code>qoderclcn</code>	<code>qoderclcn --acp</code>
Qwen Code	<code>qwen</code>	<code>qwen -p (stream-json)</code>
QwenPaw	<code>qwenpaw</code>	<code>qwenpaw acp</code>
Reasonix	<code>reasonix</code>	<code>reasonix acp</code>
TRAE CLI	<code>traecli</code>	<code>traecli acp serve</code>

来源：[server/pkg/agent/agent.go](#) 中 `SupportedTypes` 列表和 [server/pkg/agent/agent.go](#) 中 `launchHeaders` 映射。

每款 CLI 有最低版本要求：Claude Code 需 2.0.0+，Codex 需 0.100.0+，Copilot 需 1.0.0+，Grok 需 0.2.89+，Qwen Code 需 0.20.0+。低于最低版本时守护进程不会注册对应运行时。

守护进程任务执行生命周期

守护进程是 Multica 架构中"跑在用户机器上"的部分。它通过 WebSocket 与平台后端保持长连接，以统一的机器级批量领取（batch claim）方式从服务器获取任务，然后将每个任务路由到对应运行时的 CLI 进程中执行。

运行时注册

守护进程启动时，首先检测本机 `PATH` 上安装的所有内置 CLI，通过 `--version` 获取版本号，并为每个通过了最低版本检查的 CLI 构造注册条目。注册请求携带工作区 ID、守护进程 ID、设备名称、CLI 版本以及运行时列表（名称、类型、版本、状态），发送到服务器。服务器返回注册的运行时 ID 和仓库绑定信息。

如果本机没有任何内置 CLI 可注册，守护进程不会为该工作区注册任何运行时。自定义运行时配置在守护进程启动之后同步，不能作为空白机器上的唯一启动条件。

来源：[server/internal/daemon/daemon.go](#) 中 `detectBuiltinRuntimes` 和 [server/internal/daemon/daemon.go](#) 中注册函数。

任务领取与分派

守护进程运行单一机器级批量轮询循环（`runBatchPoller`）。每一轮中，它先获取当前空闲的执行槽位数，然后以一次 HTTP 请求从服务器领取最多 `max_tasks`（上限 32）个已分派的待执行任务。每个返回的任务携带其目标 `runtime_id`，守护进程检查本地是否仍追踪该运行时——若运行时在领取与分派之间的窗口内下线，任务会以 `runtime_offline` 原因退回服务器进行重试。

来源：[server/internal/daemon/daemon.go](#) 中批量轮询循环和 [server/internal/daemon/daemon.go](#) 中 `handleTask` 入口。

任务准备阶段

`handleTask` 收到任务后进入准备阶段，受 `defaultTaskPrepareTimeout`（5 分钟）硬性截止保护。准备阶段的步骤包括：

- 1. **本地目录锁**：如果任务目标是一个 `local_directory` 类型的项目资源，先获取路径互斥锁。
- 2. **环境根目录保护**：标记执行环境的根目录为活跃，防止 GC 循环在任务中途回收。
- 3. **任务取消监听**：启动轮询 goroutine 监控服务器端任务状态变化（取消、完成、删除），收到信号即取消执行上下文。
- 4. **技能包下载**：为智能体配置的所有技能下载技能包（`skill bundle`），下载失败返回 `skill_bundle_unavailable` 可重试错误。
- 5. **执行环境准备**：创建隔离的工作目录（克隆仓库、写入上下文文件 `CLAUDE.md / AGENTS.md / CODEBUDDY.md / QWEN.md`），或复用上次任务的工作目录以实现会话恢复。
- 6. **MCP 配置生成**：对于支持 MCP 的提供方，将智能体的 MCP 配置落地为 CLI 可读取的配置文件。

来源：[server/internal/daemon/daemon.go](#) 中 `handleTask` 及 [server/internal/daemon/daemon.go](#) 中 `runTask` 的准备逻辑。

超时控制

守护进程支持多层超时机制：

超时类型	配置项	作用
任务准备超时	<code>defaultTaskPrepareTimeout</code> （5 分钟）	从领取到 <code>StartTask</code> 成功的硬性上限
执行超时	<code>ExecOptions.Timeout</code>	智能体 CLI 进程的总墙钟时间；零值关闭超时，仅依靠非活跃看门狗
语义非活跃超时	<code>ExecOptions.SemanticInactivityTimeout</code>	无有意义的语义输出时中止执行
空闲看门狗	<code>ExecOptions.IdleWatchdogTimeout</code>	进程无消息输出的超时，可覆盖守护进程全局看门狗但不可延长

超时类型	配置项	作用
握手上限	<code>ExecOptions.HandshakeTimeout</code>	长连接协议提供方的启动 RPC 超时 (当前仅 Codex app-server 使用)

`runContext` 函数在 `Timeout > 0` 时创建带截止时间的上下文；零值或负值仅使用 `context.WithCancel`，将存活性完全交给非活跃看门狗，这样持续产出的长任务不会被超时中断。

来源：[server/pkg/agent/agent.go](#) 中 `ExecOptions` 和 [server/pkg/agent/agent.go](#) 中 `runContext`。

会话恢复

守护进程支持在同一个 Issue 的连续任务之间恢复智能体会话。当任务携带 `PriorSessionID` 时，`ExecOptions.ResumeSessionID` 被设置为该值，后端将尝试恢复上一次对话。`ResumeExpected` 字段记录"本次任务意图延续对话"这一事实，供后端在恢复被拒绝时（例如会话已过期）发出连续性提示。

部分后端无法检测会话恢复拒绝：`antigravity`、`copilot`、`cursor`、`deveco`、`opencode` 被记录在 `resumeRejectionUndetectable` 映射中——它们的恢复拒绝信号无法从 CLI 输出中可靠解析，因此 `ResumeRejected` 为 `false` 时对它们仅表示"无法判断"，而非"确认未被拒绝"。

来源：[server/pkg/agent/agent.go](#) 中恢复相关字段和 [server/pkg/agent/agent.go](#) 中 `resumeRejectionUndetectable`。

统一后端接口

`agent` 包提供统一的后端接口，屏蔽 20 种 CLI 在协议、传输、参数格式上的差异。所有后端实现同一个 `Backend` 接口：

- `Execute(ctx, prompt, opts) → (*Session, error)`：执行一个提示词，返回流式会话对象。

调用方从 `Session.Messages` 频道读取中间事件，在 `Session.Result` 频道上等待最终结果。

执行选项

`ExecOptions` 结构体定义了每次执行的完整参数集：

字段	说明
<code>Cwd</code>	工作目录
<code>Model</code>	模型标识符；空字符串表示使用 CLI 自身默认值
<code>SystemPrompt</code>	内联系统提示词，仅少数提供方使用（见下文）
<code>ThreadName</code>	语义化的会话/线程名称
<code>MaxTurns</code>	最大对话轮数
<code>Timeout</code>	进程墙钟超时，零值关闭
<code>SemanticInactivityTimeout</code>	语义非活跃超时

字段	说明
IdleWatchdogTimeout	消息非活跃看门狗
HandshakeTimeout	启动协议握手超时
ResumeSessionID	恢复会话标识符
ResumeExpected	意图延续对话标志
ResumeContinuityNotice	恢复失败时的连续性提示文本
ExtraArgs	守护进程全局默认 CLI 参数
CustomArgs	每智能体自定义 CLI 参数（追加在 ExtraArgs 之后）
QwenpawWorkspace	QwenPaw 每任务工作区路径
McpConfig	MCP 服务器配置 JSON
ThinkingLevel	推理/努力级别（如 Claude 的 low/medium/high/xhigh/max ）
ServiceTier	Codex 执行服务等级（如 priority ）
OpenclawMode	OpenClaw 路由模式（ local 或 gateway ）
ClaudeSettingsPath	Claude Code 的 --settings 文件路径

系统提示词的投递策略因提供方而异：默认情况下守护进程将 Multica 运行时的任务简报写入工作目录中的上下文文件（ CLAUDE.md 、 AGENTS.md 、 CODEBUDDY.md 或 QWEN.md ），CLI 自行从磁盘读取。只有四个提供方—— openclaw 、 kimi 、 traecli 、 qwenpaw ——因其协议限制无法从磁盘读取上下文文件，守护进程才将它们列入 providerNeedsInlineSystemPrompt ，通过 ExecOptions.SystemPrompt 内联传递。

来源：[server/pkg/agent/agent.go](#) 中 ExecOptions 和 [server/internal/daemon/daemon.go](#) 中 providerNeedsInlineSystemPrompt 。

流式消息

Session.Messages 频道推送统一的 Message 结构体，涵盖七种事件类型：

类型	含义
text	文本输出
thinking	推理/思考过程
tool-use	工具调用（含工具名、CallID、Input）
tool-result	工具返回值（含 Output）
status	状态更新（含 SessionID）
error	错误消息
log	日志消息（含 Level）

来源：[server/pkg/agent/agent.go](#) 中 Message 结构体及消息类型常量。

后端工厂

`New(agentType, cfg)` 工厂函数根据类型字符串选择对应的后端实现—— `claudeBackend` 、 `codexBackend` 、 `copilotBackend` 等。 `qoder` 和 `qoderclcn` 共享同一个 `qoderBackend` 实现，通过 `defaultExecutable` 区分国际版和中国区二进制文件。

来源：[server/pkg/agent/agent.go](#) 中 `New` 函数。

MCP 配置

守护进程支持将智能体级别的 MCP（Model Context Protocol）服务器配置传递给底层 CLI。支持 MCP 配置的提供方为：

`claude` 、 `codebuddy` 、 `codex` 、 `cursor` 、 `grok` 、 `hermes` 、 `kimi` 、 `reasonix` 、 `kiro` 、 `opencode` 、 `openclaw` 、 `qoder` 、 `qoderclcn` 、 `qwen` 、 `qwenpaw` 、 `traecli`

对于不在此列表中的其他提供方，前端会隐藏 MCP 配置标签页，避免用户保存的值被运行时静默忽略。

来源：[packages/core/agents/mcp-support.ts](#) 中 `MCP_SUPPORTED_PROVIDERS` 。

执行任务时，守护进程会将智能体的 `agent.mcp_config` 与运行时的 MCP 配置合并，再传递给 `ExecOptions.McpConfig` 。对于 Cursor 等需要将 MCP 配置落盘为特定格式文件的提供方，守护进程的 `execenv` 包负责在任务工作目录中生成对应的配置文件（如 Cursor 的 `mcp.json` 和批准文件）。

思维级别调节

部分智能体后端支持在执行时指定推理/努力级别，通过 `ExecOptions.ThinkingLevel` 字段控制。当前支持思维级别调节的后端包括：

- **Claude Code** : `low` / `medium` / `high` / `xhigh` / `max` （通过 `--effort` 注入）
- **Codex CLI** : `none` / `minimal` / `low` / `medium` / `high` / `xhigh` / `max` / `ultra`
- **OpenCode** : 模型变体名称形式
- **CodeBuddy Code** : 通过 `--effort` 注入
- **Grok Build** : 通过 ACP `--effort` 注入

`ThinkingLevel` 为空时表示使用运行时/模型的默认值——后端会跳过 `--effort` 注入，让上游 CLI 自身决定。不支持的提供方静默忽略此字段。

来源：[server/pkg/agent/agent.go](#) 中 `ThinkingLevel` 字段注释。

模型与供应商管理

守护进程在运行时注册时同步发现每个 CLI 提供的 LLM 模型列表，通过 `agent.ListModels` 函数完成。发现机制因提供方而异：

- **静态目录**：Claude Code、CodeBuddy、Cursor、Kimi 等提供方使用内置的静态模型列表。
- **动态发现**：Codex CLI (`codex debug models`)、OpenCode (`opencode run --list-models`)、Pi、OpenClaw 等提供方通过调用本地 CLI 动态获取模型目录，结果以 60 秒 TTL 缓存。

模型以 `Model` 结构体表示：

字段	说明
ID	模型唯一标识符（如 <code>claude-sonnet-4-20250514</code> ）
Label	用户可见的显示名称
Provider	供应商标识，ID 为 <code>provider/model</code> 形式时用于分组
Default	显示提示：UI 为运行时宣称的首选模型打上标记。执行时不影响行为——当 <code>agent.model</code> 为空，守护进程传递 "" 给后端，由每个提供方 CLI 自己解析默认值
ServiceTiers	运行时原生执行等级（如 Codex 的 <code>priority</code> ，显示为 "Fast"）
Thinking	每模型推理级别目录。 <code>nil</code> 表示该模型无思维级别控制

来源：[server/pkg/agent/models.go](#) 中 `Model` 结构体和 [server/pkg/agent/models.go](#) 中 `ListModels` 函数。

思维级别目录是**每模型**而非每提供方的：Codex 的 `codex debug models` 本身就是每模型的，Claude 的 `--effort` 超集也有已知的每模型缺口（`xhigh` 仅 Opus 可用，`max` 仅会话可用）。前端据此在每个模型的选项旁显示可用的思维级别选择器。

服务等级

某些提供方（当前为 Codex CLI）支持运行时原生执行等级，通过 `ModelServiceTier` 表示：

字段	说明
ID	等级标识符，原样回传给提供方协议
Name	用户可见的名称（如 "Fast"）
Description	可选的描述文本

UI 将 `ServiceTiers` 渲染为模型下拉框旁的等级选择器。未携带服务等级的提供方不展示此控件。

自定义运行时

除了守护进程自动检测的 20 种内置 CLI，Multica 还支持**自定义运行时配置**（`custom runtime profiles`）。用户可以在工作区设置中指定一个自定义命令路径和协议族（`protocol_family`），该命令将

作为该协议族的后端被守护进程拉起。

自定义运行时的 `protocol_family` 必须是 `SupportedTypes` 白名单中的值——也就是说，自定义运行时只能基于 Multica 官方支持的后端协议族。守护进程在注册时检查自定义运行时的命令是否在 `PATH` 上可解析（或通过每机器命令路径覆盖），不可解析的跳过。

来源：[server/pkg/agent/agent.go](#) 中 `SupportedTypes` 注释和 [server/internal/daemon/daemon.go](#) 中 `appendProfileRuntimes`。

版本检测与自修复

守护进程在启动时通过 `--version` 检测每个内置 CLI 的版本。检测到的版本会在运行时注册时发送到服务器，纯粹用于观测——后端从不基于版本来选择行为。

当钉住的可执行文件路径因就地升级（如 Homebrew Cask 或 `nvm/fnm`）而消失时，守护进程实现**自修复**：`resolveAgentEntry` 通过登录 shell 重新解析原始命令名，记录新路径和匹配的版本，后续任务启动和模型发现都复用此结果，避免每次任务都支付一次重新探测的开销。

来源：[server/internal/daemon/daemon.go](#) 中 `resolvedPaths` 和 `healGroup` 字段的注释。

与相关模块的关系

- **技能管理**：守护进程在任务准备阶段通过 `SkillBundleCache` 下载智能体关联的技能包。下载失败会以 `skill_bundle_unavailable` 错误标记任务，触发平台端重试。[skill-management](#)
- **CLI 命令行工具**：Multica CLI 提供 `multica daemon start/stop/status/logs` 命令管理守护进程生命周期，以及 `multica setup` 一键完成配置、认证和守护进程启动的全流程。[cli-tool](#)

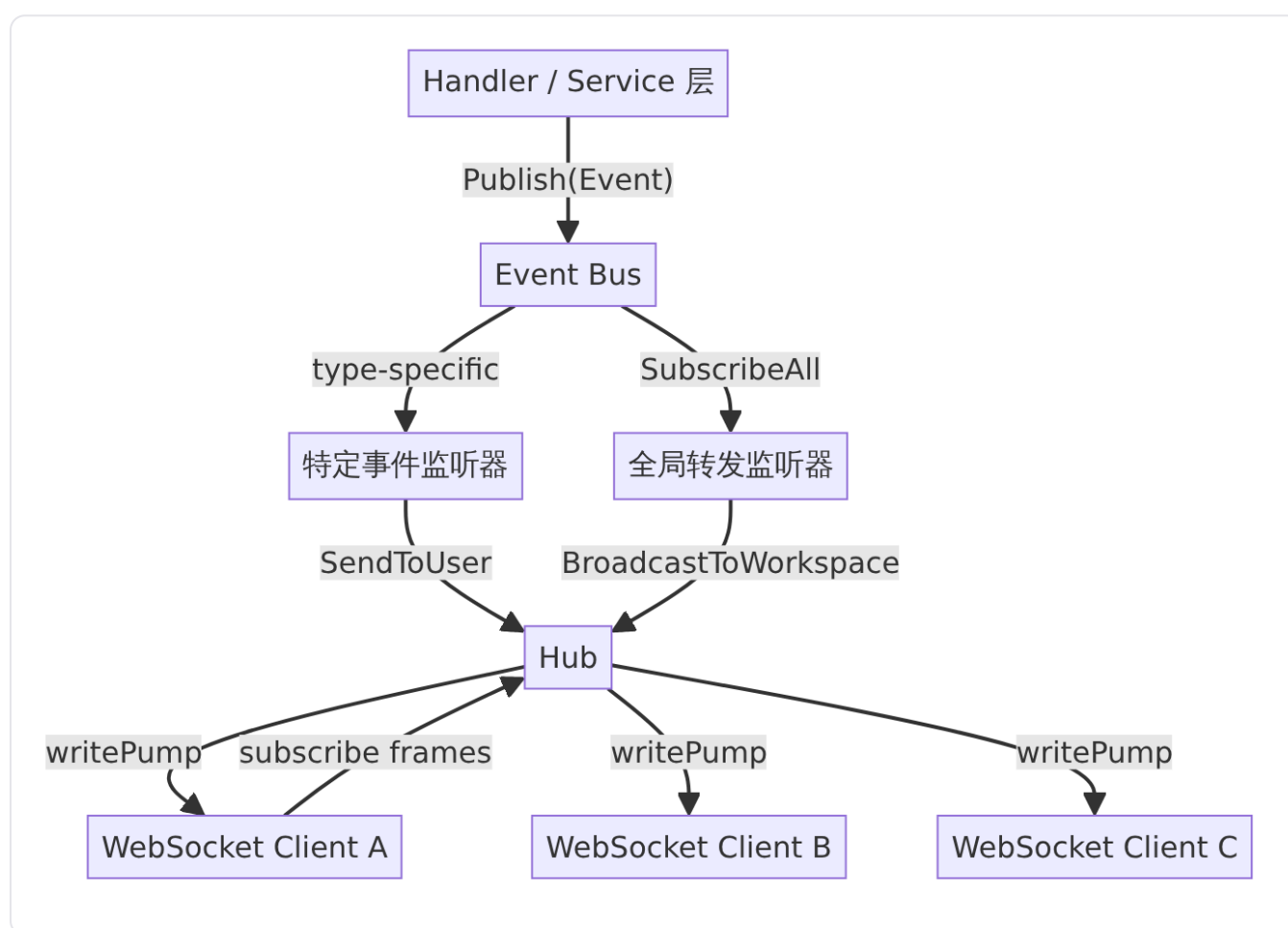
实时通信基础设施

detail · realtime-infrastructure.md

实时通信基础设施

实时通信基础设施是 Multica 服务端负责将领域事件即时推送到用户客户端的核心子系统。它由两个松耦合的组件构成：基于 [gorilla/websocket](#) 的连接中枢 `Hub`，以及进程内同步发布/订阅的 `Bus`。`Hub` 管理所有 WebSocket 连接的生命周期并按作用域路由消息；`Bus` 则负责将 handler/service 层发出的领域事件分发给注册的监听器，最终由 `registerListeners` 桥接层将事件转换为 WebSocket 帧推送到客户端。

整体架构



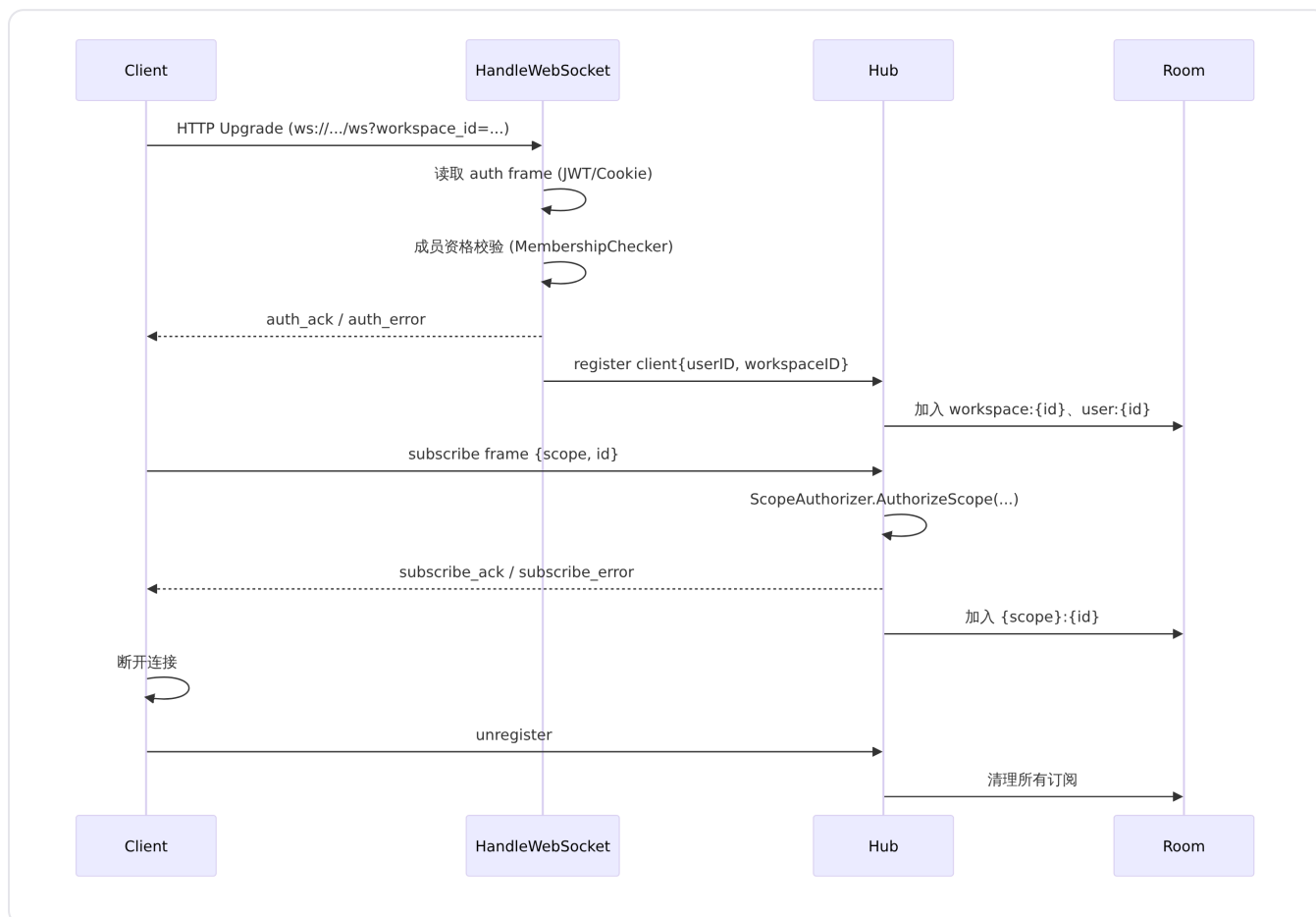
事件产生的路径是：Handler 或 Service 调用 `Bus.Publish(e Event)`，事件总线依次执行对应类型的监听器和全局监听器。桥接层 `registerListeners` 位于 [server/cmd/server/listeners.go](#) 中，在应用启动时注册了所有事件的转发逻辑，将领域事件序列化为 JSON 并通过 `Broadcaster` 接口写入 `Hub`。

WebSocket 连接中枢

Hub 核心结构

Hub 是 WebSocket 连接的管理中心，在 [server/internal/realtime/hub.go](#) 中定义。它通过 `NewHub()` 创建，通过 `Run()` 启动事件循环，以 `register / unregister` 通道模式管理客户端生命周期：

- **register 通道**：新连接通过认证后注册到 Hub，同时自动加入 `workspace:{id}` 和 `user:{id}` 两个默认作用域房间。
- **unregister 通道**：连接断开时清理该客户端在所有作用域房间中的订阅记录。
- **BroadcastToScope**：向指定作用域房间的所有连接发送消息，是广播能力的基础原语。



客户端生命周期

每个 WebSocket 连接在服务端由一个 `Client` 结构体表示，核心字段包括：

- `conn *websocket.Conn`：底层 `gorilla/websocket` 连接。
- `send chan []byte`：写协程 `writePump` 从此通道消费待发送字节。
- `userID / workspaceID`：在认证阶段确定，贯穿连接整个生命周期。
- `subscriptions map[scopeKey]bool`：记录当前连接订阅的所有作用域，由 `hub.mu` 保护。

`Client` 启动两个 goroutine：

- **readPump**：循环读取 WebSocket 帧，解析 `subscribe / unsubscribe` 等客户端协议帧。定义于 [server/internal/realtime/hub.go](#)。

- **writePump** : 从 `send` 通道消费字节并写入连接，同时处理 PING/PONG 以维持连接活性。定义于 [server/internal/realtime/hub.go](#)。

两个协程中任一退出都会触发连接的彻底清理。

连接建立与认证流程

`HandleWebSocket` 是 WebSocket 升级的入口函数，由路由层在 `/ws` 端点调用：

1. **解析工作区标识**：优先从查询参数 `workspace_id` 获取工作区 UUID；若为空则尝试 `workspace_slug`，通过 `SlugResolver` 将其转换为 UUID。
2. **跨域 Origin 检查**：通过 `checkOrigin` 函数验证请求的 `Origin` 头。白名单由 `ALLOWED_ORIGINS`（或 `CORS_ALLOWED_ORIGINS`、`FRONTEND_ORIGIN`）环境变量配置，同时支持通过 `X-Forwarded-Host` 处理反向代理场景。受信代理的 CIDR 列表由 `MULTICA_TRUSTED_PROXIES` 环境变量控制。
3. **用户认证**：支持两种策略——优先读取 `auth` cookie 中的 JWT 令牌，若不存在则等待客户端发送 `{"type":"auth","payload":{"token":"..."}}` 消息帧。JWT 解析与工作区成员资格检查均由外部注入的 `MembershipChecker` 接口完成。
4. **WebSocket 升级**：认证通过后调用 `gorilla/websocket.Upgrader` 完成协议升级，创建 `Client` 并注册到 `Hub`。

来源：[server/internal/realtime/hub.go](#)

路由层集成

路由在 [server/cmd/server/router.go](#) 中注册 WebSocket 端点，将 `Hub`、`MembershipChecker`、`PATResolver` 以及 `SlugResolver` 注入到 `HandleWebSocket`：

```
/ws → HandleWebSocket(hub, membershipChecker, patResolver, slugResolver, w, r)
```

同时，`registerListeners(bus, hub)` 在路由初始化后立即执行，将 `Bus` 与 `Hub` 桥接起来。集成测试 [server/cmd/server/integration_test.go](#) 中完整验证了 `NewHub()` → `go hub.Run()` → `NewRouter(pool, hub, bus, ...)` 的启动流程。

内部事件总线

事件总线位于 [server/internal/events/bus.go](#)，是进程内同步的发布/订阅实现。

Event 结构体

```
type Event struct {
    Type          string // 如 "issue:created"、"inbox:new"
    WorkspaceID   string // 路由到正确的 Hub 房间
    ActorType     string // "member"、"agent" 或 "system"
    ActorID       string
    Payload       any    // JSON 可序列化的负载
    TaskID        string // 可选，为细粒度路由提供 hint
    ChatSessionID string // 可选，为细粒度路由提供 hint
}
```

`TaskID` 和 `ChatSessionID` 是 MUL-1138 阶段引入的可选作用域提示。目前生产者已填充这些字段，但转发层仍使用工作区级广播——当客户端落地 `scope-subscription` 后，切换为按资源作用域路由只需修改一行转发代码。

订阅与发布

- **Subscribe(eventType, handler)**：注册针对特定事件类型的处理器。同类型的处理器按注册顺序同步调用。
- **SubscribeAll(handler)**：注册全局处理器，接收所有事件，在类型特定处理器之后执行。
- **Publish(event)**：先调用类型特定处理器，再调用全局处理器。每个处理器均在独立的 `defer/recover` 中执行，单个处理器的 `panic` 被捕获并记录错误日志，不会阻止其他处理器运行。

来源：[server/internal/events/bus.go](#)

事件转发桥接层

`registerListeners` 位于 [server/cmd/server/listeners.go](#)，是事件总线到 WebSocket 广播的核心桥接。它采用两层订阅策略：

第一层：针对个人的事件

通过 `Subscribe` 注册类型特定处理器，将通知直接发送到特定用户：

事件类型	路由策略
<code>inbox:new</code>	从嵌套 <code>payload.item.recipient_id</code> 提取接收者，通过 <code>SendToUser</code> 定点发送
<code>inbox:read</code> 、 <code>inbox:archived</code> 、 <code>inbox:unarchived</code>	从顶层 <code>payload.recipient_id</code> 提取接收者
<code>inbox:batch-read</code> 、 <code>inbox:batch-archived</code>	同上
<code>invitation:created</code>	提取 <code>invitation.invitee_user_id</code> ，发送给被邀请者
<code>invitation:revoked</code>	提取 <code>payload.invitee_user_id</code> ，发送给被邀请者
<code>member:added</code>	发送给新成员，同时传入 <code>excludeWorkspace</code> 避免已在工作区房间内的连接收到重复消息

这些事件被标记为 `personalEvents`，后续在全局处理器中被跳过，避免工作区广播造成重复。

第二层：工作区级广播

通过 `SubscribeAll` 注册全局处理器，处理所有非个人事件：

- 1. 序列化事件为 JSON，格式为 `{"type":..., "payload":..., "actor_id":..., "actor_type":...}`。
- 2. 调用 `projectOutbound` 过滤内部专用字段（如 `issue:updated` 的 `prev_description` / `prev_title`，`task:failed` 的 `error` 字段）。
- 3. 若有 `WorkspaceID`，通过 `BroadcastToWorkspace` 推送到该工作区所有连接；若无 `WorkspaceID` 且事件类型以 `daemon:` 开头，则通过 `Broadcast` 全局广播。

来源：[server/cmd/server/listeners.go](#)

出站负载过滤

`projectOutbound` 函数声明了 `internalOnlyPayloadKeys` 表，移除仅用于进程内监听器的字段后再序列化。例如 `issue:updated` 的 `prev_description` 和 `prev_title` 仅被通知监听器和活动日志监听器消费，不会泄露到 WebSocket 客户端。该设计将内部/外部负载边界集中在一处可审查的声明表中，防止未来新增的大字段被静默广播。

来源：[server/cmd/server/listeners.go](#)

作用域与广播器接口

作用域常数

`Broadcaster` 接口定义了五种作用域类型，位于 [server/internal/realtime/broadcaster.go](#)：

作用域	路由目标	说明
<code>workspace</code>	工作区内所有连接	当前默认的事件广播粒度
<code>user</code>	特定用户的所有连接	个人通知、邀请等
<code>task</code>	订阅了特定任务的连接	MUL-1138 细粒度路由（服务端就绪，待客户端落地）
<code>chat</code>	订阅了特定聊天会话的连接	MUL-1138 细粒度路由（服务端就绪，待客户端落地）
<code>daemon_runtime</code>	守护进程 WebSocket 连接	运行时唤醒帧，不发送给浏览器客户端

Broadcaster 接口

`Broadcaster` 是所有实时事件生产者应依赖的抽象，而非直接依赖 `*Hub`：

- **BroadcastToScope(scopeType, scopeID, message)**：向指定作用域房间广播，是底层原语。

- **BroadcastToWorkspace(workspaceID, message)** : 等价于 `BroadcastToScope("workspace", workspaceID, message)`。
- **SendToUser(userID, message, excludeWorkspace...)** : 等价于 `BroadcastToScope("user", userID, message)` , 可选的 `excludeWorkspace` 参数用于 `member:added` 去重场景。
- **Broadcast(message)** : 向本节点所有连接广播, 仅用于 `daemon:*` 无工作区上下文的全局事件。

*Hub 在编译期通过 `var _ Broadcaster = (*Hub)(nil)` 断言满足接口。

来源 : [server/internal/realtime/broadcaster.go](https://github.com/containers/gvisor/blob/master/server/internal/realtime/broadcaster.go)

扩展点

MembershipChecker

定义于 [server/internal/realtime/hub.go](https://github.com/containers/gvisor/blob/master/server/internal/realtime/hub.go) , 用于在 WebSocket 连接建立时验证用户对目标工作区的成员资格 :

```
type MembershipChecker interface {
    IsMember(ctx context.Context, userID, workspaceID string) bool
}
```

SlugResolver

一个函数类型, 将工作区 Slug 转换为 UUID, 在查询参数 `workspace_slug` 存在时被调用 :

```
type SlugResolver func(ctx context.Context, slug string) (workspaceID string, err error)
```

PATResolver

解析个人访问令牌 (Personal Access Token) 到用户 ID, 用于非浏览器客户端 (如 CLI、守护进程) 的认证 :

```
type PATResolver interface {
    ResolveToken(ctx context.Context, token string) (userID string, ok bool)
}
```

ScopeAuthorizer

决定连接是否有权订阅特定作用域, 典型实现会查询数据库确认资源 (任务、聊天会话) 属于当前工作区 :

```
type ScopeAuthorizer interface {
    AuthorizeScope(ctx context.Context, userID, workspaceID, scopeType, scopeID string) (bool, error)
}
```


通过 `Hub.SetAuthorizer(authorizer)` 注入。当鉴权失败时，客户端会收到 `subscribe_error` 帧，其中 `error` 字段为 `"lookup_failed"`，连接不会被添加到目标作用域。

订阅生命周期回调

`Hub` 支持 `onFirstSubscriber` 和 `onLastSubscriber` 两个可选回调，在作用域房间的首个订阅者加入和最后一个订阅者离开时触发，可用于 `Redis` 跨节点中继等场景。

可观测性

`realtime.M` 是包级 `Metrics` 单例，位于 `server/internal/realtime/metrics.go`，使用 `atomic.Int64` 和 `sync.Map` 提供轻量级计数器：

- **连接指标**：`ConnectsTotal`、`DisconnectsTotal`、`ActiveConnections`、`SlowEvictionsTotal`、`InboundTooLargeTotal`
- **消息指标**：`MessagesSentTotal`、`MessagesDroppedTotal`
- **按事件类型计数**：通过 `sync.Map` 存储 `*atomic.Int64`，`RecordEvent(eventType)` 递增对应计数器
- **订阅指标**：按作用域类型的 `subscribeTotal`、`unsubscribeTotal`、`subscribeDeniedTotal`、`scopeRooms`
- **Redis 中继指标**：`RedisXAddTotal`、`RedisXAddErrors`、`RedisConnected` 等

事件监听器在每次广播前调用 `realtime.M.RecordEvent(e.Type)` 记录发送计数。

与其他模块的关系

关联模块	交互方式
任务与问题管理	任务的创建、更新、分配等操作通过 <code>Bus.Publish</code> 发出 <code>issue:*</code> 事件，经桥接层广播到工作区所有连接
智能体对话	聊天消息通过 <code>chat:*</code> 事件实时推送到会话参与者
通知中心	通知的创建、已读、归档等 <code>inbox:*</code> 事件通过 <code>SendToUser</code> 定点推送到接收者
用户认证体系	<code>WebSocket</code> 连接建立时依赖 <code>JWT</code> 令牌校验和工作区成员资格验证
自动化规则	自动化触发产生的任务状态变更同样经事件总线实时广播

用户认证体系

detail · authentication-system.md

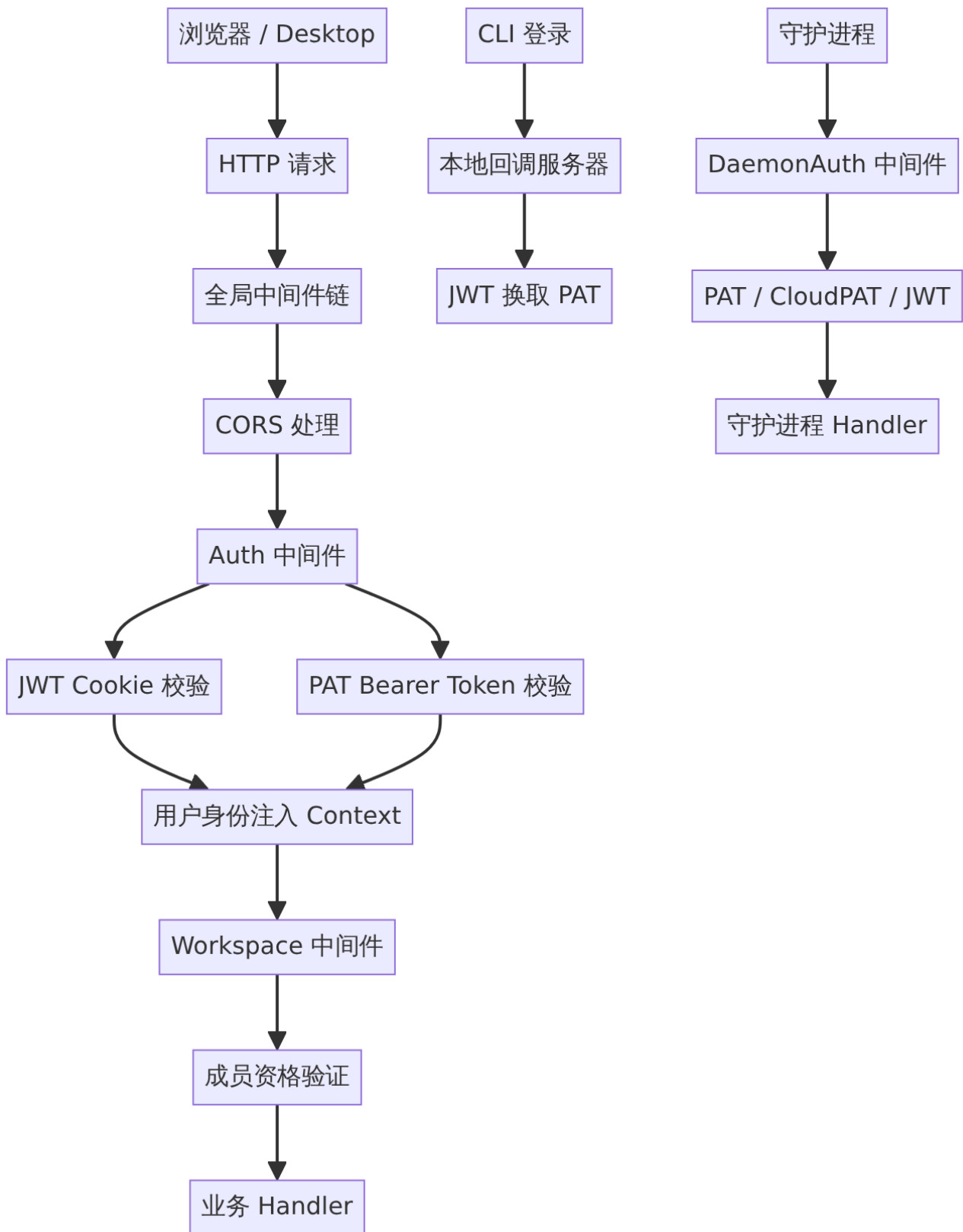
用户认证体系

Multica 的用户认证体系采用分层设计，支持浏览器登录会话、API 个人访问令牌（PAT）、守护进程认证协议和智能体执行临时凭据四种不同场景。认证状态跨 Web 和 Desktop 客户端共享，均由服务端统一签发和校验。

整体架构

认证体系由以下核心模块组成：路由层（`server/cmd/server/router.go`）定义公开和受保护路由并挂载中间件；`server/internal/auth` 包实现了 JWT 签发与校验、Cookie 管理、PAT 缓存和成员资格缓存；`server/internal/middleware` 包提供 `Auth`、`Workspace` 和 `DaemonAuth` 三个中间件，分别负责用户认证、工作区成员资格解析和守护进程认证。

来源：[server/cmd/server/router.go](#) — 路由定义与中间件挂载



浏览器登录会话

Cookie 管理

登录成功后，服务端将 JWT 写入名为 `multica_auth` 的 `HttpOnly` Cookie，浏览器自动携带但 JavaScript 无法直接读取。同时写入名为 `multica_csrf` 的 CSRF 防护 Cookie。常量定义于 [server/internal/auth/cookie.go](#)。

会话默认有效期为 30 天（`defaultAuthTokenTTL = 30 * 24 * time.Hour`），自托管管理员可通过环境变量 `AUTH_TOKEN_TTL` 调整后续签发会话的有效期，支持 Go duration 格式（如 `720h`）或整数秒数。
[server/internal/auth/cookie.go](#) 中的 `parseAuthTokenTTL` 函数完成解析，`AuthTokenTTL` 函数返回最终生效的 TTL 值。

登录方式

系统支持两种登录方式，路由定义于 [server/cmd/server/router.go](#)：

路由	Handler	说明
<code>POST /auth/send-code</code>	<code>h.SendCode</code>	向邮箱发送 6 位验证码，10 分钟有效
<code>POST /auth/verify-code</code>	<code>h.VerifyCode</code>	验证邮箱验证码，成功后签发 JWT 并写入 Cookie
<code>POST /auth/google</code>	<code>h.GoogleLogin</code>	通过 Google OAuth 回调完成登录
<code>POST /auth/logout</code>	<code>h.Logout</code>	清除当前浏览器的认证与 CSRF Cookie

邮箱验证码通过 Resend API 或 SMTP 中继发送，两者同时配置时 `SMTP_HOST` 优先。本地开发可设置固定验证码 `MULTICA_DEV_VERIFICATION_CODE`（仅在 `APP_ENV=development` 时生效）。Google 登录需要配置 `GOOGLE_CLIENT_ID`、`GOOGLE_CLIENT_SECRET` 和 `GOOGLE_REDIRECT_URI`，回调地址必须与 Google Cloud Console 中完全一致。

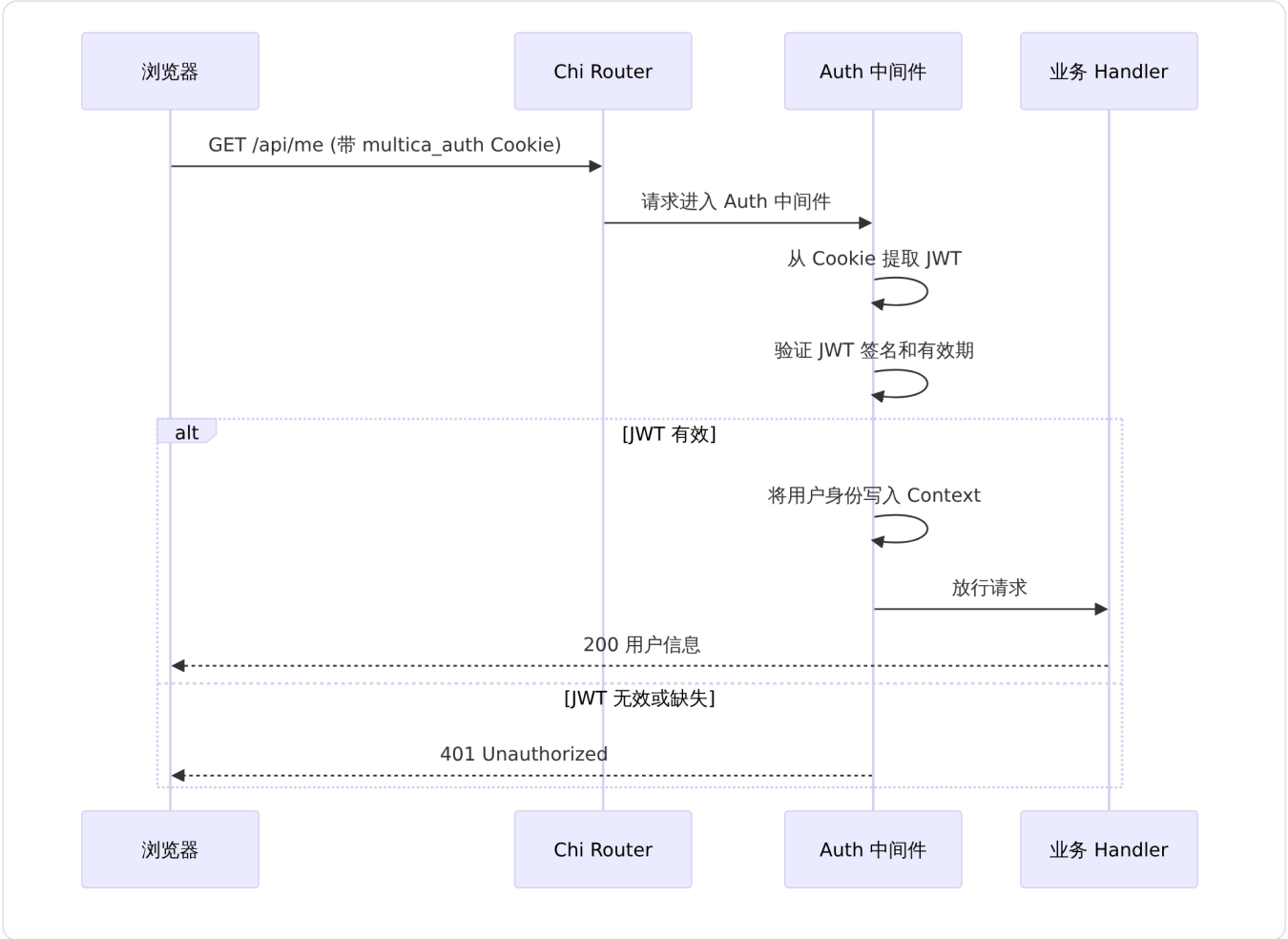
JWT 令牌签发与校验

JWT 的签发和校验逻辑集中在 [server/internal/auth/jwt.go](#)。JWT 密钥通过环境变量配置，开发环境使用默认密钥 `multica-dev-secret-change-in-production`，生产环境必须显式设置。令牌在签名时嵌入用户标识和过期时间，校验时验证签名有效性及是否过期。

`Auth` 中间件（[server/internal/middleware/auth.go](#)）使用 `github.com/golang-jwt/jwt/v5` 库，同时支持两种认证方式：

- 1. **Cookie 模式**：从 `multica_auth` Cookie 中提取 JWT，校验签名和有效期，将用户身份注入请求上下文。
- 2. **Bearer Token 模式**：从 `Authorization: Bearer <token>` 头部提取 PAT，通过 PAT 缓存或数据库验证令牌有效性。

两种模式通过统一的上下文键传递用户身份，下游 Handler 无需区分认证来源。



个人访问令牌（PAT）

个人访问令牌以 `mul_` 开头，代表用户账户，可访问用户有权访问的全部工作区和 API。令牌创建后仅展示一次完整值，服务端只存储令牌哈希值、前缀、名称、创建时间、过期时间和最近使用时间。

PAT 缓存

[server/internal/auth/pat_cache.go](#) 实现了 PAT 的 Redis 缓存层。`AuthCacheTTL` 控制令牌哈希查找结果在缓存中的保留时间，避免每次请求都查询 PostgreSQL。缓存命中时直接返回已验证的用户身份，未命中时回退到数据库查询并回填缓存。

令牌前缀体系

系统识别多种凭据前缀，分别用于不同场景：

前缀	用途	管理者	典型 TTL
<code>mul_</code>	个人访问令牌	用户在设置页面创建	30 天 / 90 天 / 1 年 / 永不过期
<code>mat_</code>	智能体执行临时令牌	服务端自动创建	最长 24 小时，task 结束时清理
<code>mcn_</code>	Multica Cloud Node 连接	Multica Cloud Fleet 管理	—

前缀	用途	管理者	典型 TTL
mdt_	工作区守护进程认证协议	服务端内部流程	—

Cloud PAT 验证

[server/internal/auth/cloud_pat.go](#) 实现了 `mcn_` 前缀令牌的远程验证逻辑。当请求携带 Cloud Node PAT 时，中间件通过 HTTP 调用 Cloud Fleet 端点校验令牌有效性，返回关联的用户和权限信息。此机制用于 Multica Cloud 托管场景中的节点认证，普通自部署安装无需使用。

工作区成员资格验证

用户认证通过后，`Workspace` 中间件 ([server/internal/middleware/workspace.go](#)) 负责解析目标工作区并验证成员资格。工作区标识按以下优先级解析：

1. 请求上下文已注入的工作区 ID（由上游中间件或 Handler 设置）
2. URL 路径参数中的工作区 Slug（通过 `chi.URLParam` 提取）
3. `X-Workspace-ID` 请求头
4. URL 查询参数

`ResolveWorkspaceIDFromRequest` 函数是工作区解析的单一真相来源，被中间件保护路由和无中间件路由（如 `/api/upload-file`）共同使用。

成员资格通过 [server/internal/auth/membership_cache.go](#) 缓存，TTL 为 5 分钟（`MembershipCacheTTL = 5 * time.Minute`）。Redis 键前缀为 `mul:auth:member:`，仅缓存成员资格存在性，不缓存角色等详细信息。缓存命中时避免一次 PostgreSQL 查询，对于守护进程心跳（约每 15 秒一次）等高频场景，将数据库压力从每次请求一次查询降低到每个 TTL 窗口一次。

注册开关控制

注册限制由三个环境变量共同决定，定义于 [server/cmd/server/router.go](#) 中的 `signupConfig` 结构体：

变量	作用
<code>ALLOWED_EMAILS</code>	允许注册的完整邮箱，逗号分隔
<code>ALLOWED_EMAIL_DOMAINS</code>	允许注册的邮箱域名，逗号分隔
<code>ALLOW_SIGNUP</code>	未配置白名单时是否允许注册，默认 <code>true</code>

判断顺序为：邮箱命中 `ALLOWED_EMAILS` 即允许；域名命中 `ALLOWED_EMAIL_DOMAINS` 即允许；未命中时若 `ALLOW_SIGNUP=false` 则拒绝；`ALLOW_SIGNUP=true` 但配置了任意白名单且未命中，仍拒绝。邀请不会自动绕过注册限制——被邀请人邮箱必须符合注册规则。

守护进程认证

守护进程通过 WebSocket 连接服务端，并使用 [server/internal/middleware/daemon_auth.go](#) 中的 `DaemonAuth` 中间件进行认证。该中间件支持三条认证路径：

- `pat` : 用户 PAT (`mul_` 前缀)，通过 PAT 缓存验证
- `ccloud_pat` : Cloud Node PAT (`mcn_` 前缀)，通过 CloudPAT 验证器远程校验
- `jwt` : 浏览器 JWT，通过 JWT 签名验证

认证成功后，守护进程 ID、工作区 ID 和认证路径标签被注入请求上下文，后续 Handler 可通过 `DaemonWorkspaceIDFromContext` 和 `DaemonIDFromContext` 等函数获取。

CLI 登录流程

CLI 的 `multica login` 命令支持两种登录模式，定义于 [server/cmd/multica/cmd_login.go](#)：

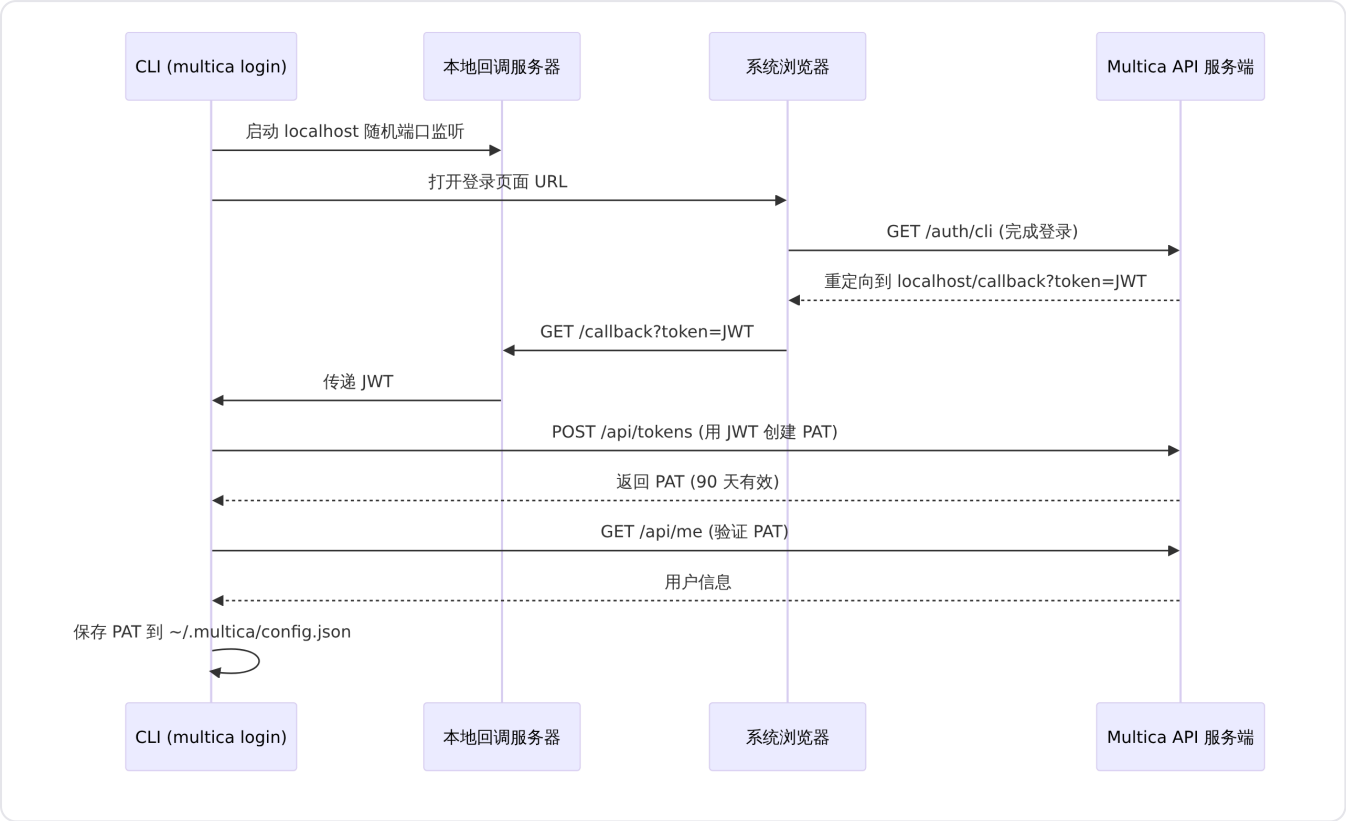
浏览器登录模式（默认）

CLI 启动本地 HTTP 回调服务器监听 `localhost` 随机端口，打开系统浏览器跳转到服务端 `/auth/cli` 页面。用户在浏览器完成邮箱验证码或 Google 登录后，服务端将 JWT 通过回调 URL 的 `?token=` 参数返回给本地服务器。CLI 接收 JWT 后在 5 分钟内通过 `/api/tokens` 创建有效期 90 天的 PAT（名称含主机名），保存到 `~/.multica/config.json` 或 `~/.multica/profiles/<name>/config.json`。

Token 直接登录模式

用于无浏览器环境，通过 `--token mul_...` 参数直接提供 PAT。不带值的 `--token`（无操作数形式）将触发交互式提示输入。CLI 验证 PAT 有效性后写入配置文件。

来源：[server/cmd/multica/cmd_auth.go](#) — 浏览器回调与 PAT 创建逻辑；
[server/cmd/multica/cmd_auth.go](#) — Token 直接登录与 `auth status` 实现。



智能体执行临时令牌

守护进程领取 Task 后，服务端自动创建以 `mat_` 开头的临时令牌。此令牌绑定当前用户、工作区、智能体和 Task，最长有效 24 小时，Task 结束时自动清理。守护进程将此临时令牌注入 AI 编程工具而非用户 PAT，确保智能体的 API 调用被记录为智能体行为，且无法调用仅限用户或 Owner 执行的操作。此类令牌由服务端自动管理，用户无需保存或吊销。

认证中间件管线

请求经过认证体系时的完整中间件链路如下：

- 1. 全局中间件 (`server/cmd/server/router.go`)：`RequestID` → `ClientMetadata` → `RequestLogger` → `HTTP Metrics` (可选) → `Recoverer` → `ContentSecurityPolicy` → `CORS 处理`
- 2. 认证路由层 (`server/cmd/server/router.go`)：登录相关路由 (`/auth/send-code`、`/auth/verify-code`、`/auth/google`) 附加速率限制中间件
- 3. **Auth** 中间件：校验 JWT Cookie 或 Bearer PAT，注入用户身份
- 4. **Workspace** 中间件：解析工作区标识、验证成员资格 (针对工作区级路由)
- 5. 业务 **Handler**：接收已验证的用户身份和工作区上下文

公开路由 (`/auth/*`、`/health`、`/readyz`、`/api/config`、`/api/attachments/{id}/signed-download`、`/api/avatars/{sig}/*`、`/uploads/*`、`/ws`) 跳过 Auth 中间件，其中 WebSocket 升级 (`/ws`) 在实时通信基础设施层自行处理 JWT 认证。

跨客户端认证状态共享

Web 应用和 Desktop 应用共享同一套认证机制。Desktop 应用基于 Electron 构建，其渲染进程与 Web 应用本质上共享相同的浏览器引擎认证模型——`multica_auth` Cookie 在 Electron 会话中同样有效。CLI 和守护进程则通过 PAT 独立认证，不与浏览器 Cookie 共享会话状态。退出 Web 登录仅清除 Cookie 不影响 PAT，反之亦然。

定时调度器

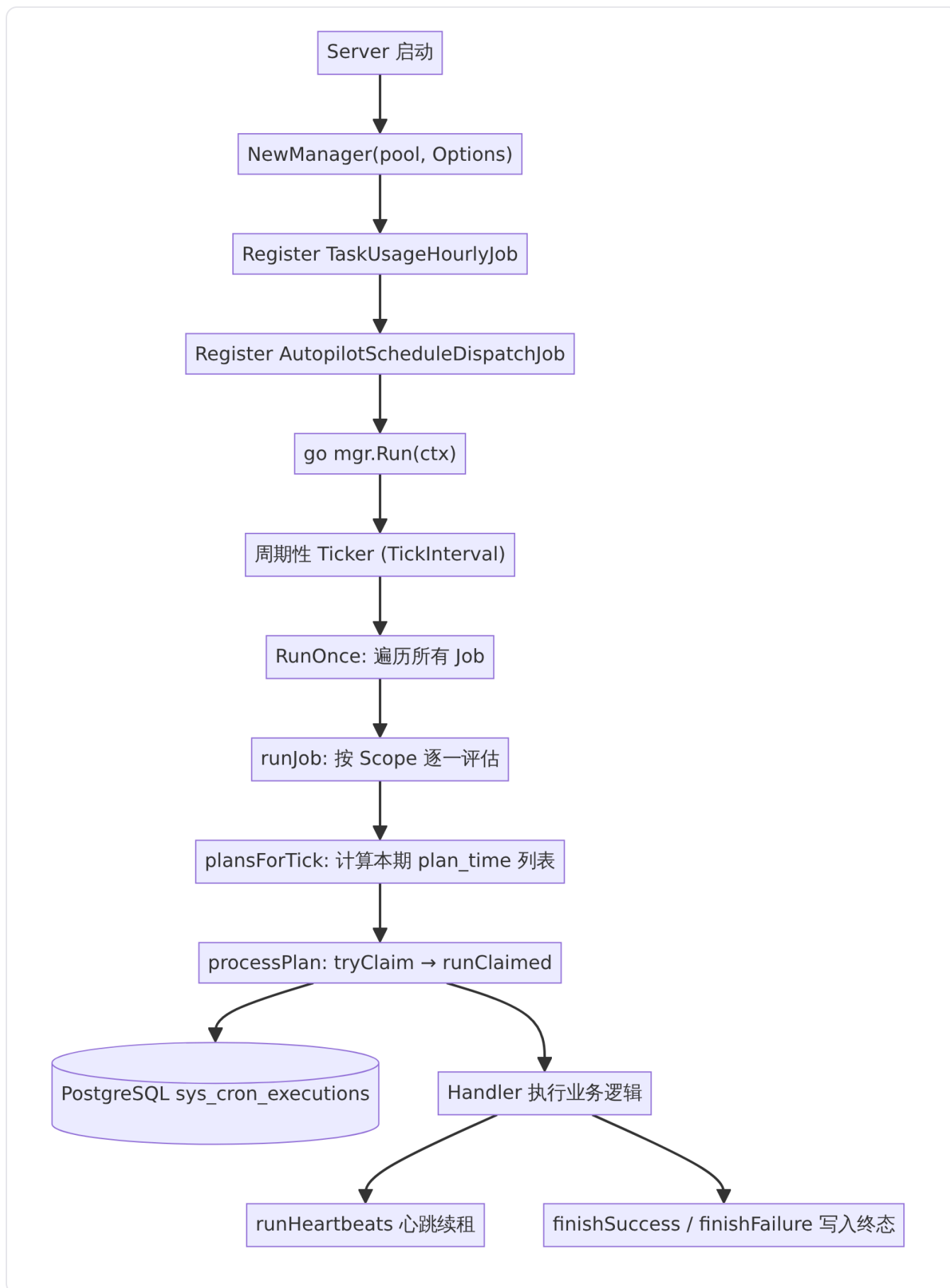
detail · scheduler.md

定时调度器

定时调度器 (Scheduler) 是 Multica 服务端的进程级周期任务执行引擎。它以 PostgreSQL 的 `sys_cron_executions` 表作为分布式锁和审计日志的统一后端，支持注册多个定时 Job，并按 `TickInterval` 周期性评估到期计划。每个进程实例由唯一的 `RunnerID` 标识，天然支持多副本部署下的单执行者语义。

架构概览

调度器位于 `server/internal/scheduler` 包中，核心组件为 `Manager`。应用启动时创建 `Manager` 实例，注册一到多个 `JobSpec`，随后在独立 goroutine 中调用 `Run`，由可取消的 context 控制生命周期。



来源：[server/cmd/server/main.go](#) 展示了 Manager 的创建、Job 注册以及 goroutine 启动流程。

核心类型

Manager

Manager 是调度器的顶层管理器，持有连接池、配置选项、Job 注册表以及读写锁保护的并发控制。

server/internal/scheduler/manager.go 定义了 Manager 结构体：

字段	类型	说明
pool	*pgxpool.Pool	数据库连接池，指向包含 sys_cron_executions 表的数据库
opts	Options	配置选项，包含 RunnerID、TickInterval、Logger
jobs	map[string]*JobSpec	已注册的 Job 映射，键为 Job 名称
mu	sync.RWMutex	保护 jobs 映射的读写锁
logger	*slog.Logger	结构化日志记录器，自动注入 component=scheduler 和 runner_id 上下文

Options

server/internal/scheduler/manager.go 定义了配置项：

字段	类型	默认值	说明
RunnerID	string	自动生成 UUID	标识当前进程实例，写入审计行的 runner_id 列
TickInterval	time.Duration	30 秒	Manager 唤醒评估到期计划的间隔
Logger	*slog.Logger	slog.Default()	结构化日志记录器

JobSpec

JobSpec 描述一个完整的注册 Job，是调度器的核心配置单元。所有字段在 Register 时通过 validate() 进行前置校验。

server/internal/scheduler/spec.go 定义了 JobSpec 结构体：

字段	类型	说明
Name	string	全局唯一标识，必须稳定（跨版本不可改名），使用 snake_case ASCII
Cadence	time.Duration	计划桶大小（如 5 分钟），调度器将 db_now - ScheduleDelay 向下取整到 Cadence 的倍数得到 plan_time

字段	类型	说明
ScheduleDelay	time.Duration	推迟 eligibility 窗口，例如 5 分钟延迟意味着 12:00 的计划在 12:05 才变为 eligible
CatchUpMode	CatchUpMode	追赶模式：CatchUpLatestOnly 或 CatchUpEveryPlan
CatchUpWindow	time.Duration	追赶窗口上限，超过此窗口的历史计划被忽略
MaxPlansPerTick	int	单次 tick 最多声明的计划数（仅 CatchUpEveryPlan 使用）
RunTimeout	time.Duration	单次 handler 执行的超时时间，必须小于 StaleTimeout
StaleTimeout	time.Duration	最后一次心跳后 RUNNING 租约被视为过期的时长
HeartbeatInterval	time.Duration	心跳续租间隔，必须大于 0 且小于 StaleTimeout
AllowStaleReentry	bool	是否允许其他 runner 窃取过期租约
MaxAttempts	int	同一 plan_time 的最大尝试次数（含首次）
RetryBackoff	[]time.Duration	重试退避间隔数组，索引超出范围时复用最后一项
Scopes	ScopeProvider	返回每次 tick 应评估的 Scope 列表
PlansForScope	func(...)	可选 hook，完全替代 Cadence 规划器，用于 cron 表达式等非均匀计划
Handler	Handler	业务处理函数，每次声称的租约调用一次

Scope

Scope 标识一次计划执行的锁定维度，决定 sys_cron_executions 唯一键的粒度。

server/internal/scheduler/spec.go 定义：

```
type Scope struct {
    Kind string
    ID   string
}
```

全局 Job 使用规范值 ScopeGlobal = Scope{Kind: "global", ID: "global"}，确保唯一键无 NULL 列。
分片 Job（如 Autopilot 调度）可返回多个 Scope，每个 scope 独立锁定。

Claim 结果

tryClaim 返回的 claim 结构体表示一次租约声明的三种互斥结果：

[server/internal/scheduler/db_ops.go](#) 定义：

字段	含义
Won	全新插入成功，本 runner 获得租约
Stole	通过 stale-steal 或 FAILED 重试路径获得租约
Conflicted	其他 runner 已拥有该计划、计划已终态、或重试次数耗尽

控制流

主循环

`Run` 方法是调度器的入口点。首次 tick 立即执行，之后按 `TickInterval` 周期性调用 `RunOnce`，直到 context 被取消。

[server/internal/scheduler/manager.go](#) 实现了主循环逻辑。

RunOnce 单次 Tick

`RunOnce` 首先通过 `dbNow` 从 PostgreSQL 获取共识时钟，然后对每个已注册的 Job 调用 `runJob`。

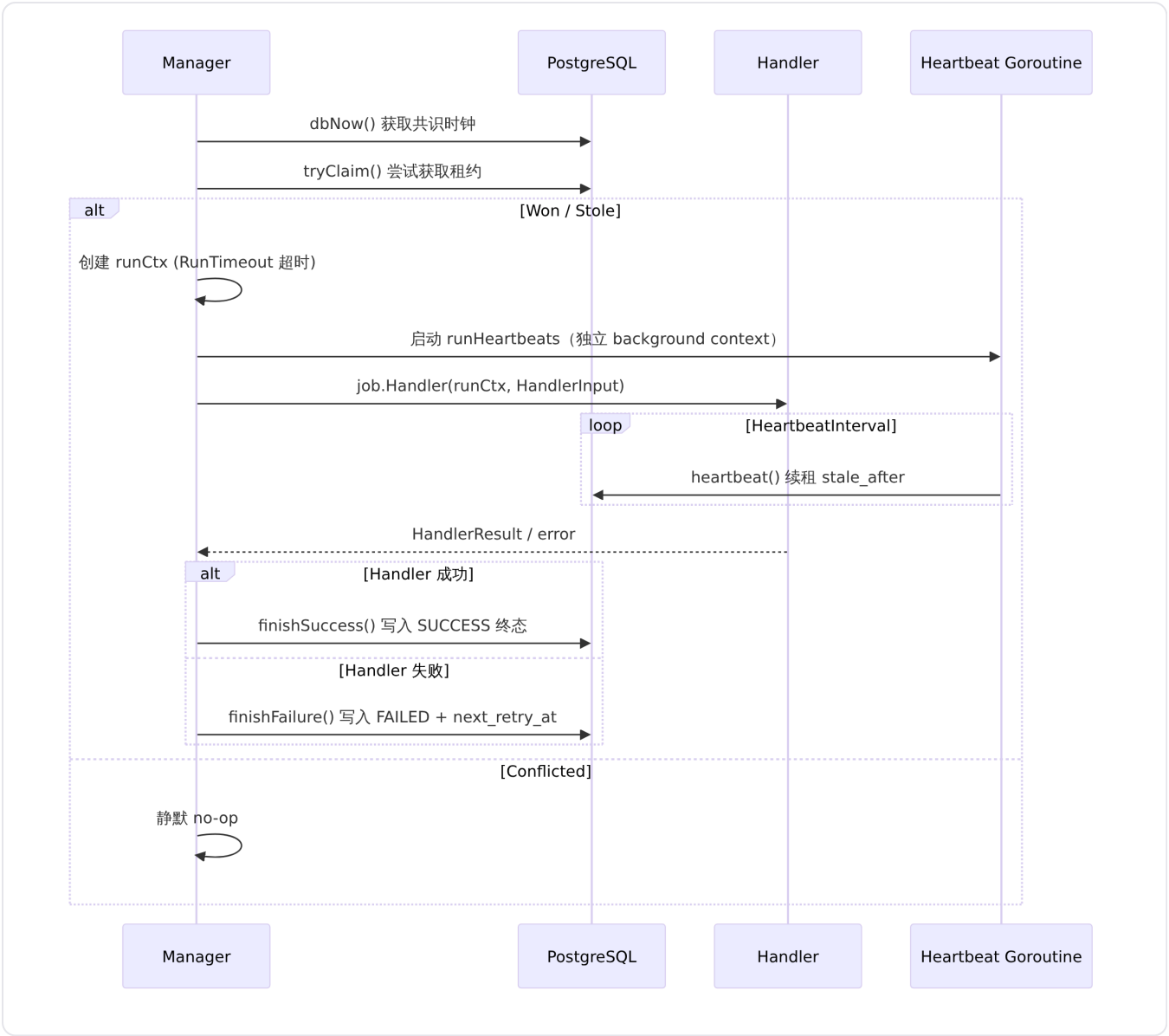
[server/internal/scheduler/manager.go](#) 实现了单次 tick。

`runJob` 的执行流程：

1. 调用 `job.Scopes` 获取当期 Scope 列表
2. 调用 `markStaleAsFailed` 清理所有已过期的 RUNNING 租约（无论 `AllowStaleReentry` 的值）
3. 对每个 scope 调用 `plansForTick` 计算待处理的 `plan_time` 列表
4. 对每个 `plan_time` 调用 `processPlan`

单计划处理

`processPlan` → `runClaimed` 流程是每个 `(job, scope, plan_time)` 三元组的完整生命周期：



来源：[server/internal/scheduler/manager.go](#) 实现了 `processPlan` 和 `runClaimed`。

心跳机制

Handler 执行期间，`runHeartbeats` 在独立 goroutine 中运行，使用 `detached background context` 确保即使 `runCtx` 被取消（超时），心跳续租也不受影响。心跳间隔为 `HeartbeatInterval`，每次调用 `heartbeat` SQL 原语续租 `stale_after`。若心跳返回 `ErrLeaseLost`，goroutine 终止并记录警告日志，Handler 应通过 `Heartbeat` 回调感知租约丢失并主动停止。

[server/internal/scheduler/manager.go](#) 实现了心跳循环。

Handler Panic 恢复

`runClaimed` 使用命名返回值 + `defer/recover` 模式捕获 Handler 中的 panic，将其转换为 `ErrHandlerPanic` 错误，确保 panic 不会导致进程崩溃或被错误记录为 `SUCCESS`。

[server/internal/scheduler/manager.go](#) 展示了 panic 恢复逻辑。

计划计算

Cadence 规划器

对于未设置 `PlansForScope` 的 Job，调度器使用 Cadence 规划器：

1. 计算 `eligible = db_now - ScheduleDelay`
2. 调用 `FloorPlan(eligible, Cadence)` 得到最近的 `plan_time` 桶
3. 根据 `CatchUpMode` 决定返回的计划列表：
 - `CatchUpLatestOnly`：仅返回最新的 `plan_time`
 - `CatchUpEveryPlan`：从数据库查询最近的存储计划，计算 `[start, latest]` 区间内所有到期的计划，受 `MaxPlansPerTick` 和 `CatchUpWindow` 限制

[server/internal/scheduler/manager.go](#) 实现了 `plansForTick`。

[server/internal/scheduler/spec.go](#) 定义了 `FloorPlan`，使用 `time.Truncate` 将 UTC 时间向下取整到 Cadence 的整数倍。

PlansForScope Hook

当 `JobSpec.PlansForScope` 非 nil 时，Cadence 规划器完全被绕过。Hook 接收当前 `scope`、DB 时间和最新的存储计划信息，返回本期应尝试的 `plan_time` 列表。这适用于 Autopilot 调度这类每个 trigger 有独立 cron 表达式的场景。

[server/internal/scheduler/spec.go](#) 定义了 `PlansForScope` 的语义。

LatestPlanInfo 与重试感知

`LatestPlanInfo` 是 `latestPlan` 查询返回的结构体，描述 `(job, scope)` 的最新一条 `sys_cron_executions` 记录。`RetryEligible` 方法判断该计划是否应在下次 tick 被重新尝试：仅当状态为 FAILED、尝试次数未达上限、且 `next_retry_at` 已过或为 NULL 时返回 true。

[server/internal/scheduler/db_ops.go](#) 定义了 `LatestPlanInfo` 及其 `RetryEligible` 方法。

数据库设计

调度器的持久化后端是 `sys_cron_executions` 表，由迁移 [113_sys_cron_executions.up.sql](#) 创建。

核心设计原则：

- **唯一键**：`(job_name, scope_kind, scope_id, plan_time)` 确保同一 (Job, Scope, 时间桶) 仅有一条记录，多副本部署下只有一个 runner 能声明 INSERT 成功
- **双重角色**：该表既是分布式锁 (RUNNING 状态 + `runner_id` + `lease_token`)，又是完整审计日志 (SUCCESS / FAILED 终态 + 执行指标)

- **租约字段**：`lease_token` (UUID) 防止终态写入竞态、`stale_after` 判断租约过期、`attempt` 和 `next_retry_at` 驱动重试

tryClaim 声明流程

[server/internal/scheduler/db_ops.go](#) 实现了 `tryClaim`：

1. **INSERT 路径**：尝试插入新行，`ON CONFLICT DO NOTHING`。成功即为 Won
2. 冲突后 **UPDATE 路径**：通过单条 `UPDATE` 语句同时处理 `stale-steal` (`stale_after < db_now`) 和 `FAILED` 重试 (`status='FAILED' AND next_retry_at <= db_now AND attempt < max_attempts`)

markStaleAsFailed

每次 tick 对每个 Job 强制调用，将 `stale_after` 已过期且仍处于 `RUNNING` 状态的行标记为 `FAILED`。这确保崩溃 pod 遗留的租约不会永久占用审计表，同时不依赖 `AllowStaleReentry` 即可触发告警。

[server/internal/scheduler/manager.go](#) 展示了 `sweep` 逻辑的设计决策。

dbNow：数据库共识时钟

调度器所有时间计算都基于 `SELECT now()` 返回的数据库时间，确保不同时钟偏移的实例对同一 `plan_time` 达成共识。

[server/internal/scheduler/db_ops.go](#) 定义了 `dbNow`。

错误分类

`classifyError` 将 Handler 返回的错误映射为审计行上的简短 `error_code`：

[server/internal/scheduler/manager.go](#) 定义：

错误条件	error_code
<code>context.DeadlineExceeded</code>	<code>run_timeout</code>
<code>context.Canceled</code>	<code>canceled</code>
<code>ErrLeaseLost</code>	<code>lease_lost</code>
<code>ErrHandlerPanic</code>	<code>handler_panic</code>
其他	<code>handler_error</code>

已注册的 Job

Multica 服务端当前注册了两个 Job：

rollup_task_usage_hourly

任务使用量按小时汇总 Job。每 5 分钟一个 plan 桶，延迟 5 分钟以确保数据完整到达。使用 CatchUpLatestOnly 模式，因为 Handler 内部通过 task_usage_hourly_rollup_state.watermark_at 自行驱动历史追赶。Handler 调用已存在的 PostgreSQL 函数 rollup_task_usage_hourly()，该函数内部持有 advisory lock 4246，确保与遗留的 pg_cron 调度并行运行时的安全性。

server/internal/scheduler/jobs_task_usage.go 定义了该 Job 的完整规格。

参数	值
Cadence	5 分钟
ScheduleDelay	5 分钟
CatchUpMode	latest_only
CatchUpWindow	24 小时
RunTimeout	25 分钟
StaleTimeout	30 分钟
HeartbeatInterval	30 秒
MaxAttempts	3
RetryBackoff	1m, 5m, 15m
AllowStaleReentry	true

autopilot_schedule_dispatch

自动化规则调度分发 Job，替代了旧的独立 goroutine autopilot_scheduler.go。每个启用的 schedule 类型的 Autopilot trigger 作为一个独立 Scope（scope_kind = "autopilot_trigger"，scope_id = trigger.id），通过 PlansForScope hook 根据 trigger 的 cron 表达式 + 时区计算 plan_time。

server/internal/scheduler/jobs_autopilot.go 定义了该 Job。

关键设计要点：

- 每个 trigger 是独立 Scope，plan_time 是规范 UTC cron 发生时间
- PlansForScope 枚举 [lastPlan, dbNow] 区间内的所有 cron 发生点，仅返回最近一个（错过多次时合并为一次）
- Handler 调用 AutopilotService.DispatchAutopilotForPlan，该服务方法通过 migration 124 的部分唯一索引在 (trigger_id, planned_at) 上具备幂等性
- 接受窗口 maxAutopilotScheduleLateness = 5 分钟，超过此窗口的 cron 发生点不会补发

多副本并发安全

调度器的多副本安全性由两层保障：

1. **唯一键**：`sys_cron_executions` 的 `(job_name, scope_kind, scope_id, plan_time)` 唯一约束确保同一计划同时只会有一个 runner 的 INSERT 成功
2. **lease_token 守卫**：终态写入 (`finishSuccess` / `finishFailure`) 时验证 `lease_token`，已租约被窃取则 UPDATE 影响零行并返回 `ErrLeaseLost` (内部别名为 `errTerminalIgnored`)，原 runner 的终态写入被静默忽略

`server/internal/scheduler/concurrent_claim_test.go` 中的 `TestConcurrentClaimsSingleWinner` 验证了 8 个并发 runner 同时声明同一计划时，恰好只有一个获得 `Won=true`，其余全部 `Conflicted`。

`server/internal/scheduler/stale_steal_test.go` 中的 `TestStaleStealTerminalUpdateIgnored` 验证了租约窃取后原 runner 的终端 SUCCESS 写入被 `lease_token` 守卫拒绝。

与自动化规则的关系

调度器是 [自动化规则](#) 的定时触发执行引擎。两者的关系如下：

- Autopilot 中 `kind = "schedule"` 的 trigger 通过调度器的 `autopilot_schedule_dispatch` Job 定期评估和分发
- 调度器仅负责“何时触发”，触发后调用 `AutopilotService.DispatchAutopilotForPlan` 完成实际的 Autopilot run 创建和任务入队
- `DispatchAutopilotForPlan` 具有 (`trigger_id, planned_at`) 级的幂等性，与调度器的单次执行保证形成双重安全网
- Webhook 类型的 Autopilot trigger 不经过调度器，由 HTTP handler 直接触发

扩展点

注册自定义 Job

外部代码通过 `Manager.Register(job JobSpec)` 注册自定义 Job。`JobSpec` 的所有字段均可按需配置，`Register` 会调用 `validate()` 进行前置校验。注册必须在 `Run` 之前完成（启动后注册也可，但需等到下一个 tick 循环才生效）。

PlansForScope Hook

对于计划时间不遵循均匀 Cadence 网格的 Job，设置 `PlansForScope` 可完全接管计划计算逻辑。Hook 接收 `scope`、当前 DB 时间和最近的存储计划信息，返回本期应尝试的 `plan_time` 列表。这对于 cron 表达式、自定义日历等场景是主要扩展点。

ScopeProvider

通过 `Scopes` 字段控制每次 tick 的评估维度。全局 Job 使用 `StaticScopes(ScopeGlobal)`；分片 Job 可返回动态 `Scope` 列表（如从数据库查询当前活跃的 trigger 列表）。

错误哨兵

- `ErrLeaseLost` : 由心跳和终态写入原语返回，Handler 应主动停止执行
- `ErrHandlerPanic` : 调度器内部使用的 panic 包装错误，Handler 不应主动返回此错误

配置

`NewManager` 接收的 `Options` 结构体支持以下配置：

参数	默认值	说明
<code>RunnerID</code>	新 UUID	多副本部署时建议显式设置以增强日志可读性
<code>TickInterval</code>	30 秒	应小于最短 Job Cadence
<code>Logger</code>	<code>slog.Default()</code>	自动注入 <code>component=scheduler</code> 和 <code>runner_id</code>

`JobSpec` 级别的配置见上文 [JobSpec](#) 表格。

国际化框架

detail · internationalization.md

国际化框架

国际化框架 (`@multica/core/i18n`) 是 Multica 服务端安全的国际化基础设施。它实现了零 React/DOM 依赖的核心层，可安全运行于 React Server Components (RSC)、Edge Runtime 和 Node.js 中间件中，同时为 React 客户端提供 `I18nProvider`、`useLocaleAdapter` 和 `UserLocaleSync` 等组件。

设计原则

框架严格遵循按环境分层的入口设计，将运行时敏感代码隔离到不同入口点中：

- `@multica/core/i18n` ：服务端安全核心入口。不导入 React、不访问 `document / navigator / window` ，可安全用于 RSC、Edge 和 Node.js 中间件。
[packages/core/i18n/index.ts](#)
- `@multica/core/i18n/react` ：React 客户端入口。依赖 `react-i18next` ，其模块级 `React.createContext()` 调用在非客户端上下文中会崩溃，因此必须配合 `"use client"` 指令使用。
[packages/core/i18n/react.ts](#)
- `@multica/core/i18n/browser` ：浏览器专用入口。封装了访问 `document.cookie` 和 `navigator.languages` 的适配器工厂函数，任何非 DOM 环境中的导入都会在导入站点立即崩溃。
[packages/core/i18n/browser.ts](#)

支持的语言

框架定义了四种受支持的语言，以 `SupportedLocale` 联合类型约束：

语言	标识符	内部翻译键	HTML lang 属性
英语	en	en	en
简体中文	zh-Hans	zh-Hans	zh-CN
韩语	ko	ko	ko-KR
日语	ja	ja	ja-JP

默认语言为 `en` 。当所有候选语言均无法匹配时，系统回退到 `DEFAULT_LOCALE` 。

[packages/core/i18n/types.ts](#)

HTML lang 属性使用 BCP-47 区域标签（如 `zh-CN` 、 `ko-KR` 、 `ja-JP` ），供屏幕阅读器和 CJK 字体回退栈使用。内部 `i18next` 实例仍以 `zh-Hans` 作为语言标识符，因为实际翻译资源按脚本子标签组织。

[apps/web/app/layout.tsx](#)

核心类型

SupportedLocale

```
export type SupportedLocale = "en" | "zh-Hans" | "ko" | "ja";
```

LocaleAdapter

`LocaleAdapter` 接口定义了三个方法，适配器负责从环境特定的存储中读取和持久化语言偏好：

- `getUserChoice()`：返回用户显式选择的语言标识符，或 `null`
- `getSystemPreferences()`：返回系统/浏览器的语言偏好列表
- `persist(locale: SupportedLocale)`：将用户选择的语言持久化到存储中

来源：[packages/core/i18n/types.ts](https://github.com/nextcloud/nextcloud/blob/master/packages/core/i18n/types.ts)

LocaleResources

```
export type LocaleResources = Record<string, Record<string, unknown>>;
```

表示嵌套的翻译资源对象，外层键为命名空间（如 `"common"`、`"auth"`、`"issues"`），内层键为翻译键。

语言匹配算法

语言解析通过 `matchLocale` 和 `pickLocale` 两个函数实现，底层使用 `@formatjs/intl-localematcher` 的 `match` 算法进行最佳匹配。

matchLocale

接收候选语言列表，返回最佳匹配的 `SupportedLocale`。当候选列表为空或匹配失败时，返回 `DEFAULT_LOCALE`（`"en"`）。

```
export function matchLocale(candidates: string[]): SupportedLocale {
  if (candidates.length === 0) return DEFAULT_LOCALE;
  try {
    return match(candidates, SUPPORTED_LOCALES, DEFAULT_LOCALE) as SupportedLocale;
  } catch {
    return DEFAULT_LOCALE;
  }
}
```

来源：[packages/core/i18n/pick-locale.ts](https://github.com/nextcloud/nextcloud/blob/master/packages/core/i18n/pick-locale.ts)

pickLocale

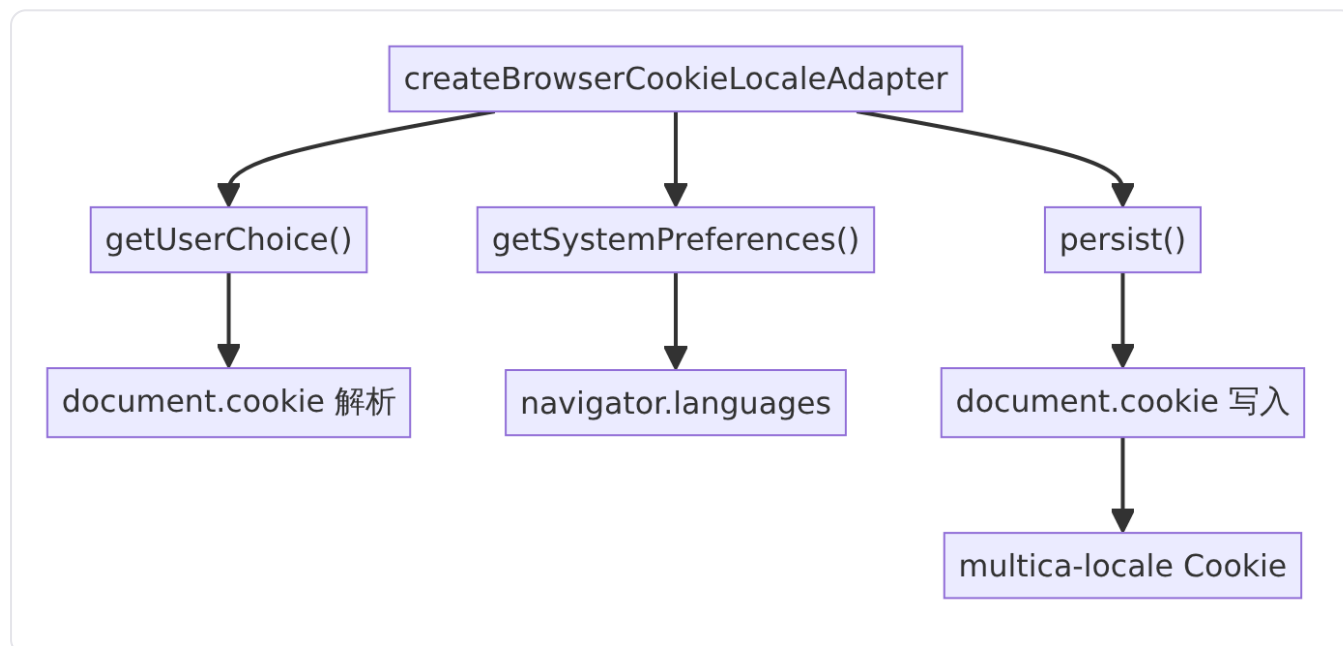
从 `LocaleAdapter` 中提取用户选择和系统偏好，按优先级匹配：先检查用户的显式选择（如 `Cookie` 值），再回退到系统偏好（如浏览器的 `navigator.languages`）。

```
export function pickLocale(adapter: LocaleAdapter): SupportedLocale {
  const choice = adapter.getUserChoice();
  if (choice) return matchLocale([choice]);
  return matchLocale(adapter.getSystemPreferences());
}
```

来源：[packages/core/i18n/pick-locale.ts](https://github.com/multica/core/i18n/pick-locale.ts)

Cookie 语言适配器 (Web)

`createBrowserCookieLocaleAdapter` 是 Web 平台的适配器工厂函数，位于浏览器专用入口 `@multica/core/i18n/browser` 中。该适配器通过 `document.cookie` 持久化用户语言选择，Cookie 名为 `multica-locale`，有效期一年（`max-age=31536000`），路径为 `/`，`SameSite=Lax`，HTTPS 下附加 `Secure` 标记。



来源：[packages/core/i18n/browser-cookie-adapter.ts](https://github.com/multica/core/i18n/browser-cookie-adapter.ts)

`LOCALE_COOKIE` 常量（`"multica-locale"`）同时从服务端安全入口 `@multica/core/i18n` 导出，使 Next.js 代理层可以在不导入任何浏览器代码的情况下读取该 Cookie 名。[packages/core/i18n/index.ts](https://github.com/multica/core/i18n/index.ts)

桌面语言适配器

桌面应用使用 `createDesktopLocaleAdapter(systemLocale)`，其适配器行为与 Web 不同：

- **用户选择**：从 `window.localStorage` 读取
- **系统偏好**：接收来自 Electron 主进程注入的 `systemLocale` 参数（通过 `additionalArguments` 和 `preload` 脚本暴露）

- **持久化**：写入 `localStorage`；Settings 语言切换器同时通过 `PATCH /api/me` 同步 `user.language` 到服务端，确保跨设备偏好跟随用户

[apps/desktop/src/renderer/src/platform/i18n-adapter.ts](#)

i18next 集成

`createI18n` 是 `i18next` 实例的工厂函数，同时被服务端（RSC）和客户端调用，接收相同的 `locale` 和 `resources` 参数以避免水合不匹配（hydration mismatch）。

关键配置：

- `initAsync: false`：强制同步初始化（`i18next v25+` 中由 `initImmediate` 重命名而来）
- `useSuspense: false`：防止水合期间的 fallback 渲染
- `fallbackLng: "en"`：缺失翻译键回退到英语
- `interpolation.escapeValue: false`：React 已内置 XSS 防护，禁用 `i18next` 的转义避免双重转义

来源：[packages/core/i18n/create-i18n.ts](#)

React 组件体系

I18nProvider

`I18nProvider` 是 React 客户端组件（"use client"），包装 `react-i18next` 的 `I18nextProvider`。它在首次渲染时通过 `useState` 惰性创建 `i18next` 实例，并通过 `useEffect` 支持 Fast Refresh 场景下的翻译资源热更新：当 `locale` 或 `resources` 引用发生变化时，使用 `instance.addResourceBundle` 增量更新资源包，然后根据需要调用 `changeLanguage` 或触发 `languageChanged` 事件。

语言切换不通过 `changeLanguage` 在客户端进行——而是通过 `window.location.reload()` 触发完整的页面重载，服务端根据 Cookie 中的新语言重新渲染。

来源：[packages/core/i18n/provider.tsx](#)

LocaleAdapterProvider & useLocaleAdapter

`LocaleAdapterProvider` 通过 React Context 向下传递 `LocaleAdapter` 实例。`useLocaleAdapter` hook 从 context 中读取适配器，若未在 Provider 内使用则抛出错误。

来源：[packages/core/i18n/adapter-context.tsx](#)

UserLocaleSync

`UserLocaleSync` 是一个不渲染任何 UI 的组件，挂载在 `CoreProvider` 内部。它在用户登录后，将服务端存储的 `user.language` 同步到本地适配器（Cookie 或 `localStorage`），确保跨设备切换时语言偏好不丢失。

同步逻辑：

1. 从 `AuthStore` 读取 `user.language`
2. 检查是否为支持的 `locale`
3. 检查是否与当前 `i18n` 实例语言不同
4. 调用 `adapter.persist()` 写入本地存储
5. 触发 `window.location.reload()` 使服务端重新解析

循环安全：重载后 `pickLocale` 从刚持久化的适配器中读取相同的值，`i18n` 语言匹配，`effect` 为 `no-op`。

来源：[packages/core/i18n/user-locale-sync.tsx](#)

CoreProvider 集成

`CoreProvider`（来自 `@multica/core/platform`）是 Web 和桌面应用共享的顶层 `Provider`，在国际化方面负责：

1. 接收 `locale`、`resources` 和 `localeAdapter` 属性（来自 `CoreProviderProps`）
2. 用 `LocaleAdapterProvider` 包装整个组件树
3. 用 `I18nProvider` 包装，传入 `locale` 和 `resources`
4. 在内部挂载 `UserLocaleSync` 以确保登录后语言同步

来源：[packages/core/platform/core-provider.tsx](#)、[packages/core/platform/types.ts](#)

服务端请求语言解析流程

Web 应用的请求语言解析涉及三个层次：Next.js 代理层、`locale-routing` 工具函数和 RSC 布局层。

代理层（`proxy.ts`）

Next.js 代理（`proxy.ts`，前身为 `middleware.ts`）在每个请求时：

1. 从 `Cookie` 中读取 `multica-locale` 值
2. 从请求头中读取 `Accept-Language`
3. 调用 `resolveLocaleFromSignals` 解析最终 `locale`
4. 通过 `x-multica-locale` 请求头将解析结果转发给上游 RSC

[apps/web/proxy.ts](#)

语言路由（`locale-routing.ts`）

`resolveLocaleFromSignals` 实现了语言信号的优先级解析：Cookie 优先于 `Accept-Language` 浏览器偏好。zh Cookie 值被规范化映射为 `zh-Hans`。`isSupportedLocale` 提供运行时类型守卫。

来源：[apps/web/lib/locale-routing.ts](#)

语言路由的单元测试覆盖了核心场景：Cookie 优先、Accept-Language 回退、中/韩/日浏览器语言信号匹配，以及旧版 zh Cookie 到 zh-Hans 的规范化。[apps/web/lib/locale-routing.test.ts](#)

请求语言读取 (request-locale.ts)

getRequestLocale 是 React Cache 函数，在 RSC 布局中被调用：

- 1. 优先读取 x-multica-locale 请求头（由代理层注入）
- 2. 若请求头无效，回退到 Cookie + Accept-Language 解析

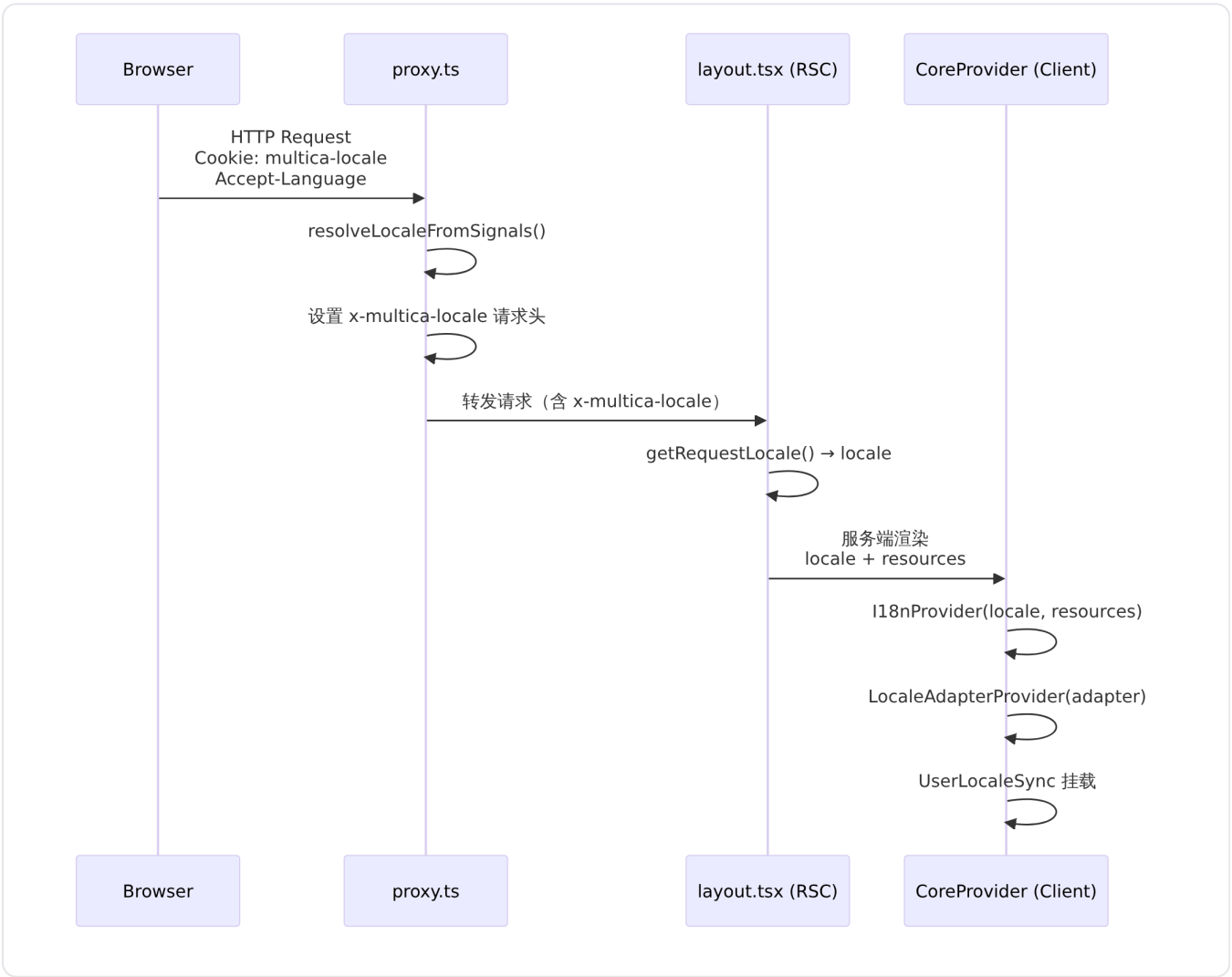
来源：[apps/web/lib/request-locale.ts](#)

布局层 (layout.tsx)

根布局调用 await getRequestLocale() 获取解析后的 locale，传递给 CoreProvider，同时在 <html lang> 属性上使用 BCP-47 区域标签映射。

来源：[apps/web/app/layout.tsx](#)

请求语言解析全链路



翻译资源管理

翻译资源集中定义在 `packages/views/locales/index.ts` 中的 `RESOURCES` 对象中。每个 `SupportedLocale` 对应一个 `LocaleResources` 对象，按命名空间组织：

命名空间	说明
<code>common</code>	通用 UI 文案
<code>auth</code>	认证相关
<code>settings</code>	设置页面
<code>issues</code>	问题管理
<code>agents</code>	智能体
<code>editor</code>	编辑器
<code>onboarding</code>	新用户引导
<code>invite</code>	邀请
<code>labels</code>	标签
<code>members</code>	成员管理
<code>my-issues</code>	我的问题
<code>search</code>	搜索
<code>inbox</code>	收件箱
<code>workspace</code>	工作区
<code>projects</code>	项目管理
<code>autopilots</code>	自动化规则
<code>skills</code>	技能管理
<code>chat</code>	聊天
<code>modals</code>	模态框
<code>runtimes</code>	运行时
<code>layout</code>	布局
<code>usage</code>	用量统计
<code>ui</code>	UI 组件
<code>squads</code>	小队管理
<code>billing</code>	计费

Web 和桌面应用均从此入口导入，确保添加语言或命名空间只需在一个位置操作。
<packages/views/locales/index.ts>

Landing 页面国际化

Landing（营销）页面使用独立的国际化方案，不走 `@multica/core/i18n/react` 的通用体系，而是通过自定义的 `LocaleProvider` 和字典工厂模式实现。

字典工厂按 `LandingDictionaryLocale`（`"en" | "zh" | "ko" | "ja"`）路由到对应工厂函数（`createEnDict`、`createZhDict` 等），工厂接收 `allowSignup` 布尔参数控制注册入口文案。
`LocaleProvider` 在语言切换时调用 `localeAdapter.persist()` 写入 Cookie，然后通过 `startTransition + router.refresh()` 触发 Next.js 路由刷新。

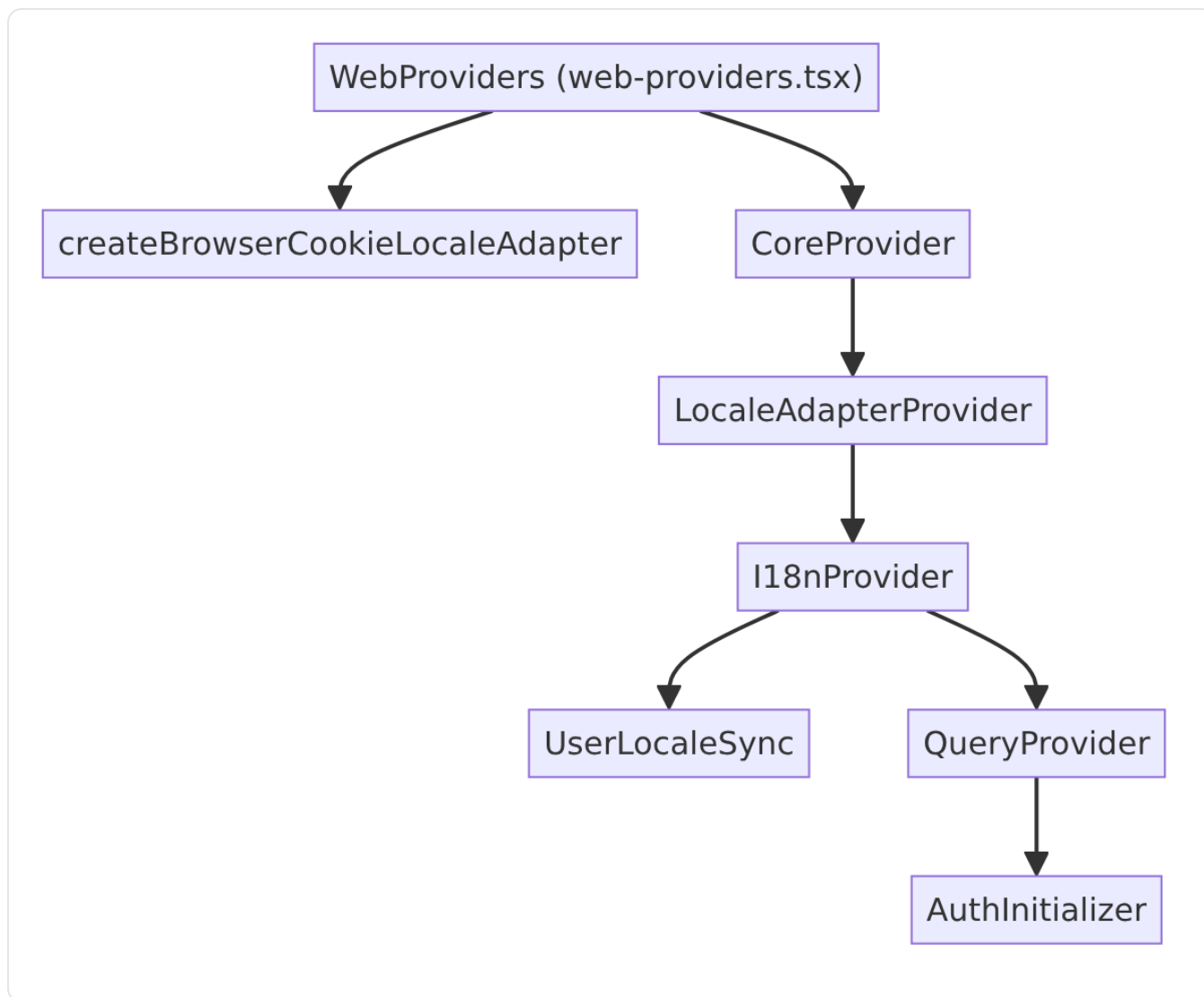
来源：[apps/web/features/landing/i18n/context.tsx](#)、[apps/web/features/landing/i18n/types.ts](#)

文档链接本地化

`docsHrefForLocale` 函数根据当前语言返回对应的文档路径：中文返回 `/docs/zh`，韩语返回 `/docs/ko`，日语返回 `/docs/ja`，其他语言返回 `/docs`。

来源：[apps/web/lib/docs-href.ts](#)

Web 端组件树



Web 端 `WebProviders` 组件通过 `createBrowserCookieLocaleAdapter` 创建浏览器适配器，然后将其与 `locale` 和 `resources` 一同传入 `CoreProvider`。[apps/web/components/web-providers.tsx](https://github.com/ant-design/ant-design/blob/master/packages/react-intl/src/web-providers.tsx)

桌面端组件树

桌面端在 `App.tsx` 中直接调用 `createDesktopLocaleAdapter(systemLocale)` 和 `pickLocale(adapter)` 解析初始语言，然后与 `RESOURCES` 一同传入 `CoreProvider`。桌面端没有服务端代理层，语言解析完全在客户端完成。

来源：[apps/desktop/src/renderer/src/App.tsx](https://github.com/ant-design/ant-design/blob/master/packages/react-intl/src/desktop-renderer/src/App.tsx)